# TeXShop Manual
# Version 1.4.8

Richard Koch

November 7, 2026

# Preface

This manual was started in early 2023. It first appeared in TeXShop 5.12, released on February 23, 2023. In August, 2023, Uwe Schmock added reference numbers to illustrations, reworded the text in a few spots, and improved formatting in other places. In January 2024 he contributed Chapter 28 to this manual. Thanks! I asked for permission to include his email address; it is schmock@fam.tuwien.ac.at.

The version number of the manual increases when changes are made. Small changes are made with most new TeXShop releases, including additional chapters from time to time.

Version 1.4.8 updates the manual for TeXShop 5.60. Starting with TeXShop 5.59, TeXShop is maintained by Yusuke Terada; the passages written in the first person are by Richard Koch.

# Contents


**1 About TEX, LATEX, and TeXShop** **1**
- 1.1 Introduction . . . 1
- 1.2 Donald Knuth and the Creation of TEX . . . 4
- 1.3 TeX and LaTeX . . . 5
- 1.4 Outline Fonts . . . 6
- 1.5 MetaFont Today . . . 7
- 1.6 Front Ends and TeXShop . . . 8
- 1.7 Brief History of TeXShop . . . 8

**2 Installing TEX on a Macintosh** **10**
- 2.1 TeX Live . . . 10
- 2.2 Installing TeX Live and TeXShop . . . 11
- 2.3 Capabilities of TEX . . . 11
- 2.4 Understanding TeXShop . . . 12

**3 Getting Started** **13**
- 3.1 First Steps with TEX and TeXShop . . . 13
- 3.2 A Few Unfortunate System Preferences . . . 16
- 3.3 Second Steps with TEX . . . 16

**4 Behaviors Inherited from macOS** **18**

**5 Configuring TeXShop** **19**
- 5.1 TeXShop Preferences . . . 19
- 5.2 ∼/Library/TeXShop . . . 20
- 5.3 Templates . . . 20

**6 Editing Preferences** **22**
- 6.1 Preference Design . . . 22
- 6.2 Source Font and Font Size . . . 23

6.3 Other Source Preference Items . . . 24
6.4 Other Editor Preference Items . . . 25
6.5 Parenthesis Pairs in the Source Editor . . . 28

**7 Encodings** **31**
7.1 About Encodings . . . 31
7.2 More on Encoding Issues . . . 34
7.3 Changing Encodings of Files . . . 35

**8 Typesetting Engines I** **36**
8.1 History . . . 36
8.2 Configuring TEX and LATEX Typesetting . . . 37
8.3 Bibtex . . . 39
8.4 MakeIndex . . . 41
8.5 Other Typesetting Preference Settings . . . 41

**9 Typesetting Engines II** **44**
9.1 More History . . . 44
9.2 Engines . . . 45
9.3 Typesetting with an Engine . . . 46
9.4 latexmk . . . 47
9.5 latexmk History . . . 48
9.6 The Inactive Folder . . . 49
9.7 Creating a New Engine . . . 49
9.8 Useful Engine Commands . . . 50
9.9 Recent Additional Engine Commands . . . 52

**10 Projects with Multiple Input Files** **54**

**11 Goto Error, Trash Aux Files, SyncTeX** **56**
11.1 Goto Error . . . 56
11.2 Trash Aux Files . . . 57
11.3 SyncTeX . . . 58

**12 Synchronizing Page Numbers** **60**
12.1 Synchronizing . . . 60
12.2 Activating the Fix . . . 61
12.3 Outputting the Crucial Line . . . 62
12.4 Earlier Way to Fix . . . 62

**13 Organizing Windows in TeXShop** **64**
13.1 Introduction . . . 64

13.2 Single Window Mode . . . 64
13.3 Multiple Windows with Tabs . . . 65
13.4 Single Window Mode and Tabs . . . 68
13.5 Obsolete Tab Commands . . . 70

**14 Macros** **71**
14.1 Introduction . . . 72
14.2 Default Macros . . . 73
14.3 Rearranging Macros . . . 73
14.4 AppleScript Macros . . . 74
14.5 Included AppleScript Commands . . . 74
14.6 Defining AppleScripts . . . 77
14.7 Dialogs . . . 79
14.8 Writing Complete AppleScripts . . . 80
14.9 Writing Command Scripts . . . 81

**15 Editing Aids** **85**
15.1 Emacs Mode . . . 85
15.2 Key Bindings . . . 87
15.3 Command Completion . . . 89
15.3.1 Introduction . . . 89
15.3.2 Herbert Schulz and Command Completion . . . 89
15.3.3 Completing Using First Few Characters . . . 90
15.3.4 Abbreviations . . . 91
15.3.5 Comments . . . 92
15.3.6 Other Environments . . . 92
15.3.7 When Typing \ is Difficult . . . 93
15.3.8 Editing the Command Completion File . . . 93

**16 Automatic Saving; Finding Documents on Disk** **95**
16.1 Automatic Saving . . . 95
16.2 Lock . . . 96
16.3 Revert To . . . 97
16.4 Finding the Disk Version of a Document . . . 98

**17 Preview Window** **99**
17.1 History . . . 99
17.2 Display Modes . . . 100
17.3 Magnification . . . 101
17.4 A Mysterious Preference Item . . . 102
17.5 Preview Mouse Tools . . . 104

17.6 Back, Forward, Rotation . . . 105
17.7 The Drawer and Two Search Fields . . . 105
17.8 Following Links and URLs . . . 106
17.9 Popup Windows and Hovering Over Links . . . 106
17.9.1 Modifying Popup Windows . . . 106
17.9.2 Too Much Magnification is Bad . . . 107

**18 Printing** **108**

**19 The Console** **110**

**20 HTML, Interactive TeX4ht, PreTeXt** **112**
20.1 Html Preview Window . . . 112
20.2 Html Files . . . 113
20.3 Help File for Html Commands . . . 113
20.4 TeX4ht . . . 114
20.5 Historical Intermission . . . 115
20.6 TeX4ht and Interactive Documents . . . 118
20.7 PreTeXt . . . 119
20.8 Special Features of TeXShop for html, xml, and ptx Files . . . 121
20.8.1 Syntax Coloring . . . 121
20.8.2 The Tags Item in the Toolbar . . . 121
20.8.3 Keyboard Tricks with Tags . . . 122
20.8.4 Command Completion . . . 123

**21 ConTeXt** **124**
21.1 ConTeXt Garden . . . 125
21.2 SyncTeX in ConTeXt . . . 126
21.3 2024 and Beyond . . . 126
21.4 Typesetting the Traditional ConTeXt in TeX Live . . . 127
21.5 Typesetting ConTeXt in TeX Live using luametatex . . . 127
21.6 Installing the ConTeXt Garden Version . . . 128
21.7 Typesetting ConTeXt from ConTeXt Garden . . . 128

**22 Typst** **130**
22.1 Introduction . . . 130
22.2 Packages and Templates . . . 131
22.3 Other Samples . . . 132

**23 Using an External Editor** **133**
23.1 Basics . . . 133
23.2 Preliminary Version: Using Sync with an External Editor . . . 135

23.3 More on External Sync . . . 136
23.4 Irrelevant Comments . . . 138
23.5 More Recent Interaction Methods . . . 139

**24 Themes** **146**

**25 expl3 Syntax Coloring** **151**
25.1 Latex3 and expl3 . . . 151
25.2 Activating expl3 Syntax Coloring . . . 151
25.3 expl3 Coloring Rules . . . 152

**26 Splitting Windows** **154**
26.1 Splitting Source and Preview Windows . . . 154
26.2 The Active View in a Split Window . . . 154
26.3 Switching Views . . . 156
26.4 Revising the Document and Typesetting . . . 157
26.5 Changing the Display Format While Using Switch Views . . . 157
26.6 Avoiding Creep When Unsplitting . . . 158
26.7 Two Options When Splitting Horizontally . . . 158
26.8 Splitting Vertically . . . 159
26.9 Final Note . . . 159

**27 Switching Preview Views** **160**
27.1 Back and Forward Arrows . . . 160
27.2 Switch Views . . . 160
27.3 Independence from Split Window . . . 162

**28 Presenting with TeXShop (Schmock)** **163**
28.1 Introduction (Koch) . . . 163
28.2 Presenting (Schmock) . . . 164

**29 Additional TeXShop Features** **171**
29.1 Experiments . . . 171
29.2 Tags Toolbar Item and Menu; Labels Item . . . 173
29.3 Latex Panel, Matrix Panel, Unicode Panel . . . 174
29.4 Show Invisible . . . 176
29.5 A Magic Line for Beamer and Other Slide Packages . . . 176
29.6 Counting Words in a Document . . . 176
29.7 Useful Items in the TeXShop Menu . . . 177

**30 Other Miscellaneous Features** **178**
30.1 Items in the TeXShop Help Menu . . . 178

30.2 Help with Style and Class Files . . . 178
30.3 Opening Other Files . . . 179
30.4 Arabic, Hebrew, Persian . . . 179
30.5 Localizations . . . 180
30.6 Redefining Menu Shortcut Keystrokes . . . 180
30.7 The Block Insertion Cursor . . . 181

**31 Spell Checking** **183**
31.1 Introduction . . . 183
31.2 TeX-aware Spell Checking . . . 184
31.3 Dictionaries . . . 185
31.4 Spell Checking LaTeX Commands . . . 186
31.5 cocoAspell . . . 187
31.6 cocoAspell Usage (Koch) . . . 188
31.7 cocoAspell Installing (Schulz) . . . 189
31.7.1 Introduction . . . 189
31.7.2 First Check for `make` . . . 189
31.7.3 Building the `cocoAspell` Dictionaries . . . 190

**32 Annotations** **191**
32.1 Annotations in General . . . 191
32.2 Who Defined Annotations? . . . 191
32.3 Using LaTeX to Annotate . . . 193
32.4 Why Add Annotations to TeXShop? . . . 194
32.5 Annotating in TeXShop . . . 194
32.6 Adding Annotations to a Page . . . 195
32.7 Editing Text in Annotations . . . 196
32.8 Changing Annotation Colors . . . 197
32.9 Changing Annotation Fonts . . . 197

**33 More on Annotations** **198**
33.1 Saving Annotated PDF Files . . . 198
33.2 Removing Image Streams . . . 199
33.3 Guard Rails . . . 200
33.4 Interacting with Adobe Acrobat . . . 200
33.5 Interacting with Preview . . . 201
33.6 History of the Annotation Code . . . 202
33.7 Arrows . . . 202
33.8 Don't Do This . . . 203

**34 Fermat as Annotator** **204**

34.1 A Difficult Theorem of Fermat . . . 205
34.2 Euclid's Formula . . . 206
34.3 Progressions of Squares . . . 207
34.4 The Method of Descent . . . 208
34.5 Fermat's Argument by Descent . . . 209
34.6 Generalizing Fermat's Area Theorem . . . 210

**35 Writing Tools** **211**

**36 Japan** **213**

**37 Updating TeXShop** **215**

**38 Opening Encrypted PDF Files** **217**

**39 Left Over Items** **219**
39.1 Menus . . . 219
39.2 Items in the Preference Pane . . . 221
39.3 Modifying the Latex Panel . . . 222
39.4 Adding to the List of Files That Can Be Typeset . . . 223

**40 Magic Comment Lines** **224**
40.1 Commonly Used Magic Lines . . . 224
40.2 Magic Lines to Control Preview in Double Page Modes . . . 225
40.3 Magic Lines for ConTeXt . . . 226
40.4 A Magic Line for Beamer . . . 226
40.5 Obsolete Magic Lines . . . 227
40.6 Pseudo-Magic Lines for Engine Files . . . 227

**41 Hidden Preferences** **229**
41.1 Introduction . . . 229
41.2 Long Standing Hidden Preferences . . . 231
41.3 Newer Hidden Preferences . . . 234
41.4 Obsolete Hidden Preferences . . . 238

**42 Unusual Bugs** **245**
42.1 Tags . . . 245
42.2 Zombie Untitled Windows . . . 245

**43 Contributors** **247**

# Chapter 1

# About TeX, LaTeX, and TeXShop

## 1.1 Introduction

TeX and LaTeX are typesetting programs available without cost on most computer platforms. The user enters instructions using symbols on an ordinary typewriter, and the program converts these instructions into a document that looks like it came from a professional book publisher.

TeX has been carefully designed to produce exactly the same output on all computer platforms. If the source file is written on a Macintosh and sent to someone using Windows, Linux, or Unix, the typeset output on both platforms will use the same fonts, have the same line breaks, have the same page breaks, and look identical. This makes collaboration easy.

TeXShop is a GUI front end for TeX. In contrast to TeX, TeXShop runs only on the Macintosh, making extensive use of the interface features available on that platform. It is available without cost.

Making TeX platform independent, but making the front end dependent on the platform is a deliberate design decision. Books, articles, letters, and slides can be read by anyone. But when an author is writing such a document, the author should be able to use all the advanced features that a particular operating system provides.

TeXShop displays an editing window where users type the text they want to typeset. It provides a button which users push to call the typesetting program, and it provides a preview window where the typeset document is displayed.

TeXShop does not do the actual typesetting. That task is performed by the TEX and LATEX command line programs not visible to an ordinary user. TeXShop does not manipulate the source in any manner before passing it to TEX to typeset. Below is a typical TEX source file.

```
\documentclass[11pt,oneside]{article}
\usepackage[parfill]{parskip}

\title{Short Sample}
\author{Richard Koch}

\begin{document}

\maketitle

\section{Introduction}
This is the first sentence. We type
{\bold this command} to get bold type and
{\emp this command} to get italics.  TeX ignores       extra
spaces because it knows how to correctly format.

A blank line starts a new paragraph.

\TeX\ produces beautiful mathematics, like
$\sqrt{x^2 + 5 x + 6}$ and $\Gamma^i_{j_1 j_2}$ and
$$\int_0^\infty e^{- x^2}  \ dx = {{\sqrt{\pi}} \over 2}$$

\end{document}
```

Below is the output when this source is processed by TEX:

# Short Sample

Richard Koch

September 13, 2022

## 1 Introduction

This is the first sentence. We type **this command** to get bold type and *this command* to get italics. TeX ignores extra spaces because it knows the rules of typesetting.

A blank line starts a new paragraph.

TEX produces beautiful mathematics, like $\sqrt{x^2+5x+6}$ and $\Gamma^i_{j_1 j_2}$ and

$$\int_0^\infty e^{-x^2}\ dx = \frac{\sqrt{\pi}}{2}$$

## 1.2 Donald Knuth and the Creation of TEX

Donald Knuth is a professor of computer science at Stanford University. In 1962 while he was a Caltech graduate student, Knuth agreed to write a book on theoretical computer science. He began the first draft in 1963 after receiving his PhD, and finished it in 1965. His initial draft had 3000 hand-written pages, which Knuth believed would shrink to about 600 printed pages. But the publisher informed him that in fact those 3000 pages would produce about 2000 printed pages. Plans changed after that, and the publisher agreed to publish the work in seven volumes, to be called *The Art of Computer Programming*. These volumes are classics, owned and consulted by most computer scientists working today. The first three volumes were published in 1968, 1969, and 1973, and work on volume four began in 1973.

A couple of years later, Knuth revised volume two for reprinting. But when he got the galleys from the publisher in March, 1977, he was unhappy with the result. He learned that the hot lead process used originally had been replaced with photo-offset printing, and thus with a different collection of fonts. Knuth told the publisher to postpone further action for six months while he consulted experts at Stanford and obtained better fonts for the system. That six month project actually took ten years, as Knuth brought typography experts to Stanford and learned the intricate details of typesetting.

Knuth wrote a specification of TEX in May, 1977. The first version was released in 1978, written in the SAIL programming language then in use at Stanford. In that same year, Knuth gave the 1978 Gibbs Lecture, an annual lecture for the American Mathematical Society. An expanded version was published by the AMS in December of 1979, titled *TeX and Metafont, New Directions in Typesetting*. That is how mathematicians first learned of the project.

In 1982, Knuth substantially revised TeX and rewrote the entire program in Pascal. This is the version that most early adopters first saw. Use of the program gradually increased during the 1980's, as experts from other countries began making important contributions. Originally, each font used in TeX had at most 128 characters. This was sufficient for the letters on an American typewriter, but not sufficient for the umlauts, accents, upside down question marks, and other characters used in Western Europe. The Europeans convinced Knuth to increase font table sizes to 256 characters for version 3.0 of TeX, released in 1989. In 1991, Knuth froze the design of TeX, believing that the stability of an unchanged system was more important than new features. As we will see, other programmers later created pdfTeX, luaTeX, XeTeX, and other systems with Knuth's approval, but if a program is named TEX, it is the 1989 version by Knuth.

Knuth returns to the program every few years to fix minor bugs. The version as of 2022 is 3.141592653. Knuth has decreed that the version number will be set to exactly $\pi$ when he dies, and the code will not change after that.

Knuth's typesetting project produced three important products: TeX, MetaFont, and the Computer Modern Fonts. He described the results in five books, standing alongside the volumes of the *Art of Computer Programming*: one book with the complete TEX program, one book with the complete Metafont program, one book showing the design of the Computer Modern Fonts with an entire page devoted to each character, one book with the instruction manual for TEX, and one book with the instruction manual for MetaFont.

After TEX was frozen, Knuth resumed work on the Art of Computer Programming. Plans for Book 4 had expanded, and Volume 4A, published in 2011, was followed by volume 4B published in October, 2022. Coming next is volume 4C and perhaps 4D. These will be followed by Volumes 5, 6, and 7, time permitting.

## 1.3 TeX and LaTeX

The TeX program understands a limited number of very primitive commands. Because these commands are primitive, several must be strung together to produce desired results. TeX allows users to define *macros*, new commands made by stringing primitive commands together, and Knuth wrote a large collection of such macros known as "Plain TeX." When users today say they use TEX, they are really claiming to use TEX and the Plain TeX macro package.

In the early 1980's, Leslie Lamport wrote a much more extensive set of macros called LATEX. Some users today think of TEX and LATEX as separate programs, but that is incorrect; instead TEX is the TeX typesetting program used with the Plain TeX macro package, and LATEX is the TeX typesetting program used with the LaTeX macro package.

Versions of LATEX were released in 1984 and 1985. In 1986 Lamport published a user-manual for LATEX titled *LaTeX User's Guide and Reference Manual*. This has remained a standard reference ever since. The manual covered LATEX 2.09, the version then being used.

Meanwhile, Michael Spivak wrote a series of macros for mathematicians called the AMS-TeX Macro Package and documented this package in his book *The Joy of TeX: A Gourmet Guide to Typesetting*.

This led to requests to combine LATEX and AMS-TeX, which turned out to be a non-trivial task. In 1989, Lamport turned over maintenance of LaTeX to Frank Mittelbach, who then formed a team with Chris Rowley and Rainer Schopf to update LaTeX to version 3. In 1994, they released a preliminary version, known as LaTeX2e. This version made it possible to merge LaTeX with AMS-TeX, and has become the standard version of LaTeX. Lamport updated his manual to cover it, and all current versions of the LaTeX User's Guide document LaTeX2e.

Since LaTeX2e accomplished the main task of the upgrade process, work on version 3 gradually subsided. But more recently the project has become active and new versions of LaTeX often appear.

## 1.4 Outline Fonts

Knuth thought his project was about two equally important programs, TeX and MetaFont. MetaFont is the program used to design font families, and to manipulate these families. In particular, it has code to rasterize fonts, that is, to convert them into an array of dots for the screen or printer.

The key feature of fonts created with MetaFont is that they are defined mathematically rather than as a series of dots. Once the mathematical description of a character is given, it can form a letter so small that it is barely visible, or so large that it covers a complete page. Mathematically-defined fonts are often called *outline fonts* and that is the term I will use, although technically Knuth's fonts weren't outline fonts.

Knuth was the second person to introduce such fonts. For an interesting history, see https://simoncozens.github.io/fonts-and-layout/history.html.

It isn't enough to design a program which can manipulate these new fonts; it is also necessary to actually produce such a font. TeX requires many variations: bold, italics, ligatures, etc. Moreover, fonts with a large number of mathematical symbols are required. The creation of these fonts is the third leg of the typesetting stool which Knuth constructed. He called his fonts *Computer Modern Fonts*.

In 1984, Adobe invented its own version of outline fonts, known as *Adobe Type 1 Fonts*, and used them in Postscript, which it licensed to makers of Postscript Printers. The first fonts in macOS were bitmap fonts, but Apple soon realized the importance of outline fonts. After attempting unsuccessfully to license Adobe Type 1 fonts, Apple created its own outline fonts, *TrueType Fonts*. These fonts first appeared on the Mac in 1991. Apple licensed these fonts to Microsoft and soon they were used on both Macs and PCs. This caused Adobe to release the specifications for Adobe Type 1 fonts, and eventually the Mac could also use these fonts. The history summarized here is actually much more complicated; see https://www.truetype-typography.com/tthist.htm.

The final step in the process was the creation of OpenType fonts by Microsoft and their adoption by both Apple and Windows. The font data in such a font can be either TrueType or Adobe-Type-1, but the font package also contains additional information used to display glyphs, like kerning tables and ligature information.

## 1.5 MetaFont Today

When outline fonts are used on a computer platform, it is essential to postpone the rasterization step until the last moment. If a typeset document is saved in pdf form, the pdf should contain the mathematical version of the fonts rather than the bitmaps because the pdf may be viewed on many devices with different resolutions.

This is not possible with fonts created by MetaFont, because neither Apple nor Microsoft adopted Knuth's formulas for outline fonts. TeX distributions still contain MetaFont, and an enormous number of fonts created for it over the years. But using these fonts brings many disadvantages, and for twenty years a major project in the TeX world has been modifying the system so it can use TrueType, Adobe Type 1, and OpenType fonts. Part of this project involved creating the LatinModern fonts, which are Adobe Type 1 versions of Knuth's Computer Modern Fonts, expanded to contain the symbols commonly used in Western Europe.

You might think that such a replacement would involve an enormous rewrite of TeX itself, but that was not necessary. The original TeX program output a *device independent file*, that is, a dvi file. This file did not contain any fonts. Instead it contained instructions of the following form:

```
select font named "CM" of size "16 pt"
print glyph 43 at position (23, 69)
print glyph 68 at position (32, 69)
etc.
```

Separate programs then handled the job of reading the dvi file, opening the corresponding font files, rasterizing them, and sending the final data to the printer. The most famous was dvips, written by Thomas Rokicki, which created a postscript file with rasterized font information from a dvi file and associated mathematical font files.

Around 2001, modified versions of TEX and LATEX named pdfTEX and pdfLATEX were created by Hàn Thế Thành. These programs contained the original TEX and could output dvi files, but they could also output pdf files directly. These pdf files could contain either mathematical descriptions of the fonts in Adobe Type 1 or TrueType form, or rasterized MetaFont font data. Nowadays, most people typesetting with TEX or LATEX are actually using pdfTEX or pdfLATEX.

Knuth thought of MetaFont, TEX and the Computer Modern Fonts as equal partners in the new typesetting system. Nowadays, MetaFont has mostly been replaced by font technology from other companies, (although the program plays an important role in some projects). and the Computer Modern Fonts are one of several commonly used font families for technical documents. TEX remains unsurpassed as a typesetting engine.

## 1.6 Front Ends and TeXShop

TEX is a command line program. A user creates a source text file, say Sample.tex, and typesets it to produce an output file Sample.pdf. The call to activate TEX can be done using Terminal or some other Unix shell, but otherwise there is no need to directly access the TEX program. Often the program is hidden away in the system and hard for a casual user to even find.

Interaction with TEX is usually done via a Graphical User Interface (GUI) front end. This is a standard program which provides an editing window used to create the source file, a button to call the typesetting engine, and a preview window to display the resulting output file. Many front ends are available, often open source and free.

TeXShop is a GUI front end for the Macintosh. It was introduced in 2001, while macOS was still in beta, and has been extended ever since. A great many programmers all over the world contributed code to the program. Since the actual TEX program is invisible, sometimes users get the false idea that TeXShop is itself doing the typesetting. Nothing could be farther from the truth. TeXShop does not modify the source in any way before typesetting. It simply calls TEX, saying in effect "Here's a source file. Typeset it and call me when you are done. In the meantime, I'm going to take a little nap."

## 1.7 Brief History of TeXShop

I first became acquainted with TEX and LATEX in 1991 when I bought a NeXt machine. This computer came with a TEX distribution, and a front end by Thomas Rokicki. His front end made use of display postscript, the graphic system on the NeXT. It was a wonderful setup, and I gradually switched to it from my previous Macintosh machines.

Around 1995, Apple ran into serious trouble. It was unable to complete its new operating system Copland, and there were rumors that the CEO was trying to find a buyer for the company. In December, 1996, Apple announced that it would use NeXtStep as its new operating system, having bought NeXT lock, stock, and barrel. And Steve Jobs would return as an unpaid consultant.

This announcement thrilled me. At last the Mac would have an honest Unix foundation, advanced user interface, and modern programming environment. As a mainstream machine, it would support standard commercial programs like Word and Excel. For mathematics it only needed a Web browser, an email program, Mathematica, and TEX, and the first three would clearly be available. I began pestering our Apple representative about the importance of TEX. He agreed, but I saw few signs of progress.

I began running beta versions of OS X, and soon had TEX itself running. I tried to compile the Rokicki front end, but Apple switched their graphic system from display postscript to

Quartz, a PDF-based system, so that was a no go. I had written a few NeXtStep programs, and finally one day I thought that it should be possible to write a front end myself.

I do not know when I started coding TeXShop. I know for sure that I had a working TeXShop by May of 2000. I suspect the program was started not much earlier, perhaps in the spring of 2000.

The 2000 Worldwide Developer Conference, WWDC, was in San Jose in May. At this conference, Apple was slated to give developers the release version of OS X, which would be sold to users several months later. I was only teaching one course that spring, a Discrete Mathematics course for Computer Science students. The authors of the book were also at the University of Oregon, so I asked them to teach my class for a week and went to the conference. Just before this conference, I bought a 17 inch PPC Portable. I immediately erased the hard disk and partitioned it so I could install OSX release at the Conference.

In the keynote address in 2000, Steve Jobs said that in the computing industry, marketing considerations sometimes dictate how software is presented. He then announced that the release version of OS X would instead be named OS X Public Beta and instead of costing $120, it would be sold for a $15 handling fee. After the keynote, a friend said to me "Wasn't that smooth? Jobs just announced that OS X is delayed a year."

By this time, TeXShop was definitely working on beta versions of OS X, but the Apple PDF software had a significant bug: the software could not read embedded fonts in files. Consequently, TeX documents written with Times Roman displayed beautifully, but any mathematics in the document would appear as blank areas. I hurried back to the motel at the end of the Monday session, installed the public beta, and ran TeXShop. Embedded fonts still did not display.

The first release of TeXShop to the open source community occurred shortly afterward, on July 23, 2000. This version called Ghostscript to rasterize pdf output so it could display mathematical fonts. This created a somewhat fuzzy final display. All the while, Apple was working on OS X, and in March of 2001 they announced the actual release version. This time, developers got their copy only a week before the official release. When I got my copy, I immediately tried TeXShop. Apple had fixed the embedded font problem. I went to the UO and bought five letter sweaters with the University of Oregon logo, and send them to Apple's developers as my thanks for the bug fix. Then I ripped out that call to Ghostscript and released the first official version of the program. That code, of course, ran on 32 bit PowerPC processors. But recently I recompiled it for 64 bit Intel and Arm processors and it still ran. The initial version, with source code, is on my web site. After the release, an enormous number of people from all over the world contributed to the program. Their names are listed in the "about file". So it is a joint effort of all of these developers, most of whom I have never met.

# Chapter 2

# Installing TEX on a Macintosh

## 2.1 TeX Live

To use TEX and LATEX on the Macintosh, you must install two items: a TEX distribution and a front end.

A TEX distribution contains thousands of files, because as TEX grew more popular, people all over the world contributed to the project. It contains the actual programs, but also hundreds of fonts, style file, class files, scripts, and documentation files.

There are several different TEX distributions for the Macintosh, and TeXShop can work with all of them. Some people install TEX using *MacPorts* or *Homebrew*, projects porting command-line Linux programs to the Mac. The most common Windows distribution, *MikTeX*, is also available on the Macintosh.

But by far the most common distribution on the Mac is *TeX Live*. This distribution works on all computer platforms: Macintosh, Windows, Linux, Unix, etc., using the same support files, fonts, etc. The command-line programs, which must vary depending on computer platform, are compiled from the same source files. TeX Live is produced by several cooperating TeX User Groups across the world, pulling sources from *CTAN*, the main repository of TeX-related files.

Installing this distribution on the Macintosh is very easy because it has been packaged as a standard install package using Apple's install software. To install from such a package, double click and answer a few straightforward questions, push a button, and watch installation proceed. At the end the system is completely configured and both TeX Live and TeXShop are ready to typeset.

## 2.2 Installing TeX Live and TeXShop

Two install packages are provided at https://tug.org/mactex/, named *BasicTeX* and *MacTeX*. Each installs vanilla versions of TeX Live; nothing has been modified for the Mac. These two distributions do not interfere with each other, so one can be installed and then later the other can be installed without erasing anything. If both are installed, both can be used and switching from one to the other is easy. But it is also easy to erase an installation by just dragging one folder to the trash.

BasicTeX is for users experimenting with TEX, who are perhaps uncertain whether they will keep using the program. The package is small, about 94 megs, but capable of typesetting with either TEX or LATEX using most facilities.

MacTeX is for users who definitely need TEX for serious projects. It is large because it installs everything that a user could conceivably need. The size of the package is 4.7 gigabytes. It can be difficult and frustrating to add extra fonts, styles, or classes to TEX after it is installed, so installing everything avoids that frustration. After MacTeX is installed, essentially any example from a book about TEX will just work.

MacTeX is sometimes criticized for installing too much, including items the user will never need. But the advice to install everything comes from talks given at TeX User Group Conferences by speakers who used TEX for less than a year. They unanimously advised installing everything and thereby avoiding frustration.

MacTeX installs everything in /usr/local/texlive/2024. BasicTeX installs everything in /usr/local/texlive/2024basic. These distributions are updated once a year, so the 2024 will become 2025 around April of 2025. To uninstall MacTeX, just drag the 2024 folder to the trash.To uninstall BasicTeX, just drag the 2024basic folder to the trash.

MacTeX installs TeXShop in /Applications/TeX. After installing, a user should select the menu item “Check for Updates” to update TeXShop to the very latest version.

BasicTeX does not install TeXShop. To obtain it, go to https://pages.uoregon.edu/koch/texshop/texshop.html. Download the program and drag it to /Applications or /Applications/TeX.

## 2.3 Capabilities of TEX

The typesetting system installed by TeX Live is capable of typesetting in virtually any language used in the world. It supports the umlauts and accents used in Western Europe. It supports Cyrillic, Modern Greek, and ancient Greek. It contains CJK fonts for work in Chinese, Japanese, and Korean. It can typeset in Arabic, Hebrew, and Persian, three languages written from right to left, and the appropriate source code will be then entered

into TeXShop also from right to left. The program has been used to typeset the complete Bible, and the complete Koran.

Many TeX contributors come from India; others come from Vietnam. These countries use entirely different scripts, but they are supported by TeX Live.

TeX Live was originally designed to support mathematical typesetting, and provides superb output of mathematics. But it is also used for poetry, novels, and the like in fields far removed from technology.

## 2.4 Understanding TeXShop

The initial versions of TeXShop were extremely straightforward, TeXShop became known as an easy to use but very basic front end. Over the years, additional facilities were added at the request of users, and today the program has features making TEX input fast and efficient. But these features make the program more complicated.

It is tempting to begin TeXShop work by examining all the menu items and opening TeXShop Preferences to understand and select the various options available. *I do not recommend this approach. Many features will only make sense after you have used TEX for several months.*

The next chapter lists everything you need to know to do serious work with TeXShop. I recommend paying close attention to these items, and then immediately starting to write TEX or LATEX code. Everyone knows how to use a basic editor, and that will suffice at first.

It will take from a week to a month to become proficient in TEX. After that, return to this manual and browse through the various chapters for extra information.

It is not necessary to read this manual linearly. Browse through it briefly to see what is covered, and then carefully read the chapters which matter at the moment.

# Chapter 3

# Getting Started

## 3.1 First Steps with TEX and TeXShop

Go to `/Applications/TeX` and find TeXShop. Drag its icon to your dock. Click this icon to run the program. You will be presented with a blank window; at the top right of this window you'll find a pull down menu named "Templates", see Figure 3.1. From this menu choose "LaTeX Template". The blank window will fill with some standard boilerplate required in each TEX source file. The red lines are comments which TEX ignores.

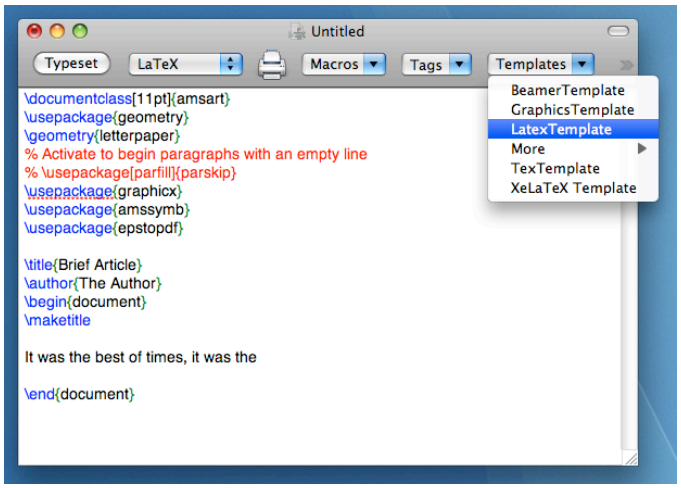



Figure 3.1: New Edit Window with LaTeX Template

New material goes between the lines

```
\begin{document}

\end{document}
```

Type some sentences there now. TEX will ignore most carriage returns because it knows how to format text, so insert them randomly if you wish. Use a blank line to indicate the start of a new paragraph.

Uncomment the source line with the words "usepackage[parfill]parskip" if you prefer to separate paragraphs with an empty line instead of an indent.

When you have some material, hit the "Typeset" button at the top left of the window. A dialog will appear asking you to save the document. You can give the document a name in a field at the top; a field below that shows the folder where the document will be saved. At the right side of this field there is a small arrow. If the arrow points down, click it and the dialog will expand to a form which allows you to navigate to other locations and create new folders.

When TEX typesets, it creates three or four additional files, so it is not a good idea to save directly to a location with many other files. Instead, create a new folder in a convenient location inside your home directory and put the source file inside this folder. This is easy using the dialog which just opened. Navigate to a reasonable location, say `~/Documents`. Then click the "New Folder" button at the bottom of the dialog. Choose a reasonable name for this new folder. Then name the document and save it. The default "Untitled" name will do, but serious projects require a meaningful name.

As soon as you save, TEX will typeset the document and open a second window showing the result, see Figure 3.2.

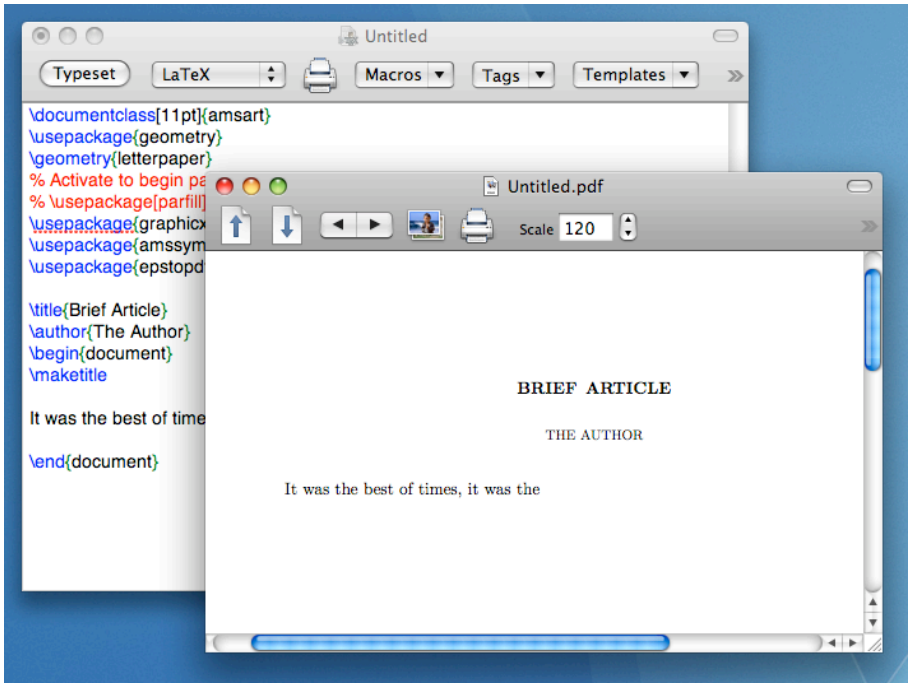



Figure 3.2: Typeset Window in front of the Edit Window

Go back to the original window and add some additional text. Hit "Typeset" again. This time TEX immediately typesets the new material and updates the output window.

Close the document by hitting the red button at the top of the edit window. There is no need to save the program first because TeXShop automatically saves regularly. Quit TeXShop.

Restart TeXShop by clicking on the icon in the dock.

It is possible to reopen the project by double clicking on the Untitled file or by starting TeXShop and navigating to Untitled in the Open dialog, but there is a much easier way. Start TeXShop and under the File menu choose the item "Open Recent". In the resulting list, choose the document's name and you will immediately return to it.

If you do not close the document before quitting TeXShop, then it may reappear automatically the next time you run TeXShop. This depends on a setting in Apple's System Preferences. Apple revised System Preferences in Ventura. If you are using an earlier system, run System Preferences and select the General module. If you are using Ventura, run System Preferences and select Desktop & Dock. In the resulting dialog, boxes appear labeled "Ask to keep changes when closing documents" and "Close windows when quit-

ting an app". If both are unchecked, then TeXShop and most other programs will reopen windows automatically if they were open when the program quit.

One configuration step is highly recommended when you first run TeXShop. Open a project which has both source and preview windows. Resize and position these windows as you prefer. Most users position the source window on the left half of the screen and the preview window on the right half of the screen, with both covering most of their half of the screen, but the choice is up to you. Select the source window and at the bottom of the TeXShop "Source" menu select "Save Source Position". Select the preview window and at the bottom of the TeXShop "Preview" menu select "Save Preview Position." After this step is done, all TeXShop windows will open in these positions.

Those are the basics. You will find further information in the TeXShop Help menu. The item "TeXShop Demos" contains two short movies. The first illustrates starting and typesetting a short document, and the second illustrates the configuration just made.

## 3.2 A Few Unfortunate System Preferences

MacOS comes with a few system preference settings that complicate working with TEX. It is useful to reset them immediately.

In the Keyboard module of System Preferences under the Text tab on systems before Ventura, and in the same module in Ventura obtained by pressing "Edit" after "Input Sources", turn off the items "Correct spelling automatically", "Capitalize words automatically", and "Add period with double-space". Automatic spell checking will remain on; the first item is more pernicious and can change text you type into completely different text. Moreover, it does not recognize TEX commands and will change them and thus break your source.

At the same spot, uncheck the item "Use smart quotes and dashes". TEX has an entirely different method of handling quotes and dashes, which this item will break.

In the General module on systems before Ventura and in the Desktop & Dock module in Ventura, the item "Prefer tabs:" presents a pull-down menu with the item "in full screen" selected. That choice is fine. The choice "always" may interfere with TeXShop's method of using tabs, to be explained much later.

## 3.3 Second Steps with TEX

The next crucial step is to learn TEX. The easiest way to do that is to pick one or two short introductions to TEX and work through them while trying examples in TeXShop. After a week or so working in this manner, it will be time to start using the program to write lecture notes and articles.

Ask friends where to start. Because it is how I started, I'd recommend Leslie Lamport's book *LaTeX, A Document Preparation System*. Be sure to get the second edition, which will say "Updated for LaTeX 2e". It would be enough to read chapters 2 and 3, a total of 52 pages.

TeX Live contains a free 139 page book on LaTeX, *The Not So Short Introduction to LaTeX2e*, by Tobias Oetiker, Hubert Partl, Irene Hyna and Elisabeth Schlegl. See `file:/Library/TeX/Root/texmf-dist/doc/latex/lshort-english/lshort.pdf`.

The link `https://learnlatex.org` leads to a series of online LaTeX lessons, with interactive examples, which were partly funded by the TeX User Group in 2020.

Another source is the first portion of George Grätzer's book *More Math Into LaTeX*. Grätzer gave permission to put this section in the TeXShop Help window, so you already have this as well.

After reading one of these short works, I'd recommend starting to use the program for serious writing. This will be frustrating at first, and you'll probably want to buy one of the excellent books on TeX for reference as you work. But don't ignore Google! A vast amount of information is available about TeX and LaTeX by asking succinct questions of Google. Try "how do I type a URL in LaTeX" if you don't believe me.

# Chapter 4

# Behaviors Inherited from macOS

The underlying Macintosh operating system has a few design features that can surprise users. TeXShop inherits these features and occasionally extends them.

For example, holding down the Option key while a menu is visible can sometimes change the names of items in the menu and add additional items. In the Finder's File menu, holding down the Option key changes "Close" to "Close All" and "Duplicate" to "Save As". The same change is made in TeXShop's File menu. Sometimes, users complain that TeXShop is missing "Save As", but it is present once you know this trick.

In the Finder's Go menu, holding down the Option key adds the item "Library" to the menu. This item opens the folder ∼/Library, which is hidden by default on the Macintosh.

TeXShop's Typeset menu has an item named "Typeset" which typesets the current document. As explained later, an error in typesetting can cause a malformed aux file. Once the error is fixed, it is necessary to remove the bad aux file before typesetting again. Holding down the Option key changes the item "Typeset" to "Trash Aux & Typeset".

TeXShop's Window menu contains the item "Split Window". As explained later, this item splits the window into an upper and lower portion, so independent pieces of the source or preview can be examined at the same time. Holding down the Option key changes this item to "Split Window Vertically", so the pieces are placed side by side rather than above and below each other.

# Chapter 5

# Configuring TeXShop

## 5.1 TeXShop Preferences

Although Macintosh computers are often used by a single person, the operating system is designed for multiple use. If your machine is used by several family members, each with their own login name, then a family member can change TeXShop's Preference settings without disturbing the settings when you use the machine.

Each user's home directory contains a special folder named `~/Library` containing the configuration files for that user. Because this folder's contents are highly technical, Apple sometimes hides the folder. To further confuse matters, there is also a system library denoted `/Library`, which has nothing to do with user preferences. TeXShop has a menu item in the TeXShop menu to open the location `~/Library/TeXShop`.

TeXShop's Preference settings are in `~/Library/Preferences` in the file TeXShop.plist. It is never necessary to edit TeXShop.plist because the items in this file are set by the Preferences Dialog opened by an item in the main TeXShop menu. In very rare situations, TeXShop.plist can be mangled when TeXShop is updated to a new version. This seems to happen to only a few people, and only when new preference items are added for a new version. When the problem occurs, the updated TeXShop refuses to open any file, or displays other bizarre behavior. To fix the problem:

- Quit TeXShop.
- Go to ∼/Library/Preferences and drag the item TeXShop.plist to the desktop.
- Restart TeXShop.
- In TeXShop's Preference panel, reset any items you may have changed.

## 5.2 ~/Library/TeXShop

Other configuration files are in subfolders of the folder `~/Library/TeXShop`. Users will definitely want to edit some of these files. Some will never be touched; others will be explained shortly. Use the menu item **Open ~/Library/TeXShop** to access these folders.

When TeXShop first runs, it creates the folder TeXShop and all of its subfolders. Users may then modify the contents of these folders. Occasionally, a user modification may hopelessly mangle the contents of a particular folder. In that case, move that folder to the desktop and then restart TeXShop. It will recreate the folder and give it default contents.

For example, the folder Macros contains the various macros available in the source window's toolbar. These macros can be edited using an editor built into TeXShop, as will be explained later. Suppose an ill-advised editing session results in corrupted macros. Move the Macros folder to the desktop and restart TeXShop. The Macros folder will be recreated with default values. Since the old Macros folder is still available on the desktop, you may be able to recover any uncorrupted changes you made earlier.

In most cases, updating TeXShop after it first runs does not modify folders because that would overwrite user edits and additions. A small number of folders *are* modified because they contain items which users can move into active locations, and we want these optional items to be up to date.

The following is a list of all folders which are rewritten during a TeXShop update:

- bin
- Documents
- Engines/Inactive
- ExternalEditorScripts
- HTML/Inactive
- New
- Scripts

## 5.3 Templates

Recall that the toolbar on the Source Window has a pulldown menu named **Templates**. Selecting one of these templates copies its contents to the current cursor location in the source. Each template is a standard .tex file, which can be created or edited using TeXShop. These files are stored in the Templates subfolder of `~/Library/TeXShop`. The Templates

folder can also contain subfolders containing template files. These become submenus in the Templates menu.

Templates can contain small snippets of code commonly used, like the code to include a graphic file, or the code to activate an interactive element in an TeX4ht document. But templates are more commonly used for the headers of standard types of documents. For instance, there can be headers for Articles, Books, Beamer Slides, Letters, XeTeX articles, LuaTeX articles, and so forth.

While TeXShop provides sample Templates, it is assumed that most users will create their own templates for articles and books. Your template should input all the standard packages you regularly use, as well as definitions and the like. As you work, you can easily add items to your standard templates to make them more robust.

Creating a new template with TeXShop is easy. If you like, start with an existing template like "LatexTemplate.tex". Duplicate it and rename the duplicate, leaving the new file in the Templates folder. Open this new file in TeXShop and modify it or add extra items. Done. The new template will automatically appear in the Templates pulldown menu.

# Chapter 6

# Editing Preferences

## 6.1 Preference Design

TeXShop has a Preference Panel, which can be opened using the **Preferences...** item in the main TeXShop menu. In macOS Ventura and later, this item is renamed **Settings**. The panel has tabs which display various related groups of preference settings. At bottom right are two buttons labeled **Cancel** and **OK**. It is necessary to push **OK** after making changes if you want these changes to take effect. Some changes take effect immediately, while others become active when documents are reopened.

TeXShop also has menu items to change its behavior. These items often duplicate items that can be set in Preferences. There is a big difference between setting a behavior with a menu item, and setting the same behavior in Preferences. *Modifications using menu commands affect only the active document, and only while it is open in the current session; modifications using TeXShop Preferences determine the default behavior when documents are first opened.*

For example, the Source menu has an item called **Show Syntax Color** and Preferences under the Edit tab has a check box labeled **Syntax Coloring**. The menu toggles syntax coloring on or off in the active source window. Select one of two source files, say file A, and toggle until syntax coloring is on. Select a second file B, and toggle until syntax coloring is off. Notice that each window remembers its state, so if you make file A active, it will be syntax colored, and if you make file B active, it will not be syntax colored. When file A is brought forward, the syntax color menu item is checked and when file B is brought forward, the syntax color menu item is unchecked.

The **Syntax Coloring** preference item does not change syntax coloring in these two files. Instead, it determines whether new documents will have syntax coloring or not when first

opened. Uncheck the item and press OK. Then open a new document and notice that it has no syntax coloring.

## 6.2 Source Font and Font Size

In a few cases, the design philosophy described above is slightly modified. An example is the method to set the font size and font in the source window. This is an important case, so we'll completely describe it in this section.

Suppose first that you want to magnify the source font temporarily, perhaps to look carefully at a formula or perhaps to show it to someone looking over your shoulder. This is easy. Activate the source window and then type command-plus to magnify the text and command-minus to shrink the text. Here command-plus means that the command key and the plus key should be depressed at the same time; command-minus means that the command key and the minus key should be depressed at the same time. Although the plus symbol is typed with the shift key, the shift key should not be used with this shortcut. Notice that only the active source window is affected by these changes. Moreover the changes are temporary; if the document is closed and then opened, it will have the standard font size.

These commands are actually menu shortcuts. The *Source* menu has a *Font* submenu, which has two items *Bigger* and *Smaller*; the key shortcuts call these menus. However making text bigger or smaller is a common task and the menu items are hidden away, so it is best to just remember and use the shortcuts.

The remarks about keyboard shortcuts in the previous two paragraphs apply to a U.S. keyboard. Many users, perhaps even a majority, will select another keyboard in the Keyboard Module of Apple's System Preferences (using the Input Sources item). These selections map physical keys on the keyboard to characters, and can change menu shortcuts. To define appropriate shortcuts for Font Sizes, read Section 30.6 "Redefining Menu Shortcut Keystrokes".

The source font itself can be changed using a menu command, but this is almost never done. Instead users make the change in Preferences so it will affect all future source windows.

To change the font in the Source Window, open the *Source* tab of Preferences. Push the *Set* button and the Font Panel will appear. Use this panel to select a new source font and its size. What happens next is one of those slight modifications of our design philosophy — the font and font size immediately change in all open windows. We do that so users can immediately see the effect of the change and experiment until they get exactly the correct behavior. Selection is not yet finished. In the end, push *Cancel* or *OK*. Pushing *Cancel*

reverts to the old font and size in all open windows, while *OK* accepts the change for all open windows and for windows opened in the future.

It is useful to choose a monospace font for the source file, since then all characters have the same width and list items and the like will line up.

**A Tricky Special Case** I once got a bug report from a user in Israel. He was writing a physics book in Hebrew. When the size of the book reached about 100 pages, TeXShop's editor slowed to a crawl. He would enter a couple of words and several seconds later those words appeared on the screen.

His source file was interesting. There were standard LaTeX commands in English starting from the left margin and flowing right, followed by Hebrew source starting from the right margin and flowing left. Each page contained many such switches in direction. While trying to debug the problem, I happened to change the source font and to my surprise the editor was suddenly snappy again. What happened?

The Macintosh offers many "virtual keyboards", which transform your existing keyboard to a different language. An item in the bar at the top of the screen can rapidly switch keyboards. He typed commands with the English keyboard, reached up and switched to the Hebrew keyboard, and typed the contents of the book.

Most standard Macintosh fonts do not contain Hebrew letters. But Apple's programmers know that and provide a special trick. When a user selects the Hebrew keyboard, the Macintosh silently switches to a font containing Hebrew. Thus to process his source file, the Macintosh was silently switching the source font many times on each page. When new text is entered into the source, some processing needs to be done to the entire source. So each new word required thousands of font switches.

The solution was simple: choose a source file which contains Hebrew. The Apple Font Book can list languages supported by each font on the system, and many of these fonts support a large number of languages, so finding a suitable source font was easy.

## 6.3 Other Source Preference Items

This is a useful time to consider a few other Preference items which affect the source window. These items occur under the first tab, Source. But several items in this tab are better explained later and will be skipped.

**Document Font** This was just explained.

**Source Window Position** This preference was set in chapter 3, when you were asked to fix the location and size of the source window and then select the item *Save Source Position*

in the Source Menu. That action selected the item *All windows start at fixed position*. I do not recommend the other choice, but try it if you wish.

**On Startup** If the first item is checked, TeXShop opens a blank document if it is opened without selecting a file. I recommend checking this when you first use TeXShop. Eventually it may become annoying and you can uncheck it. (External editors will be discussed in a separate chapter.)

**Parens Targets & Highlight Color** A preference item to be discussed in the next section briefly flashes an opening parenthesis when the corresponding closing parenthesis is typed. This preference determines which parenthesis pairs follow this rule. The behavior is useful for ( ), [ ], and { } pairs. It doesn't work well for angle brackets because inequalities don't produce matching pairs.

**Find Panel (after restart)**

TeXShop provides three different Find Panels. Select each and try it. After making a selection, TeXShop must be restarted before the new panel becomes active. The Apple Find Panel was the original choice in TeXShop twenty years ago. It is very basic. The Apple Find Bar is the newer method by Apple, used in Safari, XCode, and many other official Apple programs. It looks very minimal, but a close inspection shows that it has unexpected features. The OgreKit Find Panel is an open source project from Isao Sonobe in Japan. It is full featured and supports Regular Expressions. This is the panel that most users ultimately adopt. It is not the default choice when you first use TeXShop because the OgreKit Panel does not work well in the Chinese localization.

**Line Number Size**

The TeXShop editor displays line numbers in the left margin, unless these are turned off by a menu choice or second preference setting. These numbers are small so they are not distracting and work well with all font sizes. Some users found them hard to read. This setting allows users to request larger line numbers.

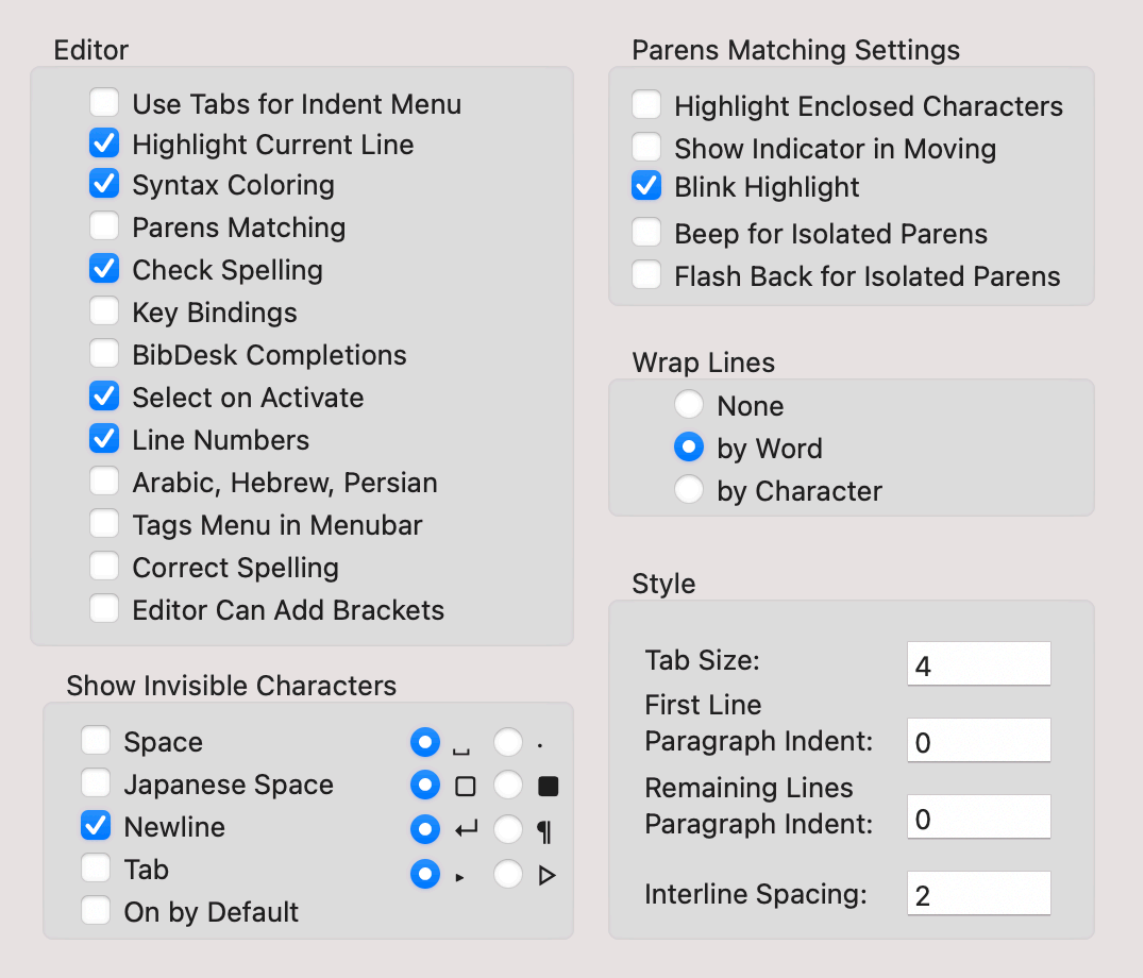
## 6.4 Other Editor Preference Items

The source window is also configured by various items in the Editor tab in Preferences. See the picture on the next page. We will discuss these items in a different order than listed in the Panel.

Several source window features can also be changed using menu items in the Source menu. We'll list these items first; the corresponding Preference items select the default behavior when a window is first opened. For example, the text in the source window is usually syntax colored, but a menu item can toggle this behavior for the active window. The **Syntax Coloring** Preference item determines whether syntax coloring is on when a window is first opened.

**Line Numbers** When selected, source lines are numbered.

**Highlight Current Line** When selected, the background of the line containing the cursor is tinted a light blue. Lines can cover several physical lines, ending when the user finally types a linefeed.



**Syntax Color** When selected, the source text is syntax colored.

**Check Spelling** When selected, misspelled words are underlined in light red.

**Show Invisible Characters** When selected, invisible characters like spaces, newlines, and tabs are shown. The preference allows users to pick which characters are shown and by what symbol. Starting with TeXShop 5.60, the symbol for spaces, Japanese spaces, newlines, and tabs is chosen from a pop-up menu next to each item. The final **On by default** item is usually left unchecked, so these characters are not shown when a window is first opened. If puzzling behavior later develops, it can be turned on using a menu item.

The remaining items are rarely changed, so they can only be set in the Preferences Pane:

**Use Tabs for Indent Menu** The third and fourth items in the TeXShop source menu are named *Indent* and *Unindent*. Select several source lines and then choose *Indent* to push these lines right by four spaces. Choose *Unindent* to push the lines back left. Indentation either adds spaces to the beginning of lines or else adds tabs. The preference item determines which are used. (Perhaps readers are familiar with the television program *Silicon Valley*, in which the hero loses a programming girlfriend because she uses spaces rather than tabs in her code.)

**Select on Activate** When checked, a click in the Source Window both selects the window and moves the cursor to that spot in the source. When unchecked, a click selects the window, but does not move the cursor.

**Correct Spelling** When selected, the Macintosh will not just underline misspelled words. It will also correct the spelling, and sometimes even change the wording. This item is present so the user can turn it off! We explain. A global preference turns correct spelling on or off for all applications. If this global preference turns it on, the **Correct Spelling** item can turn it back off in TeXShop while leaving it on in other applications.

**@ as Part of Command Names** When selected, syntax coloring treats the character @ as part of command names, as LaTeX macro authors do. It is on by default. Before TeXShop 5.60, this was the hidden preference MakeatletterEnabled.

**Color expl3 Code by Default** When selected, expl3 syntax coloring is on when a document is first opened. See Chapter 25. Before TeXShop 5.60, this was the hidden preference expl3SyntaxColoring.

The remaining items in the first column and the first block of items in the second column will be explained later in the manual.

**Wrap Lines** TeX usually ignores line feeds because it knows how to break lines. Some authors like to create a line feed near the end of each line, so their source contains a long list

of relatively short lines. Other authors like to create a line feed only between paragraphs, so their lines span multiple lines of the window. In that case, TeXShop inserts a soft line break between words so the cursors jumps down to the next line when appropriate. This line break is just for the display and isn't sent on to TeX. Said another way, if you enlarge the window, the breaks will be recalculated; they aren't permanent elements of the source. Long ago, some users didn't want breaks except if they actually typed them, hence this preference setting. I have no idea why anyone would change the default value, which is **by word**.

**Style** The first item sets the width of a tab; by default, the value is 4. The next two items format paragraphs of source created by writing several lines without line feeds. By default, TeXShop starts all lines of such paragraphs at the left margin, but it is possible to indent the first line and not remaining lines, or start the first line at the margin and indent remaining lines or format in other ways.

The last item sets the spacing between lines, so the source can be single spaced or double spaced or somewhere between these extremes.

Preference values for various style fields are integers. These values are constrained in various ways. Tab indent can be between 2 and 50, paragraph indent can be between 0 and 100, and interline spacing can be between 1 and 20. When larger or smaller values are entered, they are replaced with the closest allowed value.

If a source window is open when the style items are set, the changes will appear immediately in the source, so users can experiment with values until they find a pleasing set. However, there is a mild complication. When number entry boxes are used in preferences, changes in these boxes are not sent to the Mac until the box is deactivated. The easy way to deactivate a box is to activate another box by clicking in it. At that moment, the source window will be updated to reveal changes. It is tempting to press RETURN to deactivate a box, but the Preference dialog reacts to RETURN by accepting all changes and closing the Preference window.

*Aside:* The first version of this manual had a different explanation for the style items. It read "The first item sets the width of a tab; by default, the value is 4. The remaining items are left from an earlier version and do nothing. There are no plans to revive them." To my surprise, a user wrote telling me that the remaining items actually worked, but TeXShop had to be restarted to see the changes. Sure enough. So I revised the items to immediately show their effect in source windows.

## 6.5 Parenthesis Pairs in the Source Editor

The source editor inherits the following useful behavior from Cocoa: if you click once in a spot, the insertion cursor is moved to that spot; if you click twice in a spot, the full word

in that spot is selected; if you click three times in a spot, the full paragraph containing that spot is selected. To select a specific letter, click and hold the mouse down and slide slightly to pick up the letter.

TEX and LATEX make extensive use of the parenthesis pairs (, ) and [, ], and {, }, so double clicking on one of these characters does something different; it selects the matching member of the parenthesis pair and all text between them. Often this is just a few words or symbols, but it can be enormous sections of text set off by the pair. You will often use this feature to troubleshoot TEX errors.

Parentheses cause particular trouble when entering mathematical equations. It is all too easy to enter the wrong kind of parenthesis, or to forget a matching element. So TeXShop has several features to help with this process. Some of these come from my wonderful collaborator Yusuke Terada. The items are governed by the **Parens Matching** item on the left side of the Editing Preferences pictured earlier, and the various **Parens Matching Settings** at the top right of that picture.

Here is a brief list.

**Default** If all of these items are unchecked, then when a matching parenthesis is typed, both it and it match are colored red. After the next character is typed, these elements are given their standard syntax colors.

**Parens Matching**. If this item is checked, the previous behavior continues to hold, but in addition the matching parenthesis is very briefly given a yellow highlight. The effect is fairly subtle.

The remaining items come from the right side. Some of these items replace the behavior just described, and a few only work if the option to highlight the current line is turned off.

**Blink Highlight** When a matching parenthesis is typed, the matching parenthesis noticeably blinks.

**Highlight Enclosed Characters** When a matching parenthesis is typed, the text between it and its match flashes red for a moment. If the two parentheses are widely separated, this can be quite alarming, so use this feature only if you know what you are doing.

**Show Indicator in Moving** When a matching parenthesis is typed, the entire text between it and its match turns red. Moreover the red remains, even if the source is scrolled, until another character is typed. This requires that the current line not be highlighted, that the **Highlight Enclosed Characters** option be checked, and that the **Blink Highlight** option be unchecked. This option is also alarming and most people leave it unchecked. But if two parentheses include a large amount of text and there is a strange error in that section, the option can be temporarily useful.

**Beep for Isolated Parens** Make a short beep sound when an unmatched parenthesis is typed.

**Flash Back for Isolated Parens** Briefly flash the entire screen when an unmatched parenthesis is typed. This can also be alarming, so use this feature only if you know what you are doing.

# Chapter 7

# Encodings

## 7.1 About Encodings

In TeXShop Preferences under the *Source* tab, the top item in the second column is labeled **Encoding**. This item is a drop-down menu listing 31 choices. The default choice is **Unicode (UTF-8)** and that choice will work fine for most users. If you are just starting to work with TEX, by all means select UTF-8.

If you have older files written using a different encoding, or if other people send files in unusual encodings, it is important to know more.

A computer file contains a long stream of whole numbers, each between 0 and 255. These are called bytes, and the ability to encode virtually any kind of information into a stream of bytes defines the current digital age. Thus a byte stream might represent music on a CD, or a movie on a DVD, or a jpg picture, or a computer program, or an encyclopedia. Many computer files are ordinary text files. From the beginning of the personal computer era, text has been encoded into bytes using the ASCII (American Standard Code for Information Interchange) encoding scheme. This method encodes all the characters on a standard American typewriter: small letters, capitol letters, punctuation marks, numbers, tab, carriage return. There are less than 128 such characters, so ASCII only uses bytes between 0 and 127. In the original version of TEX, source files were standard ASCII files.

ASCII is not sufficient in Europe and in regions of the world that use completely different scripts. Scripts in Western Europe use accented vowels, umlauts, upside down question marks, and unusual Scandinavian letters. Russia and other countries use the Cyrillic alphabet. Greek is written with the Greek alphabet.

To solve such problems, programmers assigned the unused 128 entries to new characters.

Different encodings were invented for each country, each with special characters in those 128 spots. One of these encodings, IsoLatin1, contained all of the characters routinely used in Western Europe. When the European currency was introduced, IsoLatin1 was extended by adding the Euro symbol, to become IsoLatin9, and for a time TeXShop adopted that as default. Over the years, users requested additional encodings in TeXShop. This eventually led to the list of 31 encodings in the current version.

Sadly, files encoded in this way do not contain a "magic byte" defining the encoding. So a user has to remember which encoding was used to write each file, because the computer has no way to know.

But meanwhile, computer manufacturers realized that the increasing globalization of the world required a new approach to text. They formed an independent organization, which created Unicode, a standard that can represent all of the scripts of the world, including hieroglyphics, Arabic, Chinese, and mathematics. Virtually all computers have switched to Unicode; it is a central part of macOS. TeXShop is entirely Unicode-based internally. You can easily type English, Cyrillic, Chinese, Hebrew, and Arabic in a single TeXShop document; the Hebrew and Arabic will even be written from right to left.

Unicode does not define a standard method to read and write Unicode to a file. But one very popular method is called UTF-8 (Unicode Transformation Format, 8-bit). This method has the advantage that ordinary ASCII is correct UTF-8 output. The remaining unicode symbols are coded using an elaborate scheme we need not explain.

Because there is no standard way to write Unicode to disk, every routine in macOS to read or write text requires an *encoding parameter*, which describes the type of encoding to be used by the operation. If this parameter is UTF-8, then the contents of the editor will be completely preserved. If the parameter is IsoLatin9, then ASCII and Western European characters will be preserved, but Chinese, Arabic, Cyrillic, etc. characters will be lost.

There is one crucial difference between most of the encodings and UTF-8. Since most encodings just define characters for the bytes between 128 and 255, any random stream of bytes is an acceptable file. But UTF-8 files use coded input, so random streams will contain illegal bytes and a computer asked to read such a stream will reject the entire file. If TeXShop is told to read such a file as UTF-8, for instance by a magic comment line, it will put up an error dialog, and then open the file in IsoLatin9. If you save the file without changing its encoding magic comment line, TeXShop writes it back in IsoLatin9, so the bytes of the original file are kept.

Until recently, IsoLatin9 was the default encoding assumed by LaTeX. It could accept files encoded in other ways, but then a command in the source had to state the encoding. Two years ago, LaTeX switched to using UTF-8 as the standard encoding.

Users new to TEX should use UTF-8 encoding. Then anything typed in the source window will be preserved when they save the file. Moreover, modern typesetting engines based on TEX assume UTF-8 input. Luckily, very old sources from books and CDs will also work because the original TEX used ASCII and pure ASCII is UTF-8.

But people who have been using TEX for a number of years are going to have files saved with a different encoding. If the text is in English, the only problematic section may be the bibliography.

TeXShop has several features to help users deal with encoding issues.

**Automatic Detection** Starting with TeXShop 5.60, when a file without an encoding magic comment line is opened, TeXShop checks which encoding the file uses. A byte order mark identifies UTF-8 and UTF-16 files. A file containing only ASCII characters is read in the encoding selected in TeXShop Settings. Otherwise TeXShop tries UTF-8, and then the encodings used for your languages and for the default encoding, such as the Japanese, Chinese, Korean, Cyrillic, and Western European encodings. An encoding declared in the source with a line like

```
\usepackage[latin1]{inputenc}
```

is also used as a hint. Detection is a guess, and works best for files containing a reasonable amount of text. If it chooses the wrong encoding for a file, select the correct encoding in the Open dialog, and then add an encoding magic comment line to the file as explained below. Detection can be turned off with a hidden setting:

```
defaults write TeXShop EncodingAutoDetection NO
```

Then files without a magic comment line are read in the encoding selected in TeXShop Settings, as in earlier versions.

**Open Dialog** If the *Open...* item is used to open a file, a dialog appears allowing users to navigate to the file. At the bottom of this dialog, a pull-down menu can select the encoding used to read the file. By default, this will be the encoding selected in TeXShop Settings, and then the encoding of each file is detected as explained above. If a different encoding is selected, it is used for the files opened by this dialog, but only for this opening; it is not used later, for instance when a file is reloaded after an external change. An encoding magic comment line in the file still takes precedence. In Ventura, a button named *Options* must be pushed to display the pull-down menu.

**Magic Comment Line** A comment line can be added to top of a source file listing the encoding that should be used to read and write that file. This line has the following form

```
% !TEX encoding = UTF-8 Unicode
```

The line overrides all other methods used to determine the encoding. If you have a source written in an unusual encoding, you can insure that TeXShop always uses the correct encoding for that particular file. The magic line must be in one of the first 20 lines in the source file, must start at the left margin, and have exactly the form shown above. For instance, there must be exactly one space on both sides of the equal sign, and the name of the encoding must be exactly that used by TeXShop, including capitalization. Luckily, there is an easy way to achieve all of these conditions. In the TeXShop *Macro* menu, there is an item named **Encoding**. Select it and a list of known encodings will appear. Select the one you want, and the magic line will be added to the top of the source file. If the magic line names an encoding that TeXShop does not know, TeXShop reads the file as UTF-8 and says so.

**Save Dialog** Versions of TeXShop before 5.04 had an encoding menu in the Save Dialog, allowing users to select the encoding used to write the file. By default, this was the encoding used when the file was opened, so the menu was not needed in most cases. To avoid misuse, the menu was removed in TeXShop 5.04. If you updated from an earlier version of TeXShop and routinely used the menu, you can restore it. Starting with TeXShop 5.60, this is done with the item **Show Encoding in Save Dialog** under the Misc1 tab of TeXShop Settings; earlier versions used the hidden preference EncodingMenuInSaveDialog. The encoding selected in the menu is used to save the file. If the source contains an encoding magic comment line, the menu shows that encoding and cannot be changed; edit the magic comment line instead.

I do not recommend using the menu to change encodings. See the following sections for details.

**Byte Order Marks** Some programs begin UTF-8 files with a byte order mark, and UTF-16 files always begin with one. Starting with TeXShop 5.60, TeXShop keeps the byte order mark of a UTF-8 file and the byte order of a UTF-16 file when it saves the file. Encoding magic comment lines are also recognized in UTF-16 files.

**Line Endings** Files written on Windows end lines with a carriage return and a line feed, and files written on very old Macintosh systems end lines with a carriage return. TeXShop converts these line endings to the line feeds used by macOS when a file is opened. Starting with TeXShop 5.60, a notice tells you when this happens, since the file will be saved with the new line endings. The notice has a check box to stop showing it for that file.

## 7.2 More on Encoding Issues

Recall that I live in the US, where ASCII is often sufficient. Rather than using this manual, it is best to ask TEX experts in your area about encoding matters.

But I can give a minor piece of advice. Many people in the U.S. and Western Europe have

used either UTF-8 or IsoLatin9. If you have a mixture of such files, one way to proceed is to switch the default encoding to UTF-8. Starting with TeXShop 5.60, an old document without a magic comment line is then opened in the encoding TeXShop detects, which is usually correct. (With automatic detection turned off, TeXShop instead complains about a document that is not UTF-8 and opens it in IsoLatin9.) Once you know for sure that a document is encoded in IsoLatin9, use the TeXShop Macro menu's "Encoding" item to insert the magic line

```
% !TEX encoding = IsoLatin9
```

at the top of the file. From then on, the file will open in the correct encoding without guessing, and you will know for sure what encoding was used. Since LaTeX nowadays uses UTF-8 by default, the following lines should be added to the header of such IsoLatin9 documents:

```
\usepackage[T1]{fontenc}
\usepackage[latin9]{inputenc}
```

## 7.3 Changing Encodings of Files

Changing the encoding of a file is tricky. Do not attempt it without backing up the original version of the file. TeXShop users have sometimes tried to change from one encoding, A, to another, B, by using the open dialog's encoding menu to open the file with encoding A, and then using the save dialog's encoding menu to save the file with encoding B. Assume that neither file is UTF-8 and the file is currently encoded with encoding A. When TeXShop opens the file, it converts all symbols to their appropriate Unicode form. Then when TeXShop saves the file, it converts all symbols contained in B to their appropriate B form, and discards all other symbols. So symbols in both A and B are correctly moved, while symbols only in A are discarded. Since the same file was opened and saved, the original form of the file is lost.

Life is even trickier if the encoding of a file is unknown. Suppose a file was written with encoding A, but opened with encoding B, and then converted to encoding C. Suppose A encodes $\alpha$ as character 200. Suppose B encodes $\alpha$ as character 210 and encodes $\beta$ as character 220. Suppose encoding C encodes $\alpha$ as 200 and does not encode $\beta$. Then when the file with encoding A was opened with encoding B, the $\alpha$ became a $\beta$, and when this file was saved with encoding C, this $\beta$ was dropped, even though all three encodings encode $\alpha$.

There is a command line program which can convert from one encoding to another, *iconv*. This program gives different names to the input and output files, so the input is preserved. Use it instead of TeXShop if you must convert.

# Chapter 8

# Typesetting Engines I

## 8.1 History

When TeXShop was released in 2001, it could only typeset using TEX or LATEX. The actual typesetting was done by *pdftex*, an extension of TEX which directly output a pdf file rather than a dvi file.

A few months after the TeXShop release, a user suggested offering users an alternate way to typeset. Instead of calling pdftex to create a pdf file, TeXShop could call TEX to create a dvi file, and then call dvips to convert this dvi file to a postscript file, and finally call ghostscript to convert the ps file to pdf. Except for the last step, this is how TEX and LATEX originally typeset.

The advantage of offering both typesetting methods was that pdftex can accept pdf, png, and jpg illustrations, but not eps illustrations. The dvips chain can accept eps illustrations, but not pdf, png, and jpg illustrations. In addition, there is a package called *pstricks* which directly inserts postscript into the output file. This only works when the dvips route is used to typeset. Finally, back in 2001 some publishers required authors to submit a dvi file, and such a file was created by the second chain.

Nowadays, typesetting with the second method is only necessary if you use pstricks. Starting in 2011, TeX Live automatically converts eps illustrations to pdf illustrations during typesetting, and publishers almost never ask for a dvi file.

So since 2001 TeXShop could typeset in four ways: TEX with pdftex, TEX with the dvips chain, LATEX with pdftex, and LATEX with the dvips chain. Much TEX typesetting today still uses one of these original toolchains.

However, two very important computing advances are not supported by these four methods. The first of these is Unicode. As computer use spread from Western Europe and the U.S. to the rest of the world, computer companies discovered customers writing in Chinese, Japanese, Korean, Cyrillic, Arabic, Persian, Hebrew, and a host of other languages. A standard called Unicode was developed to encode these scripts, and indeed essentially all languages used anywhere in the world. It was then necessary to extend TeX and LaTeX so source files could be written in Unicode. The two most important extensions are XeTeX and LuaTeX, and more and more writers are switching from TeX to XeTeX or LuaTeX, or from LaTeX to XeLaTeX or LuaLaTeX.

A second advance in computing involves fonts. Knuth wrote a program called MetaFont that could be used to design font families, and he developed the Computer Modern font family with all characters required in Western Europe or mathematics. But Computer Modern does not contain the vast array of new characters in Unicode, and the world ultimately adopted a different standard for fonts, called open type fonts. Vast collections of such fonts became available with support for more and more languages covered by Unicode. The Macintosh comes with a large supply of such fonts. Both XeTeX and LuaTeX have been modified to let TeX users substitute the fonts which came with their systems for the Computer Modern Fonts.

TeXShop fully supports these developments. Internally, its editor works in unicode, and source files can use any Unicode script. TeXShop even allows users to insert Arabic and Hebrew from right to left rather than left to write. But if the source code uses these extra Unicode characters, then it is important to typeset with XeTeX or LuaTeX rather than with TeX or pdfTeX.

A piece of good news is that both XeTeX and LuaTeX can typeset old source files written for TeX or LaTeX without making any changes at all. So it is essentially as easy to start working with TeX using XeTeX or LuaTeX as it is to begin with ordinary TeX. Instruction books about TeX and LaTeX make perfect sense when used on a machine running XeTeX or LuaTeX.

## 8.2 Configuring TEX and LATEX Typesetting

With this history out of the way, it is time to explain how to configure TeXShop to typeset using whatever engine you decide to use. You'll do this by using Settings (that is, Preferences).

Go to TeXShop Preferences and select the **Typesetting** tab. In the first column, select either TEX or LATEX as the default command. Most people use LATEX, so that the is default choice. We'll describe the third choice in this box shortly.

Below these items is another group labeled "Default Script". Select either "Pdftex" or

"TeX + DVI" as the default script. Most people select "Pdftex". If you use pstricks or have many very old documents, "TeX + DVI" may be a better choice.

To switch between "Pdftex" and "TeX + DVI" for an occasional document, go to the Typeset menu and select the appropriate option at the bottom. This only affects the active document, so if you have two documents open, you can switch the typesetting method for one without changing it for the other.

If you want to use XeTEX rather than LATEX, choose the third option in the "Default Command" box and write XeLaTeX in the command slot. If you want to use LuaTEX rather than LATEX, write the word LuaLaTeX in the slot.

After making these choices, typesetting with TeXShop is easy. When you want to typeset, just push the "Typeset" button in the Source toolbar or select the "Typeset" menu at the top of the Typeset menu. This menu has a keyboard shortcut, command-T. After a few days, you'll ignore the toolbar item and the menu item and type command-T when you want to typeset.

At the very bottom of the first column in this Preferences tab, there is an item labeled "Use -dALLOWPSTRANSPARENCY". This is only active when using TeX + DVI mode. In this mode, when a graphic is created, it is processed by Ghostscript. In modern versions of Ghostscript, the postscript code has been converted to C and made safer, but this modernization can break transparency. This preference sets a flag telling Ghostscript to process transparency calls correctly. The preference will be irrelevant for most people.

In the second column of the preference pane, the first box is titled "After Typesetting". This preference determines which window is pulled to the front after typesetting is done. Users often want to inspect the output to make sure it is correct, so the choice "Bring Preview Forward" pulls the Preview Window to the top. But some users prefer to glance at Preview and immediately begin writing additional source. The item "Continue Editing" brings the Edit Window forward to make this task easy.

There is yet a third option. The keyboard shortcut Command-T can be typed in any of the windows Source, Preview, Console, Log and asks TeX to typeset the document. When the third option is chosen and typesetting ends, the window where Command-T was typed comes to the front.

TeXShop has the ability to sync between the source and the preview windows. Several methods to do this have been tried over the years, but the modern approach is called "SyncTeX (TeX > 2010) and should be chosen in the "Sync Method" box. Sync will be discussed in detail in a later chapter.

The box labeled "If Sync Opens a New Window" is left over from an older version of TeXShop. This item is obsolete and should not be checked. I keep old items in case someone is still using them.

## 8.3 Bibtex

The Typeset menu contains two other items, "BibTex" and "MakeIndex". These are explained in this section and the next section.

BibTeX is a tool used to create bibliographies in LaTeX documents. The tool assumes that you have a large database of references which you often quote. Simple commands allow you to cite certain of these references in your LaTeX source, and BibTeX then creates a bibliography for your article containing only the items cited.

Here is a brief example taken from The LaTeX Companion by Goossens, Mittelbach, and Samarin. Consult this book for many additional details. Suppose the database is a file called "mybibliography.bib" containing the text shown below. In this text, the entries "Felici:1991, Knuth:WEB," and "Liang:1983" are key values used to cite the articles in the LaTeX source.

```
@article{Felici:1991,
author ={James Felici},
title ={{PostScript versus TrueType}},
journal ={Macworld},
volume =8, pages={195--201},
month =sep, year = 1991 }
@techreport{Knuth:WEB,
title ={{The \textsf{WEB} System of
Structured Documentation}},
month =sep, year = 1983,
author ={Donald E. Knuth},
address ={Stanford, CA 94305},
number ={STAN-CS-83-980},
institution={Department of Computer
Science, Stanford University} }
@phdthesis{Liang:1983,
author ={Franklin Mark Liang},
month =jun, year = 1983,
school ={Stanford University},
address ={Stanford, CA 94305},
title ={{Word Hy-phen-a-tion by
Com-pu-ter}},
note ={Also available as Stanford
University, Department of
Computer Science Report
No. STAN-CS-83-977} }
```

Suppose the LaTeX source file is called "myfile.tex" and contains the following text:

```
\begin{document}
Consider the argument of Felici~\cite{Felici:1991}
in light of these comments.\nocite{Liang:1983}
We provide further remarks later.
\bibliographystyle{plain}
\bibliography{mybibliography}
\end{document}
```

When this source is typeset, a reference to Felici's article will appear in the text, and a bibliography will be created at the end of the text containing the articles of Felici and Liang, but not the article of Knuth.

TeXShop can be used with this example in the following way. First edit and typeset the document "myfile.tex" as usual. Citations will appear in the output as "[?]" and the bibliography will be missing. Then select "BibTeX" under the Program button and run BibTeX. Next select "LaTeX" and typeset again. Citations will still appear as "[?]", but the bibliography will be added to the output. Typeset a final time, and citations will have their correct values.

Notice that the BibTeX menu command has a keyboard equivalent.

The file "mybibliography.bib" can be opened and edited by TeXShop. If you use TeXShop to create the file "mybibliography.bib" in the first place, use the pulldown tag labeled "file format" to save the file as a bib file rather than as a tex file.

In recent years, additional programs like biber have been written to handle bibliographies. Japanese typesetting is done using a modified TeX and still other programs like pbibtex and ubibtex are available there for bibliographies. So TeXShop has a preference setting to set the program called when the BibTeX menu item is chosen. It is at the bottom of the Engine tab. Fill in with the appropriate name, and any flags that should be used when calling this command line program.

Often users use different bibliography programs for different projects, or when working with different coauthors. TeXShop allows the bibliography program to be set on a file-by-file basis. To set it for a given source file, include the following line within the first twenty lines of the source, replacing "biber" with the desired program names and any desired flags.

```
% !BIB TS-program = biber
```

For compatibility with other platforms, the following syntax is also acceptable

```
% !BIB program = biber
```

If this line is missing from the source file, the corresponding Preference entry will be used when the BibTeX menu command is selected.

## 8.4 MakeIndex

MakeIndex is a tool used to add an index to a LaTeX document. To create such an index, you should add the command below to the top of the source file:

```
\usepackage{makeidx}
```

Put the command below in the document preamble:

```
\makeindex
```

Put the following command where the index should appear, often just before "enddocument":

```
\printindex
```

Put index references in the source document, as in the example below:

```
There are many animals in the world\index{animal}.
Examples include bears\index{animal!bear}
and tigers\index{animal!tiger}.
```

To create the index, typeset the document as usual. Then run MakeIndex. Then typeset again.

Many additional details can be found in The LaTeX Companion by Goossens, Mittelbach, and Samarin.

## 8.5 Other Typesetting Preference Settings

If you installed using MacTeX or BasicTeX, you can ignore this section because TeXShop is preconfigured for those distributions.

Otherwise open Preferences and select the **Engines** tab. The first item in this dialog is a full path to the TEX Distribution command line programs. When TeX Live is installed, it creates a symbolic link to these binaries with path /Library/TeX/texbin. That is the default value for the preference. An advantage is that when TeX Live is updated for a new year, the link is also updated and thus the preference setting does not change.

If you installed TEX using MacPorts, Homebrew, or MikTeX, the location of the binaries will be different. Consult the documentation for these distributions to discover the required value.

The second item in the tab is a full path to the Ghostscript binary. This is /usr/local/bin for the copy of Ghostscript installed by MacTeX. The standard build sequence for Ghostscript will also install it at this location. If you use the Ghostscript in MacPorts or Homebrew, consult their documentation for the proper location. Many typesetting engines do not use Ghostscript, so this is not a crucial preference item.

Starting with TeXShop 5.60, TeXShop can set these two items for you. When TeXShop is first used, and whenever the button **Auto Setup...** in the Path settings box of the Engine tab is pressed, TeXShop looks for TEX and Ghostscript in the usual places: the folders listed in /etc/paths and /etc/paths.d, /Library/TeX/texbin, the TeX Live folders in /usr/local/texlive and /Applications/TeXLive, and the folders used by Homebrew, MacPorts, and Fink. It checks that the programs run, asks each one for its version, and lists the installations it finds, newest first; TEX Live installations are ordered by year and then by pdfTeX version. The newest one is selected, but another one can be chosen, separately for TEX and for Ghostscript. Nothing changes until **Use These Paths** is pressed, and in Settings the change can still be undone with **Cancel**. If nothing is found, the current settings are kept. Custom engines and the Alternate Path are never changed. Installations in other places can still be entered by hand.

The Alternate Path preference will be explained later and is not used by the toolchains discussed in this chapter.

Below these items, the command line calls are listed for each of the four typesetting methods discussed so far that call pdftex. These items are irrelevant if you switched to XeTEX or LuaTEX. Suppose you have a LATEX source file named Galois.tex and you have opened Apple's Terminal program and changed to the directory containing this source. You could typeset the source file with pdflatex by typing the following line in Terminal:

```
pdflatex --file-line-error --synctex=1 Galois.tex
```

TeXShop calls the typesetting engine in exactly the same way. These four preference settings list the appropriate four calls.

The pdftex defaults use two flags. The first, –file-line-error, tells pdftex how to display errors in the console; TeXShop uses this to take you directly to the source lines with the reported error, as explained later. The second flag, –synctex=1, activates code by Jerome Laurens which tells pdftex to generate sync data during typesetting. TeXShop later uses this data to sync between spots in the source file and corresponding spots in the preview output, as explained later.

There is one other flag you might want to add, –shell-escape. This flag gives pdftex permission to call other programs during typesetting. If you typeset source files which other people send you, this could be dangerous because pdftex will call any program requested, even a program which erases your entire home directory. So we recommend avoiding

the flag. But if you use it, you might want to check the item **Shell Escape Warning** underneath. After that, when you typeset a file for the first time, a warning dialog will appear reminding you of "shell-escape" and asking if you still want to proceed.

Typesetting with "TeX + DVI" calls a script named "simpdftex" which is part of TeX Live. If you want to know how the script works, open it in TeX Live and read it.

# Chapter 9

# Typesetting Engines II

## 9.1 More History

The design discussed in the previous chapter uses built-in code to call one of four programs: TeX, LaTeX, BibTeX, or MakeIndex. Around 2004, it became clear that this design was inadequate because other typesetting programs exist: XeTeX, LuaTeX, ConTeXt, TeX4ht, and on and on. Consequently, a new approach to typesetting was adopted. In a perfect world, the methods described in the previous chapter would have been ripped out, but I didn't want to disturb previous users. So the old and new methods are both present, but users should concentrate on the new methods. All typesetting, including the original TeX and LaTeX support, fits naturally into the new design.

The basic idea of the new design is simple. Each typesetting job is determined by a small shell script, which can be created and edited directly in TeXShop. This script might be very simple and call a single TEX binary. Or it might be complicated and call several programs one after another, say pdflatex followed by bibtex followed by two more calls to pdflatex.

Typesetting scripts are called "engines" and are stored in ∼/Library/TeXShop/Engines. Next to the typeset button in the TeXShop source window is a pulldown menu listing all available engines. Select one and then press the Typeset button to typeset with that script.

It is also possible to name the script which should typeset a given source file in a magic line at the top of the source. A typical magic line might read

```
% !TEX TS-program = LuaLaTeX
```

Then command-T applied to this source would typeset with LuaLaTeX.

The event that triggered this new design was the release by Jonathan Kew of XeTEX. His version of TeX accepted UTF-8 Unicode input and thus could typeset source written in any script: Chinese, Japanese, Korean, Hebrew, Arabic, and many, many others. XeTEX also supported using ordinary Macintosh fonts alongside fonts in the TEX distribution. This was important for Unicode support since Macintosh fonts were available for most languages. Finally, some languages are difficult to typeset because of complicated rules governing the script. For example, Arabic is written in cursive and letter forms can vary depending on whether a character is in the middle of a word or the end. To support such languages, XeTeX used AAT, Apple Advanced Typography. A few years later, XeTEX was was extended to work on all platforms and AAT was replaced by equivalent open source libraries for all platforms.

## 9.2 Engines

In the TeXShop menu, select the item "Open ~/Library/TeXShop" and in the resulting Finder window open the Engines folder. You will find several files with extension ".engine". Each engine file contains a list of typesetting instructions for TeXShop. Each engine can be opened and edited in TeXShop. If you want to create a new engine, the easiest way to do so is to duplicate an existing engine, rename it, and then open and edit that file. Engine files must have their execute bit set, and this procedure does that automatically.

Open the file XeLaTeX.engine. Its contents are shown below.

```
#!/bin/tcsh

set path= ($path /Library/TeX/texbin /usr/texbin /usr/local/bin)
xelatex  -file-line-error -synctex=1 "$1"
```

This file is what is known as a "shell script." It is a series of commands which could be typed in Terminal. This particular script starts with a line telling Terminal to run the tcsh shell. Various shells are available, including zsh, bash, tcsh, sh, and others. Each has its own particular language. Recently, Apple selected zsh as the default shell, so we recommend using that shell.

Then the script tells Terminal where to search for TeX binaries. This may not be necessary since most of these locations are already in a user's path. The key line is the final one. It tells the system to typeset the file "$1" using xelatex with our two standard flags. The only difference between using this engine and using the built-in LaTeX command is that this engine calls xelatex rather than pdflatex.

When TeXShop calls a shell script, it executes "cd" to change to the directory containing the source file, and then it sends the shell script several pieces of information, which the script can access as "$1", "$2", etc. The most important of these is "$1", which contains

the name of the source file, including extension. So if TeXShop typesets Galois.tex, which is found in the folder ∼/Sources/GaloisStuff, the directory will be

```
/Users/koch/Sources/GaloisStuff
```

and "$1" will equal

```
Galois.tex
```

In a later section we will have more to say about creating engine files.

## 9.3 Typesetting with an Engine

Active engine files are listed in the pull-down menu to the right of the Typeset button in a source window toolbar. To typeset, choose the appropriate engine in the pulldown menu and then either push the typeset button or type command-T.

If most of the time you typeset with a particular engine, you can make it the default engine. In TeXShop Preferences under the Typesetting tab, the first entry selects the default typesetting command. Select the item **Command Set Below** and type the desired engine name. This name must have exactly the capitalization and spelling of the desired engine, without the "engine" extension.

The pulldown menu changes typesetting only for the source in its window. You can have two files open, one typeset with your default engine and the other typeset with xelatex. You only need to select xelatex the first time you typeset that file.

There is another way to choose the typesetting engine for particular source files. Near the top of the source, include a magic line

```
% !TEX TS-program = XeLaTeX
```

Then this particular source will always be typeset with the listed engine.

This command must start at the left margin and be written exactly as above, and the name of the desired engine must have the correct spelling and punctuation. There is an easy way to achieve all of this. Select the macro named "Program" from the Macro menu. A dialog will appear listing all active engines. Select the desired engine. The appropriate line will be added to the top of the source file, replacing a previous engine determination if one exists.

If you get in the habit of adding magic lines to the top of sources files, then you can use a variety of engines to typeset and you'll never have to think about which is appropriate. Simply type command-T and the correct choice will automatically be made.

In all of these places you select one of the four original typesetting methods using the following scheme:

| | |
|---|---|
| tex | TeX with TeX and DVI |
| pdftex | TeX outputting pdf |
| latex | LaTeX with TeX and DVI |
| pdflatex | LaTeX outputting pdf |

In rare cases, you might want to invent your own engines for some of these four built-in methods; that will work fine, just don't use the four specific names on the left side of the above list.

That is all you need to know to use typesetting engines. The rest of the chapter will discuss particular engines provided by TeXShop, and then explain how you can construct your own engines. Less than half of TeXShop users construct new engines, so you can skip that step until you need something unusual.

## 9.4 latexmk

TEX is a single-pass compiler; it reads the source file just once as it typesets. But some typesetting tasks cannot be finished in a single pass. If a book has a table of contents, TEX cannot create the table of contents until it knows the names of the book chapters and the pages where they start. To solve this sort of problem, users typeset twice. In the first pass, TEX writes a file called the "auxiliary file" to the source directory; in the second pass, this file is consulted to find the required extra information.

Sometimes other programs are run between the passes. If a file has an index, an idx file is constructed during the first pass. Then the program "makeindex" is run to create an index from the information in this file. Finally TEX is run again to incorporate this index.

Some tasks require three or more passes through TEX before the job is complete.

There is a script in TeX Live named **latexmk**, by John Collins, which automates this process. It calls LATEX as many times as necessary to produce an up to date document, with intermediate calls to other programs if necessary. My colleague Herbert Schulz is an expert on this script and works with Collins to insure that it is well-supported on the Mac. He has created an engine which calls pdflatexmk to typeset. Many TeXShop users make this their default typesetting engine. The extra typesetting time with smaller projects is almost unnoticeable. The extra work on large books is essential, because the author can then preview a complete version of the work.

The pdflatexmk engine is already a possible choice when you first install TeXShop. The subfolder Engines/Inactive/latexmk contains several documents by Herbert Schulz with

more details, and related engines if you want to replace pdflatex with XeLaTeX or LuaTeX or other typesetting methods and use the script.

If typesetting speed is an issue, TeXShop has an obscure trick to help. While a document is being written, use pdflatex, which only typesets once. But every so often, switch to pdflatexmk so all features are correct. To make this easier, there is a special typesetting shortcut, control-option-command-T, which typesets with pdflatexmk. Therefore you can set the typesetting engine to pdflatex, typeset with command-T as a rule, but typeset with control-option-command-T occasionally.

This trick fails if you typeset with unusual engines. For example, suppose you typeset with XeLaTeX rather than pdflatex. Then there is a special engine in Engines/Inactive/latexmk which calls XeLaTeX rather than pdflatex. So the special command for control-option-command-T should call that XeLaTeX form of pdflatexmk rather than ordinary pdflatexmk. In TeXShop Preferences under the Misc2 tab there is a setting named "Alternate Engine" allowing pdflatexmk to be changed to any other desired engine, including the special XeLaTeX version of pdflatexmk.

## 9.5 latexmk History by Herbert Schulz

When I started using **latexmk** it only supported latex and pdflatex directly (via command line options). To make it work with latex, pdflatex, xelatex and lualatex with the default command line options needed, I redefined some internal **latexmk** commands (**$latex** for the latexmk engine and **$pdflatex** for the other engines) to work the way I needed them to work for TeXShop (e.g., using synctex and file-line-error).

Over time the engines have been extended to use Apple's new default **zsh** and TeXShop's

```
% !TEX parameter =
```

magic comment; e.g.

```
% !TEX parameter = -shell-escape
```

for adding shell-escape to the **latexmk** based engines. You can even use more than one option; e.g.,

```
% !TEX parameter = -shell-escape -auxdir="aux files"
```

which tells the latexmk based engine to store all auxiliary files in the (possibly created) `aux files` folder. Note: not all packages that create auxiliary files will work this way.

The engines also use a set of default **latexmk** settings so that several popular packages work out of the box. That information is found in `~/Library/TeXShop/bin/latexmkrcedit`

which is initially copied from `~/Library/TeXShop/bin/tslatexmk/latexmkrcDONTedit` if it doesn't already exist the first time any of the engines are run. It is left alone after that since it is meant to be edited by you for use by all engines.

## 9.6 The Inactive Folder

The Engines folder contains a subfolder named **Inactive**. This folder contains a large number of subfolders with additional typesetting engines for special workflows. The folders often contain documentation and some contain sample documents.

When TeXShop is updated, the files directly in Engines are not touched because users may have modified these files. But the Inactive folder is completely rewritten, so this material is as up to date as possible. This is important because some engines support Sage, ConTeXt, PreTeXt, and others projects under active development, where major changes can occur.

## 9.7 Creating a New Engine

The rest of this chapter is about creating engines. You may want to skip this material until you need a special engine.

Engine files must have extension ".engine" and have the execute bit set. The easy way to meet this requirement is to duplicate an active engine, rename it, and open and edit it in TeXShop.

Engine files are shell scripts, and the first line should name the appropriate shell. A sample line might be

```
#!/bin/zsh
```

Since zsh is the current default shell in macOS, I'll use that in this description, but macOS includes many shells including sh, tcsh, bash. Use the shell you know best.

I'll confess that I'm not a shell expert, so I often have to consult other engines to see how to accomplish various tasks. Google is also a useful source of help. The main tricky point is that shells use different syntax to accomplish the same task, so a line which works in one shell may fail in a different shell.

When an engine is called by TeXShop, it is given the name of a source file, say Galois.tex. Each line of the engine is then executed. After all lines are executed, TeXShop looks in the source folder to see if there is a file with the same name as the source, but extension ".pdf". For instance, it might look for Galois.pdf. If this file exists, it is opened in the Preview window. Otherwise nothing happens. This default behavior can be changed, as explained later in this section.

When engines are called, TeXshop gives the engine three pieces of information. Each is a string, denoted "$1", "$2", "$3". The first, "$1", is the name of the source file. For instance, it might be Galois.tex. The other two parameters are often not used, but might be employed in special circumstances. The parameter, "$2", will usually be just a space " ". But if further information is needed by an engine, it can be sent using a magic comment line near the start of the source file. This comment can have one of the following forms:

```
% !TEX TS-parameter =
```

```
% !TEX parameter =
```

The equal sign must be followed by a space, and then a word or sequence of symbols without spaces. If a project has a root file and several include files, the magic comment must be near the top of the root file.

The magic line might be used to pass one or more flags to engines. For instance, Herbert Schulz used this line to add a –shell-escape flag when pdflatex needs to call an external program during typesetting.

Finally, "$3" contains a path to the TEX binaries. Therefore "$3" will usually contain "∼/Library/TeX/texbin". However users who install TeX using MacPorts or MikTeX or something else will have set a different location for binaries at the top of the Engine tab in TeXShop Preferences, and then "$3" will contain that location. For the cases of TeX Live, MacPorts, and MikTeX, this parameter is not needed because the Cocoa call to a binary contains its location. So "$3" is usually not mentioned in an engine.

But there is one special situation when it is needed. ConTeXt is a set of macros by Hans Hagen providing a different approach to typesetting. It is usually used with LuaTeX, another version of TEX. The ConTeXt community has its own distribution of both ConTeXt and LuaTeX, which is small and updated often. So TeXShop has a preference item under the Engine tab named **Alternate Path** listing the location of this distribution. The ConTeXt users then add a magic comment to the top of their source code

```
% !TEX useAlternatePath
```

which causes TeXShop to set "$3" to this alternate path. The ConTeXt engines use this information to find the appropriate ConTeXt binaries in their special distribution.

## 9.8 Useful Engine Commands

When writing an engine, it is sometimes useful to decompose the string "$1". Imagine this string equals "Galois.tex". It can be useful to strip off the extension and obtain "Galois", and then add a different extension, like "Galois.pdf" or "Galois.aux".

Here are recipes to do this using the shell zsh. These recipes may or may not work in other shells.

```
#!/bin/zsh

filename="$1"
basename="${filename:r}"
extension="${filename##*.}"

echo $filename
echo $basename
echo $extension
echo $basename.pdf

if [[ "$extension" = "ptx" ]]; then
 echo "This is a PreTeXt file"
 exit
fi

if [[ "$extension" = "tex" ]]; then
 echo "This is a TeX file"
 exit
fi
```

If this script was used as an engine script and the file Galois.tex was typeset, no typesetting would occur but the console would display the following messages

```
 Galois.tex

 /Library/TeX/texbin
 Galois.tex
 Galois
 tex
 Galois.pdf
 This is a TeX file
```

The second line is blank because $2 contains nothing. The word filename is a variable, defined and given a string value in the script, and $filename is the value of that variable. Thus $filename, $basename, and $extension are the full name of the source with extension, the name without extension, and the extension. The string $basename.pdf equals the old basename with new extension ".pdf".

Unfortunately, shell syntax is very picky. The line

```
filename="$1"
```

will not work if there are spaces around the equal sign, and the line

```
if [[ "extension" = "ptx" ]]; then
```

will not work if the spaces just after [[ or just before ]] are omitted. I am not a shell script expert, so I tend to copy lines from other scripts, consult Google often, and pay attention to spaces in the examples I copy.

## 9.9 Recent Additional Engine Commands

In TeXShop 5.00, an additional html-Preview window is provided, so typesetting jobs which output html can be previewed live on the web. Engines can now determine exactly which files are opened in preview windows after typesetting, using the new commands

```
!TEX-noPreview
!TEX-pdfPreview
!TEX-htmlPreview
!TEX-bothPreview
!TEX-noConsole
```

These are analogues of the magic comment lines used in TeXShop source, but there are a few differences. First, the lines can appear anywhere in an Engine file, because Engine files are short and it costs nothing to search the whole file. Second, the magic lines do not have to start at the beginning of a line; they can appear anywhere in the text.

It is crucial, of course, that the lines look like comments to the shell interpreter. So for standard engines, each line should be prefaced with the # comment symbol. But engines can be written in a variety of scripts, and some use different comment symbols. Therefore the comment symbol is not part of the new commands.

The standard behavior of TeXShop still applies if an engine has none of these lines. After executing all lines of the engine, TeXShop looks in the document directory to see if there is a file with the same name as the source, but extension ".pdf". If so, it opens this pdf file in the Preview Window, unless the Option Key is depressed.

If one or more commands are present, then

- if the line "!TEX-noPreview" is present, no preview window appears even if some of the other lines are also present;
- otherwise if the lines "!TEX-pdfPreview" and or "!TEX-bothPreview" are present, TeXShop searches the document directory to see if there is a file with the same name as the source, but extension ".pdf"; if so, it opens this pdf file in the Preview Window;
- if the lines "!TEX-htmlPreview" and or "!TEX-bothPreview" are present, TeXShop searches the document directory to see if there is a file with the same name as the source, but extension ".html"; if so, it opens this html file in the HTML Window;
- and if the line "!TEX-noConsole" is present, the console is not shown.

It is unlikely that this final line will be used with any engine except the one which directly opens html code without any intervening typesetting.

# Chapter 10

# Projects with Multiple Input Files

Many book authors arrange their sources for a book as follows. A preliminary "root document" contains the preamble, defining macros, loading various packages, and so forth. Then this document "includes" or "inputs" separate source files for each chapter. Some authors put these chapter sources in separate folders which also contain the illustrations for that chapter. This organization dates back to Lamport himself, whose famous manual on LaTeX contained the "include" command and an "includeonly" command, so authors working on a subset of chapters could only typeset those chapters to save time, and then typeset everything at the end.

Not all book sources are organized this way. I know several authors who created a single source file for a 700 page book. But the organization is not unusual. When it is used, TeXShop creates a separate window for the source of each chapter, opening only the window for the chapter currently being written. But the typeset button on such a window cannot just typeset the source in the window because that source has no preamble. Instead, it should typeset the root document, and create a single preview window showing the entire book, or possibly only chapters listed by "includeonly".

TeXShop has a simple way to make this happen. Each chapter file (that is, each source file to be included or input by the root file) should have a magic line at the top listing the name and location of its root file. The root file should not have this magic line. When a chapter file active and the author types command-T, the main root file should be typeset. The magic line, to appear in the first twenty lines of the source, has the following form, where in this case Final.tex is the root file.

```
% !TEX root = /Users/koch/TeX/MyBook/MyRoot.tex
```

The first letters of this magic line must be exactly as written above. This name of the root can have several forms illustrated below. Here MyRoot.tex should be replaced by the name of the root file.

```
MyRoot.tex
../MyRoot.tex
../../MyRoot.tex
/Users/koch/TeX/MyBook/MyRoot.tex
```

Use the first form if the root file is in the same folder as the chapter file. Use the second form if the root is in a folder which contains a subfolder for each chapter file. Use the third form if the root is further up the folder chain. Use the fourth form if you want to give a full path to the root file. Note that the fourth form is not portable, so moving the source folder of a project will require rewriting the magic lines of such chapters.

The Macros menu has an item labeled "Root". To add the magic line to a chapter source file, activate that source file and then select the macro. A Finder window will appear which you can use to navigate to the root file. The macro will then offer to create either a relative or an absolute magic line, replacing any earlier root magic line.

If a chapter file with a root magic line is opened in TeXShop, TeXShop will also open the root file, and thus also the associated Preview Window for the entire book. It will not open remaining chapter files. Thus an author can revise the chapter text and typeset as usual; the Preview Window will show changes. During this operation, it is usually not necessary to edit the root file, so some authors prefer to immediately minimize the root source. In TeXShop Preferences under the Misc2 tab there is an item to immediately minimize the root source after opening a chapter file and its root. This action is a little surprising, so I recommend not selecting it at first. Later on, you can decide if you want to activate it. If multiple files open in the same window using tabs (as explained in a later chapter), then minimizing the root is completely unnecessary.

The root magic line is also important for the "Goto Error", "Kill Aux Files", and "SyncTeX" features to be discussed next.

# Chapter 11

# Goto Error, Trash Aux Files, SyncTeX

## 11.1 Goto Error

When TeX typesets and finds a syntax error, it stops and the console window comes to the front. TeX prints an error message at the end of the console output. Pushing RETURN causes TeX to continue typesetting until it comes to another error. Pushing RETURN many times will often lead to finishing the typesetting job and thus outputting new preview contents.

The Console window has a button labeled Goto Error. There is also a menu in the Edit menu and a keyboard shortcut for this command. The command will activate the source file, scroll to the line containing the first error, and highlight that line. Pushing the button again will go to the second error line, and so on. It is often useful to fix four or five errors and then typeset again, because one error can cause another so fixing one error may fix many.

If the project has a root file and various chapter files, GotoError will open the source file containing the error if it is not currently open, and then go to the appropriate line in that source file.

The Goto Error command parses the console output to find error messages, and requires that TeX reports these errors in suitable format. So calls to LaTeX, pdfTeX, XeTeX, and other engines should contain the flag

```
--file-line-error
```

to activate this formatting.

## 11.2 Trash Aux Files

As explained earlier, when LaTeX first typesets, it creates an "aux file" containing information needed for tables of contents, equation numbers, and the like. The information in this file is then used in a second pass of LaTeX. Several passes may be needed before the document is completely up to date. If the source is named Galois.tex, then the aux file will be named Galois.aux.

If the source has an error and the user aborted typesetting when the error was reported, the aux file may be corrupt. In that case, LaTeX will open the aux file when it typesets a second time and become confused. This often happens shortly before the "begindocument" line. A source file which claims to have an error there may instead have a corrupt aux file. The solution for this problem is to throw the aux file in the trash and typeset again from scratch.

This sort of problem tends to be rare for small documents and common for large documents or documents with many illustrations. TeXShop has a button in the console labeled "Trash Aux Files" which trashes the aux files in the document's folder. There is also a menu item in the File menu and a keyboard shortcut to trash these files. When you run into the problem, push the button or menu item and then typeset again.

Because the problem is common for large documents, there is an even better shortcut. Recall that command-T typesets. A similar shortcut, option-command-T, trashes the aux files and then typesets. If the Typeset menu is open, notice that pushing the Option key changes the item titled "Typeset" to read "Trash Aux & Typeset".

If a project has a root file and then chapter files, the aux files are associated with the root file. So if a chapter file is typeset and the chapter file has the correct magic line pointing to the root file, then the aux files of the root file will be trashed.

The aux file is not the only auxiliary file which LaTeX writes during typesetting. It also can write a bbl file, a toc file, and many others. Each saves pieces of information that are used in later passes of LaTeX. Therefore the command which trashes the aux file also trashes many other files. Here's a list:

```
aux, blg, brf, glo, idx, ilg, ind, loa, lof, log, lot, mtc, mlf,
out, ttt, fff, ent, css, idv, wrm, 4ct, 4tc, lg, xref, bcf, gtex,
gaux, glog, pdfsync, synctex, synctex(busy),latexmk,
fls, toc
```

If typesetting goes seriously wrong, even the pdf file can become corrupt. Generally users do not want to trash the old pdf file since it is the preview image they will use when a new image is not created. But if you hold down the Shift key while activating Trash AUX Files, the pdf file will also be removed.

There is a complication for projects with a root file and included chapter files. In that case, typesetting the root file can leave root.aux and root.pdfsync in the folder with the root file, and chapter.aux in the folders with the various chapter files. So more extensive cleanup is needed. To make that happen, hold down the Option key while activating Trash AUX Files. Then all files in the root folder or any of its subfolders, regardless of name, will be trashed if they have an appropriate extension.

The remaining paragraphs in this section get into the weeds for experts. Some users need to add extra elements to the list of trashed files. That can be done using a hidden preference setting. A hidden preference setting is a setting that is not shown in TeXShop Preferences, and thus must be manipulated using Terminal and the command line.

For example, to add "dvi" to the list, activate the Terminal and type

```
defaults write TeXShop OtherTrashExtensions -array-add "dvi"
```

To remove all additions and return to the original default list, type

```
defaults write TeXShop OtherTrashExtensions -array
```

To make the aggressive behavior created by holding down the Option key the standard behavior of Trash AUX Files,

```
defaults write TeXShop AggressiveTrashAUX YES
```

## 11.3 SyncTeX

SyncTeX is a feature which allows users to rapidly move from a point in the source file to the corresponding point in the output file, and vice-versa. This feature was first provided by *Textures*, a front end for the original Macintosh system written by Barry Smith in Portland, Oregon. My understanding is that Smith had rewritten the Pascal compiler and then large portions of the TeX code and used this knowledge to provide very accurate sync information. His company, Blue Sky Research, was in the process of porting *Textures* to macOS when he tragically died in 2012.

SyncTeX was first provided for regular systems on macOS by Jerome Laurens. His original method was a package which users activated in their source; this package output a synctex file containing information which front ends could use to sync between source and preview windows. Unfortunately, this package sometimes modified typesetting, so output when using it differed from output when not using it.

Later TeXShop provided a method directly in the program, bypassing Lauren's package; this didn't work very well and the less said about it, the better.

Eventually Laurens provided a version of SyncTeX which outputs sync information during typesetting directly from the typesetting engines. This no longer modifies typesetting output. His code is built into most of the typesetting engines supplied by TeX Live. When a source file, say Galois.tex, is typeset by one of these engines, and the engine is called with the flag "–synctex=1", it outputs a file named Galois.synctex containing the required sync information. Laurens provides a library written in C to interpret this information; TeXShop contains that library and uses it to provide syncing.

SyncTeX is turned on in TeXShop Preferences by an item under the Typesetting tab. Be sure to select the last item under Sync Method, called "SyncTeX (TeX > 2010)". Other items are for previous sync methods now obsolete.

To sync from the source to the preview window, hold down the Command key and click at a point. The Preview window will scroll to the appropriate spot and highlight the corresponding text in the output. To Sync the other way, hold down the Command key and click at a point in the Preview window.

SyncTeX is an essential feature of front ends, but it is not completely accurate. A short search afterward may be needed to find the precise point where changes should be made.

If a project has a root file and included chapter files, clicking on a point in the source for a chapter takes you to the corresponding point in the preview window. Clicking on a point in the preview window activates the source window for the appropriate chapter. If this window is not open, it will be opened automatically. Then you are taken to the appropriate spot in that chapter's source.

This sync feature is a useful way to open appropriate chapter files. If a project with a root named Galois and various chapter files exists, open Galois.tex to see the root source and the full Preview. Scroll to an interesting point in the Preview and sync from one of its points. The appropriate source file will open automatically.

There is one tricky feature of SyncTeX. The information in the synctex file created by the typesetting engines contains full paths to files. So if a folder containing a LaTeX project is sent to someone else or moved to another location, synctex will no longer work and Command-click will do nothing. To fix this, just typeset again. That creates a new synctex file.

# Chapter 12

# Synchronizing Page Numbers with the Toolbar Page Number Box

## 12.1 Synchronizing the PageNumber Box

Suppose a document has several pages of preliminary material before the main text begins. Perhaps these preliminary pages are numbered i, ii, iii, iv, v, ... and the main pages are numbered 1, 2, 3, 4, 5, ... Then entries in the PageNumber box will not be in sync with the actual page numbers in the text. TeXShop provides a fix for this problem. When the fix is applied, preliminary pages are numbered 1#, 2#, 3#, ... in the PageNumber box, and main pages are numbered 1, 2, 3, ... Consequently page numbers with a sharp sign list elements in the preface, and page numbers without a sharp sign list main pages and are in sync with the actual page numbers of these pages in the document. I experimented with changing the first list to i, ii, iii, iv, v, vi, ..., but these roman numerals quickly became large and overflowed the box. Moreover, using them in the reverse direction was awkward.

As just hinted, this also works when using the PageNumber box in reverse to go to a particular page. Type a regular number to go to that exact page in the main text. Type something like 3# to go to the third page in the preface.

The fix does not change the behavior of the box listing the total number of pages, since it is often useful to know how many pages altogether are in a document. If you insert the total number of pages into the PageNumber box and press enter, you will go to the end of the document, but the actual page number can be less than the total number of pages.

## 12.2 Activating the Fix

The fix is activated by adding extra output to the console and log file during typesetting. This output has the form

```
Start of main material: 17
```

where 17 is replaced by the number of the page where the main material begins. TeXShop searches for this string in the console during typesetting and applies the fix if the string is found. Also when TeXShop first opens a source file, it searches the log file for the string and applies the fix if possible. Finally if a file with the fix is open when TeXShop quits and the file appears when TeXShop is restarted, the fix will automatically be applied. Thus once a file has been typeset, the fix should always be applied during normal operations.

Fixing Page Number boxes is a standard feature of pdf display programs like Preview, Skim, and Adobe Acrobat. These programs do not have access to the source file, and often work by parsing the table of contents information in the pdf file. TeXShop is a program for authors rather than a pdf display program, so it is natural for it to find the information to fix the Page Number box in the log file.

Some users may prefer to view pdf files in TeXShop. This is possible because TeXShop can open pdf files directly, rather than having them appear indirectly when opening the source file. If these users have access to the Log file associated with the pdf, they can get Number Box correction without using the source file. Just place the Log file in the same location as the pdf file, and TeXShop will automatically open and parse it, and adjust Page Numbers accordingly.

This feature is even available if you do not have access to the Log file. For example, the Driver Manual for my car is a pdf file with two preliminary pages and then the main pages numbered 1, 2, 3, ... Using Apple's Text Edit in plain text mode, I created a small file with the same name as the pdf manual, but extension ".log". The complete contents of this file were

```
Start of main material: 3.
```

After this file was placed in the location of the manual, opening the manual in TeXShop provided PageNumber boxes with the correct numbering.

## 12.3 Getting TeX and LaTeX to Output the Crucial Line

To activate this fix when using LaTeX and related engines, add the line

```
\write128{Start of main material: \the\ReadonlyShipoutCounter.}
```

to the spot in the source just after the main material is introduced.

If instead you are using plain TeX or related engines, add the line

```
\write128{Start of main material: \the\pageno.}
```

to the spot in the source just after the main material is introduced.

My colleague Herbert Schulz has a large project using the amsbook document class. Among other things, that class defines two commands "frontmatter" and "mainmatter". These commands may also be defined in the LaTeX book document class. Schulz discovered that he could activate the fix by just redefining mainmatter in the header as follows:

```
\let\OLDmain=\mainmatter
\renewcommand{\mainmatter}
    {\OLDmain\write128{Start of main material:\ReadonlyShipoutCounter.}}
```

The last two lines above should be combined to form one line with no spaces between the end of the second line and the start of the final line. This has not yet been extensively tested, but it provides a very easy fix when it works.

TeXShop users need to find a way to remember these additional lines. Some may add items to the Macro menu, while others may modify their templates. It is also possible to add appropriate items to the Command Completion dictionary.

## 12.4 An Earlier Way to Activate the Fix

Version 3.55 of TeXShop first introduced a fix for the PageNumber box, but using a more primitive method to activate the fix. This method is still available, but should seldom be needed.

The old method required adding the following magic comment line to the source:

```
% !TEX numberingCorrection = 0 + current - desired
```

where "0" should be replaced by the number of preliminary pages in the document. If this magic line is present, then the earlier fix is turned off and the comment is used instead.

This method will get out of sync when additional pages are added to the preliminary material. But most editing is done with the main text and sync problems are then relatively

rare, caused for instance when a table of contents flows onto an extra page. TeXShop 3.55 had a trick to make fixing this sort of problem easy. Suppose you suddenly notice that you are on page 72 of the actual text, but the PageNumber box contains 78. Return to the magic comment line, which might read

```
% !TEX numberingCorrection = 3 + current - desired
```

The end of this is a formula telling you how to fix the problem. Think of "current" as the current PageNumber entry, 78. Think of "desired" as the entry you want, 72. The above formula then reads 3 + 78 - 72 or 9. This is new number you should enter in the magic comment to replace the original 3.

There is one final problem. Magic comment lines are read when TeXShop first opens a document, but TeXShop does not keep track of later changes to the lines. Thus editing the numberingCorrection line will not immediately fix page number synchronization. But the TeXShop Source menu has a new item named "Rescan Magic Comments" which will complete the process. Use this menu whenever you edit the magic comment.

# Chapter 13

# Organizing Windows in TeXShop

## 13.1 Introduction

A user working on a typical TeX project will have several windows open at once: the source window, the preview window, and the console. The number of open windows will increase if a project has a root file and individual chapter files. If several projects are open at once, many windows will be open.

Some TeXShop users cope with these multiple window by working with a large monitor, and I'll confess that I use that strategy. Other users prefer a different solution. There are essentially four ways to configure TeXShop for multiple window use:

- Use multiple windows in the normal way
- Switch to single window mode, with the source and preview in a single window
- Use multiple windows with tabs
- Use single windows with tabs

Users should select one of these strategies and stick with it. We have only discussed the first option so far. It is time to introduce the other methods.

## 13.2 Single Window Mode

Open a project which has one source window and one preview window. With one of these windows active, go to the TeXShop Window menu and select the item "Use One Window".

The two windows will be merged into a single window with the source on the left and the preview on the right. The first time this item is used, this window will be small and awkwardly placed on the screen. Resize it and place it in a convenient place. From then on, all single mode windows will appear in this location with this size.

If you prefer the preview on the left and the source on the right, go to the Preview tab in TeXShop Preferences and reset the appropriate item.

To return to ordinary mode, activate the single window and select "Use Separate Windows" in the Window menu.

The commands listed so far work best for isolated moments when it is convenient to switch one or more projects to single window mode temporarily. But if you close a project in single window mode, and later reopen it, it will appear in standard mode. If you configure TeXShop so it remembers windows that are open when it quits and reopens them when it restarts, then a source and preview in the same window when TeXShop quits will reopen in two separate windows with unusual sizes. They must be resized before being used.

In short, if you use a mixture of standard mode windows and single window mode windows, you may run into unusual behavior when quitting and restarting.

These problems do not occur if you always work in single window mode. To configure TeXShop for that behavior, select the option to "Use One Window" when "Opening Source and Preview" in TeXShop Preferences under the Preview tab. Then all documents will open in single window mode and restoring windows after quitting and restarting will work as expected. In short, you will have completely converted to One Window Mode for everything.

There are just two cautionary notes. First, projects with a root file and separate chapter files do not work in single window mode, so do not switch to single window mode if you use that facility. And second, the File menu's "New" command always produces a single source window (because there is no file to preview at first). So start new documents by writing a skeleton source file, typeset that file, and then switch that project to single window mode with the Window menu's "Use Single Window" command. Do the same thing if a colleague sends you the source for a document, but no pdf.

## 13.3 Multiple Windows with Tabs

Several years ago Apple introduced support for tabs in windows. Recently a user, Szilágyi Zsolt, wrote that TeXShop has a problem with tabs, and he sent a movie showing the problem in action. A few days later he solved the problem himself. I looked at the movie anyway, and Zsolt was doing things with tabs in TeXShop that I never dreamed possible. So I asked how he did them and was told that in Apple's System Preferences under the

General tab, there is an item labeled "Prefer tabs in full screen when opening documents." Here the words "full screen" appear in a pull-down menu whose other choices are "never" and "always". Selecting "always" greatly improves the way tabs work in TeXShop.

Next I discovered that some applications have their own similar preference. This makes it possible to improve tab support in these applications without forcing it upon the full system. For example, Safari has an item labeled "Open pages in tabs instead of windows" with possible values "Never", "Automatically", and "Always". The choice "Automatically" causes Safari to follow the global preference item in System Preferences.

TeXShop now has a similar preference setting. The item is at the bottom of the Source tab in TeXShop Preferences, with the same choices as Safari.

When separate source and preview windows are used:

- Set the new preference to "Always" and close all windows. Open a document from disk. A source window and a preview window appear. Open another document. The source and preview for the new document are added as tabs to the existing source and preview windows. We still have only two windows. Open a third document, perhaps this time one based on a root file with various chapter files. A tab for the root file appears in the source window, and a tab for the preview appears in the preview window. We still have only two windows.
- Select one of the source tabs and typeset. A console window appears; when typesetting ends, the appropriate preview tab is activated. Select a different source tab and typeset again. Rather than creating a new console window, a tab is created in the existing console; after typesetting ends, the appropriate preview tab is activated. We still have only three windows: source, preview, console.
- Try sync from source to preview or preview to source. These work and activate the appropriate tab in the destination of the sync.
- Select the log file for a document. A new window appears containing the log. Change to a different document and show its log file. A tab appears in the log window for the new log. We only have four windows: source, preview, console, log.
- Open a document in the Help menu. A new window appears containing the document. Note that the document does not appear as a tab in the source or preview windows. This is deliberate since help pages are often used in conjunction with TeX tasks and both need to be visible. We don't want to clutter the source and preview windows with documents which aren't source or preview items. Each Help document gets a separate window because multiple items aren't often used simultaneously and some of the items are displayed by other programs like Apple's Preview and not even governed by the new TeXShop preference.

- Select the menu item "About TeXShop." It is displayed in a separate dialog, as we would hope.
- In the preview window, open the Drawer. Then switch to another tab and notice that the Drawer disappears. That is because each tab controls its own drawer. Switch back to the original tab and notice that the drawer opens again. Activate drawers for two different preview tabs. Notice that when switching from one tab to another, the contents of the drawer change to match the selected tab.
- Select a tab for a document containing a root file and various chapter files. Sync from the preview to the source. Notice that the appropriate chapter file opens and a new tab for it is created. Using sync in this way, you can obtain tabs for all chapters currently being edited.

So far, things work in the expected manner. We end with a couple of cases where surprising things happen.

- Suppose we have source and preview windows with three tabs for three projects. Activate one of these tabs and select "Use One Window." A new window is created containing both source and preview panes, but only for the active tab, not for all three projects. If we select "Use One Window" again, another of the documents is converted to single window mode, but this time that single window becomes a tab in the single window for the first document. Selecting "Use One Window" a third time converts the remaining document, and now at last we have what we expected to get immediately, namely a one-window object with three tabs for the three documents. Unwrapping these windows using "Use Separate Windows" works the same way, unwrapping one document at a time.
- The surprising behavior of tabs and "Use One Window" is actually useful because it solves a problem with full screen mode. This mode is essentially useless if you are a "separate window mode" person, because it is impossible to write TeX if you only have access to the source, or only have access to the output. But suppose you are writing a very tricky portion of a document and would like to concentrate only on it without distractions. Convert the document to a single window with "Use One Window." Then click the green button to make this window full screen. You now have both source and text on the screen, which is exactly what you need. Moreover, you can typeset because when you do that, the console becomes a floating window on top of the screen. Use it to fix errors and then close it. Continue editing. When you are done, click the green button to return to a single window, and then select "Use Separate Windows" to return your document to tabs where it started.
- You can also move two different source tabs to tabs in a single window and then convert this window to full screen. This makes it possible to work on two separate documents at the same time in full screen mode.

- There is only one really bad piece of news for people who use separate source and preview windows. If you have a document with a root file and various chapter files, tabs work fine as long as you stay in this mode. But converting such a document to single-window mode and then taking it full screen will completely fail. Single windows contain ONE source file and ONE preview file. If your project also has chapter files and you try to convert it to single window mode, you end up with a single window for the root source and the preview output, and then lots of source windows for the individual chapters. This isn't something I can fix.

So you have to make a choice. If you like to split your projects into a root and chapters, fine. This decision is completely reasonable. Tabs may make your screen less cluttered; give them a try. Never convert projects with a root and various chapter files to one window mode, and never use full screen mode for this kind of project. This is actually how I often work.

The other choice is to write books with just a single source file. Many famous authors work this way. I used to use ROOT-CHAPTER, but recently I've written even longer documents with just a single source. In that case, the full TeXShop interface is available to you.

## 13.4 Single Window Mode and Tabs

If you work exclusively in single-window mode and want to use tabs:

- I recommend activating the preference item "Opening Source and Preview: Use One Window" at the bottom of the Preview Tab in TeXShop Preferences. Then documents immediately open in one window mode.
- Although the Window menu has items "Use One Window" and "Use Separate Windows", there are only two circumstances when these are helpful. The first occurs when you start a new document. The second occurs when someone sends you a TeX source document without an associated pdf. In both cases, create or open the source file. Typeset. Now you have both windows. Activate one of the two and select "Use One Window".
- Set the new tab preference to "Always" and close all windows. Open a document from disk. A window opens containing the source and the output of the document. Open a second document. A new tab appears in the window, showing the source and output of the second document. We still have one window. Open a third document. Another tab appears and our window contains all three documents.
- We do not get one set of tabs for the preview portion and a separate set for the output portion. Instead we only get one set of tabs because tabs select entire documents.
- Select a tab and typeset. A console window appears and the new output appears in

the window. Select a different source tab and typeset again. Rather than creating a new console window, a tab is created in the existing console. We still have only two windows: project and console.

- Sync works fine.
- Select the log file for a document. A new window appears containing the log. Change to a different document and show its log file. A tab appears in the log window for the new log. We only have three windows: project, console, log.
- Open a document in the Help menu. A new window appears containing the document, so the help document does not appear as a tab in the document window. This is deliberate since help pages are often used in conjunction with TeX tasks and both need to be visible. Note that each Help document gets a separate window.
- Select the menu item "About TeXShop." It is displayed in a separate dialog, as usual.
- Open the drawer. Then switch to another tab and notice that the Drawer disappears. That is because each tab controls its own drawer. Switch back to the original tab and notice that the drawer opens again. Activate drawers for two different tabs. Notice that when switching from one tab to another, the contents of the drawer change to match the selected tab.
- So far, things work in the expected manner. There is one surprise. Suppose we still have three tabs. Activate one of these tabs and select "Use Separate Windows" Two new windows are created containing the source and preview for the active tab, not for all three tabs. If we select "Use Separate Windows" again, another of the tabs is converted source and preview files, this time as tabs in the source and preview windows. So we work one project at a time, rather than converting all tabs at once. However, this is a task that is rarely done if you use one-window mode.
- Select a window with tabs. Click the green button to convert the window to full screen mode. You can still typeset because the console will appear as a floating window over the screen, and can be closed to give a full screen view. Now tabs allow you to work on several projects at once in full screen mode.
- There is only one bad piece of news. If you work in one-window mode, you cannot work with projects that have a ROOT file and then various CHAPTER files. But you couldn't do that before tabs either. Many authors of books keep the full source in a single file.

## 13.5 Obsolete Tab Commands

Before the changes described above were present, TeXShop had a few other features dealing with tabs. These features are now obsolete and should not be used.

In TeXShop Preferences under the Typesetting tab, there is an item labeled "If Sync Opens a New Window" and then a check mark "Open as Tab in Root Window." Do not check this.

Version 3.82 and 3.84 introduced magic comment lines dealing with tabs. Read the Changes document for details. These methods are untested with the modern tab support and should not be used.

```
% !TEX useTabs
% !TEX useTabsWithFiles
% !TEX tabbedFile{chapter1/Introduction.tex} (One)
```

# Chapter 14

# Macros

TeXShop users can write and execute AppleScript commands using the Macros feature of TeXShop. But what is AppleScript? Let me quote Apple's *Introduction to AppleScript Language Guide*:

> AppleScript is a scripting language created by Apple. It allows users to directly control scriptable Macintosh applications, as well as parts of macOS itself. You can create scripts—sets of written instructions—to automate repetitive tasks, combine features from multiple scriptable applications, and create complex workflows.
>
> A scriptable application is one that can be controlled by a script. For AppleScript, that means being responsive to interapplication messages, called Apple events, sent when a script command targets the application. (Apple events can also be sent directly from other applications and macOS.)

Macros and the Macro Editor are by Mitsuhiro Shishikura. Default Latex macros were created by Mitsuhiro Shishikura and Hirokazu Ogawa. Default Context macros were created by Hans Hagen.

I should confess that I find AppleScript difficult to use. The language was designed to be "conversational", but this creates ambiguities, at least for me. The current AppleScript expert for TeXShop is Michael Sharpe, who wrote a great document about it named *Notes on AppleScript in TeXShop*. His document is provided as an item in the TeXShop Help Menu.

An example will explain why AppleScript is useful. Around 2009, Ramon Figueroa-Centeno wrote three crucial AppleScript commands named "Program", "Encoding", and "Root". These are available near the top of the Macros menu, and I use them every day and recommend them to all TeXShop users. Each creates a magic comment line. For instance,

the "Program" command displays a list of all active engines. Select one of these, and the corresponding magic line similar to

```
% !TEX program = xelatex
```

is written at the top of the source, replacing any similar existing line if necessary. Magic comment lines are picky about spaces and capitalization of engine names, but the Macro handles all those details automatically.

The Macros feature of TeXShop has been stable for many years, so the material below is taken directly from the TeXShop Help Panel.

## 14.1 Introduction

TeXShop Macros and the Macro Editor were written by Mitsuhiro Shishikura. The available Macros depend on the typesetting engine chosen. If this engine is LaTeX, then a series of macros suitable for LaTeX will be displayed. If the engine is ConTeXT, then macros for ConTeXt will be displayed. When TeXShop is first installed, it has macros for only these two engines; the LaTeX macros will be displayed for all engines except ConTeXt.

Macros come in two kinds. Some insert strings into the TeX source file; these macros are similar to buttons in the Latex panel. Other macros run AppleScript scripts.

Macros are stored in

```
~/Library/TeXShop/Macros/Macros_Latex.plist
```

and

```
~/Library/TeXShop/Macros/Macros_Context.plist.
```

If these files are missing when TeXShop runs, default macro files are created there. These files are ordinary text files, so it can be opened and inspected with TeXShop. This is usually not necessary because the files are best manipulated with the Macro editor.

The Macro editor modifies macros stored in Macros_Latex.plist and Macros_Context.plist when one of these typesetting engines is chosen. But if another typesetting engine like TeX is chosen, then the Macro editor will replace the default Latex macro set with a set defined by the user for TeX, storing the results in Macros_Tex.plist and leaving Macros_Latex.plist unchanged.

When you define useful Macros and wish to give them to others, simply open the Macro Editor and choose "Save selection to file..." in the Macro menu. This will create a file in any location you wish. To distribute your macros, send this file.

To add macros created by others to your list of macros, open the Macro Editor and choose "Add macros from file..." Then use the Macro Editor to arrange the macros as you desire.

New items, new submenus, and new separators can be created by the buttons at the bottom left. A little practice illustrates the basic behavior of the editor. Submenus can be created to any level.

Any particular item can be assigned a keyboard equivalent using the buttons on the right side of the menu. Note that the command key is automatically part of this shortcut. The result will immediately appear in the outline view. However, these assignments cannot duplicate key combinations already used by TeXShop menus. If they do, the new keyboard shortcut will be ignored.

## 14.2 Default Macros

The best way to understand default macros is to examine their definition with the Macro Editor. For example, consider the macro titled "Begin/End". Suppose you wish to use an environment like the theorem environment. Type the word "theorem" and select it. Then choose the "Begin/End macro. The word "theorem" will be replaced with the text

```
\begin{theorem}
\end{theorem}
```

with the cursor placed on the line between this pair.

Now examine the begin/end macro code:

```
\begin{#SEL#}
#INS#
\end{#SEL#}
```

Text in the macro will be inserted into the source file. Each occurrence of the string #SEL# will be replaced by the text selected when the macro was invoked. If no text was selected, #SEL# will be replaced with an empty string. The cursor will be placed at the end of the inserted text unless the text contains the string #INS#, in which case the cursor will be placed at that location.

Using this knowledge, it is easy to understand and modify the default macros.

## 14.3 Rearranging Macros

Open the Macro Editor and examine the macro menu on the left. It is shown in an outline view similar to the view of the file system obtained by choosing the middle button of the

View tab in the Finder. To rearrange items, drag them from one spot to another. Notice that items can be placed at different levels by sliding left and right. To rename an item, select it in the outline view and type a new name in the field at the top right.

## 14.4 AppleScript Macros

If a macro begins with the string

```
--AppleScript
```

or

```
--AppleScript direct
```

then the resulting AppleScript code will run when the Macro is chosen. For example, consider the macro titled "View pdf with Acrobat". Choosing this macro when the source file is active (and the pdf file has been created) will start Adobe Acrobat Reader and open the output pdf file in that program. The corresponding AppleScript code reads

```
--AppleScript direct

  tell application "Adobe Acrobat Reader"
     activate
     open POSIX file #PDFPATH#
  end tell
```

Commands beginning with "–AppleScript direct" are run directly by TeXShop. During the time this AppleScript is running, TeXShop's event loop is not active. Consequently, the script cannot call TeXShop to perform an action which might require user input. For example, it cannot ask TeXShop to run LaTeX, because if the LaTeX file has an error, the console will appear and wait for user input, but such input would not be recognized.

Commands beginning with "–AppleScript" are run by a small auxiliary program in the TeXShop application bundle. Such commands can ask TeXShop to perform actions which might require user input, because in that case the auxiliary program will temporarily halt but TeXShop will remain active.

Thus the syntax "–AppleScript direct" is suitable for routine macros, but "–AppleScript" may be needed for fancy ones.

## 14.5 Included AppleScript Commands

TeXShop comes with an extended collection of AppleScripts by Will Robertson, Claus Gerhardt and others. Some of these scripts automate workflow when a series of typesetting

commands must be issued in sequence. By copying and modifying the scripts, users can construct scripts appropriate for their own workflow.

In this section, we will describe some of these scripts.

### Column Macros, Insert Reference, Open Quickly

These macros are by Will Robertson. The first provides a very convenient way to construct an arbitrary matrix or table. The second searches through the current file for

```
\label{...}
```

commands and then pops up a list from which you may insert a reference label, wrapped in an (optional) customizable LaTeX command. The final macro allows you to rapidly open any file in the directory of the current source file.

### Paste Spreadsheet Cells

This macro (by Alan Munn) provides cut-and-paste functionality from spreadsheet applications into your LaTeX source, using various popular table styles. Select a range of cells in Excel or other spreadsheet application, and copy them to the clipboard. Within TeXShop, choose Paste Spreadsheet Cells in the Macros menu. You will be asked to choose a table style. The styles available are the following:

- **cells**
  converts the clipboard text simply to latex table cells
- **simple**
  creates a simple tabular environment around the cells
- **booktabs**
  assumes the first line is a header, and places “toprule”, “midrule” and “bottomrules” in the appropriate places within a tabular environment
- **longtable**
  assumes the first line is a header, creates the appropriate longtable header information and places the cells in a longtable environment

Note that the script modifies the contents of the clipboard to contain the LaTeX formatted text, and not your originally copied cells. This means that if you need to change cell formats, you must re-copy the cells from your spreadsheet before choosing a different cell format to paste.

### Convert to Mac, Convert to Unix, Convert to Windows

In the early days, teletype terminals were used for communication. The ascii character set still has remnants from those days; for instance 0x07 rings the teletype bell. The character 0x0a was a line feed which turned the carriage to the next line and the character 0x0d was a carriage return which pulled the carriage back to the start of the line. To get a line feed, one used the sequence 0x0d 0x0a.

After the age of teletypes ended, computer manufacturers selected different subsets of these characters to indicate a line feed. In the Unix world, 0x0a was used, in the Windows world 0x0d 0x0a was used, and in the old Macintosh Classic world 0x0d was used. Apple's guidelines for Mac OS X state that programs should be able to automatically open files using any of these conventions. Most programs including TextEdit, TeXShop, etc., follow these guidelines. In TeXShop, new files are created using Unix conventions, but if you load a file created with Mac OS Classic and then add extra lines, old portions of the document will be saved with the Classic convention and new sections will have the Unix convention.

These line feeds cause no trouble until you send a file to a friend using a different operating system. Many editors now understand multiple line feed conventions, so often you'll have no problems. But if you do, use the scripts above. Suppose the source file for a document is named MyFile.tex. Then "Convert to Mac" will create a new file named MyFile_Mac.tex with the same source code and Macintosh line feed conventions. "Convert to Unix" and "Convert to Windows" work the same way. All of these scripts call a binary program by Craig Stuart Sapp named "flip" in `~/Library/TeXShop/bin`.

### Other Scripts: Bibliography

Processing a file with a bibliography requires multiple typesetting operations. First Latex is run to create an .aux file. Then Bibtex is run and uses this file to create .bbl and .blg files. Latex is run again to add the bibliography to the document. Latex is run a final time to update the references to the bibliography in the text.

The "Bibliography" command does all of these things one by one. First it saves the file. Then it runs latex->bibtex->latex->latex. Finally it updates the preview display.

### htlatexc, htlatexr

Tex4ht is a TeX program which converts a latex document into a web page. The source can be a standard Latex file and can include eps illustrations. The end result is an html page and a large number of gif files. The script command "htlatexc" saves the source documents, runs htlatex, and opens the resulting html file in Safari. Thus it behaves like a new TeXShop typesetting command, except that it was constructed by a user without

waiting for new TeXShop code! (Note: In 2023, TeXShop was modified to directly typeset with TeX4ht and display the result in an internal HTML Preview Window rather than Safari. See a later chapter for details.)

When using these scripts, it is not necessary to use the package tex4ht since this package will automatically be loaded. The script htlatexc calls htlatex without additional options. The script htlatexr calls it with the option "-r", which tells the program to recreate any .gifs which already exist.

**pdfselectc**

This script runs the shell script pdfselect to create pdf files containing only certain selected pages of a document. When it is run, a dialog appears asking for the number of pdf files to be created. Suppose we answer 3. Next a dialog appears asking for the range for the first document. Suppose we answer 5:8. A dialog appears asking for the range of the second document. Suppose we answer 10. A dialog asks for the range of the third document. Suppose we answer 20:30. We will then obtain three documents, one containing pages 5 - 8 of the original, one containing page 10, and one containing pages 20 - 30.

**mpostc, mpostcpl, latex-makeindex-mpost**

The script mpostc runs mpost and then pdflatex; the script mpostcpl runs pdflatex, and then mpost, and then pdflatex again.

The script latex-makeindex-mpost saves the source and runs pdflatex, makeindex, mpost, and pdflatex again, opening relevant log files in the process.

## 14.6 Defining AppleScripts

AppleScript syntax is a subject all its own. To learn more about it, read one of several books on the subject. If you installed the developer tools which come with Mac OS X, there is an online book about AppleScript in the Developer folder.

When TeXShop interpretes an AppleScript macro, it first replaces any string #FILEPATH#, #PDFPATH#, #DVIPATH#, #PSPATH#, #LOGPATH#, #AUXPATH# with the complete path name of the source tex file, pdf file, dvi file, ps file, log file, or aux file respectively. Similarly, the strings #INDPATH#, #BBLPATH#, and #HTMLPATH# are replaced with the complete path name of the ind file, bbl file, or html file. Each of these items refer to the frontmost document when the AppleScript actually runs.

Starting with TeXShop 5.23, the string #FOLDERPATH# is also available and will be replaced with the complete path to the folder containing the tex, pdf, log, and aux files of

a document. So if #FILEPATH# gives /Users/koch/Projects/Fourier/Fourier.tex, then #FOLDERPATH# will give /Users/koch/Projects/Fourier/.

In addition, any string #NAMEPATH# is replaced with the complete path name of the source tex file minus its extension, and any string #DOCUMENTNAME# is replaced with the display name of the current document. This last replacement is somewhat subtle; it gives the title of the document as shown at the top of the source window. If a document was saved with the "hide extension" box checked, #DOCUMENTNAME# will contain only the document name. But if the document was saved without checking "hide extension", #DOCUMENTNAME# will contain the document name and extension. This information can be used to locate the calling document for AppleScript code as follows:

```
tell document #DOCUMENTNAME# of application "TeXShop"
  latex
end tell
```

There are at least two ways to write AppleScript commands. AppleScript can run shell commands, so after preliminary processing with an AppleScript, a shell command can be called to do the actual work. TeXShop comes with several examples of this technique; some of these examples will be explained in a later help section. AppleScript commands can also call built-in TeXShop commands and thus work directly. A later section will give examples of this technique.

TeXShop understands the following commands:

```
typeset
latex
tex
context
bibtex
makeindex
metapost
typesetinteractive
latexinteractive
texinteractive
contextinteractive
bibtexinteractive
makeindexinteractive
metapostinteractive
taskdone
refreshpdf
refreshtext
goto line
```

The first seven commands call TeXShop typesetting routines. These commands typeset continuously without stopping at errors. The next seven commands also call TeXShop typesetting commands, but this time if there is an error, the user is allowed to interact with the console. When a typesetting command is called, control returns to the AppleScript immediately without waiting until the operation is complete. The "taskdone" call returns NO if typesetting is still running, and YES when it is done. The calls "refreshpdf" and "refreshtext" cause pdf and text documents to display the latest version of their files on the screen. The "goto line" command tells the editor to select a given line; for example:

```
tell document #DOCUMENTNAME# of application "TeXShop"
  goto line 37
end tell
```

## 14.7 Dialogs

AppleScript can create dialogs and act on user input. For example, insert the following script and test its behavior.

```
-- AppleScript <br>
-- Use a dialog to enter user defined information into an AppleScript.
-- There are two feedbacks possible 'text' and "choice of buttons".
-- The returned information can be used to define corresponding variables.
-- The number of buttons may not exceed three.
-- In the three examples below it is shown how to use text return,
-- button return, and both.

activate
display dialog "Test dialog: type something below" default answer "Hello World!"
   buttons {"A", "B", "C"} default button "B"
set theText to (the text returned of the result)
display dialog "Test dialog: type something below" default answer "Hello World!"
   buttons {"A", "B", "C"} default button "B"
set theButton to (the button returned of the result)
display dialog "Test dialog: type something below" default answer "Hello World!"
   buttons {"A", "B", "C"} default button "B"
set {theText, theButton} to
  {the text returned of the result, the button returned of the result}
```

One command in this example, "activate", requires comment. When TeXShop is asked to run an AppleScript, it calls a separate program to run the script. That program, Scriptrunner, is in the TeXShop bundle. Any dialogs created by the script will be displayed by Scriptrunner rather than by TeXShop. The "activate" command brings Scriptrunner to

the front so dialogs will appear on top of other windows.

## 14.8 Writing Complete AppleScripts

If a TeX project contains a bibliography, a sequence of typesetting commands must be run to update the bibliography. Latex is run first to create an .aux file. Bibtex is run to create .bbl and .blg files from this .aux file. Latex is run again to add the bibliography to the document. Latex is run a final time to update the references to the bibliography in the document.

Your projects may contain similar sequences of typesetting commands. It is possible to use AppleScript to automate these sequences.

To see how to do that we'll examine the OtherScripts: Bibliography command which comes with TeXShop. Here is the body of that command:

```
--AppleScript

set fileName to #FILEPATH#
if fileName is equal to  ""
  activate
  display dialog "Please save the file first"
       buttons {"OK"} default button "OK"
  return
end if

set frontName to #DOCUMENTNAME#
tell application "TeXShop"
  save document frontName
end tell

tell document frontName of application "TeXShop"
  latexinteractive
  repeat
    delay 2
    if taskdone
      exit repeat
    end if
  end repeat

  bibtex
  repeat
```

```
      delay 2
      if taskdone
        exit repeat
      end if
    end repeat

    latex
    repeat
      delay 2
      if taskdone
        exit repeat
      end if
    end repeat

    latex
  end tell
```

The first line of this command indicates that this is an AppleScript macro. The next lines check #FILEPATH#, a parameter which gives the full path to the tex source. This parameter is an empty string when a new document has been created and not yet saved. In that case the user is asked to save the document and the script ends.

The next line tells TeXShop to save the document. Notice that we use #DOCUMENT-NAME# to refer to the document in question.

The remaining commands run latexinteractive, bibtex, latex, and latex. Recall that control returns to AppleScript immediately after calling a typesetting command before the typesetting job is over. The repeat loop tells the script to check whether the job is complete before running another typesetting task. The line "delay 2" causes AppleScript to pause rather than continually asking if the task is done and thereby slowing the entire computer down.

## 14.9 Writing Command Scripts

We will use the AppleScript "pdflatexc" by Claus Gerhardt to illustrate the principles involved in calling a Unix shell script from an AppleScript. This particular shell script isn't very interesting; it adds the location of the TeX binary files to the $PATH variable and calls pdflatex to typeset.

The shell script itself is independent of TeXShop and can be run from the Terminal by typing "pdflatexc myfile.tex" provided the directory holding pdflatexc is in the search path for binaries. Here is that shell script:

```
#!/bin/tcsh
# pdflatexc
# Claus Gerhardt
#
# Usage
# pdflatexc filename.tex

set path= ($path /Library/TeX/texbin /usr/texbin /usr/local/bin)]
pdflatex --shell-escape "$1"
```

Of course you are likely to write a more complicated script which performs several operations in sequence. That is when these techniques become useful.

The AppleScript used to call this shell script is more interesting. Here it is:

```
--AppleScript
-- Apply only to an already saved file.
-- Claus Gerhardt, Nov. 2003

set scriptPath to (do shell script "dirname " & "~/Library/TeXShop/Scripts/ex")
set scriptPath to scriptPath & "/setname.scpt"
set scriptName to POSIX file scriptPath as alias
set scriptLiB to (load script scriptName)
tell scriptLib
    set frontName to setname(#NAMEPATH#,#TEXPATH#)
end tell

set fileName to  #TEXPATH#
set n to (number of characters of contents of fileName)
set fileNamequoted to quoted form of fileName
set baseName to do shell script "basename " & fileNamequoted
set m to (number of characters of contents of baseName)
set dirName to quoted form of (characters 1 thru (n - m - 1) of fileName as string)

set shellScript to "cd " & dirName & ";"
set shellScript to shellScript & "~/Library/TeXShop/bin/pdflatexc  " & baseName
do shell script shellScript

tell document frontName
    refreshpdf
end tell
```

Ignoring the introductory comments, the first seven lines of this script are a magic recipe

by Claus Gerhardt to save the source file and find the name of the document in question, setting "frontName" to this name. This recipe uses a compiled script named "setpath.scpt" in `~/Library/TeXShop/Scripts` to do all the hard work. Careful reading shows that these lines find the path `~/Library/TeXShop/Scripts/setname.scpt` and call this script with parameters #NAMEPATH# and #TEXPATH#.

In many AppleScripts all of this could be accomplished in an easier way using the commands

```
set frontName to #DOCUMENTNAME#
tell document frontName of application "TeXShop"
     save
end tell
```

However, Gerhardt's script has two advantages. First, his script can be called when the front document is a log file, say /Users/koch/Examples/myfile.log, that is, in a case when the front document is not the document which should receive later commands. Second, if the file has never been saved, Gerhardt's script returns an error when trying to save, while the "save" command would cause TeXShop to hang after it put up a save dialog (see the help document Writing Scripts with TeXShop Typesetting Commands for details.)

The next six lines of the script define the variables dirName and baseName. If the source file is "/Users/koch/This directory/Stuff/myfile.tex", dirName is " '/Users/koch/This directory/Stuff' ", including the single and double quotation marks, and baseName is "myfile.tex." A lot of this work is required so spaces can occur in the names of folders. As always, tex does not allow spaces in the final file name.

The next three line call the shell script. The result is the same as typing "cd dirName; /Library/TeXShop/bin/pdflatexc baseName" in a Terminal, although, to be absolutely precise, dirName would have to be replaced by its unquoted form, i.e., if we use the example above, by '/Users/koch/This directory/Stuff' including the single quotes because of the space in the name of one directory.

Shell commands in AppleScript are issued in the form

```
do shell script "cmd input"
```

If one wants to combine several shell scripts, it is better to write the command in an equivalent form

```
do shell script "cmd " & "input"
```

Notice the space behind cmd and the quotes. The ampersand is a binary concatenation operator, i.e.,

```
"cmd " & "input" = "cmd input"
```

When the shell is called via an AppleScript, the default working directory is the root directory. The command

```
do shell script "cd " dirName
```

changes the directory to the directory specified in dirName; dirName is already quoted so that additional quotes are not needed.

If one would like to keep the working directory and issue further commands, then these commands may not be stated in the form "do shell script", since then a new shell would be invoked. In the terminal one would separate consecutive commands by semicolons, in AppleScript one does it by concatenating ";", i.e.,

```
do shell script "cmd(1) " & "input(1)" & ";" & "cmd(2) " & "input(2)"
```

This is done in the above example: cmd(1) = cd, "input(1)" = dirName, cmd(2) = ~/Library/TeX/bin/pdflatexc - calling the shell script -, and "input(2)" = baseName.

The lines

```
<div id="source">
set shellScript to "cd " & dirName & ";"
<br>set shellScript to shellScript & "~/Library/TeX/bin/pdflatexc  " & baseName
```

are simply a convenient way to concatenate this sequence of commands and input into one variable.

The final three lines update the Preview window.

To be sure, several of these lines are complicated, but these lines can be copied without change into new scripts.

# Chapter 15

# Editing Aids

TeXShop has several methods to speed the entry of source text in the editor. Each of these methods is optional, and each is customizable. Some users will ignore the methods (I'm one of them because I write slowly), some users will use one or more methods in the default form we provide, and some users will spend time customizing each to suit their exact entry style.

## 15.1 Emacs Mode

There is a famous text editor named **emacs**. This editor was written before computers had mice, so it had elaborate commands to move the cursor. There were keystrokes to move forward one character, back one character, forward one word, back one word, forward one sentence, back one sentence, etc. Additional keystroke commands were available for copy and paste. The great disadvantage of emacs was that the editor came with a two page table of instructions and took weeks to master. The advantage is that it allowed rapid entry of text without lifting a hand to reach the mouse. (Modern versions of emacs accommodate mice.)

The Cocoa text editing code used by TeXShop and many other programs on the Macintosh was originally written by engineers at NeXT; many of these engineers used emacs. So they added a vast number of emacs keystroke shortcuts to the editor. I don't know the history of this code, but I like to imagine that the emacs keystrokes were added secretly without telling Steve Jobs. Most modern Mac users are completely unaware of this feature.

For a small sample, hold down the control key and type the letter "f". Notice that the cursor moves forward one character with each keystroke. Replace "f" by "b" and the cursor moves backward.

Herbert Schulz has a useful website with many items for TeXShop users: https://herbs.github.io. One of these items is a folder with Key Binding documents and samples. Don't be misled by the folder title; much of the information in this folder is about emacs. The folder has a link to a site at Harvard with very extensive information, but sadly that site is gone. However, the material is still available at https://github.com/jrus/cocoa-text-system#readme. Using the green Code button, download the ZIP file and decompress. The resulting folder has several files including three crucial html files.

Emacs keystrokes were completely customizable and the shortcuts for the Cocoa editor are similarly customizable. This is done by placing a file named DefaultKeyBinding.dict in `~/Library/KeyBindings`. The structure of this file can be found with Google searches; some sites contain sample customization files. Note that this DefaultKeyBindings.dict file has nothing to do with TeXShop's Key Bindings, and instead is about customizing the Cocoa support for emacs features.

Essentially, the DefaultKeyBindings file is a list of keystrokes and calls to procedures in NSTextView, the Cocoa editing code. For example, one line might read

```
"^f" = "moveForward:";
```

Here ^f stands for Control-f and moveForward is the Cocoa procedure which moves the cursor one character forward, documented as follows:

```
- (void)moveForward:(id)sender;
```

A professor of mathematics named David Wang asked me to add a keyboard shortcut to TeXShop which moved the cursor past the next dollar sign. I wrote back that the appropriate way to do that would be to use the emacs key bindings system. It turned out that Wang knew about and used the emacs cursor moving shortcuts. But unfortunately, searching for a string is not a procedure that the Key Bindings system can use. So I added four calls to TeXShop:

```
- (void)moveForwardTo$:(id)sender;
- (void)moveForwardTo$$:(id)sender;
- (void)moveBackwardTo$:(id)sender;
- (void)moveBackwardTo$$:(id)sender;
```

These calls move the cursor from its current position to just after the next $, just after the next $$, and so forth. Using the calls, the required shortcuts can easily be obtained. Below is a sample DefaultKeyBinding.dict file. If you want to test the feature, create this file and place it in /Library/KeyBindings. Then push the option key and f or b to move to an appropriate $, or push the control key and f or b to move to an appropriate $$.

```
/* ~/Library/KeyBindings/DefaultKeyBinding.dict */

{
    /* Sample Math Mode bindings */
    "~f" = "moveForwardTo$:";
    "~b" = "moveBackwardTo$:";
    "^f" = "moveForwardTo$$:";
    "^b" = "moveBackwardTo$$:";
}
```

The above DefaultKeyBinding file is not appropriate for permanent use. For one thing, it shadows other commands already available in the default system. Most users should just ignore this new feature, which is only provided for users who have already customized the Key Bindings system. If they want to add the new item, they will need to edit their DefaultKeyBinding.dict file and choose appropriate keystrokes to add the new search.

Note that DefaultKeyBinding.dict adds keyboard shortcuts to *all* editors which use the Cocoa code, not just TeXShop. This file is read when a program first starts. It turns out that commands like "moveForwardTo$" not in a program's code base are ignored. So, for example, the above sample does not harm Apple's TextEdit.

User-defined edit commands used by Key Bindings cannot have parameters. Thus I could not expose general procedures like

```
moveForwardTo: (String *)s;
moveBackwardTo: (String *)s;
```

However, these two procedures are now in TeXShop, and the four procedures listed earlier just call them. This means that if you would like to write a keyboard shortcut which searches for a different string like "begin" or "end" and moves the cursor appropriately, you will need to add appropriate routines to TeXShop and recompile. But writing these routines would be trivial because you can just call my generic routines.

I'd be happy to add such special routines to TeXShop upon request, so they remain available after program updates. Just write.

## 15.2 Key Bindings

This section is about the TeXShop *Key Bindings* feature, which is independent of the previous Emacs material.

The idea behind Key Bindings is simple. Some rarely-used keys can be reprogrammed in TeXShop to enter something else, and this "something else" can be several characters rather than just one.

For example, a quotation on a typewriter would be written

```
He replied "that is not the point".
```

But when typeset by LaTeX, this input would become “that is not the point” rather than the desired “that is not the point”. Key Bindings can reprogram the " character to input the correct symbols to start a LaTeX quote and the correct symbols to end that quote with the cursor positioned in the middle. If text is selected in the editor and then " is typed, Key Bindings can insert LaTeX quote symbols before the selection and after it.

There is an item in TeXShop Preferences under the Edit tab to turn Key Bindings on or off when documents are first opened. To be frank, I leave it off because I find it confusing to type a key that is supposed to produce one symbol and have it produce something else. But if you can get around that hangup, turn it on.

There is also an item in the TeXShop “Source” menu to toggle Key Bindings on or off. So if you use Key Bindings but suddenly need an unusual character that has been remapped, turn Key Bindings off, type the character, and turn it back on. If this situation occurs often for you, there is an optional tool for the Source Window toolbar which toggles Key Bindings on and off.

Finally there is another item in the TeXShop “Source” menu called “Edit Key Bindings File ...” This brings up an editor listing keys in a column on the left and the corresponding output in a similar column on the right. Extra items can be added, and existing items can be edited or removed.

Note the special symbols #SEL# and #INS#. If there is selected text when a remapped key is pressed, #SEL# will be that selection in the resulting output. The insertion cursor will be placed at the spot #INS# in the resulting output.

Although many items are remapped by the default Key Bindings, my guess is that only a few items are used, and users aren’t even aware of the others. This seems to do no harm.

If you are one of the few users who wants to rewrite the Key Bindings list, it might be best to take a screen snapshot of the existing list first. Then if an item is accidentally mangled or deleted, it can be added again. However, there is an easy way to get back to the default list if editing breaks things. Go to ∼/Library/TeXShop and drag the folder “Keyboard” to the desktop. The next time TeXShop starts, it will recreate this folder with default contents. The folder contains a single file named “autocompletion.plist”. Despite the name, this is the list of Key Binding modifications.

## 15.3 Command Completion

### 15.3.1 Introduction

The Command Completion feature was written by Greg Landweber. Command Completion in TeXShop allows you quickly enter commands and environment structures into your source document. It also allows you to easily move from one argument for a command to another using built-in menu commands. Command Completion is always on; there are no Preference settings or menu commands to turn it off. But it is unlikely that a casual user will accidentally trigger it.

In a TeXShop document, type

```
\begin{th
```

and then press the Escape key. TeXShop knows several standard LaTeX constructions which begin with these characters, and it cycles through them with each push of Escape. One of the elements in the cycle is the characters just typed; the other possibilities are

```
\begin{thebibliography}
\begin{theindex}
\begin{theorem}
```

However, TeXShop does not just finish the command word; it adds more. For instance, for the "theorem" choice it writes

```
\begin{theorem}
•
\end{theorem}•
```

and the first of the two stars is selected. The user can immediately type the theorem, and the first keystroke will replace the initial star. When the theorem text is complete, type control-command-F to jump to the second star and continue with the remaining source. This move can also be done by holding down the Option key and typing Escape.

### 15.3.2 Herbert Schulz and Command Completion

I've been too lazy to learn Command Completion. The expert on that system (and several other aspects of TeXShop) is Herbert Schulz. This section of the manual has been copied word for word from one of his documents. Schulz wrote a much larger and more detailed document as well, *Command Completion for TeXShop*. That document and other support material is in `~/Library/TeXShop/CommandCompletion`.

In the "Source" menu there is a submenu CommandCompletion/Marks with the items: Next Mark, Previous Mark. These items have keyboard shortcuts control-command-F and control-command-G. A regular user of CommandCompletion will remember these keyboard

shortcuts and thus easily enter source, jumping from star to star, without lifting a hand from the keyboard.

Some users prefer to use Tab rather than Escape for triggering operations. TeXShop has a Preference setting under the Source tab to make this change.

It takes practice to learn which commands have completions and which do not. For example,

```
\begin{
```

has no completions because many TeX commands start with "begin" and it makes little sense to list them all. However

```
\begin{t
```

has many completions, and

```
\begin{theo
```

only has one.

While the list of abbreviations is fairly extensive, just learn a few that you need all the time and slowly pick up others as you need them. Press the trigger key multiple times to get the same commands and environments with differing numbers of options.

Some commands are abbreviated with the first few letters of the command. Others are created with a sort of nickname. The following are just a few samples of each type.

### 15.3.3 Completing Using First Few Characters

On a new line type `\newc` and then press `Esc` to get:

`\newcommand{•}{•}`

while successive presses of `Esc` give

`\newcommand{•}[•]{•}`

and finally

`\newcommand{•}[•][•]{•}`

all with the first "Mark" ( • ) selected.

You need only start to type the new command's name to replace the selected •. Then select the next mark by using the `Source` → `Completion` → `Marks` → `Next Mark` (`Ctl-Cmd-F` (or, alternatively, `Opt-Esc` ) menu item and enter the next argument, etc.

You can complete many commands by starting to type them and pressing the Escape key. Variations on the commands with differing numbers of optional arguments are generated by additional presses of Escape. One example: typing `\sec` on a new line and then the Escape key produces

```
\section{•}
```

while a second press of Escape gives

```
\section*{•}
```

the *-variant of the command and a final press of Escape gives

```
\section[•]{•}
```

with the optional argument.

### 15.3.4 Abbreviations

In addition to command completion there also exist many abbreviations for commands. The principal difference is that an abbreviation is a mnemonic, a short name, rather than the start of a command name. There are many abbreviations for different environments; all of them start with a 'b'.

For example typing `benu` and then `Esc` at the beginning of a line will produce the complete enumerated list environment:

```
\begin{enumerate}
\item
•
\end{enumerate}•
```

as you might expect; the final Mark is there so it is easy to skip to the end of the environment using the `Next Mark` menu item. You can add an additional item to the list by typing `ite` and press `Esc` at the start of a new line to get and additonal

```
\item
•
```

inserted at that spot. Additional handy environment abbreviations are `bite`, `bali`, `bfig`, `btabl` and `btab` for the `itemize`, `align`, `figure`, `table` and `tabular` environments respectively; see the full list in the documentation.

Sectioning and font variation commands also have abbreviations. Typing `sec` and pressing `Esc` on a new line produces

```
\section{•}
```

while additional presses of Esc give the variations

```
\section*{•}
```

and

```
\section[•]{•}
```

in turn. Additional sectioning abbreviations are ssec, sssec, par and spar for sub-section, sub-sub-section, paragraph and sub-paragraph respectively.

The abbreviations for font changing commands can come in-line with text so you should use them with a leading \; \tt, \sf, \sc, \em, \bf followed by Esc, give \texttt{•}, \textsf{•}, \textsc{•}, \emph{•} and \textbf{•} respectively.

### 15.3.5 Comments

It is easy to remember the arguments for commands that are used fairly often but forget them for those rarely used; these are the perfect candidates for comments. An example is the order of the arguments for the \rule command; type \rul and the Escape key to get

```
\rule{•<width>}{•<height>}
```

and an additional press of Escape gives

```
\rule[•<lift>]{•<width>}{•<height>}
```

the version with the optional argument. Another example is the wrapfigure environment, from the wrapfig package, which has multiple versions with differing numbers and positions of optional arguments. To see the variations with the comments type \bwr on an empty line and press Escape to get:

```
\begin{wrapfigure}{•<placement: r,R,l,L,i,I,o,O>}{•<width>}
•
\end{wrapfigure}•
```

with the versions with optional arguments on succeeding presses of Escape key.

### 15.3.6 Other Environments

Environments that aren't built into the CommandCompletion.txt file can always be added if you use them a lot but there is an alternative for occasional use. Built into the completion algorithm is a way to complete environments. First press \b and Esc to get \begin{, enter the environment name and the closing } and then Esc again; the closing \end{} with the corresponding environment name will be generated on a separate line.

### 15.3.7 When Typing \ is Difficult

Some keyboard localizations make it difficult to type \ directly; e.g., it takes multiple keystrokes to do so using the French keyboard localization. Hope isn't lost! In most cases an abbreviation or start of command doesn't have to start with a \ but rather any 'white space character' (i.e., the start of a fresh line, a space or tab). So instead of

```
\sec
```

and the trigger key to produce

```
\section{•}
```

you can use

```
sec
```

at the start of a line and the trigger key will produce the same completed command.

Similarly, writing `tt` and Escape will give `\texttt{•}` since it is preceded by a space character. However `` `tt `` will *not* work since the `tt` *isn't* preceded by a 'white space character'; in that case you will have to use `` `\tt ``. The simplest way to make that easier is to create a macro that does nothing by insert a \ and assign it to a simple `Cmd` based keystroke.

### 15.3.8 Editing the Command Completion File

The location ~/Library/TeXShop/CommandCompletion contains a file named CommandCompletion.txt. This is the actual list of command completion elements defining the system. TeXShop has a built-in Command Completion editor, so consulting the file CommandCompletion.txt is never necessary. To use the built-in editor, select the menu item Source/CommandCompletion/Edit Command Completion File.

This is not the place to explain how to add items in detail; consult the larger Schulz document for that. Studying and copying items in the existing file will be enough to get started. But some questions will arise immediately because the file contains a large number of • symbols.

Recall the menu Source/CommandCompletion/Marks. This menu has three commands

```
Next Mark
Previous Mark
Insert Mark
```

But if the Option key is pushed, the items change to

```
Next Mark (Del)
Previous Mark (Del)
Insert Mark
```

and if the Control key is pushed, the items change to

```
Next Mark
Previous Mark
Insert Comment
```

Each of these items has a keyboard shortcut. The Schulz document explains how these items can simplify entry of new command completion elements using the editor.

The creation of the existing Command Completion list was the joint work of several people. A few users have created their own versions of the file, and thus their own systematic ways to remember and name command abbreviations. Some of these are in ∼/Library/TeXShop/CommandCompletion.

# Chapter 16

# Automatic Saving; Finding Documents on Disk

## 16.1 Automatic Saving

Automatic Saving is a powerful feature introduced by Apple in Lion, macOS 10.7. TeXShop uses this feature, which is entirely implemented by code in Cocoa written by Apple. Only a single line of TeXShop source code mentions it.

With Auto Save, you can essentially forget about saving files. Files being edited will automatically be saved every few minutes. Only changes are saved, so there is minimal disk activity and saving is completely unnoticeable.

You do not need to save files before typesetting because this will happen automatically. If you quit a document or shut down the computer, all files will automatically be saved. If you copy a file, or drag the entire file to mail, or do similar operations, the file will automatically be saved and users will get the latest version.

Since the Macintosh is only saving changes, it has a complete record of all the versions of the document. Theoretically, you could go back to earlier versions and reconstruct the document as it was yesterday, or last month. As we will see in a moment, this isn't just theoretically possible — it is actually easy.

You might wonder if the file containing a document has all the old versions hidden away somewhere. Could I send someone the TeX source of a letter of recommendation which currently says "Paul is a creative student", but whose initial version said "Paul hasn't had a new idea in years"? Happily, no. Files on disk only contain the latest version.

Before AutoSave, files that had been edited were marked as "dirty" with a black circle

in the red close button at top left of the window. Now edited documents are marked by appending the word "- Edited" to the document's name in the window's title bar. The file menu still has a "Save" command. If you save in this way, the "- Edited" notice goes away. Apple sometimes calls this operation "Saving a version".

What happens if you decide to experiment, get confused, end up with a source document that makes no sense, and then discover it has been automatically saved out from under you, and the previous version of the file is gone? Since *undo* works across file saves, just undo the bad changes.

When I give talks about TeXShop at TeX User Group Conferences, users of Linux and Windows like to make fun of features of the Macintosh. So when I talked about Automatic Saving, I was sure the Linux users would warn about the danger of losing data. To my surprise, their actual comment was "we've had that for years!" A user has never complained to me about losing data due to automatic saving and I cannot imagine living without it.

Just in case, TeXShop has a way to turn AutoSave off. Starting with TeXShop 5.60, this is the item **Enable Autosave and Versions** under the Misc1 tab of TeXShop Settings; it takes effect after TeXShop restarts. In earlier versions it was the hidden preference AutoSaveEnabled. Turning AutoSave off is strongly discouraged. I never test TeXShop with AutoSave disabled.

When AutoSave is off, the item **Keep the previous file as "name~" when saving**, just below it, makes TeXShop keep the previous version of a file as a backup each time the file is saved. The backup has the name of the file followed by a tilde. This item replaces the hidden preference KeepBackup.

## 16.2 Lock

Often users have a folder of old papers which they haven't edited in a long time, but refer back to from time to time. If you open such a document and click the title, a dialog will open allowing you to "lock" the document. When a document is locked, it cannot be saved or changed. If a document is locked, clicking on the title and unlocking the document makes it possible to edit again.

## 16.3 Revert To

If you work on a document for several hours and then notice that a crucial paragraph or illustration is now gone, there is an easy way to recover. In the File menu, select the item "Revert to" and select the option "Browse All Versions." The Macintosh screen then switches to a Time Machine view showing the current document on the left and a receding stack of earlier views on the right, see Figure 16.1. You can leaf through these earlier versions, noting the date and time that each was created. All of these views are alive, so you can copy and paste between versions, or restore an earlier version. This works even if you have not activated Time Machine. So it is a sort of "personal Time Machine just for TeXShop".

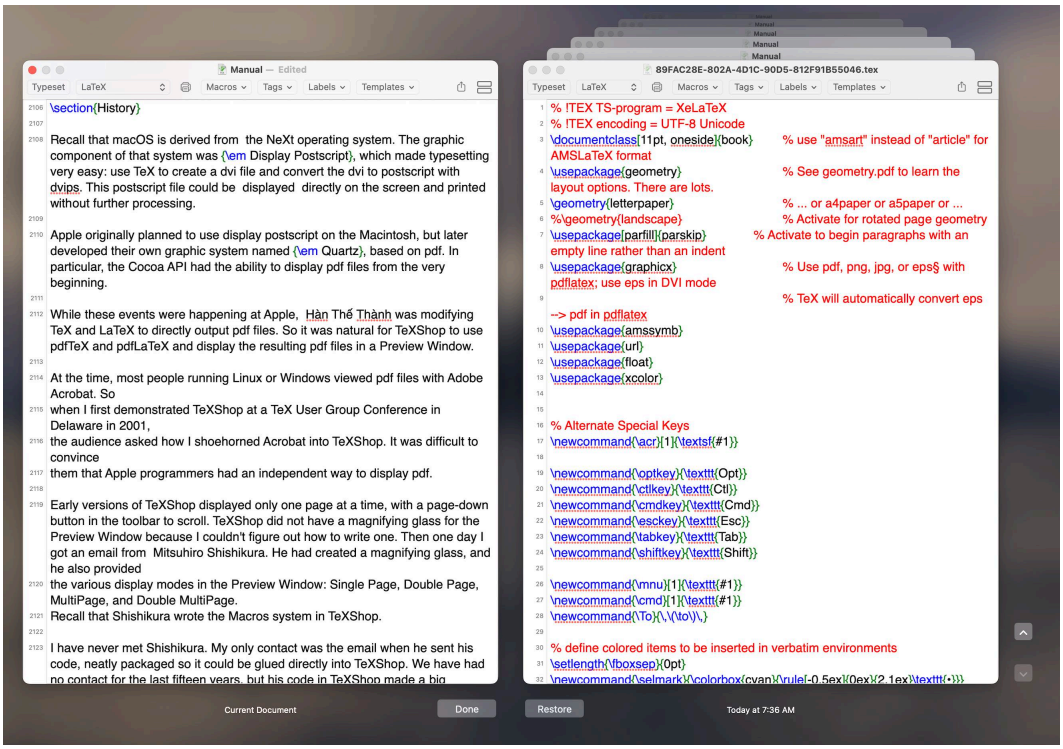



Figure 16.1: Revert To, a Document-Specific Time Machine View

## 16.4 Finding the Disk Version of a Document

In TeXShop and most other programs on the Mac, you can click the title of a window while holding down the command key and see a complete path to your document, listed as a series of entries in a drop down menu. By selecting an item in this menu, you can directly go to that spot of the file system.

# Chapter 17

# Preview Window

## 17.1 History

Recall that macOS is derived from the NeXt operating system. The graphic component of that system was *Display Postscript*, which made typesetting very easy. Use TeX to create a dvi file, convert the dvi file to postscript using *dvips*; display the postscript file.

Apple originally planned to use display postscript on the Macintosh, but later developed their own graphic system named *Quartz*, based on pdf. In particular, the Cocoa API had the ability to display pdf files from the very beginning.

While these events were happening at Apple, Hàn Thế Thành was modifying TeX and LaTeX to directly output pdf files. So it was natural for TeXShop to use pdfTeX and pdfLaTeX and display the resulting pdf files in a Preview Window.

At the time, most people running Linux or Windows viewed pdf files with Adobe Acrobat. So when I first demonstrated TeXShop at a TeX User Group Conference in Delaware in 2001, the audience asked how I shoehorned Acrobat into TeXShop. It was difficult to convince them that Apple programmers had an independent way to display pdf.

Early versions of TeXShop displayed only one page at a time, with a page-down button in the toolbar to scroll. TeXShop did not have a magnifying glass for the Preview Window because I couldn't figure out how to write one. Then one day I got an email from Mitsuhiro Shishikura. He had created a magnifying glass, and he also provided the various display modes in the Preview Window: Single Page, Double Page, MultiPage, and Double MultiPage. Recall that Shishikura wrote the Macros system in TeXShop.

I have never met Shishikura. My only contact was the email when he sent his code, neatly packaged so it could be glued directly into TeXShop. We have had no contact for the last

fifteen years, but his code in TeXShop made a big difference.

The Shishikura code was used until Apple introduced macOS 10.4, Tiger. In that system, Apple made available a framework named PDFKit, which worked more or less the same as Shishikura's, providing the same display modes and the like. I replaced his code with PDFKit because it is important for future developments to keep up-to-date with Apple technologies. The magnifying glass has gone through many iterations as version after version broke when Apple made API changes. Finally I talked directly to the author of PDFKit at WWDC and found a stable technique.

In the early days of macOS, strange bugs would sometimes surface in Apple's pdf display technology, but the Apple engineers were quick to fix these bugs. Very rarely these problems can surface even today. Users should be aware of a trick to help locate these bugs. Most pdf software on the Macintosh uses PDFKit, including Apple's Preview and Skim. However Adobe Acrobat Reader and Adobe Acrobat use Adobe's own software to display pdf. So a bug which occurs in TeXShop, Preview, and Skim, but not Adobe Acrobat Reader, is a bug in PDFKit. A bug which occurs in all four of these programs is a bug in the software which created the pdf file. And a bug which occurs only in TeXShop — perish the thought.

## 17.2 Display Modes

The Preview window can display a pdf as one long scrollable document; this is called MultiPage Mode, and is the recommended mode. It can also display two pages side by side in a long scrollable document. This is called Double MultiPage Mode. It can display one page at a time, called Single Page Mode, or two pages at a time, called Double Page Mode. Menu items in the Preview menu move to the Previous Page, or to the Next Page. Items in the Preview Toolbar also perform these tasks.

As usual, items in TeXShop Preferences under the Preview tab select the behavior to use when documents first start, but the mode for a particular document can later be changed using an item in the Preview menu.

The modes showing two pages side by side are controlled by a TeXShop Preference item under the Preview tab, which allows users to select "Pages Flow from Left to Right" or "Pages Flow from Right to Left". Select the first item for languages written from left to right and the second one for languages written from right to left, that is, Arabic, Hebrew, and Persian. By default, the first page is shown by itself, followed by pairs of double pages. This is called *Book Mode* by Apple because most books start with an odd number of preliminary pages, followed by the regularly numbered pages. Therefore the pairs of numbered pages correspond to the pairs of visible pages when a book is opened at random.

Starting with TeXShop 5.60, the item **Show first page alone as cover** under the Preview tab of TeXShop Settings selects whether the first page is shown by itself. Earlier versions

used the hidden preference DisplayAsBook for those very rare users who do not want that isolated starting page. The Settings items apply to documents opened later. In addition, two special comment lines are provided so the property can be set on a document by document basis. These overrule the Settings and turn book display mode on or off for that document:

```
% !TEX bookDisplay
% !TEX standardDisplay
```

In Japan, text can be written horizontally or vertically. When it is written vertically, pages should appear from right to left. Therefore we also provide special comment lines to set "displaysRTL" on a document by document basis:

```
% !TEX PageDirectionL2R
% !TEX PageDirectionR2L
```

If a document has no such comment line, TeXShop 5.60 and later follow the reading direction recorded in the pdf file, when the file records one.

TeXShop 5.60 also adds two menus that can be placed in the Preview window's toolbar. **Page Layout** selects the display mode of the current document, like the corresponding items in the Preview menu. **Two-Page Arrangement** selects, for the current document only, the direction in which pages flow and whether the first page is shown alone. When these are changed, the page being viewed and the magnification are kept. The arrangement lasts until the document is closed, or until **Restore Document Settings** is chosen in the Two-Page Arrangement menu, which returns to the Settings and the magic comment lines of the document. If the menus are not in the toolbar, use "Customize Toolbar..." to add them.

## 17.3 Magnification

The magnification of the Preview Window can be set to "Fit to Window", "Fixed Magnification", and "Actual Size". The recommended choice is "Fit to Window", which adjusts the magnification of the window so the text exactly fits side-to-side. On large monitors, it is convenient to adjust the window so it shows exactly one full page of output. On smaller monitors, adjust the window so it is wide enough to read easily; only a portion of each page will be shown, but scrolling easily reveals the remaining parts of the page.

The other choices are "Fixed Magnification" and "Actual Size". In a sense, these choices are the same. The first allows the user to select the amount of magnification, which is set in TeXShop Preferences under the Preview tab by an item named "Fixed Magnification Amount". This item can be reset for an individual window using the "Scale" item in the window toolbar. The "Actual Size" item behaves the same way, except that the

computer now selects the magnification amount based on the actual size of pages in the pdf document.

Note that "Fit to Window" and "Fixed Magnification" are orthogonal choices. If you select "Fit to Window" in TeXShop Preferences, then the "Fixed Magnification Amount" just under it in Preferences will be ignored, because the magnification is actually set by the size of the window. If you open a window in "Fit to Window" mode, and then modify the magnification with the "Scale" tool in the toolbar, the magnification mode will silently change to "Fixed Magnification Mode" so that resizing the window will no longer change the magnification. To restore "Fit to Window" mode for that window, reset the "Magnification" method in the Preview menu.

If you use "Fit to Window" mode, but want to temporarily magnify the image so a portion can be studied in detail. I recommend using an Apple gesture rather than the Scale tool. Click in the window, place two fingers on the track pad, and spread them to magnify or bring them together to shrink. When you shrink back to the original "fit to page" magnification, the display will snap back to this configuration. Note, however, that this gesture changes to "Fixed Magnification" mode and resizing the window will not change its magnification. To restore "Fit to Window" mode for that window, reset the "Magnification" method in the Preview menu.

A very small number of people use TeX to create intricate pdf diagrams which they then view in TeXShop for long periods of time at very high magnification. TeXShop is not really designed for that sort of work flow and if bugs occur, I recommend searching for other appropriate tools.

## 17.4 A Mysterious Preference Item

TeXShop Preference items for the Preview Window are mainly found under the Preview tab. This page also contains items for Single Page mode, and for the HTML Preview window. Single Page mode has already been discussed in this document, and the HTML Preview window will be covered later.

At the top of the second column of the Preview tab, there is an item labeled "Change Page Buttons" with a checkbox titled "Do not scroll". What does this item do? I haven't got the slightest idea.

At the start of this chapter, I explained that the original TeXShop used a preliminary version of Apple's PDF code to display pdf documents, together with extensions by Mitsuhiro Shishikura. Some of that old code is still in TeXShop, used to display graphic files and dvi files, as explained later in this document. But most of the code has been replaced by code which calls Apple's PDFKit. The old code is in a source file called MyPDFView, and

the new code is in a source file called MyPDFKitView, and for years and years I have only edited the MyPDFKitView file.

I looked at the code in Preferences which controls the mysterious "Change Page Buttons" item. It controls a global preference which is only consulted by the code in MyPDFView. Whatever this code did was not ported over to MyPDFKitView during the transition to PDFKit in Tiger. So you can ignore that preference setting.

## 17.5 Preview Mouse Tools

The Preview toolbar has an item determining the behavior of the mouse in the window. There are five choices:

- **Scroll**
  scroll the pdf document in directions allowed by the scroll bars
- **Select Text**
  select text in the pdf for copying and pasting into another text document
- **Magnify**
  a small magnifying glass
- **Magnify Large**
  a larger magnifying glass
- **Select Pdf**
  select a region of the pdf, for copying and pasting into another graphic program

An item in TeXShop Preferences under the Preview tab selects the default tool when the Preview window first opens.

The two magnifying glasses offer more options than first imagined. Click and hold down the mouse at a spot to magnify the region near that spot. Double click and hold down the mouse to magnify a larger region, and triple click and hold down the mouse to magnify the complete window. While the mouse is down, move it to magnify different regions of the window.

Hold down the command key, or option key, or control key to increase the magnification by various amounts.

Hold down the shift key and the command key, option key, or control key to decrease the magnification by various amounts.

The "select pdf" tool offers more options than first imagined. Select any portion of the pdf document. This can contain text, illustrations, or both. Copy using command-C and then paste into any document which accepts pdf data. For instance, open Preview and select "New from Clipboard" to open the selection in Preview.

Drag and drop also works with graphic selections. For instance, click and hold down the mouse inside the selection and then drag everything into Apple's Keynote to copy the information to a slide. Dragging the selection to the desktop creates a file containing the pdf information, with a default name. Rather than dragging and dropping the selection, select the item "Save Selection to File.." in the Preview menu to name the corresponding file and the location on disk where it is saved.

An item in the Misc1 tab of TeXShop Preferences determines the type of graphic data created for selections. By default, pdf data is created, but various types of tiff, jpeg, and png data are also possible. This choice can also made in the Preview menu for an individual selection using the "Copy Format" menu.

## 17.6 Back, Forward, Rotation

The toolbar has two arrows pointing up and down for scrolling by a page, and two arrows pointing left and right for moving through the history of a document. These left and right arrows act like the corresponding arrows in a browser. If you use the "Goto page" tool to go from page 3 to page 5, and then page 10, and then page 25, the left arrow will take you back to page 10, and then to page 5, and at that point the right arrow will take you back to page 10.

The Preview menu also has menu items to rotate individual pages clockwise or counter-clockwise.

## 17.7 The Drawer and Two Search Fields

The Preview Window has an associated Drawer, which can be toggled visible or invisible with an item in the window toolbar or the first item in the Preview menu. This drawer appears on the right side of the window if there is room there, and otherwise on the left side of the window. It's contents are divided into two pieces. The top pieces shows an outline of the document, listing chapters, sections, and subsections if available. Clicking on an item takes you to that point in the actual Preview Window. This top portion is blank if the pdf document does not contain an outline. Outlines are automatically created by the hyperref package in LaTeX. Here is a typical command to activate the hyperlinks package:

```
\usepackage[colorlinks=true, pdfstartview=FitV, linkcolor=blue,
  citecolor=blue, urlcolor=blue, hyperfigures=true]{hyperref}
```

Adding the extra item "bookmarksnumbered" to this call causes hyperlink to number the chapters in the outline. This is useful because TeXShop then activates keyboard shortcuts to select chapters in the drawer, as explained later. The revised call would be

```
\usepackage[colorlinks=true, pdfstartview=FitV, linkcolor=blue,
  citecolor=blue, urlcolor=blue, hyperfigures=true,
  bookmarksnumbered]{hyperref}
```

The bottom of the drawer is a search field. Type an entry in the search field and a list of appearances of the entry will be created below the entry. The page where the entry occurs,

and some of the surrounding text will be shown. Click on an entry to be taken to that spot.

The Preview Window has an alternate search tool in its toolbar. Enter an entry in this field and the first appearance of the entry will be selected. Type command-G to go to the next entry and shift-command-G to go to the previous entry. If the edit window is active, command-F will go to its Find Panel or Find field. If the Preview window is active, command-F will go to the search field in its toolbar.

## 17.8 Following Links and URLs

The package hyperref can create links from one portion of a document to another. These links are active in the Preview window, so a reference to a theorem can be followed by a link to the actual text of the theorem, a reference to the bibliography can be followed by a link to the actual item in the bibliography, and so forth. In addition, URL's are active, so clicking a URL will open the corresponding page in your default browser.

The Backward/Forward arrows in the Preview window toolbar work like similar arrows in Safari and other browsers. If you click on a link and jump to the linked page, the Backward arrow will take you back to the original page with the link, and the Forward arrow will take you again to the linked spot. These arrows maintain a "stack of locations", so if you follow several links, you can follow the chain backward to the original starting spot.

## 17.9 Popup Windows and Hovering Over Links

Hovering over a link will bring up a small view of the linked text. For instance, hovering over a section number in the table of contents will display the beginning of that section in the text. Normally the popup is on screen for four seconds and then disappears. This behavior can be changed with a modifier key, as explained below.

There is a menu item in the Preview menu called "Link Popups", suggested by Uwe Schmock. This menu is a toggle turning the hover behavior on or off. When it is on, the menu item is checked. The hover behavior is on when windows are first opened.

If you click in the contents of the Preview window while the Control key is down, a contextual menu opens. The "Link Popups" item is also in this contextual menu.

### 17.9.1 Modifying Popup Windows

The details of the small window created by hovering over a link can be changed by holding down any combination of three modifier keys before moving a mouse to the link. These modifying keys have been selected so their names give a mnemonic for the effect they create.

The SHIFT key shifts the small window to appear above the link, rather than below it. The COMMAND key commands that a bigger window with larger text be displayed. The OPTION key selects the optional behavior that the small window will remain on the screen until the mouse moves. (If a user forgets to push the option key, it can be pushed later; if it is down at the four second mark, the window will remain open.)

### 17.9.2 Too Much Magnification is Bad

One user liked to increase the default magnification significantly in the Preview Window, and in addition use the magnifying glass for extremely close inspection of fine details. This caused TeXShop to run out of memory and crash. TeXShop now protects itself by refusing to activate the magnifying glass if the default magnification is too large. The default limit is a preview magnification of 250. On your own risk you can change it with a hidden preference, see the chapter on Hidden Preferences below.

To understand the problem, it is useful to know how the magnifying glass works. When a user asks for the glass, TeXShop glues an extra view on top of the Preview window's content, but this view is transparent so the user does not notice that it is there. Then TeXShop creates an offscreen image of the entire active page being displayed on the screen. Next TeXShop reads the data from this offscreen image which is contained in a small rectangle centered on the mouse location, and draws this data to a larger rectangle in the transparent view. The result is a magnified view of the portion of the screen under the mouse; this magnified image is in the mostly transparent view rather than in the original Preview page.

If the user moves the mouse, TeXShop erases the transparent view and repeats the procedure with the new mouse location. The magnified image appears to move to a new location, but the uncovered data need not be redrawn.

If the Preview window has a large default magnification, then only a small portion of the window is visible on the screen. But the offscreen image of the active page will contain the full page in intricate detail, taking a gigantic amount of memory. Hence the out of memory bug.

# Chapter 18

# Printing

To print a TeX or LaTeX document, select "Print" in the File menu. A standard print dialog will appear, allowing you to print everything or selected items, and set several parameters depending on the capabilities of the printer. This can also be activated by typing command-P.

To print the TeX source for a document, select "Print Source" in the File menu. Users do not often perform this task, so it has no keyboard shortcut.

Bruno Voisin called my attention to new support in LaTeX for PDF files, added as of the June, 2022 release. He sent the following illuminating source file. The source creates a document with three pages. The first and third pages contain text formatted in the standard way, but the second page is in landscape mode showing a wide picture. If this output came from a book, all three pages would be standard pages and the user would rotate the book to look at the picture. But when the author is creating the pages on a computer screen, the second page should be rotated for easier viewing. This source file causes the second page to be appropriately rotated. If you want to try this example, supply your own graphic file.

```
% !TEX program = XeLaTeX
\DocumentMetadata{}
\documentclass[12pt]{article}
\usepackage{graphicx}
\usepackage[figuresright]{rotating}
\ExplSyntaxOn
\AddToHook{env/sidewaysfigure/end}{\pdfmanagement_add:nnn{ThisPage}{Rotate}{90}}
\ExplSyntaxOff
\begin{document}
```

```
Some text.
\begin{sidewaysfigure}
\centering
\includegraphics[width=\textheight]{YourPicture.jpg}
\caption{Then a nice rotated figure.}
\label{fig-stprof}
\end{sidewaysfigure}
\clearpage
And more text.
\end{document}
```

When TeXShop prints pdf files containing rotated pages, all pages are printed in portrait mode and information is lost on rotated pages. The solution to this problem is easy. Before printing, comment out the three lines starting with "ExplSyntaxOn" and typeset. Then no page needs to be rotated in the printed output.

One problem remains. Suppose you are creating a document which most users will read on the screen, but a few will print. Should pages with sideways-figures be rotated or not? This is a problem I cannot solve, since the fix must be in the software your readers use to display pdf, not the software you use to create it.

# Chapter 19

# The Console

A special tab in TeXShop Preferences contains preference items for the console. The font used by the console, the console position, and other items can be set there.

Many typesetting engines format output text to the console with lines of fixed length. A setting allows the console to be resized arbitrarily, or only vertically. Some users like to temporarily resize it arbitrarily and adjust the size to be slightly wider than this conventional fixed width. Then they reset the preference to only allow vertical resizing. This makes it impossible to accidentally change the width.

A final setting controls the placement of the console. One possibility is for it to appear in front of all other windows during and after typesetting. Another possibility is to bring it forward, typeset, and then push it behind the edit and source windows if there is no error.

When console output temporarily stops, as it does when the console reports an error, it is not possible to copy console text. But after typesetting ends, it is possible to copy sections of console text and paste it elsewhere, like an email document asking for help. It is not possible, of course, to edit text in the console.

Starting with TeXShop 5.60, the menu item **Show Console** has the keyboard shortcut `Cmd`-0. It brings forward the console of the active document. Pressing `Cmd`-0 again in the console returns to the source or preview window from which the console was called, or to the source window if there is no such window. This is similar to `Cmd`-1, which switches between the source and preview windows.

Console output and log files sometimes mix encodings, for instance when a document in a Japanese encoding uses packages written in UTF-8, or contain broken characters. TeXShop 5.60 reads such output line by line, trying UTF-8 first and then the encoding of

the document, with IsoLatin9 as a last resort, so that unreadable data in one line does not spoil the other lines. When TEX breaks a long line in the middle of a UTF-8 character, TeXShop joins the pieces again.

# Chapter 20

# HTML, Interactive TeX4ht, PreTeXt

## 20.1 Html Preview Window

Starting with TeXShop 5.00, a new preview window is available in TeXShop, able to display html source as a live web view. The window is provided because several TeX and LaTeX engines produce html output. For the moment, we ignore these engines and examine the window.

Select the item "Show HTML Window" in the Preview menu. A default web document will open with links to to several active web sites. Click the last link to see an illustration, and click the TUG link to confirm that the window can access live content. When the page first appears, it may be small and in an awkward location. Resize it and move it to a reasonable spot, perhaps where the Preview Window usually appears. Select it and choose the item "Save HTML Window Position" in the Preview menu. The window will then always open with this size and position.

In TeXShop Preferences under the Preview tab, the item "Home URL" allows you to set the web page that opens by default when the window is selected.

Although the window contains a full featured web view, the toolbar items are minimal. Our window only provides Back and Forward buttons, a field to enter a new URL, and a search field. The search field currently does nothing. You already have Safari and perhaps other browsers; TeXShop has no obligation to provide a duplicate.

When actually using the Html Window, you will almost never call "Show HTML Window." What is the purpose of the window?

## 20.2 Html Files

TeXShop has a large "Changes" document, available in the Help menu. "Changes" is clearly a pdf document because when it is resized, the text shrinks or enlarges rather than reflowing. However, this document is actually written in html, and that html version is available on the TeXShop web page as https://pages.uoregon.edu/koch/texshop/changes_3.html. Notice that this web document reflows when it is resized. To create the pdf form, I print the document and then ask that the resulting pdf be saved in a file.

Until recently, I wrote Changes source using TeXShop and opened the html file in Safari to check for mistakes. This was tedious; for one thing, Safari opened the document at the beginning and I would have to scroll down to find the section currently being written.

But now this and similar html files can be previewed directly in TeXShop. Check to see if **html.engine** is one of your active engines. If not, go to `~/Library/TeXShop/Engines/Inactive/html` and move a copy of the file html.engine to the active folder `~/Library/TeXShop/Engines`. Then find an arbitrary html file; I'll use Changes.html. Html files begin with a mandatory first line. Just below this line, add the comment shown below.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" \\
        "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<!--
% !TEX TS-program = html
-->
```

Since this addition is an html comment, it is ignored on the web. But TeXShop sees the line inside, which tells it to typeset using the html.engine. No actual typesetting is done by this engine; instead, it just opens the html file in the new Html Preview Window. On the left side of the screen we have the source in an edit window, and on the right side we have the resulting web view.

Type new content on the left and then type command-T. The new content immediately appears on the right and no scrolling is needed because the revised html opens scrolled to the correct spot. What a time saver!

If you create complicated web pages, use special software designed for that purpose. But for small tasks like the Changes document, TeXShop has become an adequate authoring tool.

## 20.3 Help File for Html Commands

I'm no html expert, so when writing Changes I often have to use Google to find appropriate commands. The TeXShop Help menu now has a document named **HTML Commands** to

make this easier. This opens a window with a list of tasks and appropriate html solutions. But it has got to be the ugliest help document ever, because it is written in html.

Notice, however, that the top of the document contains an instruction to typeset using the html.engine. You are supposed to open this source and then typeset to get a readable document. Try that now. The web version is easy to read and explains, among other things, how to embed images, movies, and UTube videos in a web page. In particular, if you right click on a UTube video in a browser like Safari, a contextual menu appears allowing you to copy code which can be pasted directly into an html document and thereby provide that video for viewers of your page. I found a lecture by John Maynard, one of the four Field's Prize winners at the International Congress of Mathematicians for 2022. It is fun to watch this video for the depth and clarity of his mathematics. Here is the code provided for this video, which is Greek to me:

```
title="13. Large gaps between primes in subsets - James Maynard
(University of Oxford) [2017]" frameborder="0"
allow="accelerometer; autoplay; clipboard-write;
encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>
```

When the html help document is edited in TeXShop; the changes will automatically be saved. So when you need a command not listed, look it up using Google and then add that example to the help file. The initial document provided in TeXShop doesn't list many commands because I don't know much html, but if you expand it, it will gradually become more useful.

The actual source for the help document is contained in a new directory `~/Library/TeXShop/HTML`, and the source is named "Help.html". This folder can also contain support files. It currently contains an image "topdot.jpg" used by Help.html and a few other support files, and you can add similar support files. When TeXShop is updated, Help.html and the various support files will not be touched, because you may have edited these items. But the HTML folder also contains a subfolder named "Inactive" and that folder will be updated. So more extensive help files may be provided in the future, which you can merge with your own additions to the default file. I encourage users to contribute better initial Help.html files for this Inactive folder.

## 20.4 TeX4ht

TeX4ht is a program written by Eitan M. Gurari. It uses a standard LaTeX source file as input, but outputs an html file instead of a pdf file. In 2009, Gurari was preparing a talk for the TeX User Group conference about an extension which outputs braille, when he suddenly died. His TeX4ht project has been taken over by other volunteers and is now being very actively developed by Michal Hoftich.

Check to see if **TeX4ht.engine** is one of the active engines. If not, go to `~/Library/TeXShop/Engines/Inactive/TeX4ht` and drag the engine TeX4ht.engine to the active folder `~/Library/TeXShop/Engines`. The TeX4ht folder also contains a subfolder titled Example. Make a copy of this entire folder and drag it to your home directory or another directory used for TeX projects. Inside this folder is a LaTeX file named Sample.tex. Open it in TeXShop. A Source Window and a pdf Preview Window will open. Look carefully at the source file and notice that the first line is a magic comment saying to typeset using the TeX4ht engine.

Typeset. Now alongside the pdf Preview Window there is an Html Preview Window. The pdf window displays the output when typeset by pdfLaTeX and the Html window displays the output when typeset by TeX4ht. This makes it easy to compare the results. Resize both windows. They behave differently. If you have a trackpad, resize both window contents with a finger gesture. Both resize. In particular, finger gestures can magnify the html image. Notice that the mathematical equations are clear and precise in both versions.

Each document contains an active link at the end. Click these links. The link in the PDF Preview Window causes Safari to open and display the new page. But the link in the HTML Preview Window opens the linked page directly in this window. The Back arrow can take you back to the original content.

Add extra material to the source and typeset again. Notice that both windows are immediately updated. Neither window jumps to the top of the document when reloaded. For HTML, this is a major advantage over the old method of opening the output in Safari and clicking the reload icon when changes are made.

Math4ht has many parameters controlling its operation. Originally it created a large number of small pictures for mathematical equations, and the web page displayed these pictures. It can also output MathML and many modern browsers can interprete these commands. An even nicer approach is to render mathematics using MathJax. This is the approach recommended by the American Mathematical Society and produces very acceptable typeset equations. Users will want to create their own engines to explore the advantages of these various approaches. The TeX4ht engine uses MathJax, but an alternate engine is also provided in Inactive/TeX4ht which uses MathML.

## 20.5 Historical Intermission

The material below, copied directly from the Changes document, tries to put this new feature in a historical context.

The TeX User Group (TUG) was founded in 1980, shortly after the appearance of TeX. It publishes a journal, TUGboat, three times a year and creates the distribution TeX Live,

which works on almost all computer platforms. TeX Live is created in collaboration with similar TeX User Groups in other countries. TUG holds an annual meeting each year, sometimes in the U.S. and sometimes abroad.

In 2001 after MacOS X was released, I was invited to speak at the national TUG conference in Delaware by Wendy McKay; TUG offered to pay my way. I knew almost nothing about TUG, but imagined a conference of 2,000 users. We would meet in a large auditorium and listen to talks on solving common typesetting issues. Perhaps 100 Mac people would be there, excited to hear anything about MacOS X. When I got to Delaware, I discovered that there were 45 people at the conference and almost every one was on the program. They paid their own way. I sheepishly refunded the travel money back to TUG. There were only three or four Mac folks, but they all became significant in my life.

Wendy worked for Jerrold Marsden at CalTech. Marsden was a leading authority in classical mechanics, and is the author of many well known mathematical textbooks. He used a Macintosh to write mathematics very rapidly, and invited TeX experts to CalTech to solve TeX problems. Among these was Ross Moore from Australia, who was also at the Delaware conference. He knows more deep facts about LaTeX than anyone I know. Hans Hagen, author of ConTeXt, was also at Delaware.

Delaware had a special honored guest, Hàn Thế Thành. His pdftex program added code to Knuth's TeX which output pdf files rather than dvi files. His project was absolutely crucial for TeXShop because otherwise I'd have to make a dvi viewer, far beyond my capabilities. Thành came from Vietnam and his pdftex work was part of his PhD thesis from Masaryk University, Brno, Czech Republic. He talked about that thesis at the conference, beginning with a detailed image of a page from the Gutenberg bible. This remarkably beautiful page had absolutely straight right margins. But when you measured with a ruler, they turned out not to be straight; instead commas and periods were pushed slightly into the margin, creating the illusion of a straight side. In his thesis, Thành added that ability to pdfTeX and pdfLaTeX.

I began my talk in Delaware by installing TeX onto a new MacOS X machine using Gerben Wierda's installer. In the process, the Finder crashed and I spent ten seconds restarting it. After the talk, a participant said to me "I'm not interested in your program, but it is amazing that you could restart the Finder without rebooting the machine."

I have been to many TUG conferences. In Hawaii, the organizers pulled us out to look at the sunset because Mercury was visible close to the horizon. A few days later they pulled us out at noon because on that one day at that one time, the Sun was directly overhead. In a recent conference at Palo Alto, I met my collaborator Yusuke Terada, from Japan, for the first time. Shortly after we met, I looked over his shoulder at another participant, and asked Terada "do you know who that is?" It was Donald Knuth.

Already in Delaware, there were talks about XML. The speakers directly typed XML code

during their talks, and the indentation grew and grew until it was two pages right of the original margin. It took me several years to understand the goal of these talks. XML is easily parsed by computers and LaTeX is not. Authors submit papers to publishers already formatted in a personal way, but the publishers need to reformat these articles to conform to the standard of a journal or book series. This can be done automatically by computers if the paper is in XML, but requires work by hand for articles in LaTeX. All well and good, but I stopped paying attention to XML talks.

There were also pessimistic talks by famous figures in the TeX world, predicting that TeX would be dead in another five years. Most of these talks were from England, and I gradually realized that many speakers had jobs in the Open University system. For these jobs, they didn't want to produce static articles, but rather interactive documents allowing students to input information and try experiments. Their students had portable machines and worked remotely. HTML web documents were much more useful than static LaTeX documents. Eventually I stopped listening to these talks as well. It is now twenty years later and LaTeX is very much alive.

In my little corner of the world, TeX has created a revolution. Our mathematics department used to employ three secretaries to type mathematical manuscripts. They are gone. Today if you go into a mathematician's office, there will be a computer running LaTeX and a chalk board. Research is done with these two devices and immediately typeset and sent to collaborators in Europe. All of our PhD students use TeX for quizzes and exams, and when we hired younger hot shots as Assistant Professors, they required graduate students to submit homework written in LaTeX. If research articles in some other field must be in Microsoft Word, that's their problem, not mine.

Then Covid hit. Luckily I'm retired, but I watched my mathematical colleagues switch to remote teaching with only a week's notice. I don't know how they did it. Gradually I realized that those pessimistic talks contained an important message.

Some of my colleagues at the University of Oregon switched to PreTeXt to write lecture notes; I'll describe that project in a moment, but suffice it to say that one goal of that project is to allow authors to write interactive material which can be accessed over the web. Authors write using XML. Perhaps I should have listened to those talks after all.

Here's my "dream setup for the future." A faculty member would write lecture notes in LaTeX. These notes would be typeset by an engine which outputs both pdf and html. The pdf document would contain the course in final polished form, so students could look back at the clear logical direction of the lectures. The html document would contain interactive material: movies, questions with student input, etc., so students would be actively involved, even if working remotely.

The HTML Preview window has been added to support such software, and to encourage new software with interactive material in mathematical documents.

## 20.6 TeX4ht and Interactive Documents

After the first version of TeXShop with an HTML Preview Window appeared, I communicated via Karl Berry with Michal Hoftich, and then directly with Michal. Recall he is the author of TeX4ht. Essentially I asked if "interactive" documents can be written using TeX4ht. That boiled down to the following concrete questions:

- Can TeX4ht source code be written which sends some material only to the pdf document, and other material only to the html document?
- Can the material for the html document be written directly in html?
- Writing mathematical equations in html is difficult. MathML is extremely clumsy if users have to write it by hand. If material for TeX4ht can be written in html, can the mathematical portion of this material be written in standard LaTeX?
- If TeX4ht can accept source written directly in html, can that source contain movies and UTube videos? Can it contain interactive Sage programs? Can it contain graphing programs where the user gets to enter the formula of the function to be graphed? Can the document contain three dimensional graphics which can be rotated directly by the user?

The amazing answer to these questions is yes, yes, yes, and yes.

There is no need to explain these answers in detail, because TeXShop contains a demonstration program which illustrates all of these features. Using a TeXShop menu item, go to `~/Library/TeXShop/New`. This directory contains a subdirectory named **Demo**. Drag a copy of this folder to your home directory or where you keep standard LaTeX document folders.

First read the document **Preparing-for-the-Demo** in this folder. Follow the simple instructions to make sure TeXShop is correctly configured for the demo.

Next open **Fourier.tex** in the **Fourier-for-TeXShop** folder. This document contains lecture notes on Fourier Series which I happened to have around the house. Typeset the document to convince yourself that it is absolutely standard LaTeX, illustrations and all. Leaf through the pdf to notice that there are many mathematical formulas.

The first line of this document tells TeXShop to typeset using pdflatex. Change the word **pdflatex** to **TeX4ht**. Typeset.

The result will be *two* Preview Windows, one for the pdf form of the document and one for the html form of the document. Notice first that the mathematical formulas in the html version are crisp and clear. Resize this window and notice that the text reflows. Resize the pdf window and notice that the magnification changes and the text does not reflow.

Thus standard Preview is showing a pdf document and Html Preview is showing an html document.

Skip to section 9 of the two documents. Notice that this section has a different name in the pdf and the html versions, and the contents are completely different.

Scroll down to the end of section 9 in the html document and notice that it contains some standard mathematical formulas, very clearly typeset. It will turn out that they are written in the html section of the document using standard LaTeX.

Scroll up to the subsection named **Calling Sage**. A few lines below is a command

```
plot(sin(x), (x, 0, 2*pi))
```

Push the Evaluate button to get a plot of this function. The plot was performed by the open source program Sage, acting from a remote server which provides free access.

The plot command for this function can be edited by the user. Change it to

```
plot(sin(x) + cos(2 * x), (x, 0, 2*pi))
```

and plot again.

Just below this example is a 3D plot. Evaluate that. Grab the graph and rotate it on the screen.

It is possible to run complete Sage programs. Look at **A Calculus Course Experiment** and push Evaluate. We obtain a demonstration of Riemann sums. Notice that many items can be changed by the reader: the function, the number of divisions, the method of computing the height, etc.

The Demo folder contains a file named **Interactive-TeX4ht-Documents**. Read this along with the Fourier demo to understand the details of how the sample was constructed, and thus the answers to my four questions for Michal Hoftich.

## 20.7 PreTeXt

At the 2014 TUG conference in Portland, Oregon, I heard a talk by Robert Beezer of the University of Puget Sound on Mathbook XML. A former PhD student of mine named Tom Judson was converting a textbook he had published into the new interactive XML format. I talked to Beezer, but the conversation soon switched to a description of how Beezer and Judson met, which was via very serious bicycling in France.

A couple of years later, I found myself in a group of alumni of the University of Oregon, wearing suits and talking about possible donations. As rapidly as possible, I retreated to the mathematics building where people wearing jeans talked about mathematics. One

of the faculty members there asked if I knew anything about PreTeXt, a way of writing interactive mathematics that he and other faculty were using. I didn't, but I looked it up on the internet. Turns out, Beezer's project had been renamed.

PreTeXt is a fascinating project. The central idea is that authors enter formatting commands using a special form of XML. But mathematics is entered in standard LaTeX, so writing documents is quite straightforward and not nearly as verbose as documents entirely in XML. The source is then translated into other formats using XSL. In this way, PDF, HTML, EPUB and other versions can be created automatically from the source. All of this is described very clearly on the web site `https://pretextbook.org`

In 2019, I spent a considerable amount of time revising TeXShop so it would be useful in this project. I added some new engines for PreTeXt, provided appropriate syntax coloring for XML documents, and made other changes. A description of all of this is in the folder `~/Library/TeXShop/Engines/Inactive/PreTeXt`. This folder contains several typesetting engines for PreTeXt, including an engine which shows the PreTeXt source in the Source Window, the pdf output in a Preview Window, and the HTML output in an HTML Preview Window. To get started, read the document **PreTeX with TeXShop** in this PreTeXt folder.

An enormous advantage of PreTeXt over similar projects is that Robert Beezer puts substantial effort into the interactive elements of the language. He organized a small group of authors who have published mathematical textbooks and are now converting these textbooks into interactive web documents. Their very practical demands have driven much of the recent work.

The PreTeXt project makes initial steps very easy. The folder `~/Library/TeXShop/Engines/Inactive/PreTeXt` contains a small shell script called updatemathbook.sh. Place this script in your Documents folder and run it in Terminal. It will download a folder named mathbook from Beezer's site. This contains all the software to use PreTeXt, together with many example folders. Since PreTeXt is an active project, updating is useful and easy. Just rename the old mathbook and then run the script again.

Inside mathbook is a folder named examples, which has many subfolders. Among these are sample-article and sample-book. The document **PreTeX with TeXShop** mentioned earlier carefully explains how to typeset these two projects in TeXShop, and provides appropriate typesetting engines for them. In particular, the book example uses portions of Tom Judson's Algebra Book, and thus is a real example rather than just a small taste.

## 20.8 Special Features of TeXShop for html, xml, and ptx Files

PreTeXt source files have extension .xml or .ptx. They are text files containing a series of tagged entries, just like html. TeXShop can open and edit html, xml, and ptx files, and when it does, it alters its behavior slightly, as indicated below.

### 20.8.1 Syntax Coloring

The old syntax coloring methods are used for LaTeX embedded in these files, but in addition five extra colors are used for the tags. These colors, and all other TeXShop colors, can be adjusted in TeXShop's Themes Preferences, to be described in a later chapter.

The "Comment" color is for xml comments and the "Tag" color is for xml tags. Some tags contain extra information; for instance, the minimal.xml file contains

```
<section xml:id="section-textual">
```

In this example, xml:id receives the "Attribute" color and section-textual receives the "Value" color. Finally, some rare commands have the form &lt and &amp and in these cases, lt and amp are given the "Escape" color. Here &lt represents < and &amp represents &.

### 20.8.2 The Tags Item in the Toolbar

In LaTeX mode, the Tags menu lists all

```
\chapter, \section, \subsection
```

commands in a source file, identified by associated text. Selecting an item takes the user to that location in the source. The Tags menu works the same way in xml mode, but this time it lists all

```
<chapter>, <section>, <subsection>
```

locations, with associated text. Indeed, the command recognizes eleven distinct tags. Since this is a large number, it can cause very long scrollable Tags menus. So under the Misc2 tab in TeXShop Preferences, an item lists all eleven possibilities and allows the user to select the items that create entries in the menu. It is possible to mark your own important spots in the source using the command

```
<!--!mytext-->
```

Here "mytext" can be replaced by any desired string. These locations will be listed as tags, with accompanying text. It is not necessary to remember the syntax; just select a

location in the source and choose the Macro "PreTeXt Personal Mark" to insert appropriate text.

### 20.8.3 Keyboard Tricks with Tags

Two new keystroke commands are available to aid in text entry. Here is the first. Suppose <hello> is an xml tag. Double click in the "hello" while holding down the option key to select all text between this tag and its associated </hello>. This action understands xml comments and will ignore text inside these comments. This technique also works in reverse; double click on "hello" in </hello> while holding down the option key and TeXShop will find the associated key <hello> and select all text between these tags.

The second keystroke command is called "Close Tag" and is initiated by a TeXShop menu item of the same name. The command has a keyboard shortcut option-command-period for easy entry. This command will search for an unclosed tag like <hello> and close it with </hello> at the cursor spot. The command understands xml comments and ignores text in such comments.

This command is often used by PreTeXt authors, who insert a new tag and then click on the appropriate closing spot and type option-command-period to insert its close. Close Tab works by searching backward until it finds an open tag that has not been closed. Note that "the rules of xml" limit the spots where commands can be legally closed. If a user tries to close a command at an illegal spot, a different closing tag will almost always be entered at that spot. When that happens, back up and think carefully about what you want to do.

For example, consider the text below, where <hello> has no associated closing tag. Notice that the initial <hello> can be completed immediately after it occurs, or after the ending </sage>, or after the following </p>, but not elsewhere. If you try to close it after A.rref(), Close Tag backs up and finds that <input> has not been closed, so it closes that rather than <hello>.

```
<section>
   <hello>
      <sage>
         <input>
         A = matrix(4, 5, srange(20))
         A.rref()
         </input>
         <output>
         [  1  0 -1 -2 -3]
         [  0  1  2  3  4]
         [  0  0  0  0  0]
         [  0  0  0  0  0]
         </output>
      </sage>

      <p>This is extra text.</p>

</section>
```

### 20.8.4 Command Completion

A new completion dictionary is provided for html, xml, and ptx files, listing phrases likely to occur in a PreTeXt document. When an xml file is opened, TeXShop will automatically switch to this dictionary, and the menu command "Edit Command Completion File" will open this dictionary.

Consult the document **PreTeX with TeXShop** in the directory `~/Library/TeXShop/Engines/Inactive/PreTeXt` for details about using and creating entries in this xml version of the command completion dictionary.

# Chapter 21

# ConTeXt

Recall that LaTeX is ordinary TeX used with the LaTeX macros written by Leslie Lamport. ConTeXt is TeX used with an entirely different set of macros written by Hans Hagen around 1990. Hagen works for an educational publishing company in the Netherlands, and that company needed to produce books with distinctive and colorful styles rather than the standardized documents produced by LaTeX. As Wikipedia wrote in their ConTeXt article,

> ConTeXt may be compared and contrasted with LaTeX, but the primary thrust of the two are rather distinct. ConTeXt from the ground up is a typography and typesetting system meant to provide users easy and consistent access to advanced typographical control - important for general-purpose typesetting tasks. The original vision of LaTeX is to insulate the user from typographical decisions - a useful approach for submitting e.g. articles for a scientific journal.

As ConTeXt developed, it faced limitations in the underlying TeX engines, and Hagen was also instrumental in a related project to create LuaTeX, a modern version of TeX with full support for Unicode, modern fonts, and precise control over typesetting. Today many LaTeX users typeset with LuaTeX rather than TeX or pdfTeX, and that engine is a standard part of TeX Live.

Nicola Vitacolonna is responsible for the next few developments, first calling some new features in ConTeXt to my attention and then helping me support them in TeXShop. Send thanks to him.

## 21.1 ConTeXt Garden

The binaries in the TeX Live distribution of TeX are updated once a year. All other style files, class files, and the like are updated daily. This approach is reasonable because the binaries are quite stable, while the auxiliary files are constantly being updated by thousands of contributors.

ConTeXt is a special case because its author, Hans Hagen, is constantly making improvements. So the copy of ConTeXt in TeX Live is often out of date. Luckily, ConTeXt users have access to a beautiful wiki called the ConTeXt Garden; see https://wiki.contextgarden.net/Main_Page. This page has a link titled "Install ConTeXt and start typesetting" which leads to a page with easy ways to install a complete ConTeXt system on MacOS, Windows, GNU/Linux, FreeBSD, and OpenBSD. The MacOS portion has binaries for both Intel and Arm. The distribution contains everything needed to typeset, including recent copies of luametatex, but it is quite small. It can be installed in a user's home directory. ConTeXt users typically use this distribution for ConTeXt projects and TeX Live for other TeX projects.

A recent version of TeXShop makes it easy for the two distributions to coexist. In TeXShop Preferences under the "Engine" tab, the "Path Settings" item at the top has a new item named "Alternative Distribution." Set this item to the path of your ConTeXt distribution, which depends on the location you picked to install it. Then add the magic line below to the top of your ConTeXt source files:

```
% !TEX useAlternatePath
```

These ConTeXt documents will be typeset by the ConTeXt Garden distribution, while ordinary LaTeX documents will continue to use TeX Live.

Other users may find this feature useful in special circumstances. The "Alternate Distribution" preference item can point to any TeX distribution and the magic line can be used in any source file. However, the magic line only affects typesetting "engines", and not the built-in calls to TeX, LaTeX, BibTeX, or Make Index. Recall that these engine files are just shell scripts. When such a script is called, it is called with three arguments:

- "$1": the full path to the source file
- "$2": an optional parameter determined by the magic line **!TEX parameter** in the source, or just a space if no such line is given
- "$3": the full path to the binaries in the distribution; this is usually /Library/TeX/texbin but points to the ConTeXt binaries if the line **!TEX useAlternatePath** is in the source

Existing engine files don't use "$3", so they need to be revised if you want to use an

alternate path with them.

Putting this all together, here is an engine file which calls the Magic Garden version of ConTeXt:

```
#!/bin/tcsh

setenv PATH "$3"\:$PATH
mtxrun --script context --autogenerate "$1"
```

## 21.2 SyncTeX in ConTeXt

SyncTeX is technology by Jerome Laurens; using it, a user can jump from a spot in the source file of a document to the corresponding spot in the output pdf file, and vice versa. Laurens provides code which generates a synctex file during typesetting. This code has been adopted by the authors of most current TEX typesetting engines. Laurens also provides C code for front end developers which can open this synctex file and provide the key information allowing them to sync from one window to another.

But Hans Hagen eventually rewrites everything for ConTeXt, and several years ago he replaced Laurens' code with his own version to generate the synctex file in ConTeXt. This was painful for me because Hagen used Lauren's original design of synctex and ignored a later modern version. So TeXShop had to include two versions of the C code for front end developers, one for all engines except ConTeXt and a second just for ConTeXt.

But recently Hagen also wrote code to interprete the synctex file for ConTeXt users. This is a welcome improvement, which has been in TeXShop since version 4.65. To use it, ConTeXt users must take two actions. First, near the top of the root source file of a project, they must include the line

```
% !TEX useConTeXtSyncParser
```

And second, users must add to the header section of the root ConTeXt source file the line

```
\setupsynctex[state=start,method=min]
```

These rules apply to all recent versions of ConTeXt.

## 21.3 2024 and Beyond

In 2024, ConTeXt comes in three forms. First there is the traditional ConTeXt gradually developed and extended over the years, and meant to be typeset using LuaTeX. Later the ConTeXt developers introduced a self-contained distribution typeset by a special program

named **luametatex**. Both the traditional and the new ConTeXt are in TeX Live 2024 and MacTeX-2024, and later. But since ConTeXt is still under active development, many users prefer the special distribution offered by ConTeXt Garden which can be installed in a user's home directory and is updated frequently.

The Inactive ConTeXt folder in TeXShop contains three engine files, one for each version of ConTeXt:

```
ConTeXt.engine
ConTeXt-TL.engine
ConTeXt-CG.engine
```

## 21.4 Typesetting the Traditional ConTeXt in TeX Live

Drag the engine **ConTeXt.engine** from the Inactive folder of engines in TeXShop to the active engine section `~/Library/TeXShop/Engines`. Use this engine to typeset. An easy way to do that is to include the following two magic lines at the top of the root source file for a project:

```
% !TEX TS-program = ConTeXt
% !TEX useConTeXtSyncParser
```

Add the line

```
\setupsynctex[state=start,method=min]
```

to the header of your ConTeXt source file.

## 21.5 Typesetting ConTeXt in TeX Live using luametatex

Drag the engine **ConTeXt-TL.engine** from the Inactive folder of engines in TeXShop to the active engine section `~/Library/TeXShop/Engines`. Use this engine to typeset. An easy way to do that is to include the following two magic lines at the top of the root source file for a project:

```
% !TEX TS-program = ConTeXt-TL
% !TEX useConTeXtSyncParser
```

Add the line

```
\setupsynctex[state=start,method=min]
```

to the header of your ConTeXt source file.

## 21.6 Installing the ConTeXt Garden Version

This version can be installed anywhere in your home directory. For simplicity, we will create a folder named "context" and install it in `~/context`.

Go to the Wiki, click the "Install ConTeXt" link at top right, find the MacOS box at top center, and click either the link **X86 64bits** or **ARM 64bits** to download a small folder to your downloads directory. Use the first link if you have an Intel processor and the second if you have an Arm processor. I have an arm processor and my downloads folder now has a folder named "context-osx-arm64". Drag whatever folder you got to `~/context` or the location you selected for ConTeXt.

Open Terminal in `/Applications/Utilities` and enter the following commands, revising the first line if you picked a different location for the installation:

```
cd ~/context/context-osx-arm64
sh install.sh
```

This will take some time as the script contacts ConTeXt Garden, and then downloads and installs ConTeXt. A progress report is printed in Terminal, ending with the line "The following settings were used."

To update if a new version of the distribution is made available, just repeat these instructions.

## 21.7 Typesetting ConTeXt from ConTeXt Garden

Drag the engine **ConTeXt-CG.engine** from the Inactive folder of engines in TeXShop to the active engine section `~/Library/TeXShop/Engines`. Use this engine to typeset. Before doing this for the first time, modify an item in TeXShop Preferences. Open Preferences, go to the Engine Tab, and change the "Alternate Path" item to read

```
~/context/context-osx-arm64/tex/texmf-osx-arm64/bin
```

The first word will change if you installed ConTeXt in a different location, and the remaining items will change in an obvious way if you used the Intel version of the installation rather than the Arm version.

Use the magic lines

```
% !TEX TS-program = ConTeXt-CG
% !TEX useAlternatePath
% !TEX useConTeXtSyncParser
```

The second line tells TeXShop to use the ConTeXt Garden distribution in your home directory rather than the standard TeX Live distribution. TeXShop knows the location of

the distribution in your home directory because it is determined by the "Alternate Path" preference item in TeXShop Preferences, which you set in the previous section. Include the line

```
\setupsynctex[state=start,method=min]
```

in the preamble to your ConTeXt source file.

# Chapter 22

# Typst

## 22.1 Introduction

TeX4ht and ConTeXT are examples of projects to extend TEX and LATEX is radical ways. LuaTEX and XeTEX are similar rewrites of the basic system. But there are a few even more radical projects, which rewrite both the input language used for typesetting and the code which typesets. An interesting example is Typst, a project by Martin Haug and Laurenz Mädje in Berlin. They created an entirely new typesetting program from scratch, written in the programming language Rust. The input source for Typst documents is also new, designed to simplify input for various common typesetting tasks. For information, follow the link https://typst.app.

An engine file by Jeroen Scheerder makes it possible for TeXShop to typeset these new input files using the Typst program, so TeXShop users can experiment with the new system. The folder ∼/Library/TeXShop/Engines/Inactive/Typst contains everything needed to get started. This folder has a file *AboutTypst* which explains how to download and install typst in /usr/local/bin; this turns out to be very easy. The folder also contains the engine "Typst.engine", which can be copied to the active engine directory. Finally several Typst source files are included to begin experimentation.

TeXShop has been extended so it can open and create ".typ" files, and it knows that these files can be typeset. Such source files can be added to ∼/Library/TeXShop/Templates, making them appear in the pull-down Templates menu in the Source toolbar.

Add the following line to the top of any Typst source file. This line tells TeXShop to typeset the file using the typst program. The symbol % does not indicate a comment in the Typst source language; instead comments start with //. That is why the line has two unexpected symbols at the start.

```
// % !TEX TS-program = Typst
```

If you start a new source file completely from scratch, TeXShop will display a Save Dialog the first time you typeset it. The file must be saved with extension ".typ" so Typst will recognize it. It is tempting to just type the new extension when you name the file, but that will not work because TeXShop will add an extra ".tex" to the end of the filename when saving. Instead, find the pulldown menu "File Format:" at the bottom of the dialog and select "typ"; it is near the bottom in the menu. Once the file has been saved with the proper extension, TeXShop will use that extension from then on.

## 22.2 Packages and Templates

*Packages* are the rough equivalent of sty and class files in the LaTeX world; a Typst source document can input package files to extend the capabilities of the language. An elaborate package system for Typst is under construction currently. See https://github.com/typst/packages#local-packages for details.

In particular, the Typst world has an equivalent of ~/Library/texmf where users can store their own packages and experimental packages from others not yet published for general use. On the Macintosh, this location is ~/Library/Application Support/typst/packages. This location takes precedence over others, just as ~/Library/texmf is searched first in LaTeX.

An earlier version of TeXShop added packages to this location so users could try out sample Typst documents from the Typst web site. However, this is no longer necessary because Typst now automatically downloads and caches packages when a source file includes them.

There is an easy way to try sample templates created by the Typst project. Go to https://typst.app/universe/search/?kind=packages. In the column at top left of the resulting page, select "Templates". The right side of the page then shows icons for around 390 sample templates..

For example, let us try the tufte-memo. Click the icon for *tufte-memo* and notice that the web view switches to a page about this document. On the right side there is a line in black containing the text

```
typst init @preview/tufte-memo:0.1.2
```

Open Terminal in Applications/Utilities and type the following commands:

```
cd
mkdir typst-docs
cd typst-docs
typst init @preview/tufte-memo:0.1.2
```

The first three lines create a folder in your home directory for typst projects; feel free to revise this portion. The last line is the magic line from the Typst web page which creates a folder named *tufte-memo* in the typst-docs directory and fills this folder with all files needed to run the project. The final part of this line, ":0.1.2", lists the version of the extra package which the document uses. This portion can be omitted and then Typst will download the latest version of this package.

Open the folder *tufte-memo*. Inside you will find several files, include *main.typ* and possibly *main.pdf*. These titles can be changed if you prefer a more precise name for the document. Open *main.typ* in TeXShop. At the very top, add the line

```
// % !TEX TS-program = Typst
```

Now typeset. Done.

## 22.3 Other Samples

The Typst templates web page has several other samples that are particularly recommended. These include *unequivocal-asm, charged-ieee, wonderous-book, dashing-dept-news, ilm, touying-simpl-hkustgv, letter-pro,* and *fireside*.

# Chapter 23

# Using an External Editor

## 23.1 Basics

TeXShop can be configured for an external editor in several ways. Some features below will be useful and others not depending on how your editor works. In the future, your editor may come with configuration instructions.

There is a menu item named "Open for Preview..." When this item is chosen, you will be asked to select a .tex source file. But only the associated pdf preview window opens. If the source has not yet been typeset or if the pdf is out of date, TeXShop will typeset the file as it is opened. You can open your source file in any editor. When it is time to typeset, save the changes in your editor, switch to the preview window, and typeset. In this mode, TeXShop never opens or modifies the source file. It simply passes this file to the Unix TeX or LaTeX process.

The preview window can be configured to contain a typesetting button. If such a button is not there, use "Customize Toolbar..." in the Windows menu to add one.

For users who prefer external editors most of the time, there is a preference item called "Configure for External Editor." When this preference is chosen, the "Open" and "Open Recent..." menus open tex source files in the above manner for editing with an external editor. Moreover, the "Open for Preview..." menu becomes "Open for Editing..." and opens the source file in TeXShop's internal editor for those rare occasions when the internal editor is desired. The "Open" and "Open Recent..." menus can also open jpg, tiff, ps, pdf, dvi, log, and other files; selecting the new preference does not change this behavior.

Some editors are able to call Unix typesetting commands directly. A new preference item, "Automatic Preview Update", has been added to improve your experience with these editors. When this item is active and a .tex file is opened using "Open for Preview", the pdf

preview display automatically updates whenever the pdf file is rewritten. Thus you can typeset with an external editor and the preview window will automatically show changes. The preference also applies to .pdf files opened in TeXShop, and if TeXShop is configured to use an external editor, the preference applies to .tex files opened with "Open".

Starting with TeXShop 5.60, the "Automatic Preview Update" preference also applies to source files opened in TeXShop's own editor: when the pdf file is rewritten by another application, for instance by latexmk running in Terminal, the preview is reloaded, keeping the page being viewed and the magnification. If the preview contains annotations which have not been saved, it is not reloaded, so the annotations are not lost. If the other application is still writing the pdf file, TeXShop waits and tries again.

TeXShop also watches source files open in its editor. When a source file is changed by another editor, TeXShop reloads it, and keeps watching it even when the other editor saves by replacing the file. If the source in TeXShop has unsaved changes, TeXShop does not reload it, so these changes are not lost.

The following AppleScript commands have been added to TeXShop so editors can call TeXShop directly:

```
typesetinteractive
texinteractive
latexinteractive
contextinteractive
bibtexinteractive
makeindexinteractive
metapostinteractive
taskdone
refreshpdf
open_for_externaleditor
```

The first seven commands activate TeXShop typesetting commands. AppleScript will return immediately after these calls without waiting for typesetting to complete. The "taskdone" call can be used to test completion; it returns NO while typesetting is being done, and YES when it is finished. Each of these calls refreshes the preview window at the end of typesetting. If your editor calls TeXShop to typeset, the "Automatic Preview Update" preference can be turned off. If the external editor modifies the pdf output directly, it can call "refreshpdf" to refresh the pdf display. This is only needed if "Automatic Preview Update" is off. Finally "open_for_externaleditor" opens a tex file by calling "Open for Preview".

## 23.2 Preliminary Version: Using Sync with an External Editor

Jesús Soto uses TeXShop with an external editor, TextMate, and requested that SyncTeX in TeXShop be given the ability to sync from the Preview window to the corresponding source in TextMate. I absolutely, positively, refused to honor this request. But I couldn't help thinking about it. In the end I relented.

TextMate installs a command line program in /usr/local/bin which is able to control some of the features of the editor. For instance, the command

```
/usr/local/bin/mate --line 50 /Users/koch/Syllabus.tex
```

opens Syllabus.tex in TextMate (if not already open) and highlights line number 50. This makes it easy to support syncTeX from TeXShop to TextMate because the standard TeXShop routine to implement syncTeX can still be used until the last moment when a line in the Source Window would be highlighted; this final step can be replaced by a call to "mate".

Therefore TeXShop can sync from the Preview window to TextMate in the standard way: click on a spot in the pdf display while holding down the command key. As a bonus, Goto Error works from the Console window if the user is using TeXShop to typeset. Click this button to be taken to the first error, and click again and again to cycle through the first few errors. Of course TeXShop must be in "external editor" mode to use these features; It is also necessary to activate the features with a hidden preference:

```
defaults write TeXShop TextMateSync YES
```

If you use a different external editor and want to implement the feature, you must first turn it on using a hidden preference:

```
defaults write TeXShop OtherEditorSync YES
```

This default deactivates the TextMateSync preference mentioned earlier. Instead whenever TeXShop wants to "sync from pdf to editor" or "Goto Error", it calls a shell script in /usr/local/bin named "othereditor". When it calls this script, it sends two parameters to the call: $1 contains the line number (as a string), and $2 contains a full path to the desired tex source file. The othereditor script then calls the external editor using its version of the "mate" program.

But there is no "othereditor" script in /usr/local/bin. The user needs to create this script. The script must be named "othereditor" with no extension, and have its execute bit turned on. The script should call whatever binary is used by the desired editor to open the file $2 if it is not already open, and jump to line $1.

For example here is the complete "othereditor" script file for TextMate:

```
#!/bin/tcsh
set path = ($path /Library/TeX/texbin /usr/texbin /usr/local/bin)
/usr/local/bin/mate --line $1 $2
```

A final word of warning. Since this code is preliminary, I haven't worried about paths or file names containing spaces. Avoid these spaces for now.

## 23.3 More on External Sync

In the previous section, we activated "Goto Error" and "Sync from Preview to Editor" for TextMate. We did that by calling an external script, which had to be located in /usr/local/bin, had to have its execute bit set, and had to be named "othereditor". This script received two inputs, $1 and $2. The first variable contained a line number of the source window and the second variable contained a full path to the file to be displayed by the editor. When called in this way, the script should cause the editor to open the source file if it is not already open, scroll to the line number, and mark that line in some fashion. Activating this feature also required that TeXShop run in external editor mode, and that the hidden preference item "OtherEditorSync" be set to YES.

The actual script would be written by the authors of the external editor, because only they know how to ask their editor to perform this action. The script might contains an AppleScript command, or perhaps activate a third party program which communicates with the external editor. In a couple of cases, the script is already known because these editors communicate with other front ends. Below are scripts for TextMate, and for BBEdit. Thanks to Max Horn for these.

TextMate:

```
#!/bin/sh
/usr/local/bin/mate --line "$1" "$2"
```

BBEdit:

```
 #!/bin/sh
/usr/local/bin/bbedit "$2:$1"
```

Both of these scripts ultimately call binary programs in /usr/local/bin provided by the editors: mate for TextMate and bbedit for BBEdit. So they require that the editors install their associated binaries. In TextMate, for instance, there is a preference item which causes TextMate to install the mate program.

TeXShop 4.25 also allows syncing the other direction, from the editor to TeXShop. This time the editor must call TeXShop, giving it three variables, $1, $2, and $3. The first is the

line number in the source file which the user clicked. The second is the character position in the line where the user clicked. Both are strings representing integers, so values might be 625 and 35. The final variable is a full path to the source file being displayed by the editor. This could be `/Users/koch/deRham/Chapter2.tex`.

For this to work, TeXShop must be in External Editor mode and the hidden preference OtherEditorSync must be set to YES.

Notice that TeXShop will then open the synctex file and do all processing to finish syncing. Ultimately it will scroll to the corresponding spot in the pdf Preview Window, and mark that spot.

The Editor can call TeXShop to perform this operation in one of two ways. First, there is a shell script which makes the call. Here is that script. Copy this text, say in TeXShop, and save it to a file named "ExternalSync" with no extension, and with its execute bit set. I recommend putting the file in /usr/local/bin. The location and name of the file are not important, provided the editor knows which name and location the user picked.

```
 #!/bin/bash

MyShellVar=$1
MyShellVas=$2
MyShellVat=$3

osascript  &lt&ltEOD

tell application "TeXShop"

activate
set the front_document to the front document
set MyAppVar to $MyShellVar
set MyAppVas to $MyShellVas
set MyAppVat to "$MyShellVat"

tell front_document
 sync_preview_line theLine MyAppVar
 sync_preview_index theIndex MyAppVas
 sync_preview_name theName MyAppVat
 return 0
end tell

end tell
EOD
```

Notice that this shell script receives three variables, and passes them on to an AppleScript which then runs and connects with TeXShop. Some editors may be able to run AppleScript commands directly. They can ignore the shell script and copy and run the AppleScript command.

## 23.4 Irrelevant Comments

The following discussion is a sign of my ignorance rather than a sign of my knowledge. You should skip it. I keep it in case someone wants to enlighten me.

When I was an undergraduate, my teacher claimed that Niels Bohr would say after a conversation "We clearly don't understand this theory; let's write a paper about it." In that spirit, let me write about communication between independent programs on the Mac. In Unix, each program runs as a separate process with an independent address space, so direct communication is not easy and usually involves the Unix kernel.

One form of communication occurs when a program starts another program. It can then provide that second program with an unlimited number of parameters. GUI programs on the Mac are actually standard Unix programs in disguise, so they can be opened from the Terminal and additional parameters can be prescribed at that time. TeXShop doesn't use that ability, but it certainly calls programs like TeX, MakeIndex, XeTeX, etc., providing each with additional parameters. Some editors can also typeset; they call other programs in this way.

Shell scripts are a special case of this. When such a script first starts, it can retrieve the parameters used to call it, as $1, $2, $3, and so forth. Cocoa has a special class called NSTask for starting other programs, so communicating when new programs start is easy to implement on the Mac, and in particular communicating by starting shell scripts is easy.

AppleScript is another form of communication between independent programs. Indeed, a major use of AppleScript is to make programs "scriptable", so they can be controlled by other pieces of code. Deep down inside, AppleScript implements special methods to make this communication between independent tasks possible.

There are, however, two problems with AppleScript. The first is that it can be insecure, and Apple has started to address this problem by allowing programs to refuse to accept AppleScript communication. It is reasonable to fear future additional restrictions. The second problem with AppleScript may be my fault. I don't grok the language, and it doesn't grok me. So I waste hours and hours trying to make AppleScript do something trivial, like pass more than one variable in a procedure.

Notice that TextMate and BBEdit have another method of communicating with external

programs. Each has an independent program in /usr/local/bin, which can ask the main program to perform certain tasks. I looked at the TextMate code. It called low level Unix socket commands; I admired it in the same way that I admired a 21 year old pianist who performed Liszt's Hungarian Rhapsody Number 6: fantastic, but something I could never do.

I once went to a NeXT developer conference, maybe the only one they held. At that conference, NeXT introduced "Distributed Objects", an easy way for independent programs to communicate on a NeXT. First, special "Connection Objects" established a connection between two programs. After that, one computer could run objects on the second machine and receive results back. The two programs could be running on the same machine, or different machines on a single ethernet network, or machines half way across the world. In the conference, NeXT showed slides containing the code necessary to establish such a connection. Each slide contained eight or ten lines of code.

For a time, distributed objects were also in Cocoa. I looked up the code recently. Every single routine in the API was deprecated.

However, there is a replacement, called XPC Services. I looked briefly at the code. It may be that this is the correct way to provide interconnection, particularly if AppleTalk has further restrictions. But someone else will have to take the time to decipher how it works.

## 23.5 More Recent Interaction Methods

Nicola Vitacolonna is responsible for the remaining features for external editors, to be explained below.

To support users who prefer an external editor, a new folder has been added to `~/Library/TeXShop` named **ExternalEditorScripts**. This folder will be updated with each new version of TeXShop, just as `~/Library/TeXShop/Engines/Inactive` is updated now. The new folder contains scripts contributed by users for various editors, each in a folder with the editor's name. TeXShop contains scripts by Michael Sharpe for BBEdit, and scripts by Nicola Vitacolonna for MacVim.

TeXShop contains additional commands making it easy to interact with external editors. These commands are documented below, but the text is quite technical. In practice, this information can be used to write scripts which automate the interaction, and these scripts will then end up in the ExternalEditorScripts folder. In the end, users can interact with TeXShop without understanding the details below.

If an external editor is used for ConTeXt work, TeXShop does not open the source file and thus cannot see the magic lines introduced in the previous chapter. So a method is

needed for external editors to pass this type of information to TeXShop. We provide new AppleScript commands for this purpose.

Eight new AppleScript commands have been added to TeXShop:

```
StandardBinPath
AlternateBinPath
SyncRegular
SyncConTeXt
SyncWithNoEditor
SyncWithTextMate
SyncWithOtherEditor
SyncRootName
```

These commands return nothing and have no parameters. The commands apply to the front window in TeXShop. The commands are only needed when an external editor is used; otherwise use the magic lines explained earlier.

The first line says that typesetting commands are in the standard location, `~/Library/TeX/texbin`. The second command says that the Alternate Binary Path should be used to find typesetting commands. By default, the first statement is active.

The third command says that sync commands for the active window should use Jerome Laurens' code; the fourth command says that sync commands for the active window should use the new ConTeXt sync. By default, SyncRegular is active.

Earlier, we introduced two two hidden Preference items]:

```
defaults write TeXShop TextMateSync YES
defaults write TeXShop OtherEditorSync YES
```

These items still work, but are no longer recommended. Instead, use the next three AppleScript commands. The command "SyncWithNoEditor" tells TeXShop that the editor does not communicate at all. The command "SyncWithTextMate" says that TextMate is the external editor and TeXShop should use the commands built into this program. The command "SyncWithOtherEditor" says that some other editor is being used and TeXShop should call a shell script to communicate with that editor, as described below and also in the previous sections.

The "SyncRootName" command is rarely used and will be explained later.

To clarify the use of these commands, the following AppleScript command tells TeXShop to typeset using the version of ConTeXt installed in the alternate location, to sync using the new ConTeXt methods, and to interact using the "other editors" method:

```
tell application "TeXShop"
```

```
        set the front_document to the front document

        tell front_document
             AlternateBinPath
             SyncConTeXt
             SyncWithOtherEditor
             return 0
        end tell
    end tell
```

This script can be made into a TeXShop AppleScript macro. Recall that TeXShop has a menu item to call Macros, so the Macro can be called even if TeXShop is in external editor mode. Call it just once when you work on a ConTeXt source, and after that the TeXShop typeset command will use the alternate ConTeXt distribution, will sync using the new ConTeXt sync methods, and will communicate using the "other editors" protocol.

If your editor supports AppleScript and allows users to add additional commands, you can build this command into the editor rather than creating a macro. If your editor does not support AppleScript but can call shellscripts, you can easily embed the AppleScript in a shell script:

```
#!/bin/bash

osascript  &lt&ltEOD

tell application "TeXShop"
   set the front_document to the front document
   tell front_document
      AlternateBinPath
      SyncConTeXt
      SyncWithOtherEditor
      return 0
   end tell
end tell
EOD
```

From now on, we concentrate on the actual communication between TeXShop and the external editor.

Your editor may already support sync from preview to source. When TeXShop syncs in that direction, it finds the location of the click in the preview window and processes this information using either Jerome Lauren's original sync code or the new ConTeXt Sync code. It ultimately produces two pieces of information, the line number of the corresponding

text in the source file, and a full path to the source file. This full path is required when source documents include other files, so the correct file will be opened and displayed. The remaining issue is communicating those two parameters, "$1" = line number and "$2" = source path, to the editor, so it will open "$2" if it is not already open and then select line "$1".

As it happens, TextMate contains a small file named "mate" which it can install in /usr/local/bin. If mate is called with two string parameters "$1" and "$2", it opens the file with path "$2" if it is not already open, and then selects line "$1". If the command SyncWithTextMate has been called, then TeXShop will call this "mate" program and configuration is complete. Incidentally, this call is also used when a TeXShop user selects "Go To Error".

If a different external editor is being used and the command SyncWithOtherEditor has been called, then TeXShop behaves in more or less the same way. Instead of calling "mate" in /usr/local/bin, it calls a binary named "othereditor" in /usr/local/bin. It calls this with the same two parameters "$1" and "$2".

The catch is that someone else has to write this "othereditor" file, which must have its execute bit set. However, it may be trivial to write this file for widely used editors, provided they have provided the necessary hooks. Below are two such scripts, one for TextMate and one for BBEdit. The TextMate script is provided to show that SyncWithOtherEditor also works for it; the BBEdit script shows that another famous editor supports the required communication.

For TextMate:

```
#!/bin/sh
/usr/local/bin/mate --line "$1" "$2"
```

For BBEdit:

```
#!/bin/sh
/usr/local/bin/bbedit "$2:$1"
```

If you use some other editor and discover an easy way to write an "othereditor" file for it, please communicate that discovery to the TeX on OS X mailing list or in some other way.

Finally, we must implement sync the other way from source file to preview. In that case, it is up to the editor to recognize a click in the source while the command key is down, and to supply TeXShop with three pieces of information: "$1" = the line number clicked in the source file, "$2" = the character index of the click in that line, and "$3" a full path to the source file. If the source file clicked was just the original file which TeXShop opened in external editor mode, then it already knows "$3". But if the user's projects involve

included or input files, then the third parameter is essential to tell TeXShop where to find the sync information.

The author of your editor must supply the code to convert a click into these three pieces of information. The editor should then call TeXShop, giving it the three parameters “$1”, “$2”, “$3”. TeXShop expects the information to come as an AppleScript command. Below is that command. Of course the editor has to supply the strings “linenumber”, “indexnumber”, and “full path to source file”:

```
tell application "TeXShop"
     activate
     set the front_document to the front document

     tell front_document
          sync_preview_line theLine "linenumber"
          sync_preview_line theIndex "indexnumber"
          sync_preview_line theName "full path to source file"
          return 0
     end tell
end tell
```

If your editor prefers to call a shell script, it is easy to embed the AppleScript in such a script:

```
#!/bin/bash

MyShellVar=$1
MyShellVas=$2
MyShellVat=$3

osascript  &lt&ltEOD

tell application "TeXShop"

   activate
   set the front_document to the front document
   set MyAppVar to $MyShellVar
   set MyAppVas to $MyShellVas
   set MyAppVat to "$MyShellVat"

   tell front_document
      sync_preview_line theLine MyAppVar
      sync_preview_index theIndex MyAppVas
      sync_preview_name theName MyAppVat
      return 0
   end tell

end tell
EOD
```

]Note that before using the above call, you must tell TeXShop the information it cannot

find from magic lines:

- The editor, either SyncWithTextMate or SyncWithOtherEditor
- The sync method, either SyncRegular or SyncConTeXt
- The binary location, either StandardBinPath or AlternateBinPath

Before asking the author of your editor to write the required code for syncing from editor to preview window, you may wish to test that it works. To simplify your life, you might suppose that all of the source code is in the main file. In that case, it isn't necessary to pass the full path to the source file, and so the last line of the AppleScript command which reads

```
sync\_preview\_name theName "full path to source file"
```

can be replaced with an AppleScript command with no parameter:

```
SyncRootName
```

# Chapter 24

# Themes

TeXShop uses customizable colors for syntax coloring, for various window backgrounds, and for other purposes. In the original version of TeXShop, users could modify these colors with a fairly baroque set of preference items. Then in macOS Mojave, Apple introduced "Lite Mode" and "Dark Mode" and an entirely new set of colors was required. At that point the old color preferences were ripped out of TeXShop and replaced by the Themes tab in Preferences, see Figure 24.1. While previously many colors could only be changed using obscure hidden Preference settings, now all color choices are available in the Themes tab.

The Themes portion of Preferences is shown on the next page. On the right are all colors currently set by TeXShop. Some items have an obvious meaning and others will be explained shortly. A full set of such choices is called a "Theme". TeXShop allows users to create as many themes as they like. These themes are listed in three pulldown menus on the left: Lite Mode Theme, Dark Mode Them, Theme to Edit. The first menu sets the theme used on all systems below Mojave, and the theme used in Lite Mode on Mojave and above. The second menu sets the theme used in Dark Mode on Mojave and above. The final menu sets the theme which Preferences is currently editing.

TeXShop is shipped with several themes, including "LiteTheme" and "DarkTheme". These are the default themes for Lite Mode and Dark Mode. As explained later, there is a way for users to rename or remove Themes known to TeXShop. But TeXShop will always replace "LiteTheme" and "DarkTheme" and use them if other required themes are missing.

Gary Gray contributed GLG-Dark for Dark mode. Gray then tweaked this theme, and ended up with a dark theme that was so my better than mine that I ended up using it as the default and renaming it DarkTheme. So Gray lost credit, but gained users. Thanks.

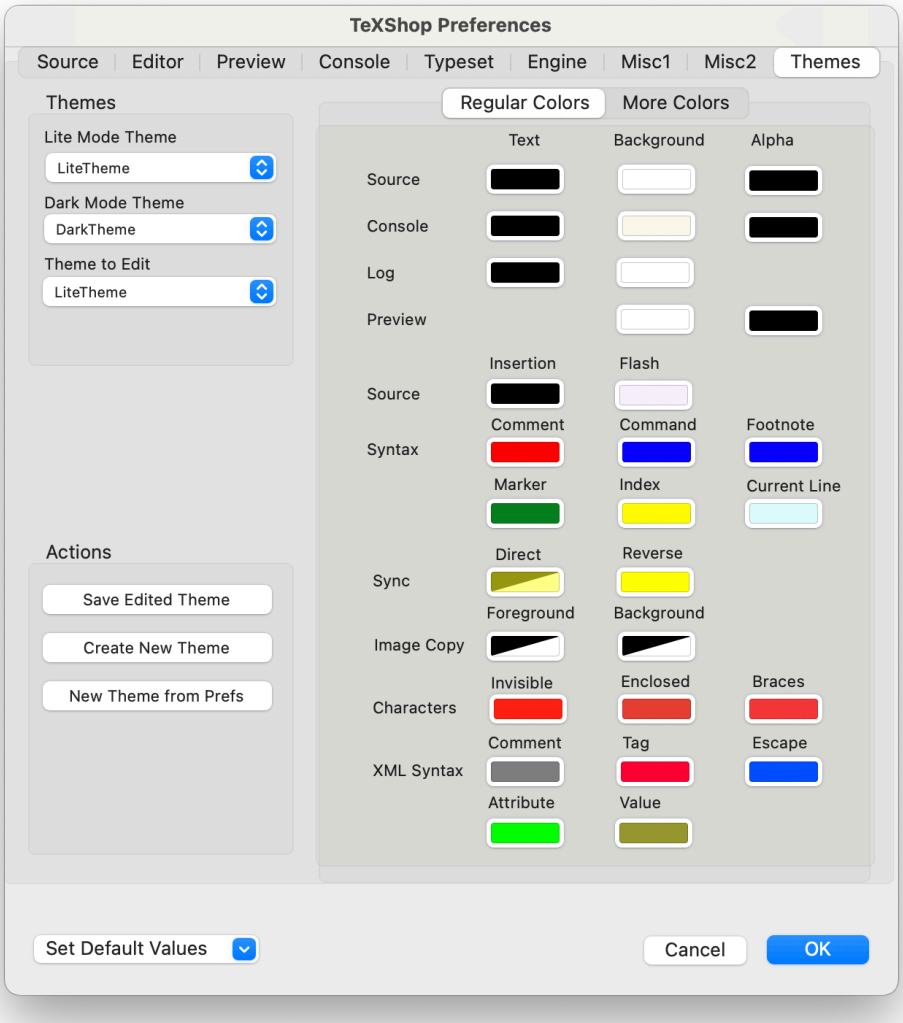



Figure 24.1: Theme Preference Items

There is no distinction between themes for Lite Mode and themes for Dark Mode. Thus nothing prevents a user from setting both Lite Mode and Dark Mode to the same theme.

After editing a theme, push "Cancel" or "OK" to end a preference session. If "Cancel" is pressed, the edited colors will not be saved and the Lite Mode and Dark Mode themes will return to choices before opening the Preference Pane. If "OK" is pressed, the edited colors will be saved and Lite Mode or Dark Mode themes will change to their new values.

Some users may want to edit several different themes during a session. When these users are finished editing their first theme, they should press "Save Edited Theme." This will save the changes for that theme *permanently*, even if the entire session is ended using the "Cancel" button. Repeat the process for other themes.

To create a new theme, first change "Theme to Edit" to obtain reasonable starting colors for your new theme. Then push "Create New Theme" and fill in the resulting dialog with a title for this theme. Do not use spaces in this title. The new theme will become the "Theme to Edit" and you can begin changing colors.

One final item in the left column must be explained, but it is essentially obsolete. When the theme panel was first introduced, some users had spent time using the original color preference system to revise the default colors, only to have that system ripped out of the program. Their color preference choices, however, still remained in the Preferences data base. The button "New Theme from Prefs" created a new color theme using these old preference choices. There is just one other use for this button which someone might conceivably still use. When the old color preference system existed, some people on the internet developed color themes for TeXShop and made them available as shell scripts which reset various TeXShop color setting preferences. These shell scripts still work, but they no longer affect the appearance of TeXShop. After running such a script, you can use "New Theme from Prefs" to convert the old scripted theme to a regular new Theme.

Recall that various TeXShop items which users can customize are set in `~/Library/TeXShop`. This folder has various subfolders. There is a new folder in `~/Library/TeXShop` named "Themes". This folder contains very small ".plist" files describing the various Themes in TeXShop. If you create a theme you like, give it to others by putting its plist file on the Internet. To install a new theme of this kind, just drop its plist form in the Themes folder.

An actual Theme plist file can be opened in TeXShop. You'll find that it has a very simple structure. The file contains a list of colors. After each color, its red, green, blue, and transparency values are listed. Each is a real number between zero and one.

It is possible to directly edit this file in TeXShop, but there is a slight complication. When TeXShop saves the edited file, the new version does not quite meet required standards for a plist file, so TeXShop cannot read the file. I haven't traced what goes wrong because there

is an easy solution. If you have downloaded Apple's free program XCode, then .plist files also open in XCode. These files can be edited in XCode, and .plist files saved by XCode have the correct structure. Moreover, if you edit a plist file in TeXShop, you can later open it and then close it in XCode, and now it will have exactly the right structure.

You can remove Themes you no longer use by removing their plist files from the Themes folder. Avoid removing themes being used for Lite Mode or Dark Mode (although TeXShop should react gracefully when it runs into this situation). As explained earlier, the themes LiteTheme and DarkTheme will be recreated if they are removed.

When a theme is selected for editing, TeXShop colors will temporarily be reset to those colors. Revising colors is then interactive; as soon as colors change in Preferences, they will also change in TeXShop's Source and Preview windows. But the new colors only become permanent after you press "OK".

Finally we explain the actual color choices. At the top of the Theme Panel, the Foreground and Background colors can be set for the Source, Console, and Log Windows, and the Background color can be set for the Preview Window. For each of these except the Log window, transparency can also be set. In color theory, alpha value sets transparency. Transparency settings bring up a full Color Well, but the colors of these items are ignored and only the alpha values of the choices matter.

Why can we set the Background Color of the Preview Window, but not its Foreground Color? That is because pdf files created by TeX have an invisible background color. When such a file is printed, the result is black characters on the actual paper. If the pdf had a background color, then the printer might print a grayish wash over the entire page, which is clearly undesirable. So TeXShop can independently set the background of the Preview Window, and it will appear except when covered by actual letters. Note that when such a document is printed, the background color setting of Preview will be ignored because this information is not in the pdf file.

Illustrations are a different matter. Most illustrations set the background color, and thus the background color of Preview will not appear behind illustrations.

However, the text color is set by TeX itself, and cannot be reset by TeXShop without rewriting the pdf file itself. The only way to change the color of the text is to set it in the LaTeX source for the document.

After these items, we see colors of the source insertion cursor and its flash. The Insertion cursor is the flashing vertical bar that is seen as we enter text. Flash refers to a feature that many users will not turn on. If a closing parenthesis like ) or } or ] is typed and it does not match an existing open parenthesis, the edit window can be told to flash its entire background briefly as a warning. Note that the default color of this flash is subtle.

Next come six colors used for syntax coloring of TeX source. In the default scheme,

comments are red, TeX commands starting with a backslash are dark blue, footnotes are blue, markers, the parentheses setting off arguments of commands, are green, and index settings are yellow.

It is possible to ask that the background of the active line be colored, and "Current Line" sets that color. After this come the direct and reverse colors used to indicate a SyncTeX match.

The colors "Invisible Chars, Enclosed Chars, Braces" are used for some features introduced by Yusuke Terada; see the menu item "Show Invisible Characters" and the item "Parens Targets & Highlight Color" in the Source Tab of Preferences, and the items "Show Invisible Characters" and "Parens Matching Settings" in the Editor Tab of Preferences. The items "Image Copy Foreground, Background" refer to rather obscure colors, used when copying a portion of the pdf file in unusual formats.rather than as pdf, jpeg, tiff, or png.

An item in the Source menu asks the editor to "Show Invisible Characters". The Invisible color is the color used for these characters.

The Enclosed and Braces characters are used for the items set in the top right of Preferences under the Editor tab.

Finally the XML Syntax colors are colors used when TeXShop syntax colors html, xml, and ptr files, as described in the chapter on Html, TeX4ht, and PreTeXt.

To be completely honest, one color is not set in the Themes panel. It is the color of the optional block cursor, set in the Misc1 tab of Preferences. This will be discussed in a later chapter of this Manual.

In September, 2025, a user requested that marker symbols be colored in a more precise manner. Before then, round parentheses were not colored, and the pairs { }, [ ], and symbols & and $ were all given the same coloring. Starting in September, 2025, only & and $ are given the standard "marker" color, and three new colors are provided for the three pairs ( ), { }, and [ ].

# Chapter 25

# expl3 Syntax Coloring

## 25.1 Latex3 and expl3

Recall the early history of LaTeX, described earlier in this manual (see Section 1.3). After the release of LaTeX2e, the original plan was to eventually replace LaTeX2e with LaTeX3. But the team later decided on a series of gradual improvements, so important changes to LaTeX have occurred in recent years without making old source files obsolete. For an overview of the current project, see https://www.latex-project.org/latex3/.

An important step in modern LaTeX programming occurred when the team designed and adopted expl3. They describe expl3 as "a new set of programming conventions that have been designed to meet the requirements of implementing large scale TeX macro programming projects such as LaTeX. These programming conventions are the base layer of LaTeX3." To read a detailed description from the team, go to ctan.org. At the top of the home page, search for "expl3". The first item to appear is named *Package expl3*. Go to this package. In the documentation section at the top, click on "Package documentation." This produces a document whose current revision as this chapter was written is January 22, 2024.

## 25.2 Activating expl3 Syntax Coloring

The TeXShop Source menu has an item named "Syntax Color expl3 Code". This menu is a toggle which turns expl3 syntax coloring on or off. When coloring is on, the menu has a checkmark. If expl3 coloring is off, TeXShop syntax coloring works as it always has. When expl3 coloring is on, the old syntax coloring still occurs, but in addition expl3 code is given special coloring. In either case, there will only be coloring if the menu item "Show Syntax Color" is checked. This menu item is just above "Syntax Color expl3 Code".

Like most TeXShop menus, "Syntax Color expl3 Code" applies to individual document source windows rather than to all windows at once. For a discussion of this philosophy, see Section 6.1. Thus expl3 coloring can be active in one source window and inactive in a second source window.

When source windows are first opened, expl3 coloring is off. This design was selected because most LaTeX programmers will use, but not write, LaTeX macros. If you are an exception and want expl3 coloring to be on when documents are first opened, select the item **Color expl3 Code by Default** under the Editor tab of TeXShop Settings. In versions before 5.60, this was the hidden preference expl3SyntaxColoring.

## 25.3 expl3 Coloring Rules

The colors used for expl3 syntax coloring can be changed in TeXShop Preferences using the Themes tab. This method works as usual, and seven additional colors are provided for expl3 coloring.

I am not a macro programmer, so my description of expl3 syntax may be a little rough. Syntax coloring is in TeXShop because it was requested by Alan Munn, and the description of the syntax comes directly from Munn. If you are a macro programmer, thank Munn for the new feature.

There are essentially six types of expl3 statements: functions, variables, double underscore functions, double underscore variables, msg commands for warnings and errors, and key setting commands and their arguments. Munn describes the difference between functions and double underscore functions as "private vs public macros, i.e., internal macros which are documented and can be relied on by other packages vs. internal macros that are not documented and should not be relied on by other packages." A similar remark applies to variables. But I'll refer to them as functions (which have a single underscore) and double underscore functions, because that is what I paid attention to in the coloring code.

As we will soon see, the string containing a function name and its argument has no spaces. This entire string is assigned one color, called the "function" color. The same rule holds for variables, double underscore functions, double underscore variables, and msg commands. The resulting colors are named

```
function, variable, __function, __variable, msg
```

Commands which set the value of a key have a slightly different syntax. Below is an example of such a command.

```
thiskey .tl_set:N = {...}
```

Such a command always starts with the name of the key, followed with optional spaces

which are ignored and then a period. The period begins a string with no spaces giving the key argument. The argument must contain at least one underscore or at least one colon. TeXShop assigns two colors to such a command. The name of the key gets the first color. The period and all characters in the argument get the second color. Thus two colors are used, named

```
key, argument
```

Here are four functions:

```
\tl_set:Nn
\clist_pop:NN
\mymodule_function:Nn
\mymodule_function:V
```

A variable always begins with a backslash, one of the letters l, g, c, and an underscore; here are three variables:

```
\l_mymodule_localvariable_clist
\g_mymodule_globalvariable_int
\c_mymodule_constant_str
```

Here is a double underscore function:

```
\__tl_set:Nn
```

and here are three double underscore variables:

```
\l__mymodule_localvariable_clist
\g__mymodule_globalvariable_int
\c__mymodule_constant_str
```

Here is a msg:

```
\msg_this_and_that
```

and here are two key commands:

```
 akey.is_nothing = {hello}
 thirdkey  .this_and_more:N = {something}
```

Note that functions, variables, double underscore functions, double underscore variables, and msg commands all consist of a string with no spaces. More precisely, this string can contain any lower case letter ‘a’ - ‘z’, any upper case letter ‘A’ - ‘Z’, the underscore character, the colon character ‘:’, and the sign ‘@’. Any other character ends the string and is not part of the string.

# Chapter 26

# Splitting Windows

## 26.1 Splitting Source and Preview Windows

The toolbars for the source and preview window each contain an icon containing two stacked rectangles. This item "splits the window" into two independent parts. This makes it possible to examine one portion of the document while writing a separate portion. The icon is a toggle which both splits and unsplits the window.

The Window Menu contains a menu item "Split Window" which is a similar toggle for the currently active window. This menu item has a keyboard shortcut, so if you are working with, say, the Preview Window, you can split or unsplit it with a keystroke without even reaching up to the toolbar.

When the Option key is pressed, the Window menu "Split Window" changes to "Split Window Vertically." Thus the source and preview windows can be split horizontally, with one portion above the other, or vertically, with one portion to the left of the other. Pushing the option key while clicking the split window toolbar item has the same effect.

## 26.2 The Active View in a Split Window

We begin this section with a concept well-known to Macintosh programmers, but perhaps not to users. Suppose your Macintosh is in use and you type on the keyboard. How does the Macintosh know where to send those keystrokes? Which window should receive them? Which view of a split view?

It turns out that at any moment the Macintosh has selected a "first responder". This is the object that first learns of keystrokes. This first responder is the beginning of a chain of more and more general responders. Suppose, for instance, that the Preview window

has been split and its drawer is open. Then the active view could be the top view, or the bottom view, or the drawer. Suppose it is the drawer. Then any keystroke will first go to that drawer. But if the drawer cannot use the keystroke, it passes down the chain to the next object, which might be the view associated with that drawer. If the view in turn cannot use the keystroke, it passes further down the chain, perhaps to the window containing the view. If this window cannot use the keystroke, then the keystroke is lost and does nothing.

The first responder will change as a user works. Often this happens when the user clicks on a different view. If a Preview window has a split view and an open drawer and the user clicks on the bottom view, then that view becomes first responder. After that, keystrokes never reach the drawer or the top view. Instead they pass to the bottom view, and from that view to the entire window. All of this is complicated and but works "automatically" without even the programmer knowing the details of these responder chains.

Similarly when the Source window is split, then it has two views and either could be the active view. In a split Source window, the active view is the view showing the cursor. To make a particular view active, click on it.

With this background, let us consider a concrete case. Suppose both the Source and Preview windows have been split and the Preview window has an open drawer. Suppose we are editing our document and going back and forth between source and preview using synctex. To make this work, we must select one of the two views in the Source window and click on it. That view now has a cursor. Similarly we must select one of the two views in the Preview window and click on it. This time there will be no particular indication that this view is active. Now we can use synctex between the active Source view and the active Preview view.

We have to be careful not to accidentally click in the inactive view in these windows. If we do, synctex results will begin appearing in unexpected places. If we accidentally click on the drawer, it will become the active view and synctex responses will vanish because that view cannot accept them.

Here's another concrete case. Suppose the Preview window is split and its drawer is open. When we click in this drawer, it becomes the active view. We can use it to select chapters and sections in one of the two split views. Suppose it starts selecting chapters in the top view, but we want it to select chapters in the bottom view. To make that happen, click on the bottom view, and then click on the drawer. When we clicked on the bottom view, it temporarily became the active view, but this had the side effect of associating that view with the drawer. When we clicked on the drawer again, it became the active view, but now it selects chapters in the bottom view.

When the drawer is active and we select a chapter in the drawer, it is marked in blue. This blue banner indicates that the drawer is still the active view. If we then click in the related

view, that view becomes active and the blue in the drawer turns to gray to indicate that the drawer is no longer active. But as we scroll through different sections of our document, the drawer shows new chapters and sections as we scroll through them, always in gray.

When the chapters in the drawer's outline view are numbered, several keyboard shortcuts are available in the drawer. If you type Option-1, the drawer will select the first chapter. If you type Option-2 followed by Option-0, the drawer will switch to Chapter 20. Type Option-UpArrow to select the first item and Option-DownArrow to select the last item. Type Option-1.5 to select Chapter 1, Section 5. If Chapter 3 is selected, Option-RightArrow will open sub-levels and Option-LeftArrow will close sub-levels. Although these keystrokes select drawer items, it is still necessary to click these items to make the associated view scroll to them.

There is a final item related to responders and active views. If the source window is split and we then unsplit it, the active view in the Source window will expand and become the contents of the unsplit window. So if the top portion is active and then we unsplit the window, we'll see that top portion. But if the bottom portion is active and then we unsplit the window, we'll see that bottom portion.

The Preview window does not work this way. Instead, the top view of a split window always becomes the view we see when we unsplit the window. So a slightly different philosophy guides the Preview window. The top view is the important piece, which will appear whether or not we split the window. The bottom view is the secondary material, which we see only when the window is split.

There are rare situations when we want the bottom portion of the Preview window to become the primary view after we unsplit. And for that, there is a special command.

## 26.3 Switching Views

The Window menu contains another item named "Switch Views". This item is only active when the Preview Window is active. It switches the top and bottom views of a split window, or the left and right views of a split window. This is useful because when the Preview Window is unsplit, the top view becomes the active view. Using "Switch View" first allows either view to become the active window when the window is unsplit.

This feature is not available for the Source Window because it is not needed there. When the source window has been split into two views, the view with the cursor will become the active window when the window is unsplit.

If "Switch Views" is used several times in a row, the images will slowly creep up or down, rather than returning to their exact starting positions. This creep is difficult to eliminate and irrelevant, because the command is designed to be used just once in the rare case that the bottom view in the Preview window should become the main view after unsplitting.

## 26.4 Revising the Document and Typesetting

When the preview window is split, it is still easy to revise the source for either view and then typeset to see the result in Preview. Click on either view and use synctex to find the corresponding spot in the source. Edit the source. Typeset. The revision will appear in both views of the split Preview window.

## 26.5 Changing the Display Format While Using Switch Views

TeXShop configures the way documents are displayed using two preference items, called "Display Format" and "Magnification". Typical display formats are Single Page, Double Page, MultiPage, Double MultiPage. Typical magnification choices are Fit to Window, Actual Size, and Fixed Magnification. These items can be selected in TeXShop Preferences, and then affect Preview windows when they first open.

After such a document is opened, the preference items can be changed for any particular Preview window using items in the Preview menu. These choices are temporary while the document is being used, but revert back to the default choices for new documents.

When the Preview window is not split, the menu items "Display Format" and "Magnification" affect both views simultaneously. So if you change the magnification of the current view, and later split the window, both views will use the new magnification level. If you switch to "Double Page" mode in the current view and later split the window, both views will be in Double Page mode.

When the Preview window is split so both views are shown simultaneously, "Display Format" and "Magnification" changes made to the top view affect the entire document as above. Thus if the window is later unsplit, the changes will hold in the entire document.

However, when the Preview window is split so both views are shown simultaneously, "Display Format" and "Magnification" changes made to the bottom view will only apply to that portion of the split window, and only as long as the window remains split. This slight

design inconsistency makes it possible for a user working with two sections of the document to temporarily magnify the bottom section and inspect fine details without propagating that magnification change to the rest of the interface.

## 26.6 Avoiding Creep When Unsplitting

When splitting the Preview window was introduced, we imagined that users faced with writing related sections of a document would split the window and work in split mode constantly without unsplitting. To our surprise, a few users worked in a different way; they alternated between working with the split window and working with the unsplit top portion. These users noticed that when a document was split and then unsplit, the display did not return to the exact starting spot, but instead crept up or down by several lines. If the two commands were cycled again and again, each cycle produced an additional creep. This became annoying.

The problem did not occur in Single Page or Double Page display modes, but only in Multipage and Double Multipage modes. It did not occur when the window was split vertically, but only when it was split horizontally. In TeXShop 5.25, this problem was fixed by a slight design change. Just before splitting a window horizontally, the program records the position of its scroll bar. When the window is unsplit, the view returns to that recorded scroll bar position. This virtually eliminates the creep, but at a slight cost. When the user splits the window and then scrolls the top portion, unsplitting does not return to the new scroll position of the view, but instead to the original position before splitting. (This only happens when splitting horizontally in Multipage and Double Multipage modes.)

It is easy to get around this problem. Suppose you want to scroll the top portion of a split window to a new "permanent position." Unsplit the window, scroll to the new position, and split it again. The lower half will not change, but the upper half will now remember its new position.

## 26.7 Two Options When Splitting Horizontally

Two special variations have been added to the split window command in horizontal mode. Most users should ignore these variations, but one or two users may find them helpful. If the shift and control keys are held down when the Split Window menu is selected or the corresponding tool is checked, the window is forced to split so the lower and upper views are the same size. If just the shift key is held down in the same situation, the two views are forced to have the same size and to contain the same material.

The first of these variations may be useful for users who rarely move the split bar and never change the magnification of either view. These users probably already have a stable

situation without creep. In those rare cases when they do move the split bar (and pay with additional creep), the option takes them back to the stable situation.

The second variation was created for a user who lectures directly from a screen showing typeset course notes. He typically shows a full page view of the notes, scrolling through each day's material. But when he comes to a theorem, he splits the screen using the second variation. The theorem remains in the upper view, so students can refer back to it and recall the assumptions required for the conclusion. Meanwhile the lower view can be scrolled to show example applications of the theorem and then key steps of the proof.

## 26.8 Splitting Vertically

If the Option key is held down when a window is split, the split is vertical. This mode works as expected provided you know what to expect. When a window is split, the original single full window's contents are placed in the left vertical side. Since vertical sections are narrower than before, more pages are shown. The original unsplit page is at the bottom of this display rather than the top. This is by design. Similarly when the window is unsplit, the bottom portion of the left side will become the new single window. This is also by design. If the left side is scrolled during the split phase, the new material at the bottom of the left side will become the full window after unsplitting.

## 26.9 Final Note

A few users told us that as soon as something unexpected happened when splitting a window, they stopped using the feature for fear that the underlying pdf file would be damaged. So a word of reassurance is in order. None of these display modifications are written back to the pdf file. They only affect how TeXShop displays the file. If TeXShop does something strange, the file is perfectly safe. Retreat to full window mode and proceed as if nothing happened. If you like, write us to explain the surprise behavior. Then relax.

# Chapter 27

# Switching Preview Views

## 27.1 Back and Forward Arrows

TEX documents can contain related pieces of information in widely separated document locations. For instance, a reference in the text may be linked to an item in the bibliography, and a proof in one chapter may refer to a theorem from an earlier chapter. In situations like these, it can be useful to edit both items at the same time, scrolling back and forth between the two locations. TeXShop has several features to make this task easier.

The first of these comes from Web browsing. Web pages often have links to other sections or other pages and users navigate from one link to the next. A set of Back/Forward arrows keep track of this navigation and allow users to return to earlier links, or revisit later ones. This set is different than the Page Up/ Page Down pair, which navigate by page number.

The hyperlink package provides links for a TeX document; this document can then be navigated from link to link using the Back/Forward arrows. This sort of navigation is easy to learn since it replicates an experience already familiar while browsing the web.

Another tool to handle related sections is splitting the Preview window, as discussed in the previous chapter.

## 27.2 Switch Views

The "Switch Views" menu item used when Preview windows are not split provides yet another way to handle related sections of a document, although it only navigates between two views. We already discussed this menu item for split Preview windows. But the use case when windows are not split may prove to be more important than the original reason

for introducing the command. Suppose you want to work on two related portions of a document. In TeXShop's Preview Window, do not split the view. Instead, just scroll to one of the two interesting portions of your document. Then select the menu item "Switch Views" or type its keyboard shortcut Option-2, and scroll to a second related section of the document. You can now switch between these two portions using "Switch Views" or Option-2.

Although this operation is a variant of splitting the Preview window, it is better to think of it as an entirely independent way to make use of the small amount of window space available when several documents are open at once. Some users may split the Preview window but never use this variant. Others may use the variant and never split a window.

Although "Switch Views" does not work in the Source window, you can easily edit and revise the two related portions of your document because synctex works between the two Preview views and the source file. Suppose text in one of the two portions needs revision. Sync from that text to the source, edit the source, and typeset.

When this command was introduced in TeXShop 5.23, it had a "creep problem". The two views would creep up gradually when switched. This problem has been fixed in TeXShop 5.25.

The two views are not fixed; they can be scrolled if desired. So while one view is visible, we can scroll down to a new position. Then Option-2 will produce the second view. If we use Option-2 again, we'll go back to the newly scrolled position.

## 27.3 Independence from Split Window

When this feature was introduced in TeXShop 5.23, the two views for the full window were also the two views displayed by splitting the window. This is no longer the case. Instead if either of the views for the full window is split, it will become the top view of the split window. The bottom view of this window is a completely separate view, which can be scrolled to any position desired. Unsplitting the window returns to the original view before it was split. If we then type Option-2 to get the second full view and then split that view, the second full view will appear as the top view of the split window, and the bottom view will be the same view obtained when we first split. This means that users who use the new Switch Preview Views feature can also use the older Split Window feature. Then they will deal with three independent views, the two views for the full window (which become the top view when splitting), and the bottom view when either view is split.

Any "Display Format" and "Magnification" change made to a switched Preview Window affects both views. Thus if the window is changed to Double MultiPage mode and later switched with Option-2, the new view will also be in Double MultiPage mode. Small adjustments of the alternate view may be needed after switching to it for the first time with Option-2, but changing Display Format or Magnification is rarely done in the middle of an editing session, so the adjustment should not be annoying.

# Chapter 28

# Presenting with TeXShop (Schmock)

## 28.1 Introduction (Koch)

When TeXShop was first released at the turn of the century, visiting colloquium speakers at the University of Oregon sometimes connected their portable Macintosh to the projector and lectured directly from the program. That was fun for me, but it didn't last long. Mathematicians soon discovered *beamer*, which makes it easy to construct slides using LaTeX. Presenting at another place probably means that you have to use a computer of your host institution, running on a different operating system, and maybe even in a language not familiar to you. Then it's usually best to create PDF slides with beamer or another LaTeX slide package. If possible, email them to the organizer in advance so the organizer can test whether everything works. Carry a USB stick with the PDF slides as a backup. Hopefully, you will get a remote control with back/forward buttons and a laser pointer (but bringing your own with a set of spare batteries is a good idea).

If you are the only speaker or if you are supposed to teach a minicourse, then it's reasonable to inquire if you'll be able to connect your portable to the projector. If the answer is yes, then bring your Macintosh and your own set of adaptors. You may be tempted to use Apple's Keynote and its spectacular transition effects. In that case, investigate the shareware program *Présentation.app* by Renaud Blanch, which can convert Beamer output to Keynote format. See http://iihm.imag.fr/blanch/software/osx-presentation/.

The covid pandemic brought to the foreground an entirely new set of issues as faculty members rapidly transitioned to teaching remotely using *Zoom* and other tools. Luckily, I was retired by then. It is a complete mystery to me how faculty managed to cope with this calamity, and it would be ridiculous for me to offer advice for that situation.

More recently, I began to get requests from Uwe Schmock in Vienna, Austria, who was using TeXShop as a presentation tool while lecturing from an elaborate set of typeset notes he had written for the course and expanded during the covid pandemic. He requested additional features in TeXShop to make this process smoother.

The students had a copy of those notes on their own machines, and could follow along during the lecture. Lecturing directly from a typeset article is dangerous because the speaker is tempted to scroll rapidly from one spot to another as students ask questions or unexpected connections between theorems suddenly pop to mind. Watching rapid scrolls during a lecture is mind-numbing. But Schmock had tricks up his sleeve to avoid such scrolls. For example, when the lecture notes were straightforward, he would show full pages of the Preview window and talk his way through key points. But when a new theorem appeared, he would split the screen so the theorem remained in the top portion for easy reference, and slowly scroll the bottom half as he discussed major points in the proof. This allowed him to scroll past trivial calculations in the bottom view, and then stop as the next key idea in a proof emerged.

This use of the program was so unexpected that I asked Schmock to write a chapter of the manual describing what he does in detail. His chapter follows.

## 28.2 Presenting (Schmock)

When it comes to presentations, you might think of creating and showing slides using commercial software, like Apple's Keynote or Microsoft PowerPoint. You might prefer free software, LaTeX, and slides in PDF. For presenting the PDF slides, the full screen mode of Apple's Preview, Skim, the free Acrobat Reader, as well as the commercial Adobe Acrobat are possible choices. The aim of this chapter is to show that TeXShop – in tandem with LaTeX's `hyperref` package – has some unique features, which can make it your top choice for not just creating PDF files (including slides) but also for presenting them.

When presenting PDF documents, there are several situations and settings. The type of document can be

1. PDF slides, preferably created with LaTeX's

   ```
   \documentclass{beamer}
   ```

2. a given PDF file, which you might be able to edit,
3. a PDF file, for which you can modify the LaTeX source code.

Furthermore, there are at least three variants of presentation:

(a) Presentation at another place, like an invited or contributed talk at some university or conference.

(b) Presentation in a class room or lecture hall, where you can connect your mobile computer directly to a projector and display your entire screen.

(c) Online presentation with proprietary software like Skype or commercial software like Zoom or Microsoft Teams, where you can share a specific window.

Case (a) is off topic for my part, because TeXShop is not used for presentation, and Section 28.1 above already contains the usual advice.

In the following I will not give a detailed tutorial for using TeXShop in all remaining combinations. Instead, here is a list of remarks and hints, which you can apply, combine, and adjust as you please. Let's start with some general remarks:

- Remember that some people in your audience might not have perfect eye sight or might not get a place in the lecture hall which gives them a good view of your presentation. Aim for a larger magnification and don't present too much on a single slide and not an entire page of a paper. Make use of your entire screen real estate.
- If you want to convey mathematical ideas or even a proof, don't distract your audience with too much action on the screen, and give them time to grasp the results before removing them from their view. Slow scrolling might be a compromise when compared to waiting and changing the slide abruptly.
- When using the traditional method of writing with chalk on a blackboard, it is a challenge to present the material quickly. When you present online or with a projector, the opposite is true. So try hard to limit yourself and don't present more theory than you could cover with the traditional method. Instead, show pictures, drawing, examples, etc.
- Before you begin a presentation with your computer:
  - Quit all applications you don't plan to use, close all TeXShop and Finder windows you don't need, and clean up your desktop. In case (c) this makes selecting windows easier, and it protects your privacy is case you share your screen.
  - Switch off various notifications under "System Settings..." → "Notifications", and check – and possibly adjust – under "System Settings..." → "Lock Screen", the time until the screen saver starts or the display is turned off. Questions and discussions during or after the presentation can take a lot of time.
  - If you plan to use the mouse pointer, go to "System Settings..." → "Accessibility" and increase the "Pointer size" above its normal size for better visibility.

Now let's look at TeXShop-specific issues. First for the presentation of slides (case 1).

- With the Preview window in front, go in the Preview menu to the item "Magnification" and select "Fit to Window". Similarly, in the item "Display Format" select

"Single Page" (this step can also be done with the context menu). Then the full screen mode works best (click the green button of the traffic light).

- When the slides were created with document class beamer, then – depending on the packages used – internal links are created. However, if you use the mouse pointer, TeXShop's hover preview can be disturbing during the presentation. Use the toggle called "Link Popups" from TeXShop's Preview menu (or use the context menu of the Preview window) to switch them off, see also Section 17.9.

When presenting online, see case (c), there are advantages to sharing only the Preview window.

- All other windows and the desktop remain your private territory.
- If the toolbar of the Preview window shows the icons as well as the descriptions, control-click into the toolbar and select "Icon Only" from the context menu to recover some screen space.
- The drawer of the Preview window (see Section 17.7) counts as a separate window and is not automatically shared together with it. So the drawer can be used for navigation (but read Section 26.2 and always click into the Preview window after using the drawer to avoid surprises). With the back and forward arrows (see Section 27.1) of the Preview window's toolbar, you can easily navigate like in a browser window. In case 3, you can add further entries to the drawer by using

  ```
  \pdfbookmark{Drawer entry}{Label name}
  ```

  at the desired places. I always add at least the Table of Contents – see this manual's drawer – for easier navigation.
- Recall from Section 27.3 that the Preview window has three separate views, including two full views. Hence you can scroll to a figure in the Preview window, select "Switch Views" from the Window menu, and scroll to the beginning of your presentation. When the time arrives to show the figure, you press the keyboard equivalent Option-2 and the figure magically appears (under the assumption that you didn't change "Display Format" or "Magnification" in between). You don't have to stop sharing and start sharing again!
- By default, I use the mouse pointer for the Preview window. In the "Change Mouse Mode" icon of the toolbar, it is the second choice (text selection), see Section 17.5. Highlighting text by selecting it works fine, but can produce confusing results with bigger displayed formulas, so you should check these critical cases in advance.

When you have lecture notes for teaching a class (or a typeset solution of an exercise you present to your fellow students), then you can present these instead of taking the time to

create additional slides (and to avoid the hassle to keep the slides current when you update the lecture notes). In this setting you can profit a lot from TeXShop's features!

- With the Preview window in front, go in the Preview menu to the item "Magnification" and select "Fit to Window", which acts like a "Fit to Width". If this is not large enough, then use "Zoom In" from "Magnification" or the context menu, the keyboard equivalent Shift-Command-+, or use the "Scale" field in the toolbar. Similarly, in the item "Display Format" select "MultiPage" (in the context menu, Apple's default name "Single Page Continuous" is used for the same functionality).

- In case 3, there is another way so that "Fit to Window" is fine, and "MultiPage" is actually more useful in case a statement or a longer proof is interrupted by a page break: Remove most of the white space around the text without changing the line or page breaks (so the participants following your presentation an easily find the location to annotate their copy of the lecture notes). This manual, for example, uses

  ```
  \documentclass[11pt, oneside]{book}
  \usepackage[letterpaper]{geometry}
  ```

  In the log file I find

  ```
  \textwidth=430.00462pt
  \textheight=556.47656pt
  ```

  For removing most the white border but keeping the headline, I would use slightly increased values for the paper size and add a line containing something like

  ```
  \geometry{papersize={450pt,610pt}, textwidth=\textwidth,
  textheight=\textheight, centering, vmarginratio={4:1}}
  ```

  to get a version of this manual for presentation. This ad hoc intervention does the job, and you don't have to fiddle around with the magnification during the presentation.

  Note that in this process the magnification in the Preview window might get so large that the magnifying class stops working, see Section 17.9.2 for details.

- When you present a proof, you will probably refer to previous results, equations, definitions, etc. *Here TeXShop's exceptional hover preview comes into play, see Sections 17.9 and 17.9.1.* Make sure that the item "Link Popups" is checked in the Preview menu. If there is an internal link created by the `hyperref` package, then there will be a popup window below the link when you hover over it, reminding your audience about what is used at this stage of the proof. If this popup window covers something important, press the shift key when hovering over the link to shift the window up. By default, the popup window disappears after four seconds or when you move the mouse. This is probably long enough for you, but not for your audience. So exercise the option to display the popup window longer by pressing the option

key when hovering over the link or when the four-seconds timer is about to run out. When the default popup window it too small – no problem. You are in command, so press the command key when hovering over the link for a larger window! Of course, any combination of the three modifier keys also works as expected.

If the above mnemonics for the modifier keys don't work for you, fix a sticky note to your keyboard.

The position of the popup window is mostly relative to the location where you enter the active area of the link. Hence a shifted vertical placement of the popup window (by about a baseline skip) is easy and sometimes helpful.

It's your responsibility to arrange by scrolling for sufficient vertical space so that the interesting part of the popup window is visible, in particular when you use the shift key for an upper placement. If you use the option for an extended duration of the popup view, then there is enough time for a careful gesture on the trackpad to "scroll" the preview including the popup.

- In case 3, make sure to place an internal link to every previous location in the order you want to show these.
- Sometimes during online teaching, case (c), you might be tempted to use the mouse pointer to explain a correspondence between aspects in the popup window and the current situation – however, this doesn't work, because the popup window disappears as soon as you move the mouse. There is no old-fashioned stick or modern laser pointer to help you out. However, here is a solution, see also Section 26.7: Scroll the Preview so that the link is in the upper half of the window. Hold down the shift key and toggle "Split Window" in the Window menu (or the context menu) or click in the toolbar the symbol consisting of two rectangles stacked on top of each other. This splits the Preview window into equal parts and duplicates the upper part of the full view into the lower part. Follow the internal link in the lower part. If desired, scroll each part a litte bit and move the dividing bar. When you switch back, i.e. unsplit the Preview window (no modifier key necessary), then the lower part vanishes and you see more of the upper one (and possibly experience an upward creep of a line or two).

  TeXShop remembers the position of the dividing bar and the view of the lower part is saved. If you split again (without any modifier key), you get back the setting which you just left.
- The "Split Window" just explained is also helpful when presenting a theorem. You probably want to keep it visible while discussing examples and going through its proof. Again as above, shift-split the Preview window, scroll each part a litte bit and move the dividing bar as you please. Then treat your examples and the proof in the

lower part. When you are done with the theorem, select "Switch Views" from the Window menu (or press the keyboard equivalent option-2), then unsplit the Preview window. Up to a little bit of scrolling, you are at the end of the previous lower part, but now in full view.

- Imagine you have a given problem set (case 2) and your handwritten solutions including figures. Scan them and combine them with the problem set to produce one PDF file. Use TeXShop as PDF viewer, split the Preview window, scroll both parts to the desired locations, adjust the dividing bar. Now you are ready to recall the given problems in the upper part and to present your solutions in the lower one. Note that you can magnify the upper and lower part independently if desired!

For improved functionality of hover preview and – more importantly – for PDF files mainly read online, more accurate internal links are strongly recommended. Therefore, I propose the following adjustments when creating PDF files with the current version of the `hyperref` package:

- When you have a larger numbered formula, possibly covering several lines, then hover preview for a corresponding link might not show the beginning of the formula and clicking on the link might require some scrolling to bring it to full view. You can improve this behaviour by raising the hook (or anchor) of the internal link. Define a macro to work with the `hyperref` package, for example

```
\makeatletter
\newcommand{\RaiseHook}[1]{\renewcommand\HyperRaiseLinkHook%
{\setlength\HyperRaiseLinkLength{#1\baselineskip}%
\global\let\HyperRaiseLinkHook\@empty}\ignorespaces}
\makeatother
```

  and add a command like

```
\RaiseHook{1.6}
```

  just before you start a displayed two-line equation, for example. The number 1.6 is in multiples of a baseline skip. When the command is used, it resets itself to the default value 1. To find the suitable numbers, you have to experiment a bit. A number is perfect, when a click on the link displays the entire formula at the top of the Preview window without superfluous space above it. Usually you have to start adding the `RaiseHook` command when your displayed formula contains a fraction (1.2), an integral with an upper bound (1.4), a summation symbol with an upper bound (1.5), a `cases` or `split` construction (1.6), or a `multline` environment (2.2). These numbers usually work in simple cases with my setting of the baseline skip. I suggest that you make your own list.

- When a numbered LaTeX environment starts at the beginning of a page, then the

corresponding internal link often points to the end of the previous page. In most cases it suffices to add

```
\pagebreak[2]
```

to the end of the previous line, sometimes `[2]` has to be increased to `[3]`.

- Links to a figure or a table might lead to the corresponding caption instead of the figure or table itself. To fix this behaviour, place

  ```
  \usepackage[all]{hypcap}
  ```

  after calling the package `hyperref`. To see it working, try the link to Figure 3.1 in this manual. If there are figures without a caption (like the one at the end of Section 1.1 in this manual), enclose it with

  ```
  \capstartfalse
  \begin{figure}
     (Figure without caption)
  \end{figure}
  \capstarttrue
  ```

  For more information about the package, type `hypcap` in the window you get by selecting "Show Help for Package..." from TeXShop's Help menu.

There is always room for improvement. Feel free to send any suggestions about this chapter directly to me at schmock@fam.tuwien.ac.at.

# Chapter 29

# Additional TeXShop Features

## 29.1 Experiments

This feature was requested by Wendy McKay.

The item "Experiment..." in the Edit menu is available when the source window is active. The item allows users to experiment with short, but complicated, fragments of TeX before copying the source into the main document.

When the item is chosen, a panel appears. Type a TeX fragment into the panel, say

```
$$\sqrt{x^2 + y^2}$$
```

Push the Typeset button at the bottom of the panel, and a second panel appears showing the result of typesetting the fragment. The fragment can contain anything: a displayed formula, ordinary text, several pages of mixed material. To typeset, TeXShop creates a new source file with the header of the current document up until "begindocument", the new fragment, and a final "enddocument" This also works in a project with a root file. In that case the contents of the root file up until "begindocument" are used.

Both panels have close buttons. The ESCAPE key will also close panels when they are active. Although the two panels do not have resize buttons, they can both be resized. TeXShop will remember the new sizes and locations and use them the next time "Experiment.." is selected. The font in the source panel will be the default TeXShop source font. The keyboard shortcuts "command +" and "command -" work in the source panel to enlarge the text if desired. Key bindings and command completion are available in the source panel, but with one caveat. Command completion uses the tab key in the panel even if it uses the escape key for regular source, since the escape key in a panel closes the panel.

The "Experiment..." feature requires a latex-like engine. It will not work with ordinary plain tex. The source panel's Typeset button looks at the main source window's toolbar to determine a typesetting engine., and also uses the the magic comment line mechanism if available at the top of the source window. If "Plain TeX" or "Context" is selected, nothing happens. If "bibtex" or "make index" are chosen, pdflatex is used. Obviously "pdflatex, xelatex, and lualatex" can be used. The panel will try to use any user-defined engine selected, but some such engines may fail if they don't expect latex-like code or don't output pdf.

The preview panel understands mouse scroll commands and trackpad gesture commands to scroll and resize. If you close a panel during work and later reopen it, the contents will be remembered. But the contents are lost when quitting TeXShop. It is assumed that panels will be used for short fragments of work; when the user is satisfied, they will transfer the source to the main document using copy and paste. Panel contents are not auto-saved and cannot be manually saved except via the copy mechanism.

Each document has its own source and preview panels, so if you have multiple documents open, you could also have multiple source and preview panels open, leading to a confusing mess. I expect users to exercise common sense and only experiment with one fragment at a time. One way to avoid confusion would be to hide the panels when a document becomes inactive. I didn't want to do that because a user constructing a complicated example might want to temporarily open a second document and copy source from that document into the panel as a starting point.

After this item was introduced, it was mentioned on the web site tex.stackexchange.com, and Denis Bitouzé suggested an improvement. Following his suggestion, if a selection of source is made first and then the menu is chosen, the selection is automatically copied into the experiment window. If no selection is made and instead the cursor is simply positioned by clicking at a point, then the Experiment window opens with its previous contents. Thus if a user carefully edits an equation, closes the experiment window, and then decides on a final change, the contents can be brought back for another edit.

Typesetting for "Experiment..." uses the Preference settings for pdflatex and simpdftex latex. If the user's pulldown menu is set to latex, the three choices of 1) pdflatex, 2) tex + dvi, 3) personal script are obeyed. This should make the command work for TeX in Japan.

"Experiments" are for moments when you want to work on a single tricky problem, usually involving complicated mathematical formulas or tikz illustrations, and you don't want to risk making a mistake in the source file that would be difficult to find and fix. Instead you work in two windows that float over the main document. These windows are only there for a short, specific task, and most of the extra infrastructure provided by TeXShop still applies to the underlying Source and Preview windows. For example, the "Comment" and

"Indent" commands in the source menu continue to access the main Source window and ignore the floating Experiment window.

## 29.2 Tags Toolbar Item and Menu; Labels Item

The toolbar for the Source Window has an item labeled Tags. This menu displays a scrollable list of all chapters, sections, and subsections in the document. Select an item to be taken to that spot in the source file.

It is possible to add your own tag to a location. Include at line at that location of the form

```
%: tag_name
```

where "tag_name" can be any sequence of words.

The menu takes at least the following commands (and more if an xml file is being used):

```
part
chapter
section
subsection
subsubsection
paragraph
subparagraph
macro
environment

frame
slide
wideslide
notes
```

The item "Tags Menu in Menubar" under the Editor tab in TeXShop Preferences causes a duplicate of the Tags menu to appear in the menubar.

## 29.3 Latex Panel, Matrix Panel, Unicode Panel

The Window menu has two items labeled “LaTeX Panel” and “Matrix Panel”. Each causes an appropriate panel to open.

The LaTeX Panel, see Figure 29.1, floats above regular source windows in TeXShop and lists a large number of mathematical symbols. Activate a source window and click to position the Insertion Cursor. Then click on a symbol in the panel and the corresponding LaTeX code will be inserted at the indicated position in the source.




Figure 29.1: Latex Panel

The Matrix Panel, see Figure 29.2, is similar, but more complicated. It creates matrices and tables. The number of rows and columns can be set. Experiment with other entries to see how they work.

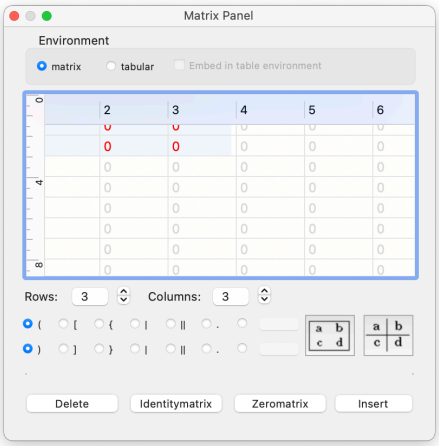



Figure 29.2: Matrix Panel

The Labels item on the toolbar works just like the Tags item, except that it lists labels in the Source Code.

The TeXShop Editor is fully Unicode capable, so unusual scripts can be entered and typeset. Sometimes it is useful to know the particular Unicode characters being typed, and for that purpose Terada Yusuke provided a Unicode Panel. Select a portion of text, and then select "Character Info.." in the Edit Menu. Figure 29.3 shows this panel examining a piece of Arabic text in a source file to be typeset with XeLaTeX.

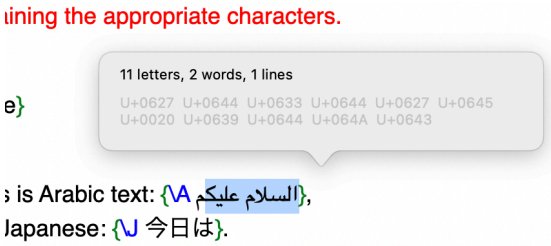



Figure 29.3: Arabic Unicode

## 29.4 Show Invisible

The Source menu has an item labeled "Show Invisible Characters". The specific invisible characters shown, and the symbols used for them, are determined by settings in the Editor tab of TeXShop Preferences. Figure 29.4 shows a short section of source code with this item applied to it.

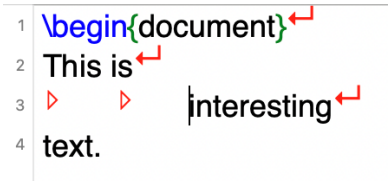



Figure 29.4: Invisible Characters

This particular example has three line feeds and two tab characters.

## 29.5 A Magic Line for Beamer and Other Slide Packages

Users often write mathematical slides using LaTeX. I highly recommend Beamer for that purpose. It is easy to use and produces beautiful slides.

I usually recommend the MultiPage Display Format for the Preview window, in which the pdf document can be scrolled from beginning to end. But when slides are produced and shown, it is preferable to use the Single Page mode and page down from slide to slide. The magic comment line

```
% !TEX pdfSinglePage
```

causes the document to display in Single Page Mode.

## 29.6 Counting Words in a Document

The menu item "Statistics" in the Edit menu counts the number of words, lines, and characters in the currently active source file. This is roughly the number of such items in the typeset document because the software is designed not to count LaTeX command words. The statistics panel calls "detex", a command line program in TeX Live, to do the actual counting.

## 29.7 Useful Items in the TeXShop Menu

### Check for Updates

TeXShop uses a third party open source framework called *Sparkle* to handle updates. This item indicates if an update is available. If so, it lists the features of the latest update, and offers to install it. If you accept, the program is updated and restarted, returning to its state just before the update happened. Sparkle is used by many other TeX related GUI programs, including BibDesk, LaTeXiT, and TeX Live Utility.

### Open `~/Library/TeXShop`

As mentioned often in this manual, important configuration files are in this folder. Since `~/Library` is hidden by Apple, it can be difficult to access the location. Using this item makes the task easy.

### Select TeXShop as Default TeX Editor

TeXShop contains a file named Info.plist which the Finder examines to discover features of the program it needs to know. One of these features is a list of filetypes that TeXShop can open and edit. The most important of these are files are TeX source files with extension ".tex".

Unfortunately, many other programs can also edit these files. The Finder more or less randomly picks a program to open files with extension ".tex" when they are double clicked. It is not unusual to install another program and discover that such files no longer automatically open in TeXShop.

The standard solution is to select a ".tex" file in the Finder and then select the menu item Get Info. A dialog opens with many pieces of informaion, and one is a pull down menu listing the program that will open the file if it is double clicked. Choose TeXShop in this menu. Below the menu is an item reading "Use this application to open all documents like this one." Select that item.

However, there is a much easier way to tell that Finder that ".tex" files should open in TeXShop. Just select the menu "Select TeXShop as Default TeX Editor".

# Chapter 30

# Other Miscellaneous Features

## 30.1 Items in the TeXShop Help Menu

The TeXShop Help menu contains several items, including a number of pdf documents. Note that the Print command works with these items, so it is easy to get a hard copy by just printing the document. The print dialog has a command which creates a new pdf file instead of printing, so it is also easy to produce an independent pdf document from any of these pdf items.

## 30.2 Help with Style and Class Files

The TeXShop Help Menu contains three items which provide information about TeX, LaTeX, and other items in the TeX Live Distribution:

**TeX Live Documentation**

The TeX Live distribution comes with a massive amount of documentation. This menu item opens a page of links to various important items in that documentation.

**Show Help for Package ...**

Most binaries and packages in TeX Live have associated documentation. Selecting this item opens a small dialog in which you can type the name of a binary or package. Usually this item is in small letters. TeXShop then calls a program in TeX Live to open the documentation in appropriate programs, often in Preview or Safari. Try this by searching for documentation for pdftex, xetex, beamer, hyperref, and other interesting software.

The search tool which this item calls can be set in TeXShop Preferences. By default it is

```
kpsetool -w -n latex tex
```

**Open Style File ...**

This menu command opens a small dialog in which you can type the name of a style file or class file. TeXShop will then open the file itself.

## 30.3 Opening Other Files

]TeXShop can open most files for editing. This facility has been provided so users can read TeX-related files with extensions ".log", ".aux", etc.

TeXShop can open files with extensions .ins, .dts, .sty, .cls, .mf, .def, .fd, .fdd, .ltx, .clo, .ctx, .bst, .bib, .engine, .htx, .sk, .dtx, .dry, .dn, .abc, .ly, .texi, .Rnw, .asy, .akt, .bbx, .cbx, .lbx, .mp, and others. It can also open any text file.

Files with extensions .jpg, .tif, .eps, or .pdf are opened as graphic files.

TeXShop can also open .dvi and .ps files. In these cases, it converts the file to pdf and displays this pdf file.

## 30.4 Arabic, Hebrew, Persian

These languages are written from right to left.

There is an item under the Editor tab in TeXShop Preferences called *Arabic, Hebrew, Persian*. When this item is checked, any line of source text which contains a character from one of these languages in the first three characters will be right justified, while other lines will be left justified.

Consequently lines starting with a TeX command in English will be left justified, even if later parameters are written in one of the languages, but source lines in any of these three languages will be right justified.

I was once shown the source for a college physics textbook in Hebrew. All along the left margin there were TeX formatting commands in English. But all along the right margin there were sentences in Hebrew.

Incidentally, there is a hidden preference setting which can be set by typing the following command in Terminal:

```
defaults write TeXShop RightJustifyIfAnyKey YES
```

If this preference is NO, then the previous behavior will hold. If it is YES, then any line with any character at all in one of these languages will be right justified.

## 30.5 Localizations

TeXShop has been localized for English, French, German, Italian, Spanish, Portuguese as used in Brazil, Chinese, Japanese, and Korean, Three other languages were briefly supported, Dutch, Portuguese for Portugal, and Romanian. The program can be run in these three languages, but many commands will be in English.

## 30.6 Redefining Menu Shortcut Keystrokes

Many TeXShop menu items have keyboard shortcuts. Eventually these shortcuts became a nightmare as users requested additional ones and no more shortcuts were available. Some shortcuts are really essential, like command-T for typesetting and command-+ and command– for changing the size of fonts. Others are used by many but not all users, and some are used by only a few.

Nowadays I ignore shortcut requests, letting users modify the shortcuts themselves if desired. There are now three ways to do this.

Starting with TeXShop 5.60, the easiest way is the item **Keyboard Shortcuts...** in the TeXShop menu, just below **Settings...**. It opens a window listing the menu commands with their shortcuts; a search field at the top finds a command by name. Select a command and press **Change...**, or double-click it. A small sheet appears; press the new shortcut and then **Apply**, or press Escape to cancel. A shortcut must use the Command or Control key, or be one of the function keys F1–F20. If another command already uses the shortcut, TeXShop says so and does not change anything. The buttons **Remove Shortcut**, **Restore Default**, and **Restore All...** remove a shortcut, restore the original shortcut of the selected command, or restore all of them. Changes take effect immediately and are kept when TeXShop restarts. They are stored in TeXShop's preferences under the key TSMenuShortcutOverrides, using names for the commands which do not depend on the language of the menus. Shortcuts for macros are still set in the Macro Editor, and key bindings which insert text are set in their own editor.

Keys used by macros take precedence. If a saved shortcut is already used by another command when TeXShop starts, for instance because a macro was given the same keys, the shortcut is kept but not used, and the window lists the command after **Shortcut Unavailable** at the top. **Restore Default** is refused while another command uses the original shortcut; change that command first. In the Preview window, command-F and command-G follow the shortcuts of **Find...** and **Find Next**; if one of them is removed, it no longer works in the Preview window either. In the sheet which records a shortcut,

Return applies the shortcut and Tab moves to the buttons. The list also contains commands which appear only when the Option key is held down, and some commands exist only in some localizations.

The two older methods described below still work. Shortcuts set in KeyEquivalents.plist become the defaults in the Keyboard Shortcuts window, and **Restore All...** does not change that file.

One method is provided by Apple in System Preferences under the Keyboard Module. The details which follow apply to Ventura and later, but similar methods are available for earlier systems. Select the Keyboard module and push the item *Keyboard Shortcuts...* On the left side, select *App Shortcuts*. In the resulting dialog, push the plus key. A new dialog appears. On the top line in the pull-down menu, select TeXShop. TeXShop may not be in the resulting list; in that case select *Other* at the end of the menu and then choose TeXShop. On the second line, type the exact name of the menu command you want to work with. This will be the menu name in the localization you use, not necessarily the English menu name. On the final line, type the desired shortcut. Press *Cancel* or *Done* to end the session.

Before Apple created this Preference Item, TeXShop provided a way for users to modify its shortcuts. This method still exists, but it is seldom used. In a pinch, you can try it.

To redefine keyboard menu shortcuts used by TeXShop, open the file `~/Library/TeXShop/Menus/KeyEquivalents.plist` with TeXShop, read the comments at the top of the file, and edit the file to redefine selected menu shortcuts. Be sure to edit and save in UTF-8 format if you use Unicode characters.

## 30.7 The Block Insertion Cursor

The macOS cursor can take many shapes: an arrow for pointing, a hand for scrolling in multiple directions, etc. In the editor, it becomes a blinking vertical line between letters, called the *insertion cursor*. Apple provides routines allowing developers to modify the appearance, size, and color of many of these shapes. Indeed in Apple's System Preferences, the Accessibility panel allows *Users* to change the size, outline color, and fill color of cursors. But the insertion cursor is special and these preference items do not change it. Also there are no developer routines to modify the insertion cursor.

A user with vision problems asked that TeXShop provide a block insertion character. Some rare programs like Terminal provide such a cursor, but no Cocoa routines mention it, and modifying Apple's NSTextEdit class to provide one would be a daunting task. On the other hand, I lost the central vision in one eye several years ago, so I was sympathetic.

TeXShop has two related features which make using the insertion cursor easier for many

users. The first is called *Highlight Current Line*. This feature can be turned on in the Source menu for the active document, and can turned on when documents first open in TeXShop Preferences under the Editor tab. When this feature is on, the line containing the insertion character is given a pale blue background. This color can be revised in Themes, where it is called a Syntax color named "Current Line".

Some authors like to issue a line feed after each source line, while other authors only issue a line feed at the end of paragraphs. So the line with the pale background may be a short single line, or a full paragraph.

For further emphasis of the insertion cursor itself, a second item named *Use Block Cursor* is available. This item is really a second mode for "Highlight Current Line", changing its default behavior. The two features cannot both be active at the same time.

To turn the block cursor on for all new documents, select it under the Misc1 tab in TeXShop Preferences. This will override the item "Highlight Current Line" in the Editor tab, which can be either on or off. To modify the block cursor using a menu item for the active document, toggle *Use Block Insertion Cursor* on or off in the Source Menu. Turning this item on will also turn on "Highlight Current Line". Thus if you later toggle the block cursor off, the current line will be highlighted instead.

The block cursor works by syntax coloring the backgrounds of the adjacent characters just before and just after the insertion line, while leaving the original insertion line to blink as before. It provides a color for this background which can be adjusted by a preference item in the Misc1 tab.

The block cursor cannot color the background of a character that does not exist. So if there is no character before the insertion character, only one character's background is colored, and if there is no character after the insertion character, only one character's background is colored. In particular, when starting a brand new line, nothing is colored and only the original insertion character shows. When editing existing text, both sides show and the insertion character is highly visible.

The features of this block cursor can be modified in TeXShop Preferences under the Misc1 tab. The defaults described above seem best to me. I recommend that users try this cursor and decide for themselves if it is useful or distracting.

# Chapter 31

# Spell Checking

## 31.1 Introduction

TeXShop uses Apple's NSTextEdit Cocoa Framework to provide editing in the Source Window. In typical Cocoa style, TeXShop adds or modifies features but an astonishing list of features come directly from Apple without any TeXShop code. This includes obvious features like copy and paste, and subtle features like the switch to entry from right to left when writing in Arabic, Hebrew, and Persian.

In particular, spell checking comes from Apple. macOS uses an array of built-in dictionaries, depending on the user's language, to identify incorrectly spelled words. It controls the GUI structures which reveal these words to the user and allow them to be corrected.

Unfortunately, standard LaTeX commands are not in these dictionaries and thus are marked as misspelled. There are several approaches to fixing this problem. Before describing these approaches, let us discuss the standard ways spell checking can be controlled in Cocoa.

Apple provides three ways to spell check text, and TeXShop inherits these three methods. The methods are activated in TeXShop's Edit menu, and some are selected by TeXShop Preference settings.

The first of these items is titled *Check Spelling*, and has a keyboard shortcut "command ;". When this combination is pressed, the first misspelled word is highlighted. Each additional press causes TeXShop to jump to the next misspelled word and highlight it. This spell check command is thus a glorified search in which only misspelled words are found.

A second way to spell check is to activate the menu item *Correct Spelling Automatically*. If this menu did not exist, the feature would be controlled in Apple's System Preferences,

where it is on by default. When it is on, your computer becomes a giant iPhone, standing behind you and changing what you type into what it thinks you ought to have typed. Instead of turning it on, TeX users are likely to want to turn it off. Therefore I added the menu *Correct Spelling Automatically* to TeXShop, and by default it is off.

The final way to spell check is to use the menu item *Check Spelling While Typing*. This item underlines misspelled words as they are typed, and the user can then go back and correct these words. This is how I spell check in TeXShop.

## 31.2 TeX-aware Spell Checking

TeXShop 5.60 adds TeX-aware spell checking, based on an implementation proposed by Ralph Martin. It is on by default, and can be turned off with the item **TeX-aware spell checking** under the Source tab of TeXShop Settings. The item does not turn on *Check Spelling While Typing*.

TeX-aware spell checking skips comments, mathematics, TeX commands, and the arguments of commands such as `\label`, `\ref`, `\cite`, and `\usepackage`, and checks the rest of the text. It works with the standard *Spelling and Grammar* panel, and with *Check Spelling While Typing*, which then does not underline the skipped parts of the source. The new rules apply to TeX source files and to new documents not yet saved; other kinds of files, including Sweave files, keep the older behavior described in the next sections.

Choose *Show Spelling and Grammar* in the Edit menu to open the panel. It goes to the first misspelled word after the cursor. Press *Find Next* in the panel, or choose *Check Document Now* (`Cmd-;`), to move to the next one. The word is selected and the window scrolls so that its line is in the middle. Hold down the Option key when choosing these items or pressing *Find Next* to use the system's own search, which does not skip any markup.

Macros defined in the document with `\newcommand`, `\def`, and similar commands are read, so an argument that a definition only passes on to `\ref`, to mathematics, and the like is skipped too. Text marked as another language with `\foreignlanguage`, `\selectlanguage`, the `otherlanguage` environment, and the like is checked in that language, but is not underlined while typing.

The panel has an additional field, *Replacement for Change All*, with a button *Change All*. Type a replacement there, or select a suggestion in the panel's list by clicking it or with the arrow keys, which copies the suggestion into the field. The document changes only when *Change All* is pressed; then every occurrence of the word in the text being checked is replaced, and the whole change can be undone in one step.

The item *Forget Learned Words...* in the Spelling and Grammar submenu of the Edit menu lists the words in the active document which were learned for the current language.

Selected words can be removed from the dictionary.

What is skipped is described by rules in the file

```
~/Library/TeXShop/SpellCheck/SpellingExclusions.txt
```

Open it with *Edit Spelling Exclusions...* in the Spelling and Grammar submenu, or with the button **Edit TeX exclusion rules...** in TeXShop Settings. The file explains its format at the top, and changes take effect the next time you spell check. For example, the following lines skip the first argument of a command `\mycommand` and the contents of the `proof` environment:

```
skip-args  1  mycommand
skip-body  proof
```

If the file is deleted, TeXShop creates a fresh copy of the default file the next time it starts.

The following sections describe the methods used before TeXShop 5.60. They still apply when TeX-aware spell checking is turned off, and to files that are not TeX source files.

## 31.3 Dictionaries

The Macintosh has many dictionaries; special commands choose the active dictionary. A "Dictionary" field in TeXShop Preferences under the Source tab has a pop-up menu which can be used to select the default dictionary. An unexpected feature is that dictionaries are listed using the ISO 639-1 and ISO 639-2 standards rather than the localized names shown in the "Show Spelling and Grammar" pane to be discussed later. These ISO values are easy to decipher. The preference menu also has an "Automatic" item in which the computer selects the appropriate dictionary based on the localization you are using.

When a new file is opened in TeXShop, it will be set to use the default dictionary. But this dictionary can be changed for that file by opening "Show Spelling and Grammar" in the Edit menu and selecting a new dictionary. In the small panel that opens, select a dictionary using the pull-down menu at the bottom.

A user who writes in English but corresponds with a French relative can easily do that when writing a note to that relative. A more unusual situation occurs if a user has several files open at once, some written in one language and some in another. Activate each source file by clicking on the text, and then select the dictionary for that file using the "Show Spelling and Grammar" panel. Click randomly on the various source files and notice that the dictionary field changes to the correct dictionary for each file.

Finally, if you intend to work on a file for an extended period of time and it does not use your default dictionary, the default dictionary for that file can be set with an instruction

at the top similar to

```
% !TEX spellcheck = de-DE
```

This particular document will then open with spelling set to German. But the "Show Spelling and Grammar" panel can later be used to switch dictionaries temporarily, in case a German letter contains an English quotation which needs to be spell checked.

After working on these features, I mentioned them to a user who told me in a disappointed tone that he really just wanted to return to the days when TeXShop entirely ignored the "Show Spelling and Grammar" panel and let it "do its thing." There is a special hidden preference for that user:

```
defaults write TeXShop OriginalSpelling YES
```

## 31.4 Spell Checking LaTeX Commands

This section is about code sent to me by Neil Sims, the Head of the Department of Mechanical Engineering at The University of Sheffield. It concerns the problem that most LaTeX commands are marked as misspelled. Thanks to Sims, TeXShop can now handle this problem — for some users — while using the standard Apple spell checker and standard Apple dictionaries.

What is the mechanism Sims uses to turn off spell checking? I wish I had thought of it. The text in the TeXShop source window is an "attributed string." This means it is an ordinary (often very long) string, with an additional data structure associated with the string that lists attributes like "text color" and "background color" for selected ranges of the string. Sims noticed that one of the available attributes is "do not spell check this selection." So Sims added lines to TeXShop's syntax coloring code which prevent the Mac from spell checking TeX commands and certain other words. This means in particular that the feature only works if syntax coloring is turned on.

In TeXShop Preferences under the Source tab, there is a box of selections labeled *Spell Checking*. These items are off by default. Leave them off if you use cocoAspell or any spell checking method except "Check Spelling While Typing." Starting with TeXShop 5.60, these items are used only when TeX-aware spell checking is turned off.

The first spell checking item turns off spell checking for all TeX command words like:

```
\documentclass, \usepackage, \begin, \alpha
```

The second item is more complicated and will be explained in the next paragraph. The third item turns off spell checking inside comments. Some users may write little essays as source comments and prefer to leave spell checking on for them.

Many TeX commands have optional parameters [...] and mandatory parameters {...}. The entries inside these parameters can also be specialized TeX words. That is true of the first example below, and it is annoying if the spell checker marks "parfill" and "parskip" as misspelled. On the other hand, in the second example the parameter is a user-supplied string which ought to be spell checked.

```
\usepackage[parfill]{parskip}
\emph{This remark has been made before}
```

The second item in the *Spell Checking* box of TeXShop Preferences turns on a somewhat crude method of handling both kinds of parameters. When this item is on, most TeX command parameters are spell checked. But TeXShop has an internal list of certain TeX commands whose parameters contain specialized words, and for these it turns off spell checking for the first two parameters (if they occur in the source).

This internal list, incidentally, is exactly the list marked for special handling by cocoAspell. But cocoAspell is more intelligent, and can mark which parameters to spell check and which to skip.

There is a hidden preference setting to extend the list of specialized TeX commands. The first command below adds one more element to the existing array of user supplied exceptions. The second command erases the array so the user can start over. Note that neither command affects TeXShop's default list of special commands.

```
defaults write TeXShop ExtraCommandsNotToCheckParameters -array-add "\\verbatim"
defaults write TeXShop ExtraCommandsNotToCheckParameters -array
```

There are some minor glitches with Sim's method. When a document is first opened, all TeX commands will be marked as misspelled. But a single click in the edit window will fix this problem. Similarly, while source is being typed, some commands may be marked as misspelled, but the mark will be removed when RETURN is pressed. Unfortunately, the new attribute is totally ignored by the "Check Spelling" search item, so it will not help when you go through the manuscript word by word looking for misspellings.

## 31.5 cocoAspell

cocoAspell is an alternate spell checker by Anton Leuski. Leuski's project allows users to replace Apple's own dictionaries with new dictionaries that can be made "LaTeX aware", while still using all the Apple spelling facilities. Leuski made the project open source recently; see https://github.com/leuski/cocoAspell. I think of the project as the "gold standard" for TeX-aware spell checking, and if you use it, then the *Spell Checking* preference items discussed in the previous section should be turned off.

cocoAspell can be difficult to install, and the latest commit to the open source project was

August 5, 2017. Leuski understandably stopped working on the project and nobody has stepped up to take it over. Users must then use the dictionaries supplied with cocoAspell, rather than dictionaries by Apple or others. Below are two documents from 2019, a short description by Koch, and a longer description on installing the code by Schulz. These are the latest documents I possess on cocoAspell.

## 31.6 cocoAspell Usage (Koch)

cocoAspell is a spell checking service which attaches to the build-in spell checker from Apple. It knows about LaTeX and does not mark LaTeX keywords as misspelled. Since it is not notarized, it cannot be provided in our install packages; obtain it directly from the author's web site.

The spell checker installs a Preference Pane named "Spelling" in Apple's System Preference, see Figure 31.1. After running the cocoAspell installer, open this pane and configure it appropriately.

The Pref Pane requires macOS El Capitan or higher, because Apple changed the requirements for Preference Panes just before El Capitan. The cocoAspell pane meets all current Apple standards.




Figure 31.1: Preference Pane "Spelling" in Apple's System Preference

On the left side, select those dictionaries which should appear in the Mac interface. For each

selected dictionary, turn on "Latex" and possibly "Context". Notice that all dictionaries are variants of English. If you use another language, go to the web page to obtain and install an appropriate dictionary for that language.

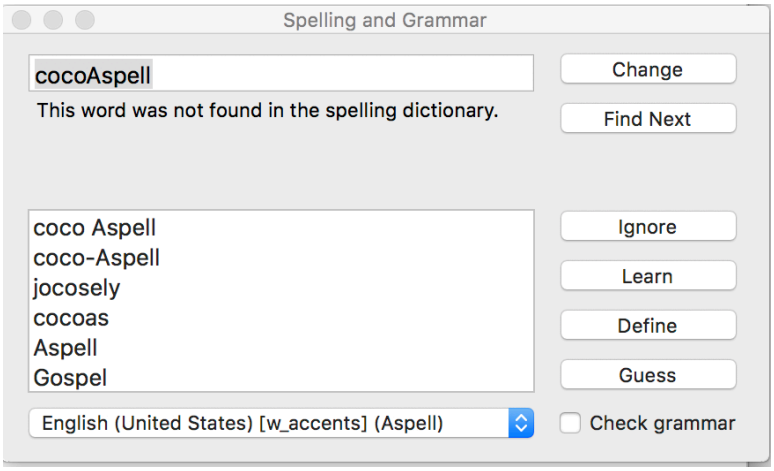



Figure 31.2: Spelling and Grammar Window

Now in the TeXShop Edit menu, show "Show Spelling and Grammar." A panel will appear, see Figure 31.2. At the bottom of this panel, select a cocoAspell dictionary instead of a built-in dictionary. Once it is selected, LaTeX keywords will not be marked as misspelled. Your editor should remember this choice across reboots.

## 31.7 cocoAspell Installing (Schulz)

### 31.7.1 Introduction

`cocoAspell` is a LaTeX-aware Spell Checker that operates through Apple's `Spell Checking` System. Version 2.5, for `macOS 10.11` (`El Capitan`) and later, is available at <http://people.ict.usc.edu/~leuski/cocoaspell/> or as part of `MacTeXtras` available at <http://www.tug.org/mactex/mactextras.html>.

If you've never installed `cocoAspell` before *and* you are using `OS X 10.13.x` (High Sierra) (or later?) the install may partially fail; the dictionaries are not completely built under that OS version (or later?). There is a fix, provided by Thaddeus Perala, but it involves using the Command Line (CL) tool called `make`.

### 31.7.2 First Check for `make`

First run the command

```
make --version
```

in Terminal. If make is already installed you will get a short paragraph that looks like

```
GNU Make 3.81
Copyright (C) 2006  Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.

This program built for i386-apple-darwin11.3.0
```

which means all is well and you can continue with the fix for the cocoAspell installation below.

If it comes up with a dialog box you don't have the CL tools installed. To install the CL tools click on the install button in that dialog box. Note: unless you *really* want to use XCode in the future don't click on the XCode button since that is a very large download and application while the CL tools take very little space. Once those tools are installed run

```
make --version
```

in Terminal again and you should get the short paragraph given above.

### 31.7.3 Building the cocoAspell Dictionaries

Once you know you have the make tool installed simply run the two commands

```
cd "/Library/Application Support/cocoAspell/aspell6-en-6.0-0"
sudo make install
```

in Terminal. You will have to give your administrator password to run the second command. After some output the prompt should return and you can open the cocoAspell Spelling pane in System Preferences to finish your configuration.

# Chapter 32

# Annotations

## 32.1 Annotations in General

TeXShop has the ability to open annotated pdf files and view the annotations, because PDFKit supports annotations and TeXShop can open arbitrary pdf files. Recently TeXShop gained the ability to *create* these annotations. The new feature has been designed in a non-intrusive manner. If the feature is not used, the only visible change in the interface is an additional item in the Preview menu named "Annotation Panel;" no new tools have been added to the toolbar and no additional code will run.

## 32.2 Who Defined Annotations?

The PDF document format was created by Adobe. The resulting API allows annotations to be added to the individual pages of such a document. These annotations become part of the pdf file and are shown by pdf display software conforming to the standards. Most annotations prescribed by Adobe are supported by PDFKit, Apple's pdf rendering framework.

Two typical pdf pages are shown in the following illustration. The page underneath was produced by pdfTeX from a standard LaTeX source file. Various annotations have been drawn on this page and are shown on the second page. Annotations can be added to the pages of a given pdf document by several programs, including Apple's Preview, Adobe Acrobat, Skim, and many others. The particular annotations on this illustration were created by TeXShop, and show the set of annotations it can provide. All are standard annotations which can be provided by the more sophisticated programs as well.

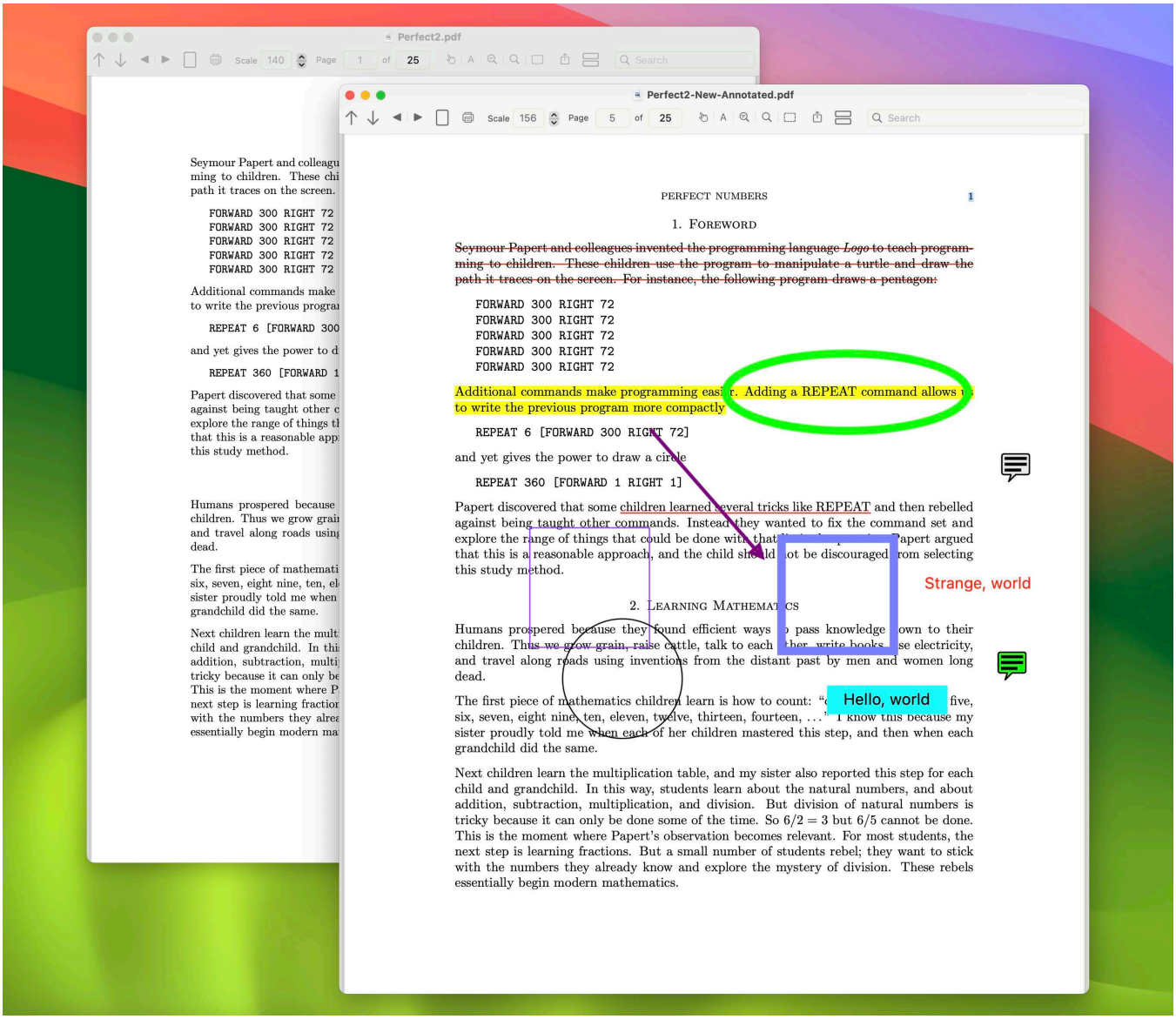



Figure 32.1: TeXShop Annotations

Notice that annotations are attached to individual pages of a document. Annotations are saved in the pdf file, but they do not appear in the LaTeX source of the file. This would not make sense, because material moves from page to page when source files are revised, but annotations are attached to particular pages and never cross page boundaries. Programs that annotate usually have access only to a pdf file and do not care how it was produced.

LaTeX files and pdf annotations are created in entirely different ways. Source files are written documents, composed on a keyboard and later typeset. But annotations are drawn. So in a vague sense, LaTeX documents are written by *authors*, but annotations are created by *editors*.

While only recent versions of TeXShop can create annotations, earlier versions can easily used by authors whose editors annotate their copy. Suppose I have written a document, and hand the pdf output file to an editor. The editor opens the pdf file in Preview, Skim, Adobe Acrobat, or some other program, and annotates the pages. The annotated pdf file is sent back to me. I open both my original document and the annotated pdf in TeXShop. Thus I see three windows: my source, the typeset output from the source with no annotations, and the annotated version. Looking at the annotated copy, I gradually revise and typeset my original document. The result is a pdf file without annotations which has been revised using the editor's annotations.

## 32.3 Using LaTeX to Annotate

When LaTeX authors hear about annotations, they often suppose that a LaTeX package could be created which adds annotations to the margins of the document. Then the third party annotation program could be eliminated and the original author and the author's editor could work entirely in LaTeX. There are indeed several packages of this form, and one of them may be exactly what you are looking for. We mention them briefly as an alternate approach.

For a package which mimics Adobe annotations, consider *todonotes, pdfcomment,* or *marginnote*. Also investigate the *marginpar* command in standard LaTeX.

Edward Tufte is a statistician famous for producing beautiful books which make statistical results easy to visualize. His books add extra comments in the margin, rather than using footnotes. The *tufte-latex* package in TeX Live and CTAN can help produce books with a similar design.

## 32.4 Why Add Annotations to TeXShop?

Since annotations can be created by many programs and then used in TeXShop, a reader might wonder why the ability to annotate within TeXShop has been added to the program. There are several reasons, none very profound. If only simple annotations are needed, it may be easier to stay in TeXShop than to switch to another program. Typical annotation programs provide great flexibility, but that can make them difficult to learn. In contrast, TeXShop provides only a few annotations and simple editing tools, so it is easy to master these tools in a few minutes. I'm hoping that some users learn about annotations with TeXShop, find the process interesting, and then investigate more powerful annotation programs.

Annotations created in another program can be edited in TeXShop. This may be a useful feature, although it comes with the warning that not all annotations from elsewhere behave well in TeXShop's editing mode. Experiment.

Finally, adding annotations was an interesting summer project for me.

## 32.5 Annotating in TeXShop

Activate TeXShop's Preview window. Suppose a particular page is visible in this window and you wish to add annotations to the page. Select the item "Annotation Panel..." in TeXShop's Preview menu. A small panel will appear, floating over the screen, This panel contains all tools needed to annotate. The eleven graphic items on the right side correspond to the eleven annotations which TeXShop can create. More about them in a minute.

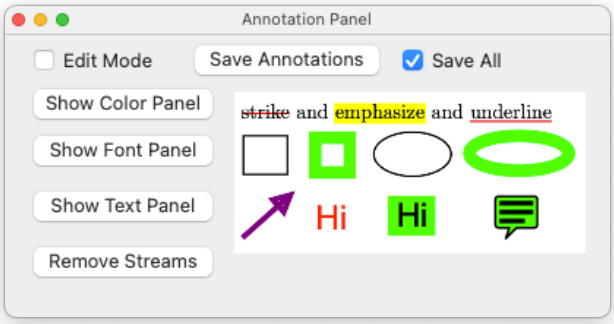



Figure 32.2: Annotation Panel

When annotations are being created in TeXShop, the user toggles the program between two modes known as “Edit Mode” and “Run Mode”. Run mode is the standard mode which TeXShop has used up to now. Annotations can be displayed in this mode, but not created. Edit mode is the mode which allows creation and manipulation of new annotations. To toggle between the two modes, click the top left “Edit Mode” button.

While this mode is off, click in the Preview window while holding down the Control key. The standard Preview contextual menu appears. TeXShop creates the first three items in this menu, but most items come directly from Apple. Repeat this procedure when Edit Mode is on. A different contextual menu appears, designed specifically for annotation. Some users may prefer annotating with this contextual menu. Experiment if you like. Annotating using this contextual menu will not be mentioned again in the manual.

The Adobe API for annotations states that each annotation must contain the name of the person who wrote it and the time when it was created. Therefore the recipient of an annotated document can determine who wrote each comment. This useful feature comes with a warning: if you tell your boss that you stayed up all night working on his document and you actually worked for half an hour, the document can reveal the truth.

## 32.6 Adding Annotations to a Page

The top three annotations are created by first selecting one or several lines of text. Next click on the words “strike”, “emphasize”, or “underline” to annotate the selection. If a selection involves several lines, each line becomes a separate annotation. Click on an annotated line to select it. Drag the mouse to drag the annotated portion left or right. Click on the small red box at the right side to resize the annotation, making the annotated portion shorter or longer. After you are satisfied, click elsewhere to deselect the new annotation.

Each remaining annotation is created as follows. If you like, select a small portion of text. Then click in the Panel on the annotation you want. This annotation will appear near your selection if you selected text first. Otherwise the annotation will appear near the bottom of the page. Click inside the new annotation and move it to its desired spot. Click on the small red rectangle at bottom right to resize the annotation. Resizing squares and circles creates rectangles and ovals. But if the Option key is down, resizing is constrained to only create larger or smaller squares and circles. To use this constraint, begin resizing as usual. Then press and hold down Option and the resized figure will immediately snap to become a circle or square.

In the above, click in the center of the icon in the Annotation Panel rather than near the boundary. Otherwise you may miss the area that registers the click.

The same dragging technique is used to move the arrow anywhere you like. In this case the small resizing rectangle is the head of the arrow and is not drawn. Click on the head and drag to fix the tail of the arrow and move the head anywhere you like. Selecting and moving arrows is slightly tricky; for details, read the section about arrows later in this chapter.

The last three annotations hold text. The first of these creates text with a transparent background; the second creates text with a colored background. The final element creates a popup symbol; it is often useful to drag this to a spot in the margin. During Run mode, a click on the popup icon will cause it to expand to a scrollable area containing text. In Edit mode, clicking on the icon while the Option key is down causes it to expand; you can then enter the initial text for the annotation.

Once a series of annotations are on the page, select one and press the tab key. This will cycle from annotation to annotation, selecting each in turn. To remove an annotation, select it and press the "delete" key.

As we later explain, the font and color used by the regular annotations showing text can be changed. This is not possible for popup annotations; the text size and font used when popups expand is fixed once and for all. But there is a useful trick which works both while creating popups in Edit mode, and reading them in Run mode. Select the text by typing command-A and then enlarge the text by typing command-= several times. This makes the text easy to read and edit. When the popup is closed, this size change is lost, so you must apply this trick each time you expand the popup.

The previous trick works on a U. S. keyboard, but fails on other keyboards. This problem can be fixed. The keystrokes call the menu items "Bigger" and "Smaller" in the Source menu's Font menu. Using System Settings, users of macOS can create their own keyboard shortcuts to menu items. See Section 6.2 and Section 30.6 of this manual.

Popups originally created a fairly small region to display text, but here there is good news. In Apple's operating system Sequoia and later, the region for text has been enlarged.

## 32.7 Editing Text in Annotations

We have already explained how to create the text in popup annotations. The text in the other two textual annotations is created using the panel which appears by pushing "Show Text Panel". A small editing panel will float over the screen. Every time one of the textual annotations is selected, the text of the annotation will appear in the Text Panel. Edit this text in the usual way, modifying words or adding new text. Push OK and the new text will appear in the selected annotation.

The text panel has three radio buttons at the bottom labeled "Left", "Center", "Right".

Select a button to determine the alignment of text in the Text Panel and corresponding alignment in the annotation.

The Text Panel contains *Rich Text*, which leads to an annoyance. In rich text, each paragraph can have its own alignment, so a centered paragraph could be followed by a left justified paragraph. We will not use this feature in TeXShop, but it causes a problem. When you select an annotation containing text, that text automatically appears in the Text Panel with the correct justification. If you click at the very end of this text and then begin adding extra lines, those lines may be left justified even if the original text is centered. To avoid this, click at the end of the text, and then delete one character. Type to add that character back and after that text entered in the panel will be correctly justified.

This trick is not needed if you click in the middle of the text to revise it, or if you select and delete all the old text and type entirely different text.

## 32.8 Changing Annotation Colors

To edit colors, push the "Show Color Panel" button in the Annotation Panel. A Color Picker will float over the page; move it near the bottom. To change the color of an annotation, select the annotation and then select a color in the Color Panel. The annotation color will immediately change. This process can modify colors of selection annotations, colors of rectangles and circles, colors of arrows, and background colors of textual annotations. To change the color of the actual text in such an annotation, hold down the option key when selecting the color.

Changing the color of a Popup annotation initially changes the color of its icon. But when the annotation pops up, the icon color becomes the color of the text background. Thus a very light gray is useful, while a dark color will make the text difficult to read. The color of the text itself cannot be changed.

The Color Picker has several modes of operation. Colors can be selected by picking a point in a circle filled with colors, or by adjusting sliders, or by selecting a particular crayon. Once you select a mode, the picker will use that mode whenever the Color Picker opens, until you again change modes.

## 32.9 Changing Annotation Fonts

Push the "Show Font Panel" button in the Annotation Panel to make a floating Font Panel appear; move it near the bottom of the page. This panel can change the font of the text with transparent background, and also the font of the text with colored background. First select the appropriate annotation, and then select a new font in the Font Panel.

# Chapter 33

# More on Annotations

## 33.1 Saving Annotated PDF Files

Recall that annotation data is not part of the source file. If you annotate a page of the preview output and then typeset again, the preview pdf will be replaced by a new typeset copy and the annotations will be lost. As we will see later, TeXShop has guard-rails to prevent this from happening, but these guards ultimately depend on saving your annotated pdf file.

To do this, push the "Save Annotations" button in the Annotation Panel. The panel will automatically switch to Run mode and a standard Macintosh Save Dialog will appear. This dialog will contain a default folder where the file will be saved, and a default file name for the file. The folder will be the folder containing your source document, so if you are working with a document titled "Fourier.tex", it will be the folder containing this file. If instead you opened a pdf file, it will be the folder containing that file.

The default name of the file being saved will be the name of the source followed by the phrase "-Annotated". So in the case of Fourier.tex, it will be Fourier-Annotated.pdf. This name can be modified to anything you desire. If a file already exists with the name you select, a warning will appear allowing you to choose a different file name, or overwrite the existing file.

If you choose a new name or new location to save the annotated file, the Save Panel will remember these changes the next time it opens. Hence if you save "Fourier-Annotated.pdf" as "Fourier-Annotated1.pdf", the next time the panel opens, it will suggest writing to "Fourier-Annotated1.pdf" and you can easily change that to "Fourier-Annotated2.pdf".

The entire pdf file, including all annotated pages, will be saved. But if you remove the check mark from the box titled "Save All" to the right of the Annotate button, only pages

containing annotations will be saved. This check mark is present when the Panel first appears because many users prefer to annotate, save, continue annotating, save again, etc., and obtain full documents with more and more pages annotated. But if you remove the check mark, it will remain off while the current document is still open, unless you turn it back on.

## 33.2 Removing Image Streams

Suppose you have an annotated pdf file like Fourier-Annotated.pdf, or perhaps an annotated document created by Adobe Acrobat, Skim, Preview, or something else. These files can all be opened in TeXShop, of course. Then by opening the Annotation Panel and switching to Edit mode, the annotations can be revised.

When annotated copies are saved to disk, it is common for the software to save an "image stream" for each annotation. This image stream is a sort of picture of the annotation, which can be displayed quickly and rapidly, without looking at the underlying data describing the annotation in detail. One advantage of this technique is that annotations created by one program on a particular computer can be opened accurately by a different program on a different computer.

But a disadvantage is that if you decide to edit these annotations, the images cannot be modified. They can only be resized and usually this process is constrained to preserve aspect ratio. So if you have an oval, for instance, and you resize, the resized oval may no longer touch all four sides of the bounding rectangle. It is impossible to change colors, or fonts.

There is a way to fix this problem. If an annotated pdf file is loaded, then soon after switching to Edit mode push the "Remove Streams" button. All image streams of annotations will then be removed, and annotations again become editable.

This trick is not important if the annotated file you create in TeXShop will be opened by someone who will only view the annotations. It is usually not important if the annotated file you create in TeXShop is opened by another program like Adobe Acrobat or Preview which can edit annotations (perhaps because these programs automatically remove image streams). But it is very important if the annotated file you create in TeXShop is being opened for further editing in TeXShop. Otherwise the editing process will be miserable.

## 33.3 Guard Rails

Recall that TeXShop automatically saves source files. For details, see the chapter of this manual titled *Automatic Saving*. Although TeXShop automatically saves source files, it is not involved in the process of saving pdf output files. In fact, these output files are created by TeX and LaTeX during typesetting and have already been written to disk by the typesetting software when TeXShop regains control of the computer. Since TeXShop is not involved in saving output pdf files, it does not automatically save any annotations users add to these files.

Therefore if pages are annotated and the user has not yet saved these annotations, several actions may cause the user to lose their annotations. These include typesetting the document again, closing the Preview Window, quitting the entire document being annotated, or quitting TeXShop altogether.

Therefore a series of guard rails have been added to the program to prevent these accidental losses of data. Recall that the annotation panel switches between two modes, "Edit Mode" and "Run Mode". If the user selects "Edit Mode" in the Panel, TeXShop decides that the user is adding new annotations or modifying annotations that already exist. So the next time the user typesets or does another of the dangerous actions listed earlier, the "Save Annotations" panel appears, allowing the user to save annotations. As soon as annotations are saved or the user chooses "Cancel", typesetting or the other action occurs. Since this is an attempt to save data, the entire annotated file is saved even if some pages are not annotated.

## 33.4 Interacting with Adobe Acrobat

Adobe Acrobat and Adobe Acrobat Reader run on most computer platforms, so when publishing companies want annotated copy from a writer, they often require annotations created on one of these programs. Preview is a standard program available on all Macintosh computers and can also create and edit annotations.

Both Acrobat and Preview can create files with additional annotations not provided by TeXShop. There is no problem viewing these files in TeXShop, but attempting to edit them in TeXShop can cause trouble. Experiment first if you intend to do this.

On the other hand, it is reasonable to create an annotated file in TeXShop and then edit that file in Acrobat or Preview, and possibly move the file back to TeXShop for further editing. We'll explain a few glitches, but this process generally works well.

When a file from TeXShop is opened in Acrobat or Acrobat Reader, it is not necessary to remove the streams first. Acrobat may complain about invalid encodings in certain fonts. It may also put up a warning with the text "Acrobat cannot open this file because a task

is still active in Acrobat." Both messages are harmless. Just save the file in Acrobat and proceed.

The file will look exactly the same in Adobe as it did in TeXShop. Now start editing. Selecting and resizing rectangles and circles with or without border accomplishes the same thing in TeXShop and Acrobat, although the method of grabbing and dragging the object is slightly different. This also holds for arrows. If the arrow is resized, the head of the arrow will change to a slightly different form. Popups work as before.

Moving and resizing text works as before with one exception. As soon as the text is touched, its alignment changes to "Left." I believe Acrobat and TeXShop set text alignment in a different way; I also suspect that the previous "task is still active" message is due to this difference. But there is a very easy fix. Double click the text annotation and a small window will pop up next to it. This window allows several features to be changed, but in particular a single click changes the alignment. After alignment is fixed once in Acrobat, it never breaks again.

That's it. Save any changes.

If an Acrobat-edited document is opened in TeXShop, it will look exactly like it did in Adobe. After switching to Edit mode, arrows may vanish. If that happens, select "Remove Streams". Arrows will immediately return, and with the TeXShop arrow head rather than the Adobe head. Editing in TeXShop then proceeds normally.

## 33.5 Interacting with Preview

Opening a TeXShop document in Preview does not require removing streams first. The document will initially look exactly like it did in TeXShop.

All objects can be selected and moved with results exactly the same in TeXShop and Preview, although the process of selecting and moving is slightly different. Resizing rectangles and circles with or without border works the same. Resizing text is slightly different because Preview only resizes horizontally. This resizing also changes the size of the text. A tool in the "Preview Annotation Toolbar" can easily resize the text if desired. (This tool can also change the font and color of the text.)

The main difference between TeXShop and Preview is their treatment of arrows. In Preview, arrows have three small selection points, at both ends and in the middle. Grab the beginning or end point to resize the arrow. As long as you do not touch the middle, all is well.

Save the file when you are done editing. Then open the file in TeXShop. It will look exactly the same. Change to Edit Mode and Remove Streams. Everything will work as before with one exception. It you select or resize text annotations, they will forget the font

choices and switch to default fonts. These fonts can be changed back to desired fonts in TeXShop.

Return to editing in Preview, and this time also resize arrows by moving the middle point. This creates curved arrows, a nifty touch. When the file is moved back to TeXShop, the curved arrows initially remain. But this time, changing to Edit mode makes the arrows disappear. Remove streams. The arrow still is gone. Select any annotation and then begin pushing the tab key. This cycles through all existing annotations. When the cycle reaches an arrow, at first you will see an empty box, but shortly after that the arrow will reappear. It has the same start and end, but is now straight. After this step, proceed normally.

## 33.6 History of the Annotation Code

The TeXShop annotation code is based on an Apple sample program named *PDFAnnotation Editor macOS Application* copyright 2017 and available from Apple. The code provided a foundation for this project, but has been considerably extended. TeXShop's annotation code would not have been possible without this start from Apple.

## 33.7 Arrows

Each annotation is determined by its bounds rectangle. In the case of an arrow, the start of the arrow will be at one corner of the rectangle and the end of the arrow will be at the opposite corner. A user resizes the arrow by resizing the bounding rectangle.

Arrow creation should be done as follows. Make an arrow. Then move the arrow until its starting point is at the desired location. Finally grab the tip and move it to where you want the arrow to end; the start of the arrow remains fixed during this operation.

In macOS, rectangles are determined by two pieces of data: the location on the page of the bottom left corner, and the width and height of the rectangle as measured from this corner. Therefore if an arrow points toward the first quadrant, there is a very simple correlation between the coordinates of the beginning and end of the arrow and the corresponding bounds rectangle.

But when the arrow is resized and its end moves to another quadrant, the origin of the bounds rectangle is no longer the beginning of the arrow and a complicated new formula is required to get from the arrow to the corresponding bounds rectangle.

Unfortunately, the definition of rectangles in macOS requires that height and width be nonnegative. But when I wrote the code, I cheated and assumed that negative heights and widths would be accepted. This worked and vastly simplified the resize code.

Currently TeXShop switches the bounds rectangle to "regular legal form" when arrows are

deselected, and switches the arrow to "easy to use slightly illegal form" when arrows are selected for editing.

Because of this design, moving and resizing arrows should be done as follows. To move an arrow, first click to select the arrow but avoid any dragging component during the click. Then move the arrow by pushing the mouse button and holding it down during the motion. If these steps are not followed, the arrow may jump when it is first clicked. Nothing is seriously wrong, so recover and continue editing.

## 33.8 Don't Do This

Each document has its own individual annotation panel, so if several projects are open and each is being annotated, confusion will ensue. Don't do this.

# Chapter 34

# Fermat as Annotator

**Warning:** I'm tired of writing this endless manual. So I'm going to write about mathematics for a while; this chapter is my only chance. The entire chapter can be skipped.

Why would a mathematician care about annotations? There is a famous case when a great mathematician's number theoretic legacy consists of annotations rather than books. That mathematician is, of course, Fermat, who left annotations in his copy of *Arithmetica*, a book written by Diophantus of Alexandria sometime around 260 A.D.

Fermat also left letters to Mersenne and others. But Fermat had difficulty writing up his proofs, and at one point tried to induce Pascal to help, without success. Fermat's work only survives because after his death, his son published a copy of *Arithmetica* which contained Fermat's annotations. The book sat on dusty shelves unnoticed until Euler asked for a copy after hearing about some of Fermat's conjectures.

Number theory is a tricky subject, because it is difficult to distinguish easy results from profound theorems. Fermat understood the distinction in his bones. So if we are going to work with annotations, let us celebrate Fermat by describing some of his results.

We'll begin with an easy example not by Fermat, to make the contrast with the depth of his results clear. Suppose a calculation by hand produces

$$247,397,414 \times 525,996,123 = 130,130,080,604,225,922$$

We want to check this calculation using a method which will often catch errors. One possible check is to look at the last digits of the two products, 4 and 3. Their product is 12, so the last digit of the product must be 2, as it is. But this isn't a very good check because it will only catch errors which occur in the last digit. There is a similar method which can catch errors at any spot.

Any large positive integer can be converted to a single digit by "casting out nines:" add all the digits, add all the digits of the new sum, repeat until the answer has only one digit. For instance,

$$247,397,414 \to 41 \to 5 \text{ and } 525,996,123 \to 42 \to 6$$

$$130,130,080,604,225,922 \to 48 \to 12 \to 3$$

The checking method in question says that the product of the two numbers obtained by casting out nines from the terms of the original product, $5 \times 6 = 30 \to 3$, must equal the number obtained by casting out nines from the answer, also 3. If these numbers disagree, the original multiplication was wrong.

To prove this, consider the quotient ring $\mathbf{Z}_9$. The nine elements of this ring are represented by 0 through 8, but since 0 is equivalent to 9, so we can also represent these elements by 1 through 9. The natural map $\mathbf{Z} \to \mathbf{Z}_9$ is given by casting out nines because 10 is equivalent to 1 in $\mathbf{Z}_9$. For example, $736 = 7(10)^2 + 3(10) + 6 \sim 7 + 3 + 6 = 16 = 1(10) + 6 \sim 1 + 6 = 7$. The check by casting out nines works because the natural map $\mathbf{Z} \to \mathbf{Z}_9$ preserves addition and multiplication.

## 34.1 A Difficult Theorem of Fermat

Fermat is known for many beautiful and important results: Fermat's little theorem, Fermat's two squares theorem, Fermat's last theorem, and so forth. In particular, the last theorem asserts that $a^n + b^n = c^n$ cannot be solved in positive integers when $n > 2$. It is enough to prove it when $n$ is an odd prime and when $n = 4$. Fermat claimed only once to have a general proof. But he often challenged others to prove the case $n = 4$. His proof of this case survives, as we will show shortly. The general result was finally proved in 1994 by Andrew Wiles.

Fermat left only two proofs. We will describe both of these proofs below. Euler had to reprove Fermat's other assertions from scratch one hundred years later, and the proofs were not easy to find.

A Pythagorean triangle is a triangle with integer sides $a, b, c$ which satisfies the Pythagorean theorem, so $a^2 + b^2 = c^2$. The area of such a triangle is $\frac{ab}{2}$. This area is always an integer because if $a$ and $b$ are both odd, then in $\mathbf{Z}_4$ both $a^2$ and $b^2$ will equal 1 and $c^2 = a^2 + b^2$ will equal 2. But no element of $\mathbf{Z}$ has square 2.

Fermat proved that the area of a Pythagorean triangle can never be a perfect square. For instance, $3^2 + 4^2 = 5^2$ and this triangle has area 6, which is an integer but not a square.

At first glance Fermat's area theorem looks like it would be easy to prove, and this seems even more likely when we recall that Euclid already had a formula which gave all possible

$a, b, c$. Indeed suppose $u > v > 0$ are integers and let $a = u^2 - v^2, b = 2uv, c = u^2 + v^2$. It is easy to check that $a^2 + b^2 = c^2$. The area of such a triangle is then $\frac{ab}{2} = \frac{(u^2-v^2)(2uv)}{2} = (u^2 - v^2)uv$. Fermat is claiming that $(u^2 - v^2)uv$ can never be a perfect square.

But it seems possible that any high school student could show otherwise. Let's try. We first make $u$ and $v$ squares, say $u = r^2$ and $v = s^2$. Then we only have to choose $r$ and $s$ so $(r^4 - s^4)r^2s^2$ is a square, and it is enough to make $r^4 - s^4$ a square, say $r^4 - s^4 = t^2$ or equivalently

$$r^4 = s^4 + t^2$$

Fermat's area theorem shows that this is impossible. If so, it is certainly impossible if we choose $t$ to be a fourth power rather than a square, so the equation $r^4 = s^4 + t^4$ has no solutions in positive integers. This is Fermat's proof of Fermat's last theorem when $n = 4$, the other Fermat proof that has come down to us.

## 34.2 Euclid's Formula

We call a Pythagorian triangle *primitive* if no prime divides all three sides. In that case, no prime can divide a pair of sides because once $p$ divides two sides, it divides all three. It is enough to find the primitive triples because all others are found by magnifying by a constant.

Earlier we proved that if $u > v > 0$ and set $a = u^2 - v^2, b = 2uv$, and $c = u^2 + v^2$, then $a^2 + b^2 = c^2$. Euclid also proved the converse: in an appropriate sense, all Pythagorean triangles have this form.

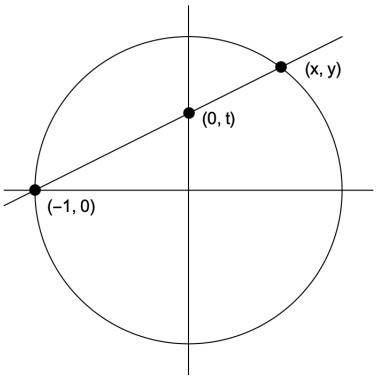



Figure 34.1: Euclid's Formula

If $a^2 + b^2 = c^2$, then $(x, y) = \left(\frac{a}{c}, \frac{b}{c}\right)$ is a point on the unit circle and both coordinates of this point are rational. We can compute the equation of the straight line from $(-1, 0)$ to $(x, y)$ by straightforward algebra; since all numbers are rational, the resulting equation

has rational coordinates. This line passes through the vertical axis, and the intersection coordinate $(0, t)$ must be rational.

It is easy to find a formula for $(x, y)$ in terms of $t$; indeed $(x, y) = \left(\frac{1-t^2}{1+t^2}, \frac{2t}{1+t^2}\right)$. Since $t$ is rational, write $t = \frac{v}{u}$ in lowest terms. Clearly $0 < t < 1$, so $u > v > 0$. Easy substitution gives

$$\left(\frac{a}{c}, \frac{b}{c}\right) = \left(\frac{u^2 - v^2}{u^2 + v^2}, \frac{2uv}{u^2 + v^2}\right)$$

From this, to prove that $a = u^2 - v^2, b = 2uv, c = u^2 + v^2$ it suffices to show that the fractions on the right side are written in lowest terms.

If a prime divides both $u^2 - v^2$ and $u^2 + v^2$, it divides their sum $2u^2$ and their difference $2v^2$. So either $p = 2$ or else $p$ divides both $u$ and $v$. The second case is impossible because we wrote $\frac{v}{u}$ in lowest terms. We conclude that either $a = u^2 - v^2, c = u^2 + v^2$, or $a = \frac{u^2-v^2}{2}, c = \frac{u^2+v^2}{2}$.

If a prime divides both $2uv$ and $u^2 + v^2$, then $p = 2$ or $p$ divides $u$ or $p$ divides $v$. But since $p$ divides $u^2 + v^2$, if it divides one of $u$ and $v$, then it divides them both. This is impossible because we wrote $\frac{v}{u}$ in lowest terms. So again the only problem is $p = 2$. We conclude that either $b = 2uv, c = u^2 + v^2$ or else $b = \frac{2uv}{2}, c = \frac{u^2+v^2}{2}$.

Putting these results together, either $a = u^2 - v^2, b = 2uv, c = u^2 + v^2$ or else $a = \frac{u^2-v^2}{2}, b = \frac{2uv}{2}, c = \frac{u^2+v^2}{2}$. But we are assuming that $a$ is odd and $b$ is even. If $b = \frac{2uv}{2} = uv$ and $b$ is even, then 2 divides $u$ or $v$. Since $\frac{u^2+v^2}{2}$ is an integer, 2 divides $u^2 + v^2$, so if it divides one of $u$ and $v$, it divides the other. This is impossible because $\frac{v}{u}$ is in lowest terms. Euclid's result is proved.

## 34.3 Progressions of Squares

We also need a beautiful result well-known in the medieval world, but somehow forgotten today. It is occasionally possible to find three squares $a^2 < b^2 < c^2$ such that $b^2$ is exactly in the middle between the other squares. We call this a *progression of squares*. Two examples are $1 < 25 < 49$ and $49 < 169 < 289$. We'd like to describe all such progressions.

High school algebra students know that $(a+b)^2 = a^2+2ab+b^2$ and $(a-b)^2 = a^2-2ab+b^2$. We can use this to find examples of progressions of squares. Choose positive integers $a > b$ and write

$$(a - b)^2 < a^2 + b^2 < (a + b)^2$$

This is almost a progression of squares because the two ends are square and these ends differ from the middle by the same $2ab$. To complete the process, we need only choose $a$

and $b$ so the middle is a square:

$$a^2 + b^2 = c^2$$

In this astonishing way, the theory of progressions of squares leads right back to the theory of Pythagorian triangles! Specifically, if $a^2 + b^2 = c^2$ gives a Pythagorian triangle, then $(a-b)^2 < c^2 < (a+b)^2$ is a progression of squares. For instance the 3-4-5 triple gives $1 < 25 < 49$ and the 5-12-13 triple gives $49 < 169 < 289$.

Notice that the common distance between adjacent terms in the progression is the distance from $(a-b)^2 = a^2 + b^2 - 2ab$ to $c^2 = a^2 + b^2$, and so just $2ab$. This is four times the area of the triangle.

All progressions of squares arise in this way. If $e^2 < f^2 < g^2$ is a progression of squares, we want to find a triangle $a^2 + b^2 = c^2$ such that $e = a - b, f = c, g = a + b$. Solving gives $a = \frac{g+e}{2}, b = \frac{g-e}{2}, c = f$. Since we started with a progression of squares, $f^2 - e^2 = g^2 - f^2$ and their sum $g^2 - e^2 = (g-e)(g+e)$ is even. So at least one of $g - e$ and $g + e$ is even. If one is even and one is odd, then their sum $2g$ is odd, but clearly $2g$ is even. So both are even and $a, b$ are integers. Finally, $a^2 + b^2 = \left(\frac{g+e}{2}\right)^2 + \left(\frac{g-e}{2}\right)^2 = \frac{g^2+e^2}{2}$. Let $d$ be the distance from $e^2$ to $f^2$, and thus also the distance from $f^2$ to $g^2$. Then $g^2 = e^2 + 2d$ and $\frac{e^2+g^2}{2} = \frac{2e^2+2d}{2} = e^2 + d = f^2$. Since $c = f$, $a^2 + b^2 = c^2$.

## 34.4 The Method of Descent

Although Fermat did not publish most proofs, he did describe his basic technique in letters to Mersenne. Fermat called the technique "the method of descent". To prove his result about areas of Pythagorian triangles, Fermat demonstrated that if he had one Pythagorian triangle whose area is a perfect square, he could use that triangle to find another such triangle with smaller area. Repeating the process again and again would lead to an infinite sequence of triangles, and their areas would be an infinite sequence of decreasing positive integers. No such sequence can exist, so the triangle which started the process cannot exist.

The method of descent was used by Fermat to prove other results. Each of these proofs requires a revision of how descent will be used, followed by a very difficult proof that the required descent can be done. So Fermat's hint of how to proceed was only useful for the very best mathematicians.

## 34.5 Fermat's Argument by Descent

We now come to Fermat's crucial argument by descent. Suppose $a^2+b^2=c^2$ is a Pythagorian triangle with square area. We need to find another such triangle with smaller area. If our triangle is not primitive, this is trivial; choose a prime $p$ which divides all three sides and use the triple $\left(\frac{a}{p},\frac{b}{p},\frac{c}{p}\right)$. The resulting triangle has area $\frac{ab}{p^2}$. This is an integer because it is the area of a Pythagorian triangle, and it is a square because $ab$ and $p^2$ are squares. So we have descent.

Otherwise our triangle is primitive and by switching sides if necessary we can assume that $a$ is odd and $b$ is even. By Euclid's theorem, we can write

$$a=u^2-v^2, b=2uv, c=u^2+v^2$$

where no prime divides both $u$ and $v$. The area of the triangle is then

$$\frac{ab}{2}=(u^2-v^2)uv=(u-v)(u+v)uv$$

This area is a square. If we factor each of the four terms into primes, we will get a factorization $p_1^{k_1}p_2^{k_2}\dots p_r^{k_r}$ in which all exponents are even.

We will show that no prime divides two of the factors, so each separate factor must be a square. Indeed the only tricky case is when a prime $p$ divides both $u+v$ and $u-v$. Then it divides their sum $2u$ and their difference $2v$. So either $p$ divides both $u$ and $v$, which is impossible by Euclid's result, or $p=2$. But one of $u$ and $v$ is even and one is odd, so 2 cannot divide $u-v$ or $u+v$.

Consider the progression $u-v,u,u+v$. Each of these terms is a square, so they form a progression of squares. By our results on such progressions, this progression is associated with a Pythagorian triangle. That triangle is the triangle Fermat will use to descend. The common distance between adjacent squares is $v$. This is four times the area of the new triangle, so the new triangle has area $\frac{v}{4}$, which is certainly smaller than the area $(u-v)(u+v)uv$ of the original triangle. Since the new triangle has integer sides, its area is an integer. Since both $v$ and 4 are squares, the area of the new triangle is a square. The proof is complete!

## 34.6 Generalizing Fermat's Area Theorem

A *rational* Pythagorean triangle is a Pythagorean triangle $a^2 + b^2 = c^2$ in which $a, b$, and $c$ are allowed to be rational numbers rather than just integers. If we start with an integer Pythagorean triangle, we can get a rational Pythagorean triangle by multiplying all three sides by a rational $q$. Conversely, we can find a rational $q$ which converts any given rational Pythagorean triangle into an integral Pythagorean triangle by multiplying all sides by $q$.

If we start with a rational Pythagorean triangle and multiply all sides by $q$, the area of the triangle changes from $\frac{ab}{2}$ to $\frac{ab}{2}q^2$. Consequently, areas of rational Pythagorean triangles are unique up to multiplication by rational squares. Any rational area is equivalent in this way to a unique positive square-free integer. Consequently, Fermat's area theorem has a natural generalization: Which positive square-free integers can be areas of some rational Pythagorean triangle? Numbers which can be areas in this way are called *congruence numbers*.

The area of a triangle is equivalent to a congruence number $C$ if and only if it equals $C$ times a perfect square. Fermat's theorem is equivalent to the statement that 1 is not a congruence number. Fermat also proved the 2 and 3 are not congruence numbers. Another way to say this is that the area of an integral Pythagorean triangle cannot be 2 times a square, and cannot be 3 times a square.

The sequence of possible square-free integers is

$$1, 2, 3, 5, 6, 7, 10, 11, 13, 14, 15, 17, 19, 21, \ldots$$

By Fermat, 1, 2, 3 are not congruence numbers, and of course 6 is a congruence number because it is the area of the $3 - 4 - 5$ triangle.

Curiously, 5 is a congruence number, a result already known before Fermat. It is the area of the rational triangle $\left(\frac{20}{3}, \frac{3}{2}, \frac{41}{6}\right)$. Another way of saying this is that the Pythagorean triangle $(40, 9, 41)$ has area $80 = 5 \cdot 6^2$.

The first few congruence numbrs are $5, 6, 7, 13, 14, 15, 21$ and thus the first few noncongruence numbers are $1, 2, 3, 10, 11, 17, 19$.

See https://kconrad.math.uconn.edu/blurbs/ugradnumthy/congnumber.pdf for a wonderful article on congruence numbers.

In 1983, Tunnell found an algorithm to determine whether a given square-free number is a congruence number. If his algorithm says that it is not, then it definitely is not. But if the algorithm says that it is a congruence number, the algorithm requires the Birch and Swinnerton-Dyer conjecture is show for certain that it is a congruence number. This conjecture is one of the seven million dollar Field's prizes offered and still without a proof.

# Chapter 35

# Writing Tools

Writing Tools is an Apple Intelligence feature of macOS 15, Sequoia. It is not present in the initial release of Sequoia, but will appear in a later release.

To activate Writing Tools, select one or several paragraphs of the source code. Then locate the Writing Tools submenu in the Edit menu and select "Show Writing Tools". The panel shown below will appear next to the source window.

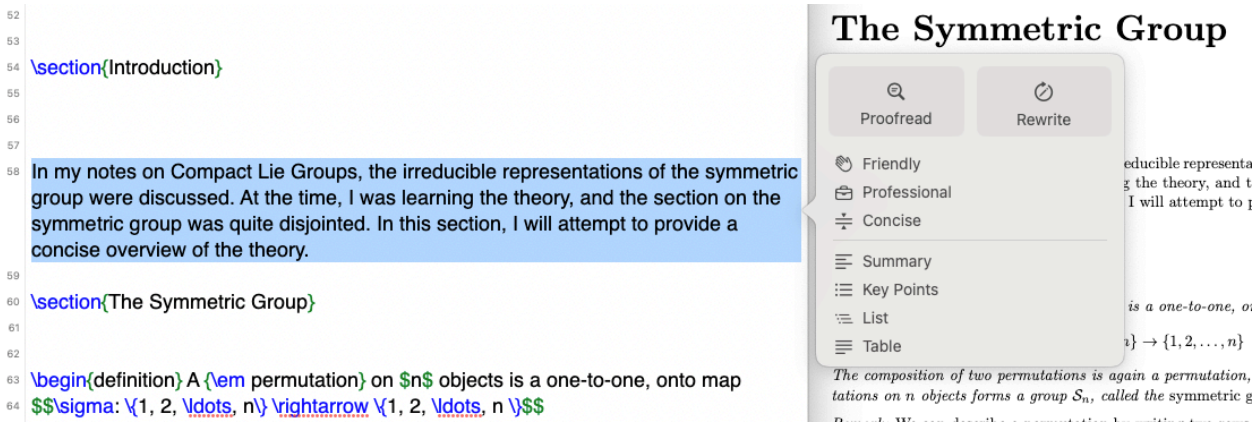



Figure 35.1: WritingTools

Writing Tools is also available in the source window's contextual menu; control-click in the source window to reveal this contextual menu.

An even easier way to activate Writing Tools is to select a portion of the source text and then move the mouse to the line number column near the top of your selection. A blue dot will appear; click this dot.

The Writing Tools items described above do not appear when TeXShop runs on an earlier system or on the initial release of Sequoia. When Apple updates Sequoia, Writing Tools will appear without updating TeXShop. The various Writing Tools interface items are supplied by macOS and are not part of the TeXShop code.

Writing tools offers to rewrite the selected section of source text in one of several styles. It can make the source more informal, or more professional, or more concise. The original text will be replaced by rewritten text; Writing Tools allows you to accept this change or revert back to the original version.

Earlier versions of macOS offered to automatically revise spelling as you type. That system often made wrong guesses about what was being typed, converting text into a mishmash of strange words and Freudian slips. Luckily the feature could be turned off, both in System Settings and directly in TeXShop.

Writing Tools is not like that. It improved several samples I gave it, and otherwise left text alone. This was quite a surprise to me, and I suspect others will find the feature useful as well. Only time will tell as Apple makes the feature widely available.

Just for the heck of it, I asked Writing Tools to rewrite an entire 40 page LaTeX document about irreducible representations of symmetric groups. The document had Young diagrams, illustrations, and very technical mathematical sections. And of course the source contained a large number of LaTeX commands. Apple Intelligence seems to have been trained on LaTeX documents, because most LaTeX commands were left unchanged.

After the revision, the document no longer typeset. But it took less than fifteen minutes to fix these problems. A typical problem was that \begin{theorem} was left alone but \end{theorem} was removed. Surprisingly, all Young diagrams and all illustrations were left unchanged, and almost all technical mathematical material remained as before. Writing Tools mostly changed informal discussions in the text. Apple surely doesn't expect users to put entire LaTeX articles through the Writing Tools wringer, but the experiment shows that it is more capable than you might expect. It will be interesting to see what others think when the code has wider release among LaTeX users.

# Chapter 36

# Japan

For reasons I do not understand, a series of very important TeXShop code contributions have come from Japan. The OgreKit Find Panel is a project by Isao Sonabe used by several open source programs. It provides support for regular expressions. The Macro support comes from Mitsuhiro Shishikura, who also created the magnifying glass and the original Display and Magnifying Modes for the Preview Window. More recently, Yusuke Terada contributed the Character Info panel to obtain Unicode information, the invisible character support, several different ways to simplify entry of matching parentheses during text entry, and other features.

All of these people sent me code with only a few connection points, and then sent very detailed instructions. Ten minutes after receiving an email, their code would be connected up and working in my copy of TeXShop!

There are special problems when typesetting Japanese text, and I understand only a tiny fraction of what is required. I first heard of problems when Japanese students at the University of Oregon came to my office with bug reports about TeXShop. These reports made no sense, and seemed to involve the yen symbol. The students were emissaries rather than users and didn't have details.

Eventually I learned the story, which is amusing. When Apple originally sold Macs in Japan, they used the standard U.S. keyboard. There was one problem. Even if the owner typed in English, a yen symbol was essential. So Apple used the backslash key, changing the symbol on the key top to a yen, and changing the backslash to a yen in all the fonts. This caused no problem in LaTeX because the input code was still ascii for backslash.

Then later, Apple grew more conscientious about unicode, and now their keyboards really output a Unicode Yen. Luckily, Donald Knuth had not hardwired the backslash character into TeX, but had made that character configurable. Since the Japanese were used to using

the yen symbol, they just recompiled TeX so commands started with yen. However, this required that all of the style and package files be slightly edited to also use a yen.

Because of the strong input from Japan, TeXShop has a few Preference items which only affect users in Japan. I do not understand these items. For instance, the list of invisible characters which can be shown contains a "Japanese space". The Misc2 tab has an item about pTeX (for Japan), and an item labeled "Automatic UTF-8-Mac to UTF-8 Conversion". The TeX Live Ghostscript installer also installs a few special files for CJK users. Other users can ignore these items.

Starting with TeXShop 5.60, when a file is saved in a Japanese encoding, the backslash and the tilde are always written as the bytes 0x5C and 0x7E which TEX expects. Before, a backslash or tilde typed in a file using the MacJapanese or Shift JIS X0213 encoding could be saved as a full-width character or as the byte 0x80, and a yen sign typed as a backslash in a file using Windows Shift JIS, EUC-JP, or ISO-2022-JP could be saved as a question mark.

# Chapter 37

# Updating TeXShop

TeXShop can be updated using the Sparkle mechanism described at the end of chapter 20. It can also be updated by going to the TeXShop web site, https://texshop.app/. Users of TeXShop 5.58 and earlier, whose updates came from Richard Koch's former web site https://pages.uoregon.edu/koch/texshop/texshop.html, must install a version from the new web site once by hand; later versions are then updated automatically. CTAN, https://ctan.org, has the versions of TeXShop released by Richard Koch; on the main page, search for "texshop" and go to the "package texshop".

After updating, there are occasionally final steps that only a user can do. If an active engine is updated, TeXShop cannot automatically update it because the user may have modified that engine. If new macros are introduced, TeXShop cannot add it to the Macros list because the user may have edited that list.

There is a document at the top of the TeXShop Help menu named *About This Release*. This document describes any final steps required when updating to the current release. Usually nothing is required. Users should get in the habit of glancing at this document after each update.

The second item in the TeXShop Help menu is *Changes*, which lists all changes in the new update, and in previous updates. Read the top of this document to learn of new features after updating. This document is also on the TeXShop web site if you'd prefer to read about changes before updating.

The complete TeXShop source code is available on GitHub:

```
https://github.com/TeXShopProject/TeXShop
```

On very rare occasions, users will update and the new version will be unresponsive, or have massive bugs. If this happens, it is useful to reboot your machine and then try the update.

This often fixes all problems. If not, the problem is almost always caused by a glitch in TeXShop Preferences; this can happen when new items are added to Preferences. (Even then, the vast majority of users have no problems.)

The solution is to rewrite the TeXShop Preferences data file from scratch. Do this in the following order:

- Quit TeXShop.
- Go to `~/Library/Preferences`, find the file *TeXShop.plist*, and drag this file to the desktop.
- Restart TeXShop.
- Reset any Preference changes you might have made previously.

If this step doesn't fix the problem and you want to return to your earlier Preference settings, you can do so using the saved copy on the desktop. But reinstalling this copy requires care because the system caches the Preference file for extra startup speed. Proceed as follows:

- Quit TeXShop.
- Replace the file "TeXShop.plist" currently in `~/Library/Preferences` with the old backup copy from the desktop.
- In Terminal type

```
defaults read ~/Library/Preferences/TeXShop.plist
```

- Restart TeXShop.

# Chapter 38

# Opening Encrypted PDF Files

Both Apple's Preview program and the full version of Adobe Acrobat can encrypt pdf files so they can only be read after supplying a password. Using this feature, authors can distribute documents to selected readers without making them available to everyone on the internet.

TeXShop can display such documents. When an encrypted document is opened, the window for the document contains only a password entry field. After the user types an appropriate password into this field, the document opens for reading. Surprisingly, TeXShop contains no code to make this happen. The initial window with a password field is created by Apple's Cocoa, and after a password is entered, decoding the pdf code as it is read from disk is handled automatically by Cocoa. When the feature was added by Apple, I didn't even know about it until a user commented on this new TeXShop feature.

There is one small problem. Users who receive such a password seldom memorize it because passwords are deliberately hard to remember. Therefore users often create a file listing pdf passwords they use often. Of course this is a threat to security.

For many passwords, Apple has a solution to this problem. The first time a user enters a password, the computer offers to add the password in the user's keychain, where it is encrypted and saved to the cloud. The next time this user (who has logged into the machine using his or her computer password) needs to enter a password, the Macintosh can supply that password automatically from their keychain.

Unfortunately, Apple has not extended this feature to passwords for encrypted pdf files. Perhaps in the future ...

Until that moment, there is a convenient technique to record passwords of documents used often in TeXShop. In the Macros menu, select the item "Open Macro Editor". An editing

window will appear listing the existing macros on the left, and their coding on the right. We are going to add a new macro which will fill in the appropriate password. Click on the item just above the spot where this new macro should appear, and then select "New Item".

On the right side of the editing window, give the item a reasonable name. In the content region just below, type the following text:

```
--AppleScript direct

    tell application "System Events" to keystroke "password"
    tell application "System Events" to keystroke return
```

The first line must start immediately without extra space at the left margin. Change the text *password* to the actual password of the file. Then select "Save" in the Macro Editor.

When you open the password-protected file associated with this password and the password entry field appears, reach up to the Macro menu and select this macro.

At first the Macro will fail because you do not have permission to send events to the System using AppleScript. The Macintosh will report an error, but then it will sometimes open System Settings to the module where appropriate permissions can be entered. If this does not happen, open System Settings, select the module "Privacy & Security", and in the "Automation" section of this module add TeXShop and turn on "System Events".

# Chapter 39

# Left Over Items

## 39.1 Menus

**Open from Stationery**

This is the second item in the File menu. It provides an easy way to start a new document. Select the type of document you want to write and a new window will appear filled with the correct starting boilerplate code. This item is customizable using `~/Library/TeXShop/Stationery`. To create or edit a piece of stationery, put the initial document in this folder with a name suggesting its purpose. Suppose this was Letter.tex. Create a second file named Letter.comment with a sentence explaining the purpose of the document. The comment will appear in the initial dialog when "Open from Stationery" is chosen, and a copy of the stationary will be created. When you typeset, you'll be asked to give a location where it will be stored.

**Save** This item appears in the File menu, but many users want **Save As ...** Hold down the option key and this new item will be added to the File menu.

**Convert Tiff** This item appears in the File menu. Pdflatex can accept pdf, png, and jpeg illustrations. In modern versions of TeX Live, it can also accept eps illustrations because the system automatically converts them to pdf during typesetting. This leaves tiff as an unsupported file type. This command converts tiff files to png format.

**Line Number** This command in the Edit menu brings up a small dialog. Type the line number of the source line you'd like to go to, and the command will take you there.

**Close Current Environment**

Suppose you have typed

```
\begin{itemize}
\item one
\item two
```

Select the menu item "Close Current Environment" and the following line will be added:

```
\end{itemize}
```

**Close Tag** Suppose you have typed

```
<start>
<bf> This is nice </bf>
```

Select the menu item "Close Tag" and the following item will be added

```
</start>
```

**Toggle XML**

When a source file with extension html, xml, or ptx is opened, TeXShop syntax colors it as an XML file rather than as a TeX file, and provides a different set of command completion items for the file. This item toggles this feature on or off regardless of the type of the file. Thus the syntax coloring of an XML file can be changed to ordinary TeX syntax coloring, and the syntax coloring of an ordinary TeX file can be changed to syntax coloring suitable for XML and html.

**Save Source Position for Portable**

Many users have a portable computer attached to a large monitor. They used the command "Save Source Position" to fix the size and position where source files open on the large screen. But when they travel, this extra room is not available. "Save Source Position for Portable" sets an alternate size and position for opening source files on the portable. TeXShop can distinguish the two cases, so source files then open in a reasonable position when traveling.

**Save Preview Position for Portable**

This commands behaves the same way for the Preview window.

**Source $\leftrightarrow$ Preview**

If a source window is active, this command brings the corresponding Preview window to the front. Conversely if a preview window is active, the corresponding Source window

comes to the front.

**Next Source Window**

This command cycles through open source windows, bringing each successively to the front.

**Previous Source Window**

This command reverses the cycle direction.

## 39.2 Items in the Preference Pane

**Distiller**

TeXShop sometimes needs to convert a postscript file into pdf. This is usually done with Ghostscript, but users who did not install Ghostscript can use a distiller provided by Apple in macOS. In case one method is unsatisfactory, it can be useful to switch to the other temporarily.

**Use -dALLOWPSTRANSPARENCY**

TeXShop uses Ghostscript in two crucial places. First, TeXShop can open and display postscript files. To do this, it converts the postscript file to a pdf file using ps2pdf, a program in the Ghostscript family, and then displays the pdf file. Second, TeXShop typesets in DVI mode by calling tex or latex to create a dvi file, and then calling dvips to convert the dvi file to postscript, and finally calling Ghostscript to convert the postscript file to a pdf file. An advantage of this chain is that PSTricks package can be used in the source to create special effects in the typeset output.

The Ghostscript program was originally written in postscript. Recently, the authors of Ghostscript rewrote the program in C. This rewrite revealed several spots where the original code introduced security bugs, and these bugs have been removed, occasionally by breaking very obscure features of postscript. Unfortunately for TeX, among the obscure features were all postscript transparency operators, so any transparency in the original postscript file is ignored in the pdf version of the file.

To fix this, the Ghostscript authors added a special flag, "-dALLOWPSTRANSPARENCY". When Ghostscript is called with this flag set, the transparency operators are recognized (while the other more serious security bugs are still quashed). This flag is available in Ghostscript 9.51 and higher.

A special preference titled "Ghostscript Options" under the Typesetting tab allows users to turn on this flag. The flag is off by default because some users have earlier versions of

Ghostscript or never use transparency, and because the flag might conceivably be removed at some future date.

The flag only affects the two TeXShop operations described in the initial paragraph of this section. It has no effect if the user has selected the Apple distiller rather than Ghostscript for conversion operations. I suspect the flag works for users who have a Ghostscript provided by MacPorts or Homebrew rather than the Ghostscript provided in MacTeX, but this has not been tested.

The DVI typesetting chain is controlled by the Preference items "TeX + dvips + distiller" under the Engine tab. These items call a script named *simpdftex* in TeXLive to control the typesetting process. This script recognizes many optional flags. A few are set by default. One of the optional flags not set by default is named *–distilleropts*; it allows a user to send flags directly to the distiller (i.e., Ghostscript) portion of the chain. If TeXShop discovers that this flag has been set by the user in TeXShop Preferences, then it does not add the -dALLOWPSTRANSPARENCY flag because the user is apparently controlling that operation directly.

**On Big Sur and Later**

In Big Sur (macOS 11), Apple introduced a new style for toolbar items. The new style only works on Big Sur and higher.

Since TeXShop runs on High Sierra (macOS 10.13) and above, it has to be able to draw toolbar items in both the old and new styles. This preference item allows users of recent systems to retreat to the old styles for toolbar items. I *strongly recommend* switching to the new style.

## 39.3 Modifying the Latex Panel

The Latex panel contains a "Custom" tab; up to sixteen user-defined buttons can be placed on this tab. Other buttons in the Latex panel cannot be changed, but the text inserted when the button is pressed can be changed.

To do so, edit the file `~/Library/TeXShop/LatexPanel/completions.plist` with TeXShop. The comments at the start of this file explain how to make these changes. Be sure to edit and save in UTF-8 format if you use Unicode characters.

## 39.4 Adding to the List of Files That Can Be Typeset

TeXShop can open a wide variety of files: tex, ltx, dvi, html, pdf, etc. The list contains many different kinds of text files, but only some of these files can be typeset. For other text files, the typeset button will be inactive, and the text will not be syntax colored.

Currently only files with the following extensions can be typeset:

```
tex, dtx, ins, sty, cls, rnw, def, ltx, mp, mf, bib, htx,
sk, skt, ly, Stew, lytex, ctx, bbx, cbx, gabc, gtex,
md, fdd, lhs, lua, xml, html, ptx, asy, lbx
```

There is a hidden preference to add file extensions to this list. For example, a user wrote me wanting to typeset TeXInfo files, which have extension ".texi". He had written an engine which called "texi2pdf" to do the typesetting; this binary is not in TeX Live, but it is available in MacPorts. However, the typeset button was inactive when he wanted to call the engine.

To add ".texi" to the list of filetypes which can be typeset, set a hidden preference in TeXShop using Terminal:

```
defaults write TeXShop OtherTeXExtensions -array-add "texi"
```

This adds texi to an internal hidden list of additional types. The command

```
defaults read TeXShop
```

will list all TeXShop Preference items, including hidden items, and their values, so the hidden list can be determined. To start over by erasing the entire list, enter

```
defaults write TeXShop OtherTeXExtensions -array
```

# Chapter 40

# Magic Comment Lines

Earlier sections listed "magic comment lines" to set the typesetting engine, encoding, root file, and other items. These lines are ignored by TeX, but seen by TeXShop. The lines should appear in the top 20 lines of a source file.

A small number of these lines are used routinely. Others are used in specialized situations. A few are obsolete. Here is a complete list.

## 40.1 Commonly Used Magic Lines

**% !TEX program =**
**% !TEX TS-program =**

Set the typesetting program used for this file, overriding all other ways to determine the program. The words ``tex'', ``pdftex'', ``latex'', ``pdflatex'' indicate the built-in TeX + DVI, pdftex, LaTeX + DVI, pdflatex engines. Otherwise use the exact name and spelling of the appropriate engine.

**% !TEX encoding =**

Set the encoding used to open the file, overriding all other methods to determine this encoding. Irrelevant if all files use UTF-8 Unicode. Important if you deal with a mixture of encodings

**% !TEX root =**

Point to the root file which includes this file. Can be a full path or a relative path. Should appear at the top of all included or input files, but definitely not at the top of the root file.

**% !TEX parameter =**
**% !TEX TS-parameter =**

Occasionally used. Equals the string sent to an engine as the second argument $2 of an engine command. If the magic line is missing, the second argument will be a single space.

**% !TEX spellcheck =**

The exact name of the Spelling Dictionary to be used with this source file. Only important when a user writes documents in multiple languages.

**% !TEX numberingCorrection = 0 + current - desired**

The number of preliminary pages before the main text numbering starts at page 1.

## 40.2 Magic Lines to Control Preview in Double Page Modes

**% !TEX PageDirectionL2R**

The choice for languages written from left to right. Pages flow from left to right.

**% !TEX PageDirectionR2L**

The choice for languages written from right to left. Pages flow from right to left.

**% !TEX bookDisplay**

The initial page stands alone, and then double pages begin. Since books start with an odd number of preliminary pages, this causes the page pairs to correspond to the two pages of an open book. This is the default behavior of the Preview Window.

**% !TEX standardDisplay**

The double pages begin immediately. Useful in Japan for certain special books.

## 40.3 Magic Lines for ConTeXt

**% !TEX useAlternatePath**

Tells the typesetting engine to find the binary program in the location "Alternate Path" set in TeXShop Preferences under the Engine tab. This is mainly used by ConTeXT, which provides an alternate distribution containing only files required by ConTeXt.

**% !TEX useConTeXtSyncParser**

The author of ConTeXt, Hans Hagen, likes to rewrite portions of the TeX infrastructure. He rewrote the SyncTeX system for ConTeXt, and provided his own file to parse the information in the resulting synctex file. This line causes TeXShop to use that alternate parsing method.

**% !TEX useOldSyncParser**

This magic line is obsolete. Hans Hagen first wrote his own code to create the synctex file output by Jerome Lauren's SyncTeX, but he based his code on the original version of SyncTeX rather than the current version. This command parsed that very old style file. This magic line might well disappear in later versions of TeXShop.

## 40.4 A Magic Line for Beamer

**% !TEX pdfSinglePage**

Sets the Preview Window's Display Mode to SinglePage style even if it is set in preferences as MultiPage. When creating and displaying slides, single page is the appropriate choice.

## 40.5 Obsolete Magic Lines

**% !TEX useTabs**
**% !TEX tabbedFile(**

```
These commands are described in the Changes document
and were created when Apple first introduced tabbed windows.
There are now better ways to use these windows described
in this manual. The old magic lines interfere with these
better methods.
```

**% !TEX projectFile =**

```
A very old method used to handle projects with a root file and
included chapter files. Hasn't been used in the last decade.
```

## 40.6 Pseudo-Magic Lines for Engine Files

Very recent versions of TeXShop introduce magic lines to be used when constructing engine files. These lines can appear anywhere in the engine, and can start at any position in a line. The first character of these lines is not a comment symbol because engines can be written in many languages, each with its own method of writing comments. But the magic lines should appear within appropriate comments so the engine itself does not see them.

Here are the magic lines:

```
!TEX-noPreview
!TEX-pdfPreview
!TEX-htmlPreview
!TEX-bothPreview
!TEX-noConsole
```

The standard behavior of TeXShop still applies if an engine has none of these lines. After executing all lines of the engine, TeXShop looks in the document directory to see if there is a file with the same name as the source, but extension ".pdf". If so, it opens this pdf file in the Preview Window, unless the Option Key is depressed.

If one or more commands are present, then

- if the line "!TEX-noPreview" is present, no preview window appears even if some of the other lines are also present;
- otherwise if the lines "!TEX-pdfPreview" and or "!TEX-bothPreview" are present, TeXShop searches the document directory to see if there is a file with the same name as the source, but extension ".pdf"; if so, it opens this pdf file in the Preview Window;

- if the lines "!TEX-htmlPreview" and or "!TEX-bothPreview" are present, TeXShop searches the document directory to see if there is a file with the same name as the source, but extension ".html"; if so, it opens this html file in the HTML Window;
- and if the line "!TEX-noConsole" is present, the console is not shown.

It is unlikely that this final line will be used with any engine except the one which directly opens html code without any intervening typesetting.

# Chapter 41

# Hidden Preferences

## 41.1 Introduction

A great many TeXShop Preferences can be set with the Preference Panel. But there are also "hidden preferences" for obscure settings that can only be changed with the command line. Changing these hidden items is discouraged, because it is easy to forget that they were set, buy a new machine, and find that the behavior of TeXShop is different.

Hidden Preferences are sometimes added when a new feature is introduced. If users don't like it, there is a hidden way for them to retreat to the previous behavior. Another use occurs when a bug is discovered that I don't know how to eliminate but do know how to mitigate. In that case, a hidden preference may be provided temporarily until the correct fix is discovered.

There is a way to discover all TeXShop Preferences and their current values. In Terminal, type

```
defaults read TeXShop
```

The document "Comment Lines and Hidden Settings" in the TeXShop Help menu lists all hidden settings which work in the current version of TeXShop, with the command to change each one. This chapter describes many of them in more detail, together with their history.

TeXShop 5.60 moved several hidden preferences into TeXShop Settings: AutoSaveEnabled and KeepBackup (Automatic Saving), EncodingMenuInSaveDialog (the encoding menu in the Save dialog), MakeatletterEnabled and expl3SyntaxColoring (syntax coloring), and DisplayAsBook (showing the first page alone). The sections below say where to find them. TeXShop 5.60 also stopped reading a number of hidden preferences which no longer had

any effect. These are listed with the obsolete preferences at the end of this chapter.

## 41.2 Long Standing Hidden Preferences

These preferences have existed for a long time. Most of them are still active, if seldom used.

### <Preferences for Line Spacing in the Source Window>

The command below sets the line spacing in the source window. The default value 1.0 corresponds to standard single spacing, the value 10.0 corresponds to double spacing, and values outside the range 0.5 < value < 40.0 will be ignored.

```
defaults write TeXShop SourceInterlineSpace 1.0
```

### <Preferences for Trashing AUX Files>

Occasionally during typesetting, .aux files, .log files, and other files created automatically by TeX become corrupted and must be removed. TeXShop has a "Trash AUX Files" menu item and associated button in the console window. When either item is selected, TeXShop moves to the trash all files in the folder containing the source file with the same name as the source name, and extensions .aux, .bbl, .blg, .brf, .glo, .idx, .ilg, .ind, .ioa, .log, .log, .lot, .mtc, .mlf, .out, .pdfsync, and .toc. Other extensions can be added to this list, one by one. For instance, to add .dvi files to this list:

```
defaults write TeXShop OtherTrashExtensions -array-add "dvi"
```

To remove all added extensions and return to the original list:

```
defaults write TeXShop OtherTrashExtensions -array
```

Sometimes a more extensive cleanup is desirable. If the option key is held down while choosing "Trash AUX Files", TeXShop uses the

```
% !TEX root = ...
```

mechanism to find the root file. It then moves all files in the folder of this file and any subfolders of this folder to the trash if they have appropriate extensions, regardless of the names of the files. This behavior can be made the default behavior for "Trash AUX Files" without using the option key using the command

```
defaults write TeXShop AggressiveTrashAUX YES
```

### <Preferences for Command Completion>

TeXShop supports command completion. Type the first portion of a complicated TeX command and push the escape key. The rest of the command will be entered into the

source. The full command can be complicated, so TeXShop types "marks" within it; menu commands jump from mark to mark. These commands have shortcuts listed in the menu.

Since jumping to the next or previous mark is a common operation, TeXShop also allows users to type option-escape to get to the next mark and control-escape to get to the previous mark.

A Preference Panel item allows users to use "tab" instead of "escape" as the key which triggers command completion. In that case, the shortcuts which jump from mark to mark become option-tab and control-tab.

Occasionally, third party utilities make use of option-escape, control-escape, option-tab, and control-tab for other purposes. If TeXShop's use of these shortcuts interferes with these utilities, you can turn off these particular TeXShop shortcuts with a hidden preference:

```
defaults write TeXShop CommandCompletionAlternateMarkShortcut NO
```

**<Preferences for the Preview Window>**

When the preview window is updated after typesetting, it comes to the front. This behavior causes trouble for uses with an X11 editor running in Apple's X11 Window Manager. For these users, the behavior can be turned off via:

```
defaults write TeXShop BringPdfFrontOnAutomaticUpdate NO
```

TeXShop can be configured to automatically refresh pdf views when the pdf file changes. Once a second it examines the date and time when the pdf was last written to see if this information has changed. The time interval between these checks can be modified. To do this,

```
defaults write TeXShop RefreshTime 1.00
```

Automatic refresh for pdf views is useful if TeXShop is configured to use an external editor. In these cases, a .tex file is opened, but TeXShop only shows the associated pdf file. TeXShop also allows pdf files to be opened directly; this is useful for a brief glance at illustrations before embedding them in a TeX document. If you want pdf files opened for an external editor to be refreshed automatically, but pdf files opened for a brief glance to be left alone, issue the following command. (The default value is YES.)

```
defaults write TeXShop PdfFileRefresh NO
```

When TeXShop opens a .tex file for an external editor, it checks the dates of the tex and pdf files to make sure that the pdf output is up to date. If this output is not up to date or does not exist at all, TeXShop typesets the .tex file again. To turn off this behavior

```
defaults write TeXShop ExternalEditorTypesetAtStart NO
```

When the TeXShop Preview Window displays a page containing links and the mouse hovers over a link, a small window pops up showing the contents of the linked area. This feature can be toggled on or off using the menu "Link Popups" in the Preview menu. By default, the item is active when a document is first opened. To change this,

```
defaults write TeXShop LinkPopups NO
```

### <Preferences for the Preview Window Drawer>

When the Preview window first appears, its drawer is hidden. A hidden preference changes this behavior so the drawer is visible when the Preview window first appears.

```
defaults write TeXShop PreviewDrawerOpen YES
```

### <Preferences for Automatic Saving>

By default, automatic saving is on. I strongly recommend leaving it on, but it can be turned off. Starting with TeXShop 5.60, use the item **Enable Autosave and Versions** under the Misc1 tab of TeXShop Settings instead of the former hidden preference AutoSaveEnabled.

Before the days of Automatic Saving, TeXShop could be configured to save a backup file whenever it saves or typesets a source file. Starting with TeXShop 5.60, this is the item **Keep the previous file as "name~" when saving** under the Misc1 tab, which replaces the former hidden preference KeepBackup.

### <Preferences for the Matrix Panel>

There is a hidden preference to set the default size of the matrix in the Matrix Panel:

```
defaults write TeXShop matrixsize 12
```

### <Preferences for Server Problems>

Ulrich Bauer added a patch to make TeXShop perform better when used with a server if AutoSave is on. Many users thank us for this patch, but it caused problems for a few. To turn it off:

```
defaults write TeXShop WatchServer NO
```

**<Preferences for the Tags Menu>**

TeXShop adds lines which begin with the words

```
\section, \subsection, \subsubsection, \chapter
```

to the Tag menu. To turn this off

```
defaults write TeXShop TagSections NO
```

ConTeXt users may want additional words to be recognized and added to the Tag menu. For instance, they want

```
\subsubsubsection, \subsubsubsubsection, \part, \title, \subject,
\subsubject, \subsubsubject, \subsubsubsubject, \subsubsubsubsubject
```

The following command adds these words

```
defaults write TeXShop ConTeXtTags
```

## 41.3 Newer Hidden Preferences

**<Preferences for the Source Window>**

There is a preference item forcing TeXShop to place the source window in front of the preview window when opening files:

```
defaults write TeXShop OpenWithSourceInFront YES
```

This manual describes magic lines which manipulate the spelling dictionary. There is a preference to return to Apple's original method of handling this dictionary:

```
defaults write TeXShop OriginalSpelling YES
```

**<Preferences for Syntax Coloring>**

Syntax coloring was modified for the commants

```
\footnote, \footcite, \autocite
```

to also color inserted text. For example, the entire source phrase

```
\footnote{This is well known}
```

is syntax colored. There is a hidden preference to turn this feature off

```
defaults write TeXShop SyntaxColorFootnote NO
```

In recent versions of TeXShop, the syntax coloring code was turning off while the source file loaded. It is possible that this code was added to fix bugs if syntax coloring starts

too soon, but experiments suggest that the bug no longer exists. So in recent versions of TeXShop, syntax coloring begins immediately. In case of trouble

```
defaults write TeXShop ColorImmediately NO
```

TeXShop gives a Command color to symbols beginning with a backslash and continuing with 'a' - 'z' or 'A' - 'Z'. These are the typical commands used by Latex authors. Latex Macro authors also use '@' in commands. The item **@ as Part of Command Names** under the Editor tab of TeXShop Settings adds those to characters receiving a Command color; before TeXShop 5.60, this was the hidden preference MakeatletterEnabled. Latex3 programmers use '_', ':' and '@' in their commands. The item **Color expl3 Code by Default** under the Editor tab turns on expl3 coloring, which adds all of these, when documents are first opened; before TeXShop 5.60, this was the hidden preference expl3SyntaxColoring.

### <Preferences to Control Spell Checking>

TeXShop Preferences under the Source tab at bottom left has an item for Spell Checking. This item is explained in the main part of this Manual. There are some hidden preferences to fine tune this behavior. Starting with TeXShop 5.60, they are used only when TeX-aware spell checking is turned off; TeX-aware spell checking is configured with the file SpellingExclusions.txt instead.

The list of specialized TeX commands can be extended.. The first command below adds one more element to the existing array of user supplied exceptions. The second command erases the array so the user can start over. Note that neither command affects TeXShop's default list of special commands.

```
defaults write TeXShop ExtraCommandsNotToCheckParameters -array-add "\\verbatim"
defaults write TeXShop ExtraCommandsNotToCheckParameters -array
```

The second series of hidden preferences is related to an earlier experiment with spelling and TeX command parameters. Originally I thought it might be better to turn off spell checking the first two parameters for all TeX commands, and then have a list of special commands whose parameters should be checked. This is the opposite of the current logic. It is possible to test this idea:

```
defaults write TeXShop ExceptionListExcludesParameters YES
```

In this case there is a very small list of exceptions, mainly containing

```
\emph, \verbatim
```

but users can again add exceptions of their own:

```
defaults write TeXShop ExtraCommandsToCheckParameters -array-add "\\emph"
```

```
defaults write TeXShop ExtraCommandsToCheckParameters -array
```

### <Preferences to Control Apple Find Bar>

The behavior of the Apple Find Bar was changed in TeXShop. Earlier it picked items one by one, but now the entire window being searched darkens during a search and all instances of the phrase being searched stand out in white, with the active element in yellow. To get the original behavior back:

```
defaults write TeXShop IncrementalSearch NO
```

### <Preferences to Control Tags and Labels>

During one update, the code to create the Tags and Labels menus was made more efficient. But in case of trouble, users could revert some of this code. Trouble never materialized, so the following preferences are obsolete:

```
defaults write TeXShop UseNewTagsAndLabels NO
defaults write TeXShop CreateLabelList NO
defaults write TeXShop CreateTagList NO
```

### <Preferences for Coloring Indices>

TeXShop has a hidden preference setting to control the "ColorIndex" tool:

```
defaults write TeXShop IndexColorStart YES
```

### <Preference for Tabs>

TeXShop has two magic comment lines for tabs, "useTabs" and "useTabsWithFiles". These magic lines are mainly obsolete, as explained in the section of this document on tabbed windows. The first command automatically added "include" files as tabs, and a user asked that this be extended to "input" files. A hidden preference does that:

```
defaults write TeXShop TabsAlsoForInputFiles YES
```

### <Preferences for Save Dialog>

The optional menu to select the encoding was removed from the Save Dialog. Starting with TeXShop 5.60, it can be brought back with the item **Show Encoding in Save Dialog** under the Misc1 tab of TeXShop Settings, which replaces the former hidden preference EncodingMenuInSaveDialog.

### <Preferences for the Preview Window>

In double page mode, the initial page is shown by itself and then pages are shown side by side. This is called Book Mode. Starting with TeXShop 5.60, it is turned off with the item **Show first page alone as cover** under the Preview tab of TeXShop Settings, which replaces the former hidden preference DisplayAsBook.

Some users select very high magnification in the Preview Window, and then use a magnifying glass. This can dangerously deplete memory, so TeXShop does not allow the magnifying glass to work when the Preview magnification is above 250. While not recommended, this limit can be raised

```
defaults write TeXShop GlassMaxMagnification 300
```

When syncTeX syncs from the source window to the preview window, it highlights the selected text in yellow. This can be a little difficult to see. Years ago, TeXShop circled the selected text with a red oval rather than highlighting it. To make this the default

```
defaults write TeXShop syncWithRedOvals YES
```

### <Preferences for External Editors>

Two hidden preferences were added, but later replaced by better methods:

```
defaults write TeXShop TextMateSync YES
defaults write TeXShop OtherEditorSync YES
```

### <Preferences for Accessibility>

Voice Over is technology for those with limited sight; it reads the screen while the user controls the computer with keyboard commands. A user reported that Voice Over works in the TeXShop Source Window, but not in the Preview Window. This bug was in Monterey, and fixed in Ventura.

Further experiments revealed that Voice Over could be tricked into working in the Preview Window by issuing the menu commands “Next Page” and then “Previous Page”. The trick requires that documents contain at least two pages. Add a blank page to one page documents to make the trick work for them. This trick was off by default, but could be turned on to work automatically with the hidden preference FixVoiceOver. Since TeXShop 5.59 requires macOS 14.6 or later, where the bug is fixed, TeXShop 5.60 no longer reads this preference.

**<Preferences for Apple Intelligence>**

Apple Intelligence in Sequoia 15.2 introduces a new feature called Math Expression Completion. If a user types "$e =$" in an editor, the editor automatically adds "2.718"; if a user types "$x + \pi =$", the editor adds "3.14159". This addition is almost never appropriate in TeX source code, so TeXShop has a hidden preference item to turn it on or off. By default, it is off.

The feature is controlled by an item in the Apple API called MathExpressionCompletion, which has three possible values. The command

```
defaults write TeXShop MathExpressionCompletion 0
```

sets this item to NSTextInputTraitTypeNo and turns the feature off. Changing the last integer to 1 sets the item to NSTextInputTraitTypeYes; changing the last integer to 2 sets the item to NSTextInputTraitTypeDefault. Both of these choices turn the feature on. Setting the last integer to any other value sets the item to NSTextInputTraitTypeNo and turns the item off.

## 41.4 Obsolete Hidden Preferences

These settings work around bugs that were later permanently fixed, or were created for older versions of the program and no longer do anything.

TeXShop 5.60 no longer reads the following preferences in this section, so setting them has no effect: DoNotFixTeXCrash, DisplayLogInfo, UseTerminationHandler, RepeatTypesetOnError13, SourceScrollElasticity, FixLineNumberScroll, YosemiteScrollBug, FixSplitBlankPages, ContinuousHighSierraFix, ScreenFontForLogAndConsole, SyncTeXOnly, ReleaseDocumentClasses, and FixPreviewBlur.

**<Obsolete Preferences Introduced To Fix a Bug>**

Several years ago, a small number of users reported that while typesetting documents with at least 700 pages, typesetting would suddenly stop in the middle without an error message. This was the most difficult bug I ever faced. The Changes document explains how it was finally fixed. Before the fix, several hidden preferences were introduced so users could help search for a solution. All are now obsolete, but we list them below.

```
defaults write TeXShop DoNotFixTeXCrash YES
defaults write TeXShop DisplayLogInfo  YES
defaults write TeXShop UseTerminationHandler YES
defaults write RepeatTypesetOnError13 NO
```

### <Obsolete Preferences Related to the Flash Bug>

When Apple rewrote PdfKit, they introduced a significant bug. When a document was scrolled so the first page was not showing in the Preview window, and then the document was typeset, there would be an irritating flash when the new version of the document was loaded. I reported this bug to Apple, who eventually told me they would not be able to fix the bug. However, they suggested that I could fix it in TeXShop using the following elaborate method. Just before reloading the page, cover the preview window with a transparent view. Since the view is transparent, users will not notice it. Now draw a picture of the screen into this transparent view. Users will not notice that because the new image matches the image underneath. Now update the document. There will be no flash because it is hidden under the new view. Finally remove the covering view.

This fix works and has been used in TeXShop for several years. But it can be turned off:

```
defaults write TeXShop FlashFix NO
```

A key parameter in this fix is the length of time the transparent view remains on top of the window. If this time is too short, the flash will be visible when it is removed. If the time is too long, users won't be able to interact with the window until it is removed. Experiments showed that the ideal amount of time was a quarter of a second. Users didn't notice with this speed, while they felt a slight delay with longer times. But this value can be reset with a hidden preference:

```
defaults write TeXShop FlashDelay 0.25
```

### <Various Obsolete Bugfixes>

A bug in TeXShop 3.38 was caused by a single line of code added to help a few users in Japan. These users used Japanese input methods, customized the background and foreground colors of the source window, and picked a dark color for the background. The bug was fixed by removing that line of code. The users requiring the bad line of code can get it back by

```
defaults write TeXShop ResetSourceTextColorEachTime YES
```

We earlier discussed a Hidden Preference named FixPreviewBlur related to a bug causing a blur in the Preview Window when shown on large monitors. This bug was fixed long ago. When dealing with this bug, an additional and now obsolete hidden preference was introduced:

```
defaults write TeXShop InterpolationValue 3
```

Here

```
0 = NSImageInterpolationDefault
1 = NSImageInterpolationNone
2 = NSImageInterpolationLow
4 = NSImageInterpolationMedium
3 = NSImageInterpolationHigh
```

In Yosemite, there was a problem with the elasticity of scrolling in the source window, and a few hidden preferences were introduced to deal with this problem. The problem no longer exists.

```
defaults write TeXShop SourceScrollElasticity NO
defaults write TeXShop FixLineNumberScroll NO
```

In Yosemite, there was a problem scrolling with left and right arrows in the Preview Window. We provided a work-around, which could later be removed if Apple fixed the problem, as they eventually did:

```
defaults write TeXShop YosemiteScrollBug NO
```

Another bug caused blank split pages in the Preview window after typesetting. Our fix could be turned off if Apple later fixed the problem, as they did:

```
defaults write TeXShop FixSplitBlankPages NO
```

When scrolling the pdf display with a gesture or mouse in High Sierra, a bug prevented the system from updating the page number box. A fix was provided in TeXShop, but fearing that it would affect performance, a hidden preference was also provided to turn it off if Apple fixed the bug:

```
defaults write TeXShop ContinuousHighSierraFix NO
```

### <Obsolete Bug for the Entire Program>

When macOS is shut down, a dialog appears with an optional check box labeled “Reopen windows when logging back in.” If this is checked, all programs running when the machine was shut down will restart the next time the machine is rebooted. One user reported that TeXShop gets into an infinite loop when restarted in this way. I could not reproduce this problem.

There is a hidden preference for users who run into this problem. With this preference set, all other programs will start when the machine is rebooted, but TeXShop will not. Restart it manually.

```
defaults write TeXShop IgnoreStartOnReboot YES
```

**<Other Obsolete Preferences>**

These items should never be used, but are listed for completeness.

Before TeXShop had a preference to set the font for the console and log windows, there was

```
defaults write TeXShop ScreenFontForLogAndConsole YES
```

The following preference causes synchronization spots to be shown in the Preview window:

```
defaults write TeXShop ShowSyncMarks YES
```

TeXShop used to support a different set of source commands to determine the typesetting engine, file encoding, and root file; examples are

```
%&latex, %&encoding= UTF-8 Unicode, %SourceDoc ../Main.tex
```

This syntax was a poor choice on my part and has been changed. If you have a lot of old documents, you can temporarily turn this choice back on using the command below. This preference change should only be made in an emergency.

```
defaults write TeXShop UseOldHeadingCommands YES
```

The left and right arrow keys scroll left and right if the preview page is narrower than the total page width, but otherwise they page up and down. A hidden preference changes this behavior so the left and right arrow keys always page.

```
defaults write TeXShop LeftRightArrowsAlwaysPage YES
```

The console text reporting typesetting behavior and errors shows black text. The text can be made to switch to red after the first error with a hidden preference.

```
defaults write TeXShop RedConsoleAfterError YES
```

In the editing window, a soft line break occurs after words. This behavior can be changed to "no line break" = 0, "line break after words" = 1, or "line break after characters" = 2.

```
defaults write TeXShop LineBreakMode 1
```

TeXShop supports SyncTeX. If synctex fails, TeXShop falls back on the Search synchronization method. For users who would like to experiment with synctex by itself, there is a hidden preference to turn off this fallback behavior:

```
defaults write TeXShop SyncTeXOnly YES
```

Early versions of TeXShop considered backslash @ to be a single command which ended with the @ sign, so lines like the one below in style files would only be partly colored. This was later fixed.

```
\ifx\@@input\@undefined\let\@@input\input\fi
```

If you prefer the original method, turn off the item **@ as Part of Command Names** under the Editor tab of TeXShop Settings (before TeXShop 5.60, the hidden preference MakeatletterEnabled).

### <Other Obsolete Items>

The items in this section are now irrelevant and mentioned only for historical reasons. When TeXShop was released on Tiger, users ran into an annoying bug which caused the program to gradually slow to a crawl after several typesetting actions. This bug was fixed a couple of days after the release. The problem occurred when a new pdf file was loaded into the PdfKitView in the Preview window. According to Apple documentation, this should have released the previous data structure from memory. The release did occur, but it caused the program slowdown. The bug fix consisted of tricking the system into believing that the data structures were still being used so the system didn't try to release them.

Later a hidden preference was added to release the data if desired. The values of this preference were 0 to release the data on system 10.4.3 or higher, where the bug was less severe, 1 to never release the data, and 2 to always release the data. The default value was 1, causing the program to use the original workaround.

```
defaults write TeXShop ReleaseDocumentClasses 1
```

Apple fixed the problem on Leopard and none of these kludges are active on that system or higher.

When macOS Mavericks was released, the Preview display of text became slightly flurry on many screens. This problem is long gone, but at the time we attempted to improve the situation with

```
defaults write TeXShop FixPreviewBlur YES
```

When macOS Mountain Lion was released, Apple optimized text and font drawing for the Retina display. This caused a few fonts to appear slightly blurry in small sizes on a non Retina display. The following preference reverted back to the earlier behavior. This problem was fixed long ago, so the preference is now irrelevant:

```
defaults write TeXShop NSFontDefaultScreenFontSubstitutionEnabled YES
```

**<Obsolete Color Preferences>**

Originally, many syntax colors, foreground and background colors, and other TeXShop colors could be modified using hidden preferences. These preference items still exist, but they now do nothing. Modern colors are determined by themes, which are stored as small files in `~/Library/TeXShop/Themes` rather than in the Preferences database.

With syntax coloring, comments are colored red, commands are colored blue, and the symbols $, {, and } are colored dark green. These colors could be changed. A color is determined by the red, green, and blue components of the color; each is a number between 0.00 and 1.00.

```
defaults write TeXShop commentred 0.0
defaults write TeXShop commentgreen 1.0
defaults write TeXShop commentblue 0.0

defaults write TeXShop commandred 0.0
defaults write TeXShop commandgreen 1.0
defaults write TeXShop commandblue 0.0

defaults write TeXShop markerred 0.0
defaults write TeXShop markergreen 1.0
defaults write TeXShop markerblue 0.0
```

The background color of the source window could be changed.

```
defaults write TeXShop background_R 0.42
defaults write TeXShop background_G 0.39
defaults write TeXShop background_B 0.77
```

The text color of the source window could be changed. This change requires that syntax coloring be on.

```
defaults write TeXShop foreground_R 0.42
defaults write TeXShop foreground_G 0.39
defaults write TeXShop foreground_B 0.77
```

The color of the insertion point in the source window could be changed.

```
defaults write TeXShop insertionpoint_R 0.42
defaults write TeXShop insertionpoint_G 0.39
defaults write TeXShop insertionpoint_B 0.77
```

The background color of the preview window could be changed. This background color will not affect printing.

```
defaults write TeXShop Pdfbackground_R 0.42
```

```
defaults write TeXShop Pdfbackground_G 0.39
defaults write TeXShop Pdfbackground_B 0.77
```

The transparency of the source, preview, and console windows could be changed. Here an alpha value of 0.00 is completely transparent and an alpha value of 1.00 is completely opaque.

```
defaults write TeXShop ConsoleWindowAlpha 0.75
defaults write TeXShop SourceWindowAlpha 0.75
defaults write TeXShop PreviewWindowAlpha 0.75
```

Version 2.38 of TeXShop introduced new parenthesis matching code and the ability to display invisible characters. The new code employs three colors: a temporary color assigned to parenthesis pairs (magenta), a temporary color assigned to the background of the text between these parenthesis pairs (yellow), and the color of invisible characters (orange). The first of these colors could be changed in the Preference Panel. The other two colors were controlled by hidden preference items:

```
defaults write TeXShop highlightContentRed 1.00
defaults write TeXShop highlightContentGreen 1.00
defaults write TeXShop highlightContentBlue 0.00

defaults write TeXShop invisibleCharRed 1.00
defaults write TeXShop invisibleCharGreen 0.50
defaults write TeXShop invislbleCharBlue 0.00

defaults write TeXShop ReverseSyncRed 1.00
defaults write TeXShop ReverseSyncGreen 1.00
defaults write TeXShop ReverseSyncBlue 0.00
```

Version 2.38 also marks matching parentheses with a yellow badge over the parenthesis which flashes briefly and then disappears. To turn the badge off:

```
defaults write TeXShop brieflyFlashYellowForMatch NO
```

# Chapter 42

# Unusual Bugs

In a small number of cases, it may be easier to work around a bug than to fix it. When cases like that arise, they will be listed here. We'll also list very rare bugs with unexpected solutions. This section should be short because bugs are regularly fixed in TeXShop.

## 42.1 Tags

Two bugs were discovered in TeXShop 5.42, but these bugs have been active since 2005. When a document is first opened, the Tags menu is populated with tags for chapters, sections, and so forth. Later as the document is edited, the Tags menu is revised; new tags are added as they are entered into the source, and tags are removed when they are removed from the source. But this revision step did not happen if the toolbar was in "Text Only" mode, and it never happened to the Tags menu in the TeXShop menu bar. Note that the menu bar version only appears if an appropriate preference item is selected.

In TeXShop 5.43 and later, this bug is fixed for the tabs menu in the toolbar. It was impossible to fix for the Tags menu in the menu bar without creating performance issues when large documents are edited. However, there is an easy workaround. If the Tags menu in the menu bar is noticeably out of date, use the toolbar's tag menu just once. Then the tags menu in both places will be up to date and either can be used.

## 42.2 Zombie Untitled Windows

If macOS is configured appropriately, then when TeXShop is quit while windows are open, these windows will appear in their old positions when the program is restarted. To restart TeXShop without opening the old windows, hold down the Option key while TeXShop restarts.

In this configuration, there is a very rare bug which can cause TeXShop to open a zombie untitled window. This window cannot be closed, and appears each time TeXShop starts. Other things begin to go wrong as well. I've been notified of this behavior exactly twice in the last eight years.

This bug occurs because there is mangled information about old window locations in the folder `~/Library/Autosave Information`. TeXShop has no code to write to this location and the mangled entries come directly from macOS. To fix the bug, go to this folder and remove all files containing the word "TeXShop" in their names. Then restart the Macintosh.

# Chapter 43

# Contributors

A great many people contributed code to TeXShop. For a partial list, see https://pages.uoregon.edu/koch/texshop/extras.html. I'd like to mention a very small subset to illustrate the different ways one can contribute to an open source project.

### Overall Architecture

At three different points in the life of TeXShop, programmers rewrote and reorganized large sections of the code. Their work was crucial; without it, TeXShop would have been a lump of spaghetti that could never be untangled. Max Horn, Dirk Olmes, Yusuke Terada.

### Specific Features

Some major features in TeXShop are contributions by a single author. Macros and the Macro Editor are by Mitsuhiro Shishikura. Command Completion is by Greg Landweber. The Latex Panel is by Geoffroy Lenglin and the Matrix Panel is by Jonas Zimmermann. Herbert Schulz provided all the TeXShop engine support for John Collin's script pdflatexmk, and wrote crucial documents explaining Command Completion and TeX Live's automatic conversion of eps to pdf during typesetting.

### Independent Projects Used by TeXShop

Some independent projects were incorporated whole into TeXShop. OgreKit and the OgreKit Find Panel are by Isao Sonobe. Line numbers in the source window are provided by the NoodleLineNumberMarker and NoodleLineNumberView Cocoa classes by Paul Kim. The Sparkle update code used by TeXShop and many other programs is by Andy Matuschak.

### Localization

Localization must be done by native speakers. Nine languages including English are currently supported. French by Jean-Claude De Soza, German by Steffen Wolfrum, Italian by Nicola Vitacolonna, Spanish by Juan Luis Varona Malumbres, Brazilian Portuguese by Emerson Ribeiro de Mello, Chinese by Linus Yang, Korean by Karnes Kim, Japanese by Yusuke Terada.

### Icon

The TeXShop program icon is by Thiemo Gamma. Designing icons is the most difficult task a programmer can face; I've never been able to come close. Thiemo's icon showed up in my mailbox one day and I immediately switched to it. I know that Apple's current style is square with rounded corners, but Thiemo's icon will remain.

Starting with TeXShop 5.60, the application icon places Thiemo's round design inside a white rounded square, the shape used by current versions of macOS, with the letters TeX adjusted to look centered. The item **Dock Icon While Running** under the Misc2 tab of TeXShop Settings selects one of five Dock icons: Original, Classic Triangle, Retina Triangle, Classic Circle, and the current icon. The selected icon is shown only while TeXShop is running; after TeXShop quits, the Dock shows the standard icon again, and the icon in the Finder never changes.